

CS222

Operating Systems

Lecture 10

Memory Management

(Section 100001)

ผศ. ดร. กษิติศ ชาญเขียว

ckasidit@tu.ac.th

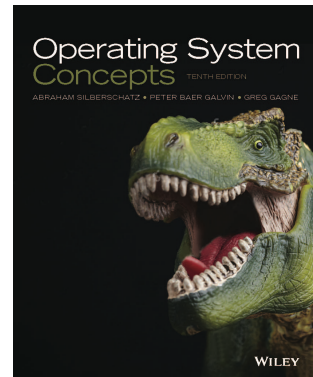
Textbook

- Avi Silberschatz, Peter B. Galvin and Greg Gagne; Operating System Concepts, 10th Edition; John Wiley & Sons, Inc; 2018;

- Chapter 9 (10th edition)
Memory Management

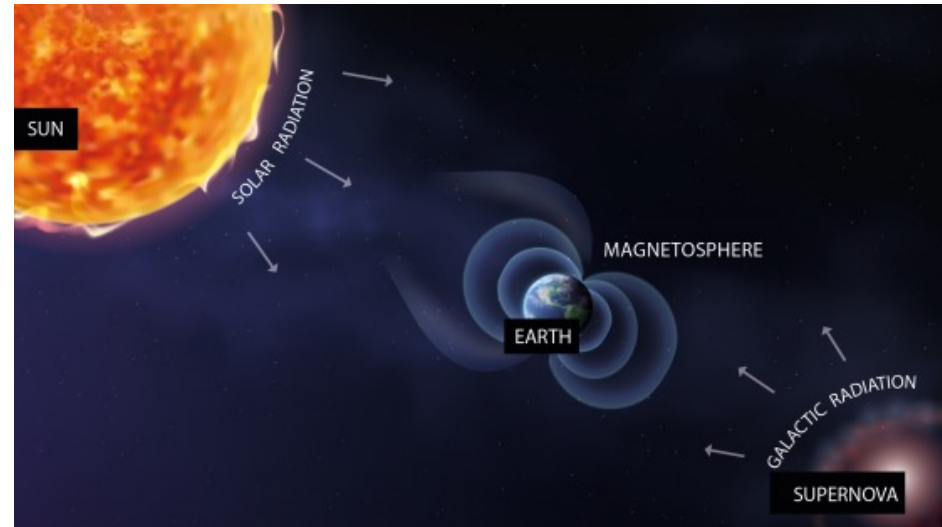
- Original Slides

- <https://www.os-book.com/OS9/slide-dir/index.html>
- <https://www.os-book.com/OS10/slide-dir/index.html>



เกี่ยวกับ cosmic rays

- อนุภาค ที่มาจากภายนอกโลก
- ชนกับชั้นบรรยากาศ แล้วทำให้เกิดอนุภาคอื่นตกมายังพื้นโลก
- อนุภาคเช่น นิวตรอน อิเล็กตรอน
- ส่งผลต่อระบบ ดิจิตอล
- ตย. คะแนนผลการเลือกตั้งในเบลเยียมเพิ่ม
- หรือ ผู้เล่นเกมส์ Supermario สามารถกระโดดได้เกินกว่าชั้นที่โปรแกรมกำหนด
- บน Space Shuttle ต้องใช้ คอม 4 เครื่องพร้อมกันและนำผลจากการคำนวณทุกอย่างมา vote เป็นค่าผลลัพธ์
- Astronaut เห็นแสงสว่าง Light Flash ในตา

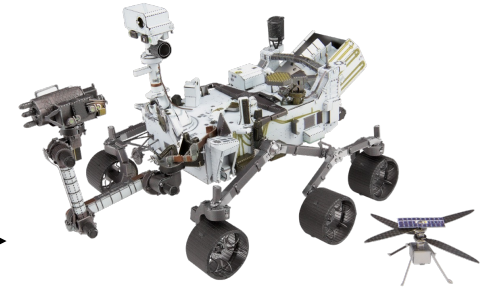


เกี่ยวกับ Radiation กับอุปกรณ์ Electronics

- ใน MARS rover ต้องใช้ Mem น้อย และซีพียูป้องกัน Radiation (รวมทั้งจาก cosmic rays)

Comparison of embedded computer systems on board the Mars rovers

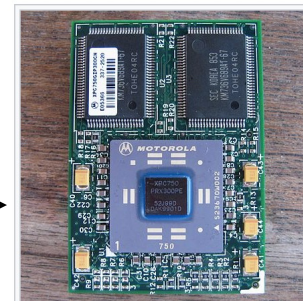
Rover (mission, organization, year) ⇄	CPUs ⇄	RAM ⇄	Flash ⇄	EEPROM ⇄	Operating system ⇄
<i>Sojourner Rover</i> (Pathfinder, NASA, 1997) ^{[1][3][4][5]}	2 MHz ^[6] Intel 80C85	512 KB	176 KB	None	Custom cyclic executive
<i>Pathfinder Lander</i> (NASA, 1997) ^[1] (Base station for <i>Sojourner</i> rover)	20 MHz MFC	128 MB	None	6 MB	VxWorks ^[7] (multitasking)
<i>Spirit</i> and <i>Opportunity</i> (Mars Exploration Rover (MER), NASA, 2004) ^[1]	20 MHz BAE RAD6000	128 MB	256 MB	3 MB	VxWorks (multitasking)
<i>Curiosity</i> (Mars Science Laboratory (MSL), NASA, 2011) ^{[1][8][9]}	200 MHz BAE RAD750	256 MB	2 GB	256 KB	VxWorks (multitasking)
<i>Perseverance</i> (Mars 2020, NASA, 2020) ^[10]	200 MHz BAE RAD750	256 MB	2 GB	256 KB	VxWorks (multitasking) ^[11]



- ระดับของ Radiation สูงกว่าบนโลกหลายเท่า

- อุปกรณ์อิเล็กทรอนิกส์จะทนต่อ Radiation

ได้พอสมควร ได้ในระดับหนึ่ง



300 MHz Motorola PowerPC 750 processor with off-die L2 cache on the CPU module from a Power Mac G3.



เกี่ยวกับ Memory (terms)

- แบ่งเป็นสองแบบได้แก่ ECC Memory และ Non-ECC Memory
- ECC Memory หรือ Error Correction Code (ECC) เป็นอุปกรณ์ฮาร์ดแวร์หน่วยความจำที่สามารถตรวจจับและแก้ไขข้อผิดพลาด หากเกิด BIT FLIP จำนวน 1 bit ได้
 - มักใช้บน Server ที่รันเป็นเวลานาน หรือ Mission Critical Computer ที่มีโอกาสที่จะถูกชนโดย ฝนอนุภาคที่เกิดจากการที่ cosmic rays ชนชั้นบรรยากาศโลก
- Non-ECC Memory คืออุปกรณ์หน่วยความจำธรรมดา ที่ไม่สามารถตรวจจับความผิดพลาดได้ มักใช้บน Desktop หรือ Notebook
- Soft error คือความผิดพลาดที่เกิดจากข้อมูลผิด อาจเกิดเพราะอุปกรณ์ผิดพลาด หรือ ไม่ได้เกิดจากสิ่งอื่นเช่น cosmic rays
- ECC ไม่สามารถป้องกันการโจมตีแบบ Row Hammer ที่ใช้วิธีการเขียน DRAM ในรูปแบบหนึ่งซ้ำแล้วซ้ำอีกและส่งผลให้ข้อมูลของ RAM ที่อยู่ใกล้เคียงตำแหน่งที่เกิดการเขียนบ่อยเปลี่ยนได้ *ถ้ามี bitflips มากกว่า 1 bit*

ความรู้ทั่วไป

อ้างอิง: ภาพและข้อมูลจาก Wikipedia

<https://www.metalearth.com/mars-rover-perseverance-ingenuity-helicopter>

<https://astronomy.com/news/2021/01/the-history-of-cosmic-rays-is-buried-beneath-our-feet>

https://www.youtube.com/watch?v=AaZ_RSt0KP8

<https://www.cashify.in/best-processor-for-mobile-phone-ranking-list>

<https://www.iaea.org/newscenter/news/cosmic-radiation-why-we-should-not-be-worried>

<https://docs.kernel.org/scheduler/sched-energy.html>

<https://docs.kernel.org/scheduler/sched-capacity.html>

Chapter 9: Memory Management

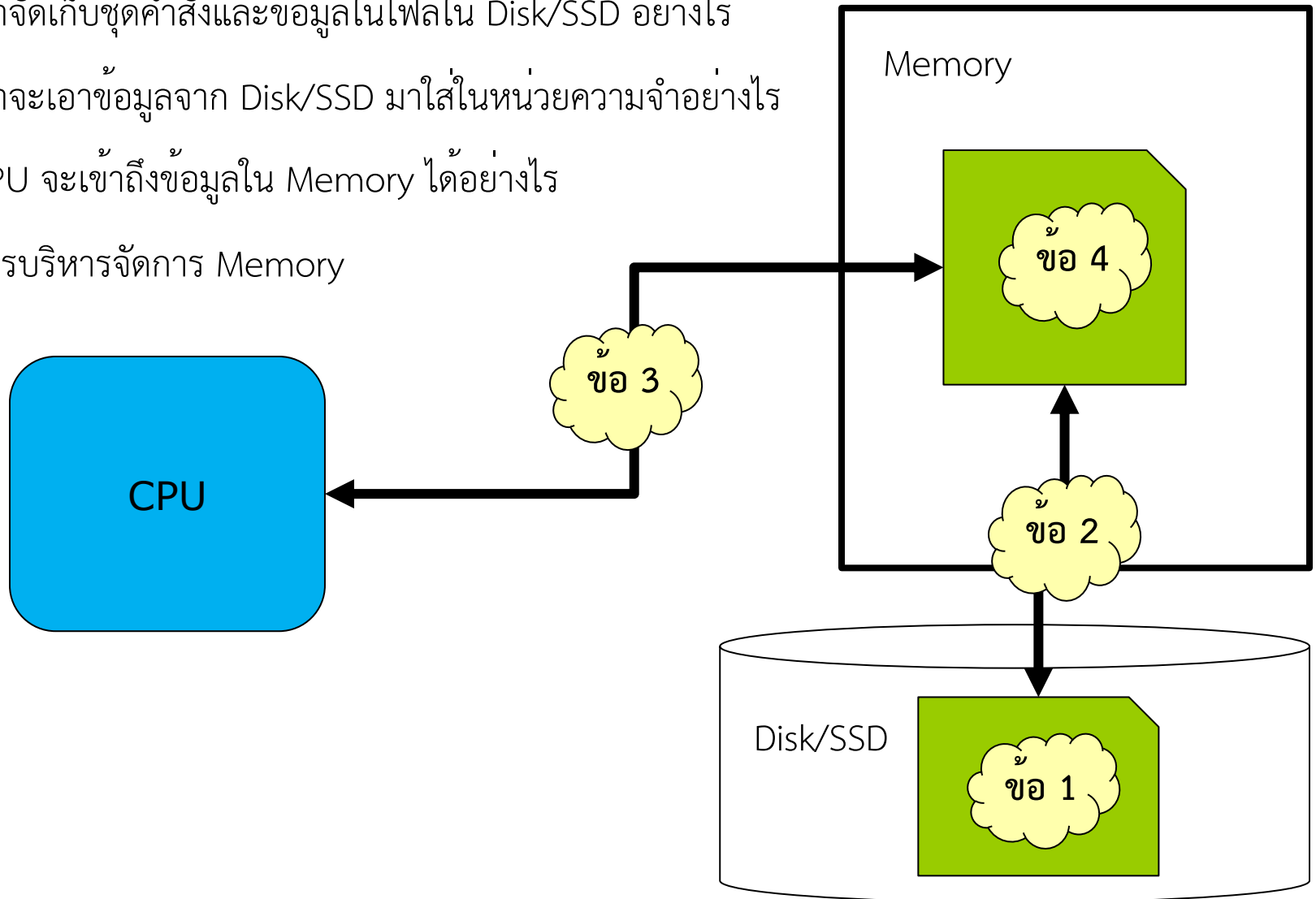
- Background
- Contiguous Memory Allocation
- Paging
- Structure of the Page Table
- Swapping
- Example: The Intel 32 and 64-bit Architectures
- Example: ARMv8 Architecture

Objectives

- To provide a detailed description of various ways of organizing memory hardware
- To discuss various memory-management techniques,

โจทย์ที่เราต้องการศึกษา

1. เราจัดเก็บชุดคำสั่งและข้อมูลในไฟล์ใน Disk/SSD อย่างไร
2. เราจะเอาข้อมูลจาก Disk/SSD มาใส่ในหน่วยความจำอย่างไร
3. CPU จะเข้าถึงข้อมูลใน Memory ได้อย่างไร
4. การบริหารจัดการ Memory



การจัดการ **Memory** แบบพื้นฐาน

Background

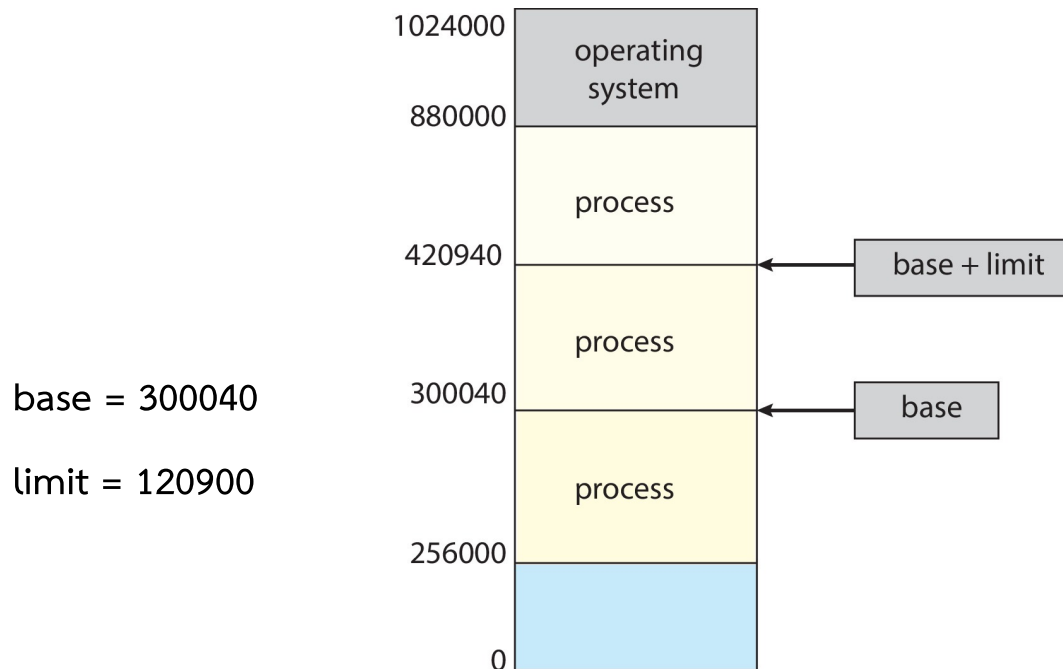
- **Program must be brought (from disk) into memory and placed within a process for it to be run**
- Main memory and registers are only storage CPU can access directly
- Memory unit only sees a stream of:
 - addresses + read requests, or
 - address + data and write requests

Background

- Register access is done in one CPU clock (or less)
- แต่ Main memory can take many cycles, causing a **stall**
- **Cache** sits between main memory and CPU registers
 - การบริหารจัดการ cache ทำโดย hardware
- **Protection of memory required to ensure correct operation**
 - ทุก Process มี Separate Memory Space
 - hardware ป้องกันการเข้าถึงข้อมูลของ Process หนึ่ง โดย Process อื่น

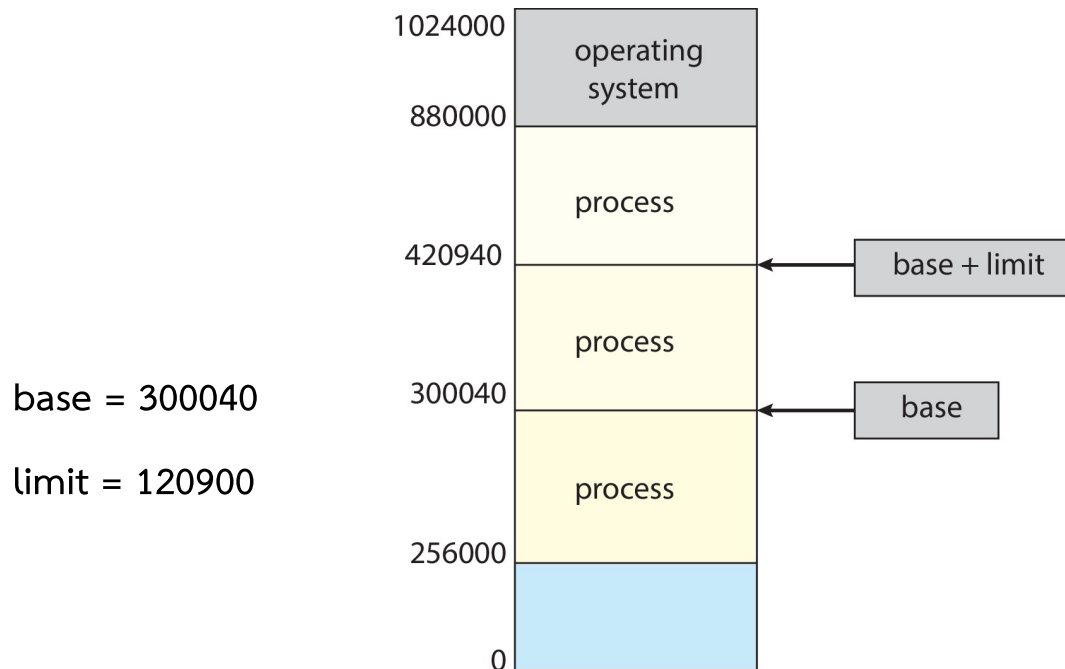
Protection

- Protection of memory required to ensure correct operation
 - ทุก Process มี Separate Memory Space
 - hardware ป้องกันการเข้าถึงข้อมูลของ Process หนึ่ง โดย Process อื่น



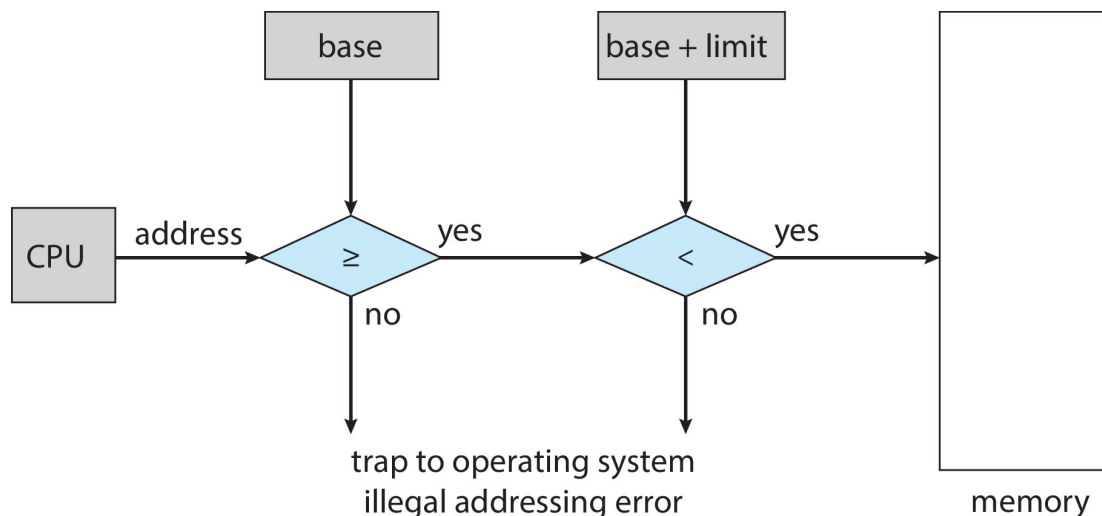
Protection

- Need to ensure that a process can access only those addresses in its address space.
- We can provide this protection by using a pair of **base** and **limit registers** define the logical address space of a process



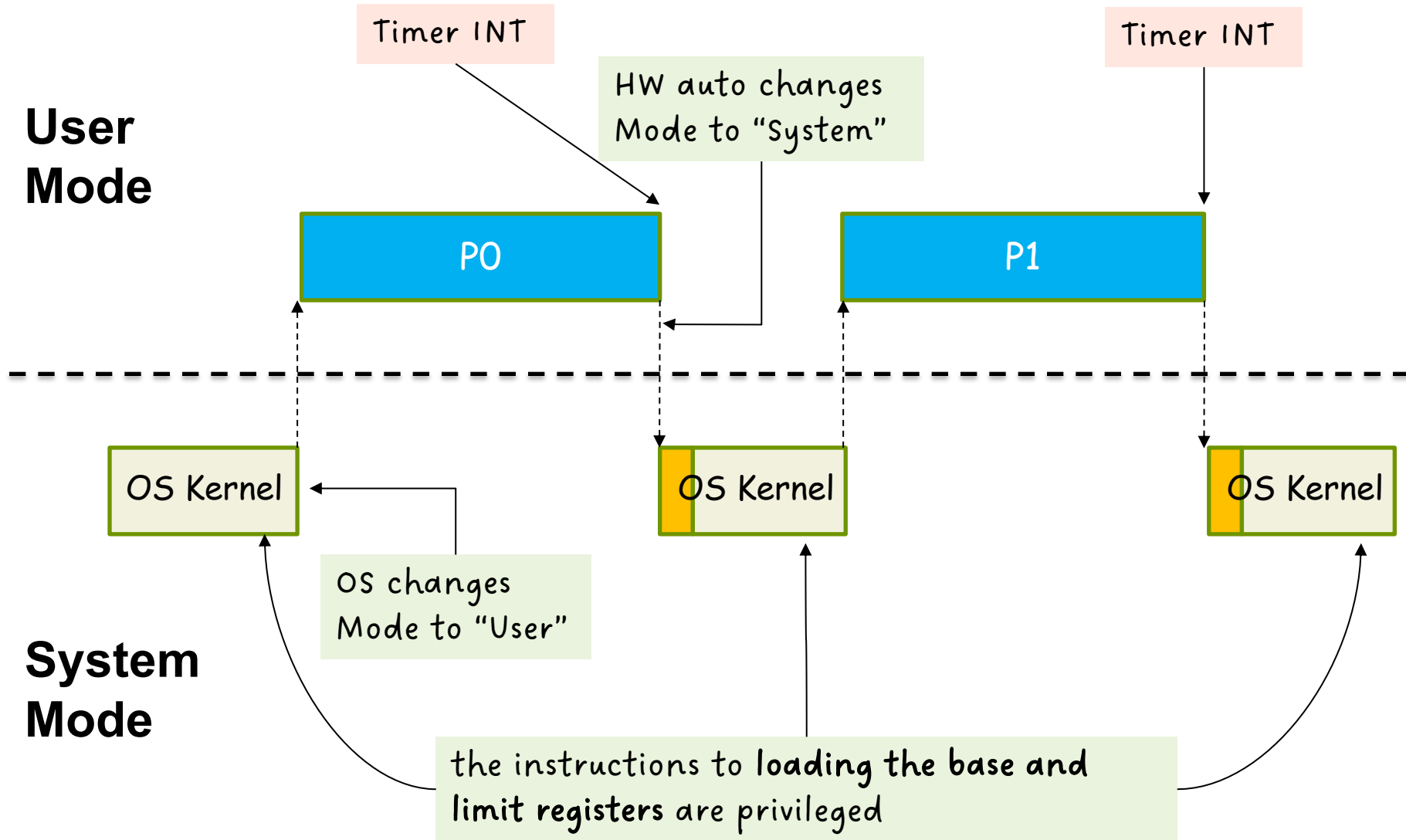
Hardware Address Protection

- CPU must check every memory access generated in user mode to be sure it is between base and limit for that user



- the instructions to loading the base and limit registers are privileged
 - CPU ต้องอยู่ใน System Mode เท่านั้น ถึงจะรันคำสั่ง Privileged Instruction ได้
 - CPU อยู่ใน System Mode เมื่อ OS Kernel ประมวลผล

CPU timeline

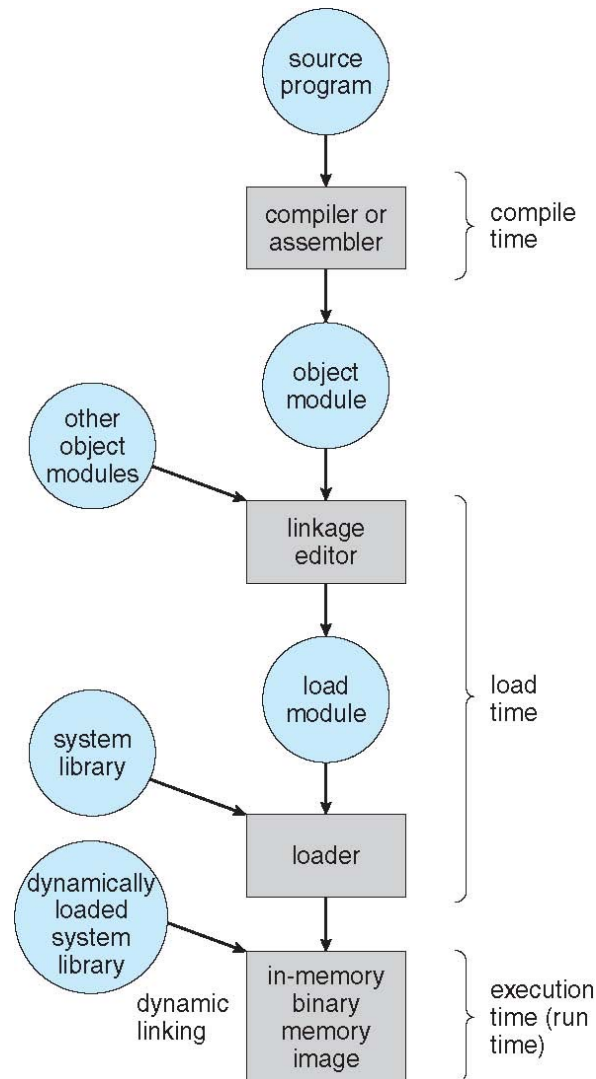


Address Binding

ระดับชั้นของ นามธรรม (abstraction)

- CPU ประมวลผลชุดคำสั่งแบบ binary code หรือชุดคำสั่งภาษาเครื่อง
- การออกคำสั่งด้วย binary code ทำได้ยากจึงเกิดการพัฒนาภาษาโปรแกรมขึ้น
- ภาษา Assembly ใช้สัญลักษณ์ขั้นพื้นฐาน
 - เราใช้ Assembler แปลงโปรแกรม Assembly ให้เป็น binary code
 - one statement ต่อ 1 machine instruction
- จากโปรแกรม C, C++, Java และ python เป็น High-level language (HLL)
 - เราสามารถใช้ Compiler แปลงโปรแกรมให้เป็น assembly code
 - และใช้ assembler แปลงโปรแกรมให้เป็น binary code
 - 1 statement มักจะถูกแปลงเป็น หลาย machine instruction
- นามธรรมของส่วนประกอบของโปรแกรมมีหลายระดับ

Multistep Processing of a User Program



ระดับชั้นของ นามธรรม (abstraction)

- ในภาษา มีการให้ตัวตนกับสรรพสิ่ง ถ้าสิ่งนั้นมีตัวตน ก็ต้องมีนาม (สัญลักษณ์)
- การให้ชื่อเกิดขึ้นในต่างระดับของนามธรรม

ตัวอย่าง	สัญลักษณ์ภาษาชั้นสูง	สัญลักษณ์ภาษา <i>assembly</i> (ทย. RISC-V, MIPS)
1	<code>int x, y, z;</code>	ตำแหน่งหน่วยความจำ (<i>address</i>) ที่ 01014 (<i>x</i> ใช้พื้นที่จากนั้น 4 bytes) ตำแหน่งหน่วยความจำ (<i>address</i>) ที่ 01018 ตำแหน่งหน่วยความจำ (<i>address</i>) ที่ 01022
2	<code>int f1(void){...}</code>	ตำแหน่งหน่วยความจำ (<i>address</i>) ที่ 01234
3	<code>z = x+y;</code>	<code>lw t0 0(01014)</code> <code>lw t1 0(01018)</code> <code>add t2 t0 t1</code> <code>sw t2 0(01022)</code>
4	<code>f1();</code>	<code>jal 01234</code>

Address Binding

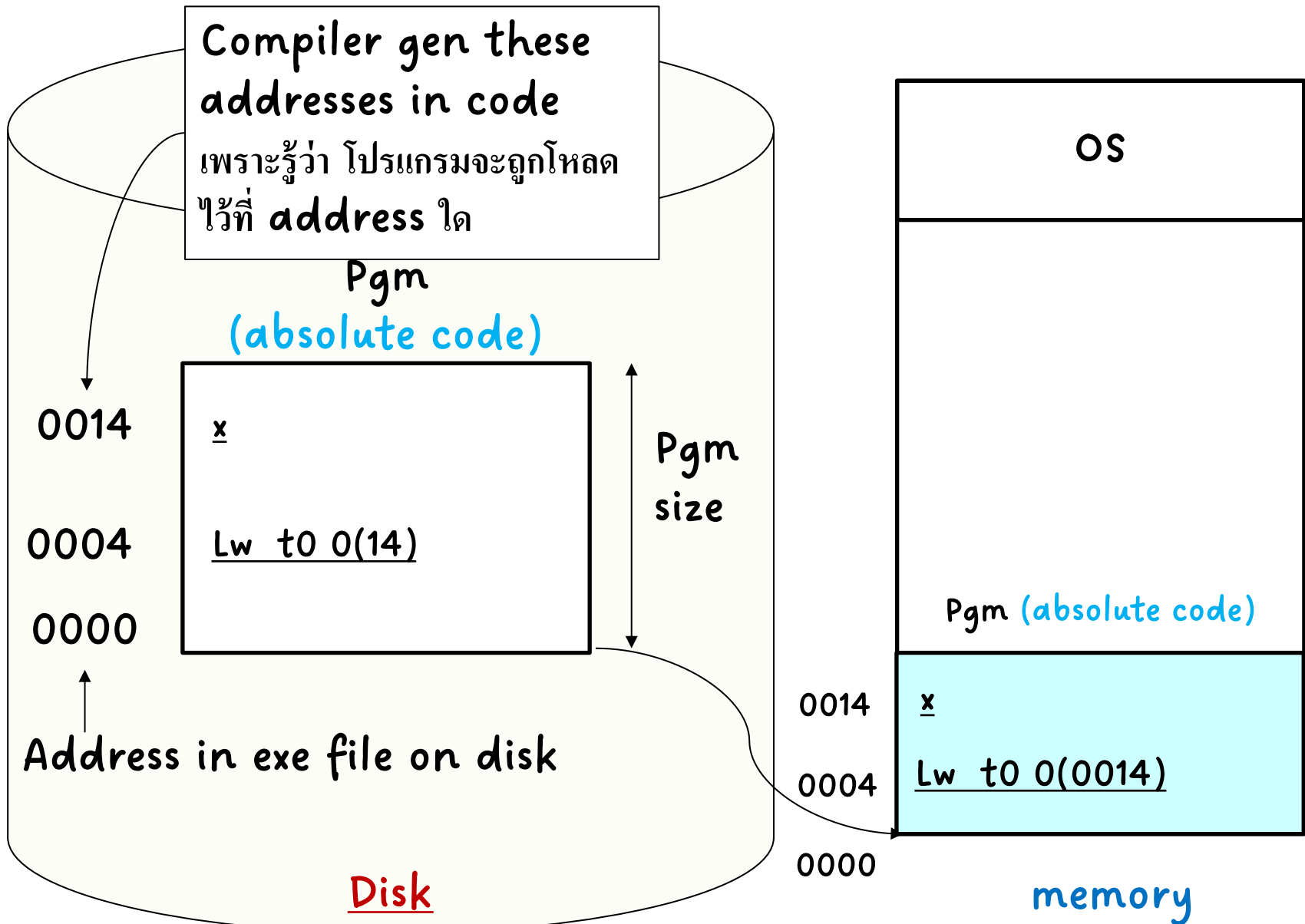
- Programs on disk, ready to be brought into memory to execute form an **input queue**
 - Without support, must be loaded into address 0000
- Inconvenient to have first user process physical address always at 0000
 - How can it not be?
- Addresses represented in different ways at different stages of a program's life
 - Source code addresses usually symbolic
 - Compiled code addresses **bind** to relocatable addresses
 - ▶ i.e., “14 bytes from beginning of this module”
 - Linker or loader will **bind** relocatable addresses to absolute addresses
 - ▶ i.e., 74014
 - Each binding maps one address space to another

Binding of Instructions and Data to Memory

- ในการรัน **Process** เนื้อหาของโปรแกรมจะต้องอยู่ใน **main memory**
- **binding** จาก **Address in exe file on disk** → **Address in memory**
- ทำได้สามตอน: ตอนคอมไพล์โปรแกรม ตอนโหลดโปรแกรม และตอน รันโปรแกรม
- หมายความว่า คุณจะรู้ค่า *physical mem addr* ของ ชื่อของตัวแปร หรือคำสั่งในโปรแกรมของคุณตอนไหน ตอนคอมไพล์ ตอนโหลด หรือตอนรันโปรแกรม

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - ต้องรู้ memory address ตั้งแต่คอมไพล์โปรแกรม
 - ใช้ในโปรแกรม MSDOS ในอดีต

ตัวอย่าง **compile time binding**



Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
 - การกำหนดค่า address จริง จะเกิดตอน โหลดโปรแกรมเข้าสู่ memory
 - ถ้า starting address เปลี่ยนก็ต้อง reload โปรแกรมใหม่ ให้ค่า address ของส่วนประกอบของโปรแกรมเปลี่ยนตาม starting address ที่เปลี่ยนไป

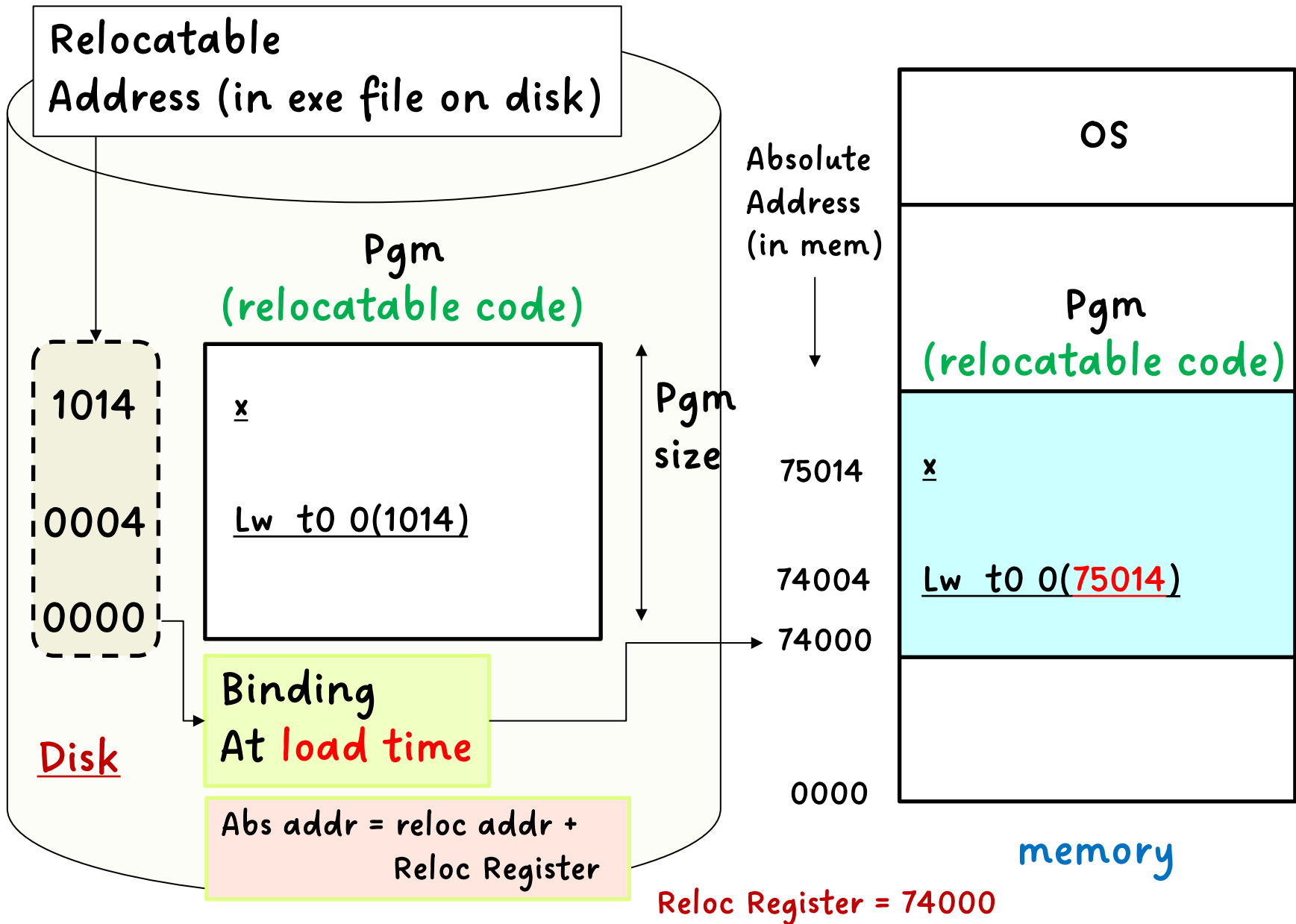
relocatable address

- ชื่อของส่วนประกอบของโปรแกรม ในระดับ Assembly และ Machine Code ล้วนแต่เป็น relocatable address
- compiler จะแปลงทุกชื่อในภาษาชั้นสูงให้เป็น “relocatable address” เหล่านี้ในไฟล์ executable
- เมื่อโปรแกรมประมวลผล โปรแกรมจะต้องถูกโหลดเข้ามาใน Hardware ที่ใช้เก็บข้อมูลแบบ array อุปกรณ์หนึ่งๆที่เรียกว่า Main Memory $M[n]$ ที่ประกอบไปด้วย i คือ index และ $M[i]$ คือค่าจำนวน 1 bytes จำนวน n ค่า

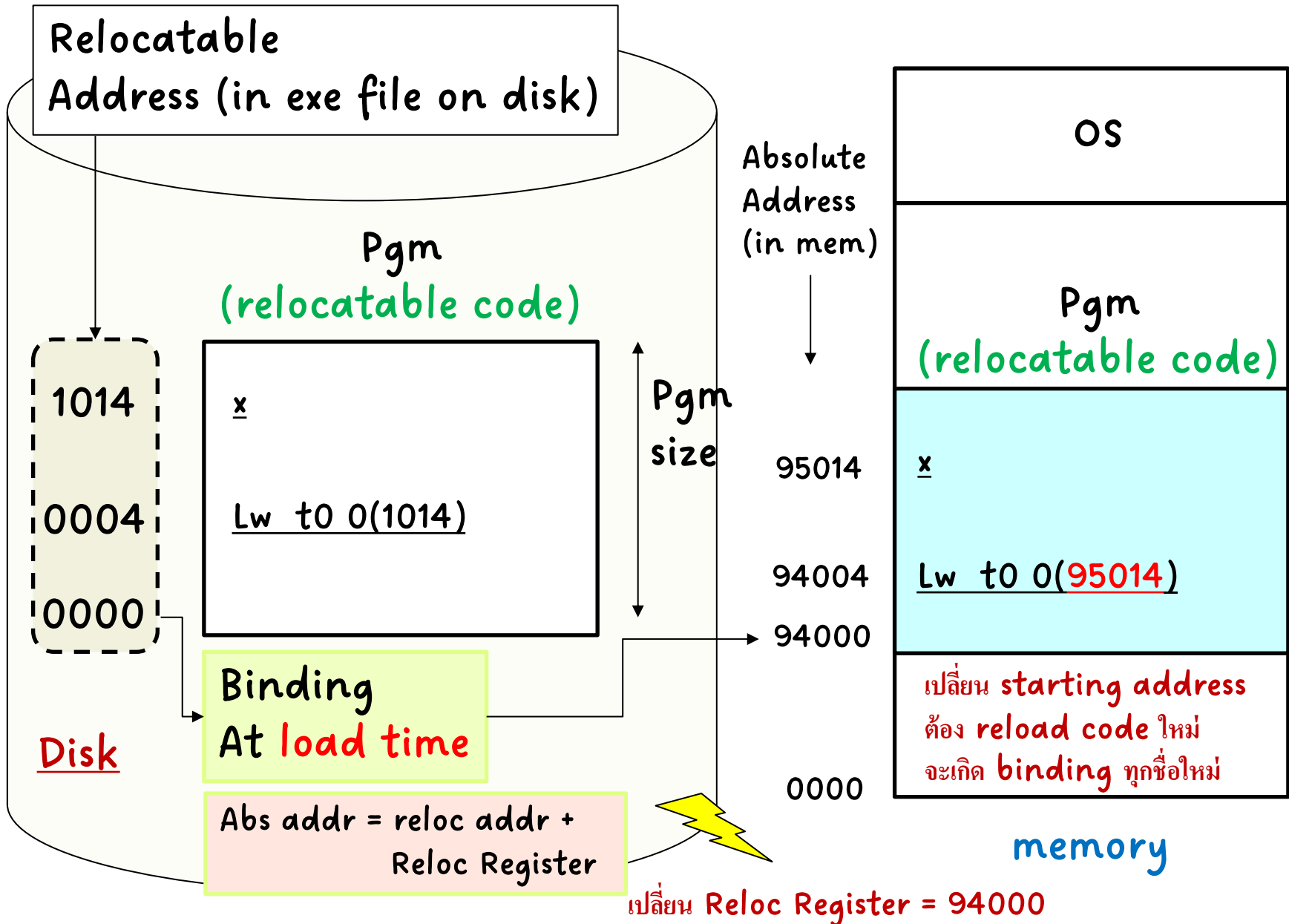
$M[i]$ $M[0]$ $M[1]$ $M[n-1]$

- การแปลง “relocatable address” ให้เป็นค่า i คือ Address Binding

ตัวอย่าง load time binding



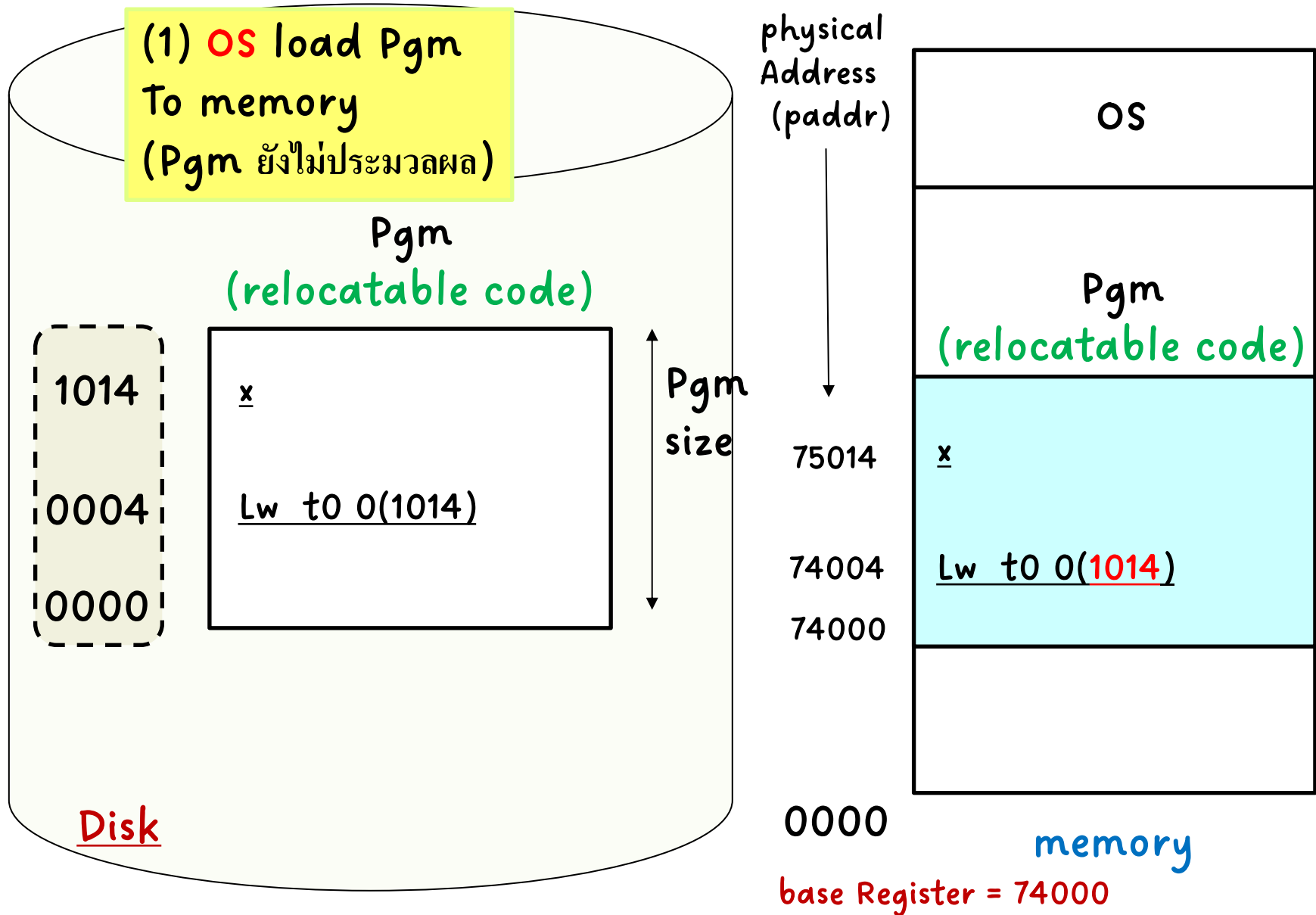
ตัวอย่าง load time binding (reload)



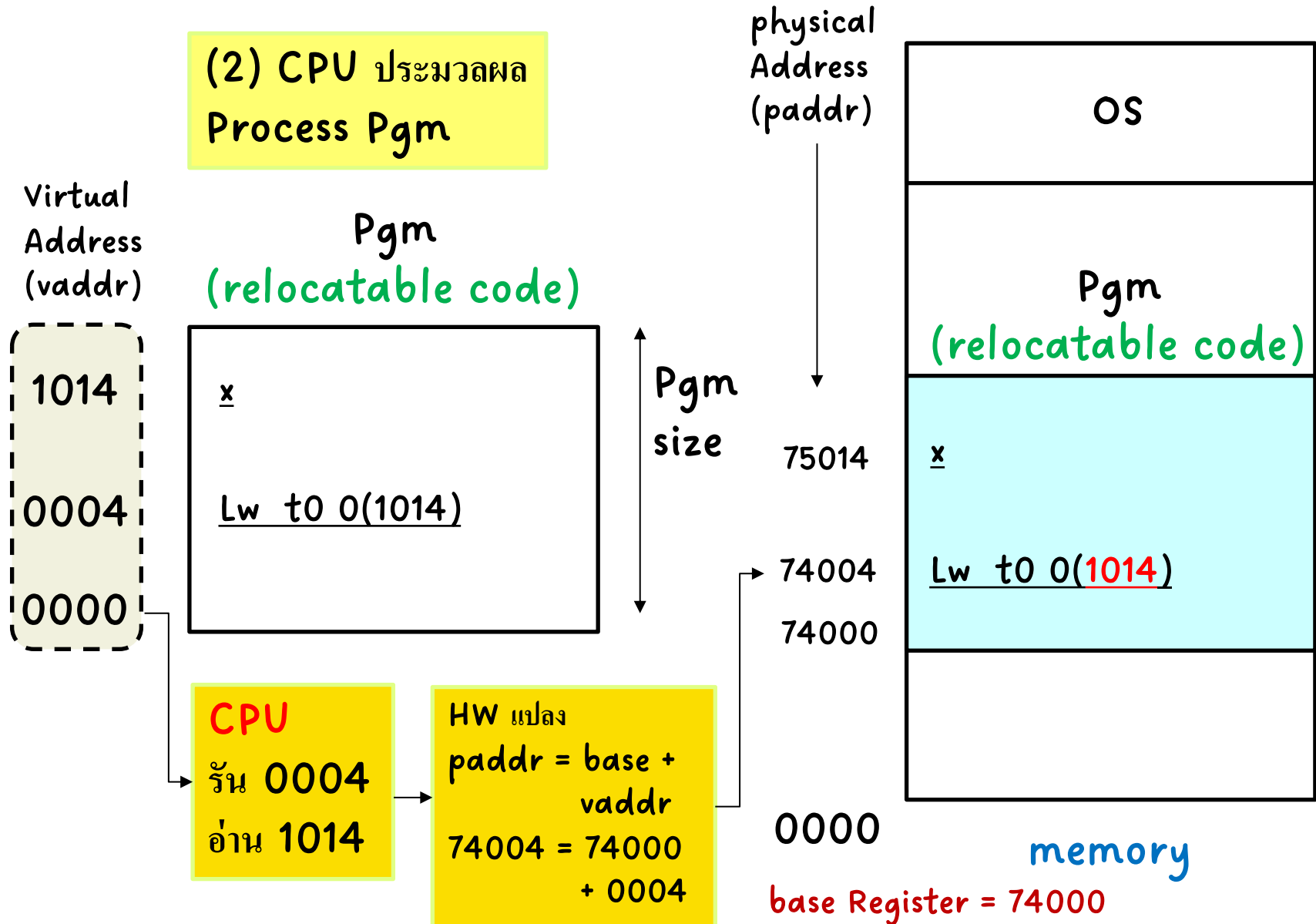
Binding of Instructions and Data to Memory

- Address binding of instructions and data to memory addresses can happen at three different stages
 - **Compile time:** If memory location known a priori, **absolute code** can be generated; must recompile code if starting location changes
 - **Load time:** Must generate **relocatable code** if memory location is not known at compile time
 - **Execution time:** Binding delayed until run time if the process can be moved during its execution from one memory segment to another
 - ▶ Need hardware support for address maps (e.g., **base** and **limit registers**)
- การกำหนดค่า address จริง จะเกิดตอน **โปรแกรมรัน** แต่ละคำสั่ง
- ในเรื่องของ “address” Process แต่ละ Process จะมีโลกเสมือน เป็นของตนเอง แยกจาก Process

ตัวอย่าง execution time binding (1)

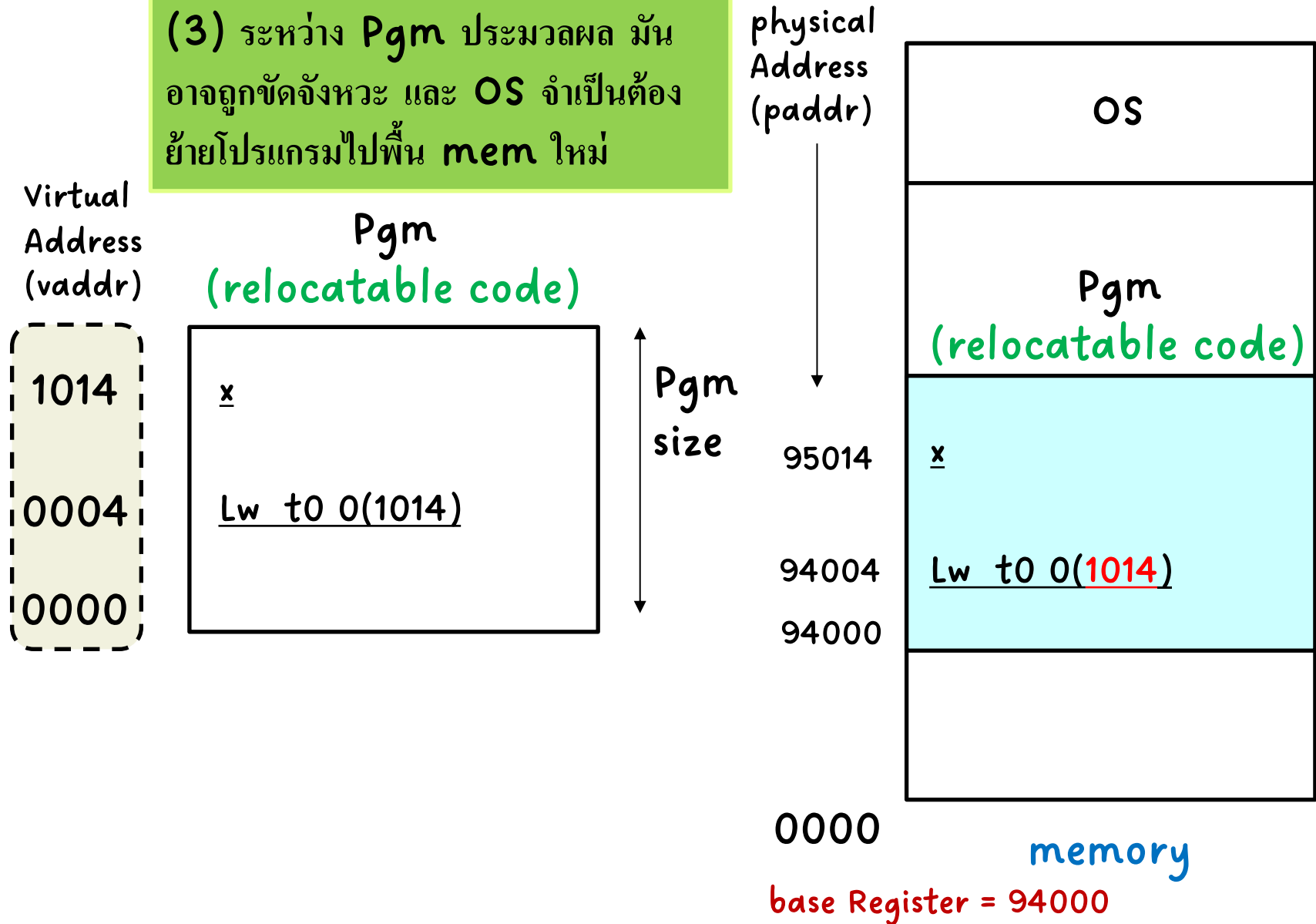


ตัวอย่าง execution time binding (2)

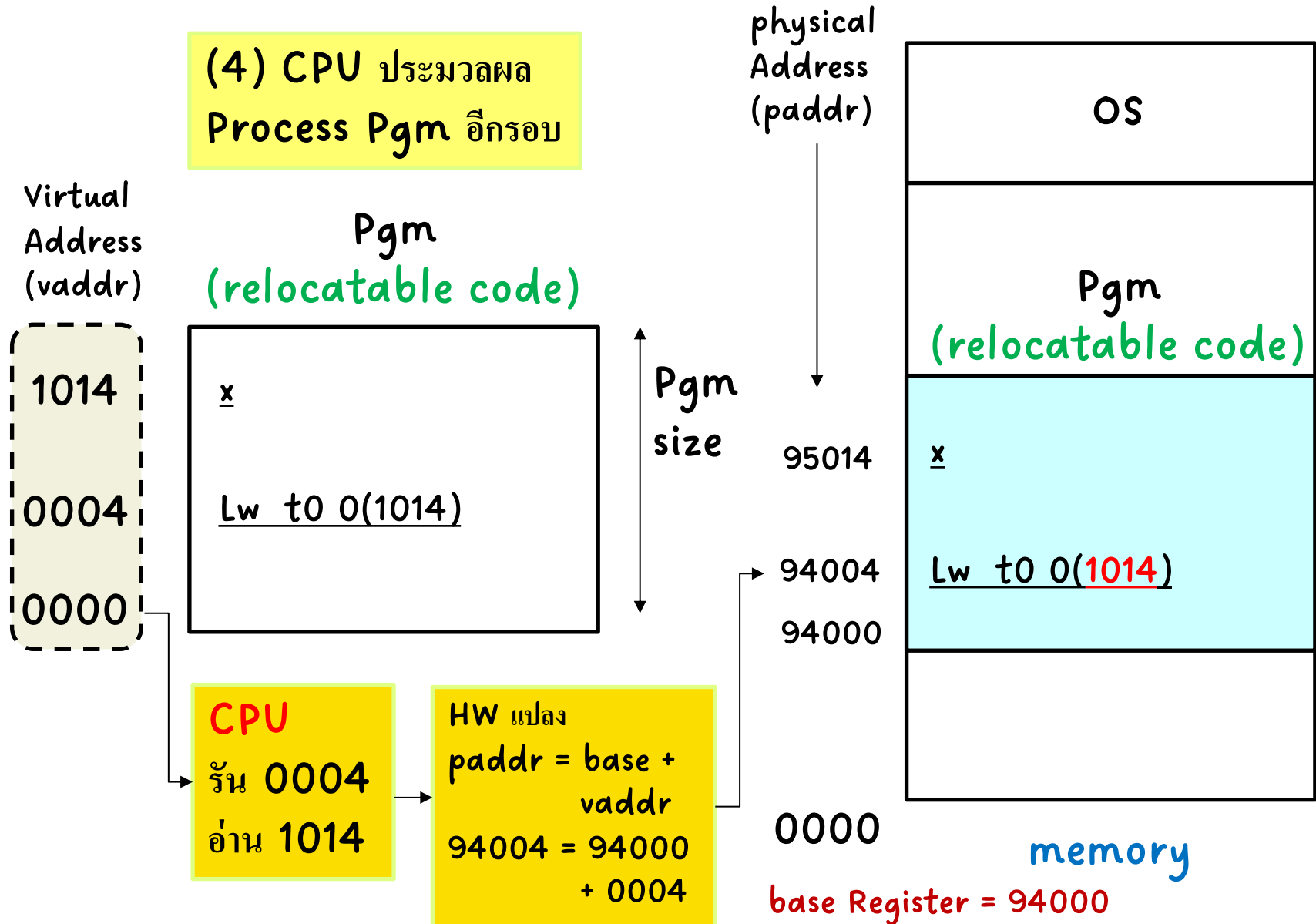


ตัวอย่าง execution time binding (3)

(3) ระหว่าง Pgm ประมวลผล มัน
อาจถูกขัดจังหวะ และ OS จำเป็นต้อง
ย้ายโปรแกรมไปพื้นที่ mem ใหม่

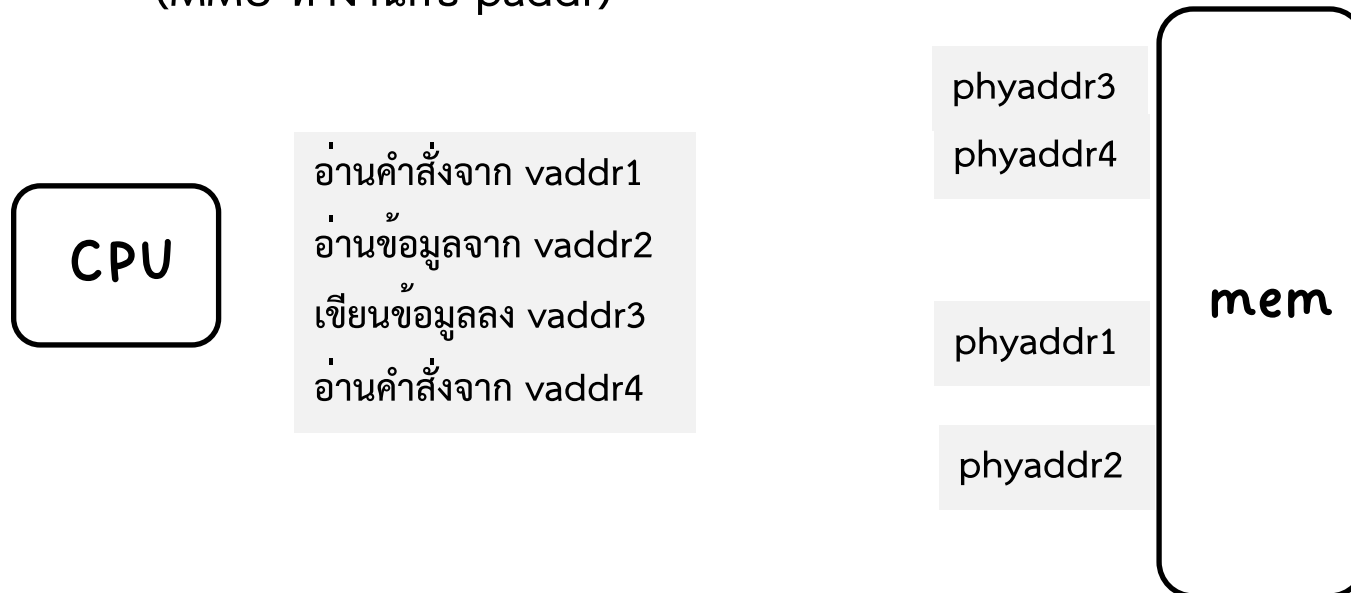


ตัวอย่าง execution time binding (4)



Logical vs. Physical Address Space

- ซีพียู อ่านคำสั่งและข้อมูลจาก Memory เข้ามารัน
- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
 - **Logical address** – generated by the CPU; also referred to as **virtual address** (ซีพียูทำงานกับ vaddr)
 - **Physical address** – address seen by the memory unit (MMU ทำงานกับ paddr)

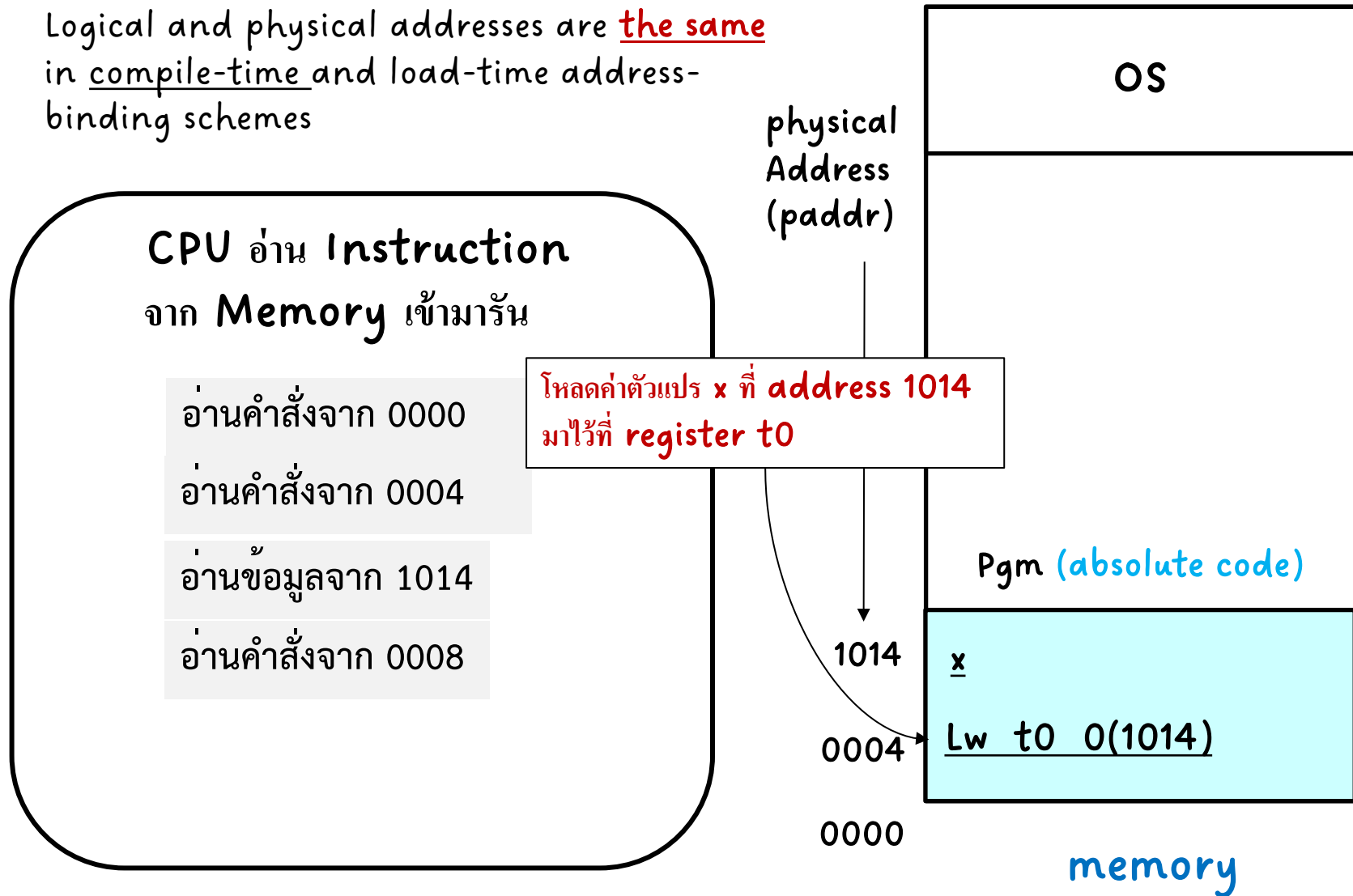


Logical vs. Physical Address Space

- The concept of a logical address space that is bound to a separate **physical address space** is central to proper memory management
 - **Logical address** – generated by the CPU; also referred to as **virtual address**
 - **Physical address** – address seen by the memory unit
- Logical and physical addresses are the same in compile-time and load-time address-binding schemes; logical (virtual) and physical addresses differ in execution-time address-binding scheme
 - หมายความว่า ซีพียูใช้ vaddr ค่าเดียวกันกับ psaddr ในกรณี compile-time & load time binding
 - ซีพียูใช้ vaddr ที่แตกต่างจาก paddr ในกรณี execution time binding

ตัวอย่าง **compile time binding**

Logical and physical addresses are **the same**
in compile-time and load-time address-
binding schemes



ตัวอย่าง load time binding

Logical and physical addresses are the same in compile-time and load-time address-binding schemes

CPU อ่าน Instruction
จาก Memory เข้ามารัน

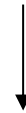
อ่านคำสั่งจาก 74000

อ่านคำสั่งจาก 74004

อ่านข้อมูลจาก 75014

อ่านคำสั่งจาก 74008

physical
Address
(paddr)



75014

x

74004

Lw t0 0(75014)

74000

0000

OS

Pgm

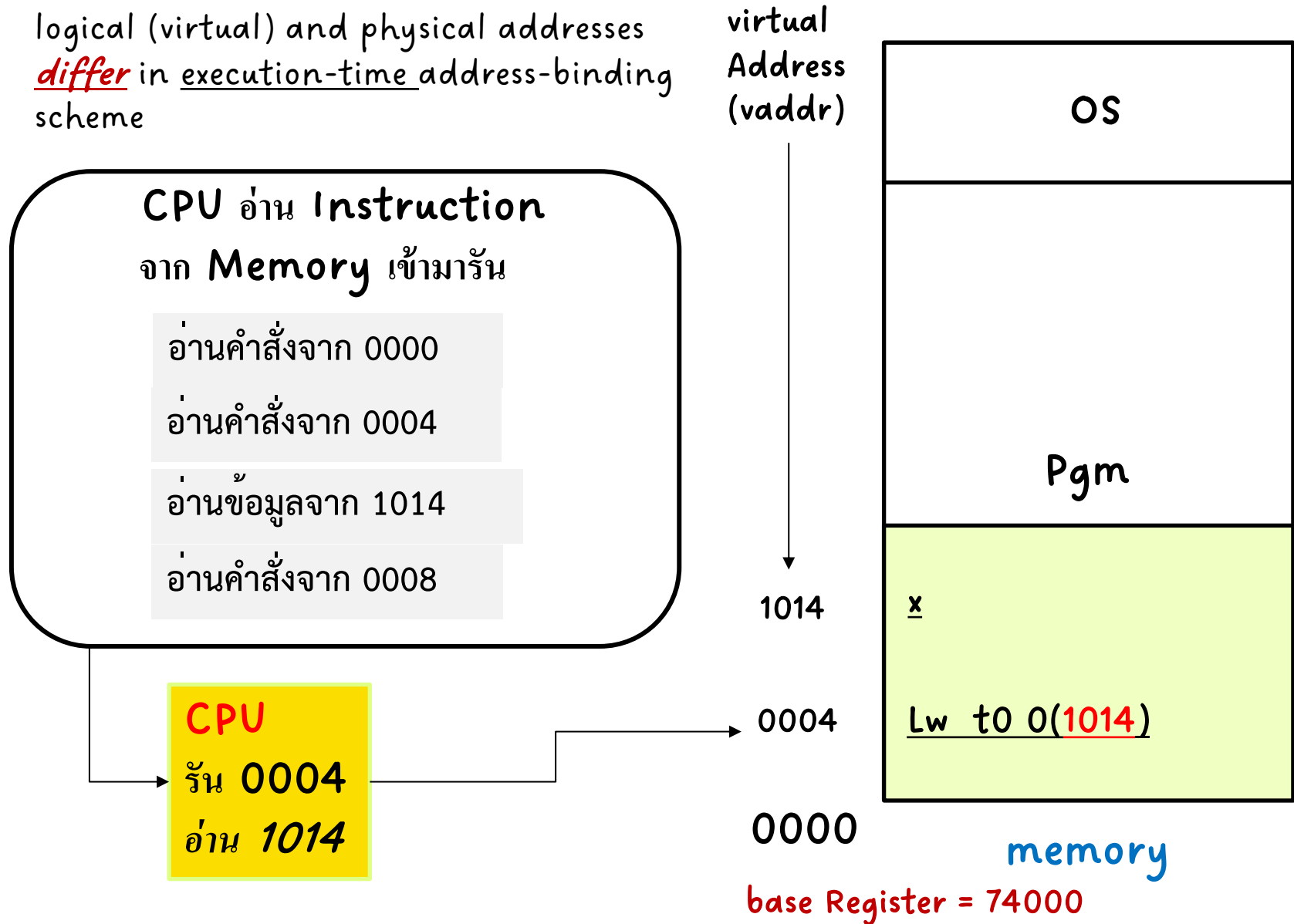
(relocatable code)

memory

Reloc Register = 74000

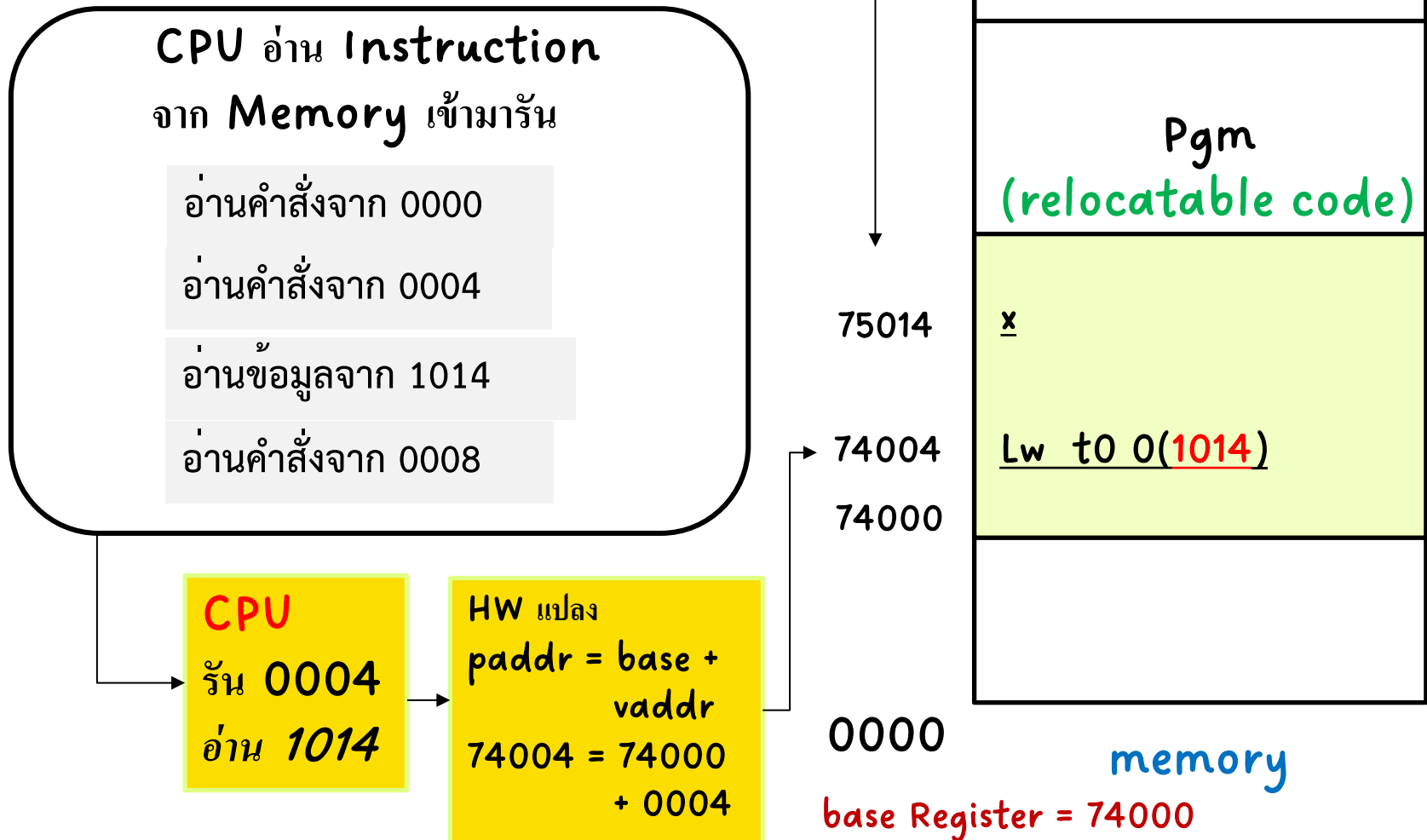
ตัวอย่าง execution time binding (1)

logical (virtual) and physical addresses
differ in execution-time address-binding
scheme



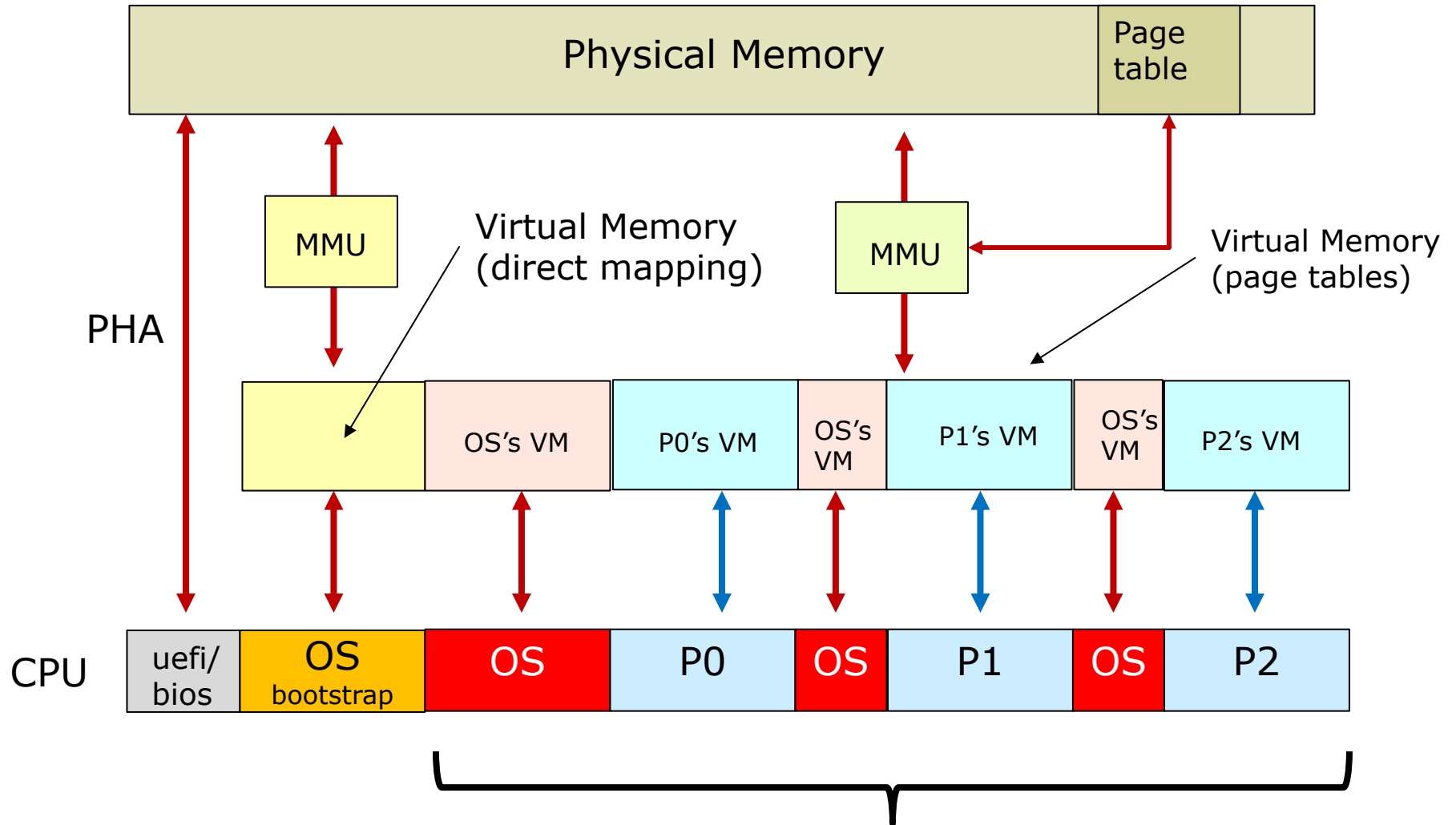
ตัวอย่าง execution time binding (2)

logical (virtual) and physical addresses
differ in execution-time address-binding
scheme



**Logical Address และ Physical Address
และ Memory Management Unit (MMU)**

เรื่องราว วิธีการเข้าถึง Memory ของ CPU



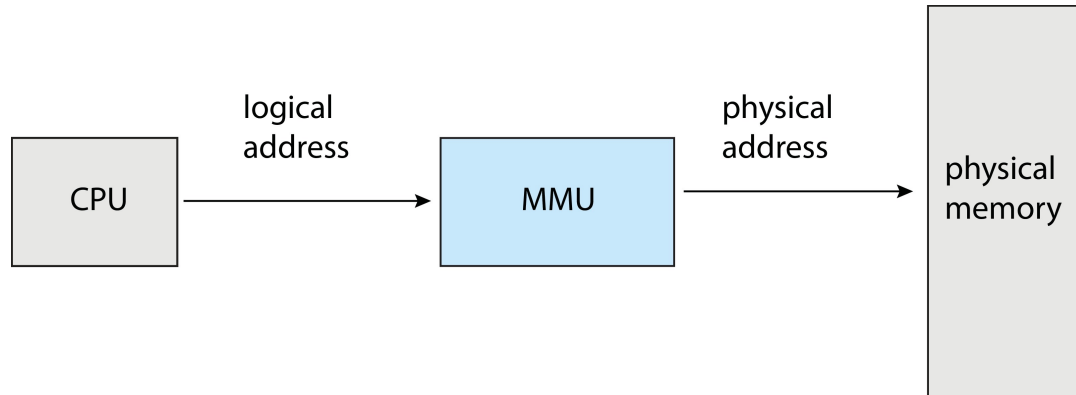
เราสนใจส่วนนี้ ณ เวลานี้

Logical vs. Physical Address Space

- **Logical address space** is the set of all logical addresses generated by a program
- **Physical address space** is the set of all physical addresses generated by a program
- ในปัจจุบัน OS และ HW ส่วนใหญ่เป็น Exe time binding
- ใช้ MMU ช่วย

Memory-Management Unit (MMU)

- Hardware device that at run time maps virtual to physical address



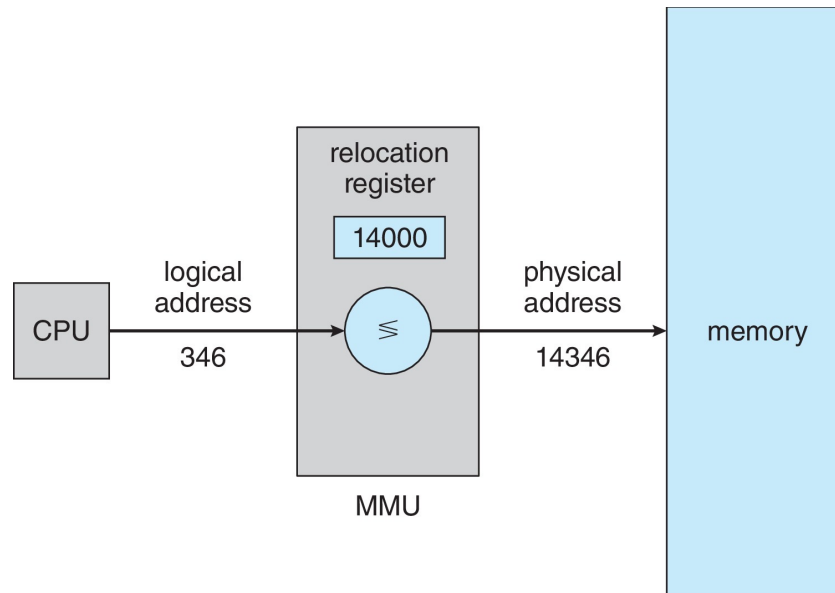
- Many methods possible, covered in the rest of this chapter

Memory-Management Unit (Cont.)

- Consider simple scheme. which is a generalization of the base-register scheme.
- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory
- The user program (and CPU) deals with *logical* addresses; it never sees the *real* physical addresses
 - Execution-time binding occurs when reference is made to location in memory
 - Logical address bound to physical addresses

Memory-Management Unit (Cont.)

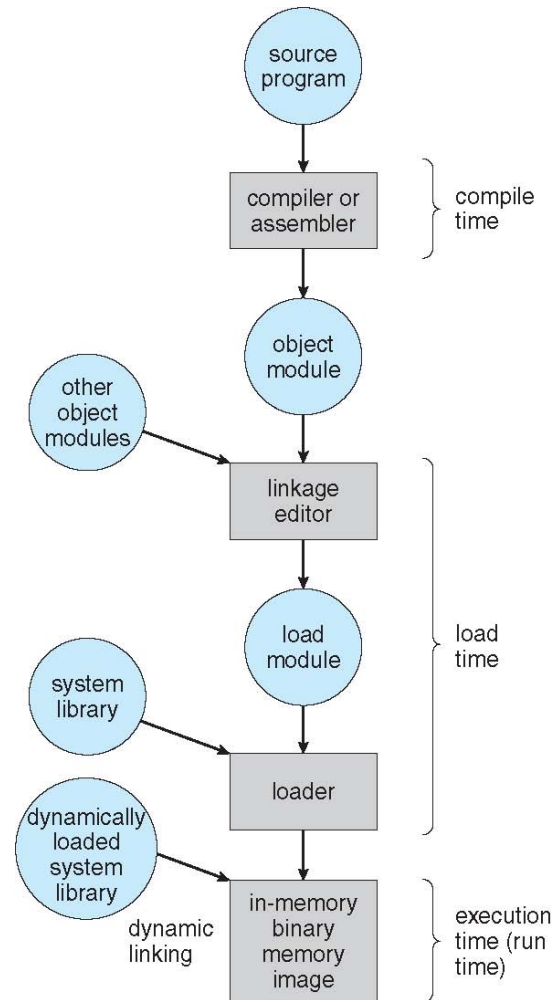
- Consider simple scheme. which is a generalization of the base-register scheme.
- The base register now called **relocation register**
- The value in the relocation register is added to every address generated by a user process at the time it is sent to memory



การโหลดโปรแกรมเข้าสู่ **Memory**

- **Dynamic Loading**
- **Dynamic Linking**

Multistep Processing of a User Program



Dynamic Loading

- The entire program does need to be in memory to execute
- Routine is not loaded until it is called
- Better memory-space utilization; unused routine is never loaded
- All routines kept on disk in relocatable load format
- Useful when large amounts of code are needed to handle infrequently occurring cases
- No special support from the operating system is required
 - Implemented through program design
 - OS can help by providing libraries to implement dynamic loading

Dynamic Loading

- โปรแกรมต้องถูก load เข้ามาใน Memory เพื่อ Execute
- Routine จะไม่ถูก load จนกระทั่งโปรแกรมเรียกใช้ routine นั้น
- ประหยัดพื้นที่หน่วยความจำ routine ที่ไม่ถูกใช้ก็ไม่ถูกนำเข้ามาใน mem
- ทุก routine จะถูกเก็บใน disk/ssd ในรูปแบบ relocatable load format
- โหลดเฉพาะ routine และ dependency ของ routine นั้น ไม่ใช่ทั้ง library
- มีประโยชน์เพราะโปรแกรมอาจไม่ได้ใช้ routine บ่อยนักแต่เมื่อต้องใช้ก็โหลดได้
- ไม่ต้องใช้กลไกของ operating system (OS ไม่ใช่คนจัดการให้)
 - การโหลดเป็นส่วนหนึ่งของการออกแบบโปรแกรม เช่นเมื่อผู้ใช้เลือก option อันใดอันหนึ่งใน menu ก็ค่อยโหลด
 - OS อาจให้บริการ libraries หรือเครื่องมือเพื่อให้โปรแกรมเรียกใช้ routine ใน libraries เพื่อทำ dynamic loading ด้วยตัวโปรแกรมเอง

Dynamic Loading (wikipedia)

- ใช้ใน IBM mainframe ตั้งแต่ IBM System/360 ใน OS/360
- ในเป็น plugin ของ Apache Web Server *.dso เป็นไฟล์ plugin แบบ “dynamic shared object”
- ใน UNIX-like OS และ Windows ใน C/C++ ก็มี dl library ให้โปรแกรมเรียกใช้เพื่อ โหลด dynamic library หรือ shared library เมื่อต้องการ

Name	Standard POSIX/UNIX API	Microsoft Windows API
Header file inclusion	<code>#include <dlfcn.h></code>	<code>#include <windows.h></code>
Definitions for header	<code>dl</code> (<code>libdl.so</code> , <code>libdl.dylib</code> , etc. depending on the OS)	<code>kernel32.dll</code>
Loading the library	<code>dlopen</code>	<code>LoadLibrary</code> <code>LoadLibraryEx</code>
Extracting contents	<code>dlsym</code>	<code>GetProcAddress</code>
Unloading the library	<code>dlclose</code>	<code>FreeLibrary</code>

https://en.wikipedia.org/wiki/Dynamic_loading

ตัวอย่าง Dynamic Loading (wikipedia)

■ UNIX

Most UNIX-like operating systems (Solaris, Linux, *BSD, etc.) [\[edit\]](#)

```
void* sdl_library = dlopen("libSDL.so", RTLD_LAZY);
if (sdl_library == NULL) {
    // report error ...
} else {
    // use the result in a call to dlsym
```

■ โหลดเฉพาะ routine และ dependency ของ routine นั้น ไม่ใช่ทั้ง library

■ Windows

Windows [\[edit\]](#)

```
HMODULE sdl_library = LoadLibrary(TEXT("SDL.dll"));
if (sdl_library == NULL) {
    // report error ...
} else {
    // use the result in a call to GetProcAddress
```

■ โหลดเฉพาะ routine และ dependency ของ routine นั้น ไม่ใช่ทั้ง library

■ java

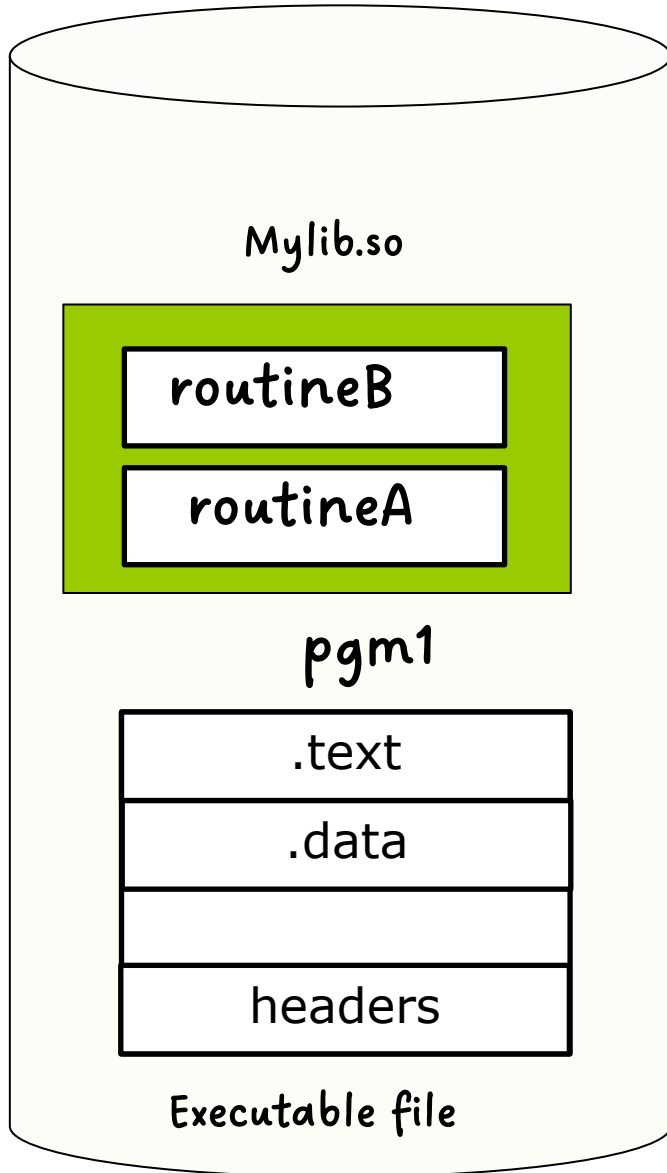
```
Class type = ClassLoader.getSystemClassLoader().loadClass(name);
Object obj = type.newInstance();
```

```
Class type = Class.forName(name);
Object obj = type.newInstance();
```

https://en.wikipedia.org/wiki/Dynamic_loading

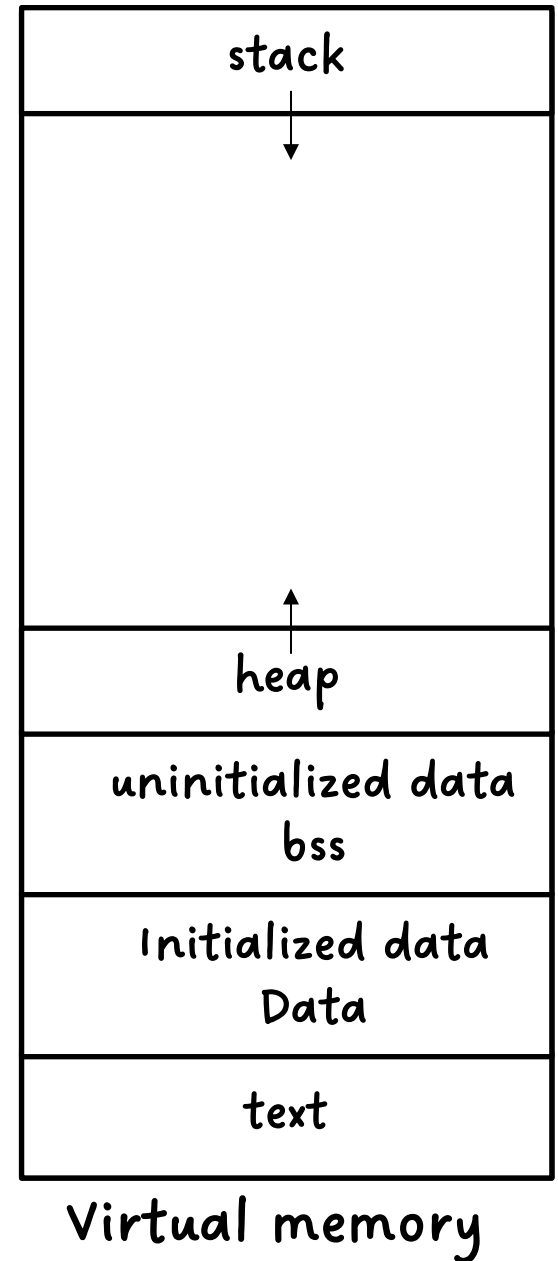
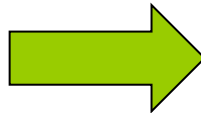
ตัวอย่าง dynamic loading

Disk



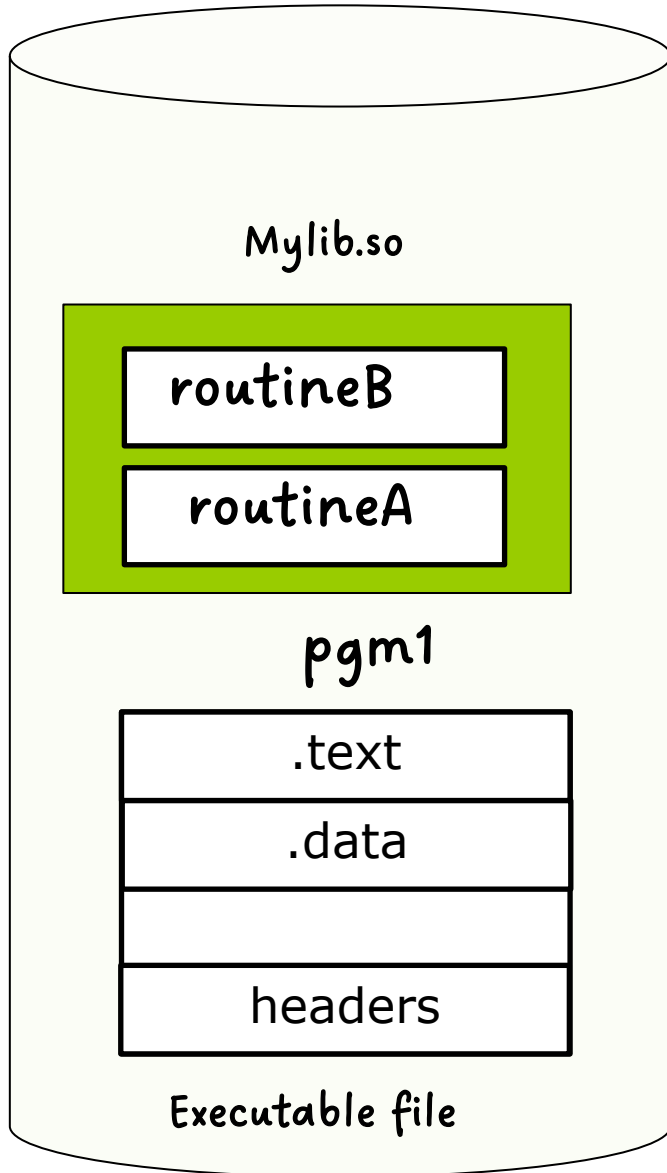
Virtual
Address
space

Shell load
Pgm1 to mem



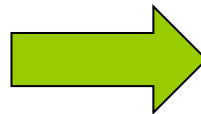
ตัวอย่าง dynamic loading

Disk

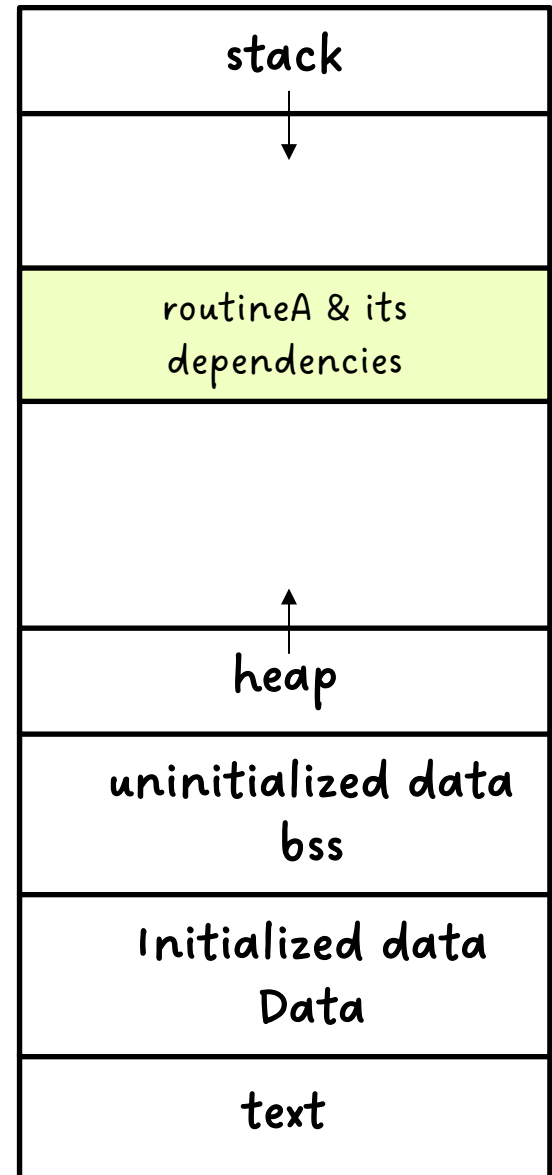


Virtual
Address
space

pgm1 call
dlopen(Mylib.so)
...
Call routineA

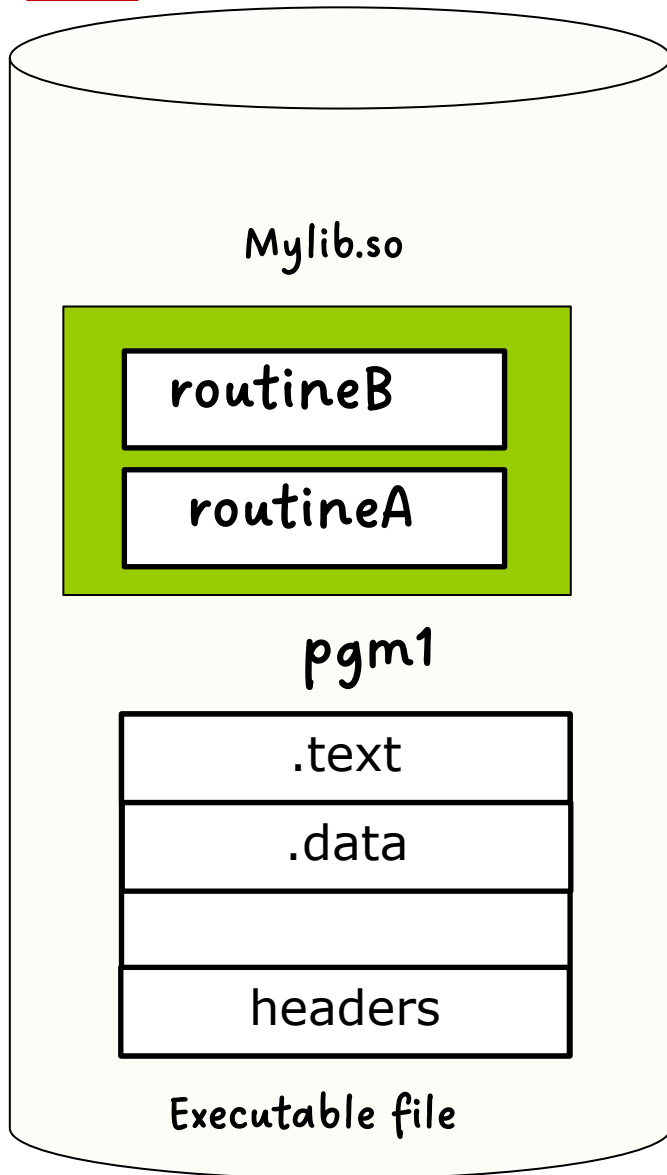


0000



ตัวอย่าง dynamic loading

Disk

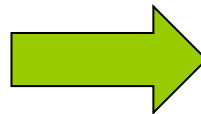


Virtual
Address
space

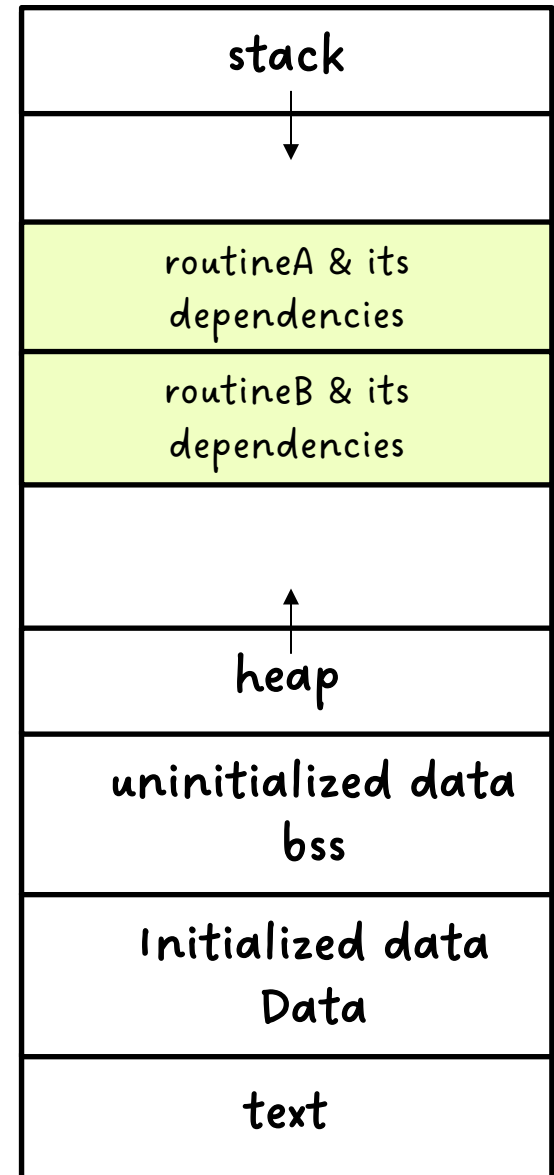
pgm1 call
dlopen(Mylib.so)

...

Call routineA
Call routineB



0000



Dynamic Linking

- **Static linking** – system libraries and program code combined by the loader into the binary program image
- **Dynamic linking** –linking postponed until execution time
- Small piece of code, **stub**, used to locate the appropriate memory-resident library routine
- Stub replaces itself with the address of the routine, and executes the routine
- Operating system checks if routine is in processes' memory address
 - If not in address space, add to address space
- Dynamic linking is particularly useful for libraries
- System also known as **shared libraries**
- Consider applicability to patching system libraries
 - Versioning may be needed

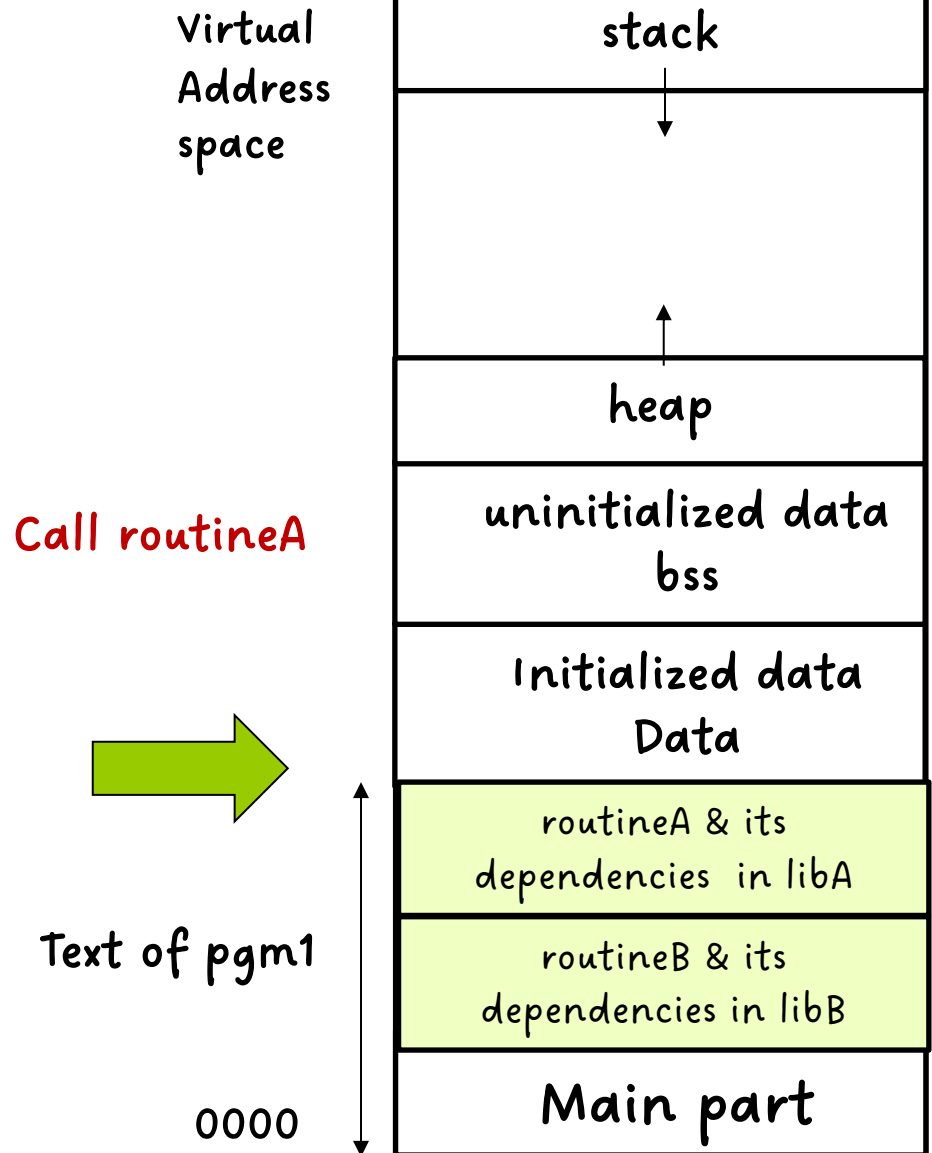
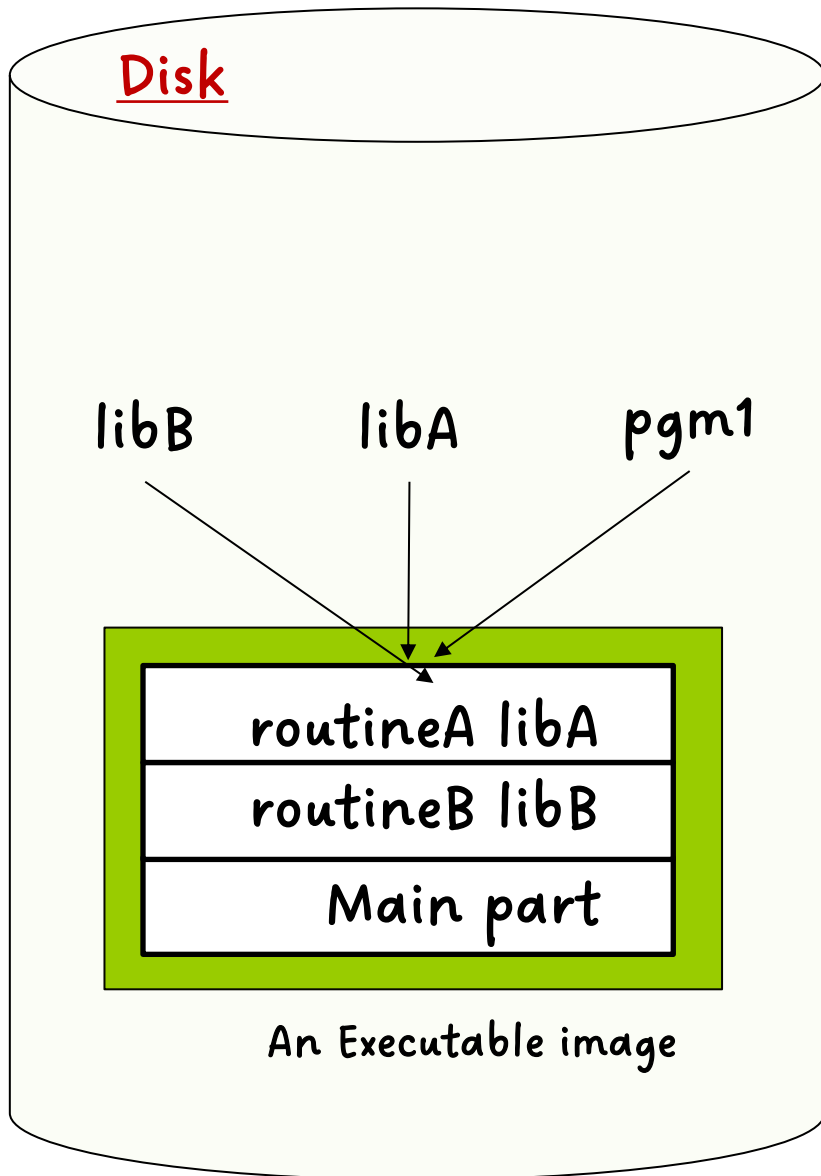
Dynamic Linking

- Static linking คอมไพเลอร์จะรวมโค้ดของ routines ที่มันต้องใช้พร้อมทั้ง dependencies ของ routine เหล่านั้นใน library ที่ statically link เข้ามาในโปรแกรม executable code
- Dynamic linking จะ link routines ที่ต้องการเข้ามาตอน execution time
- คอมไพเลอร์จะใส่ routine ปลอมที่ไม่มีเนื้อหาจริงเรียกว่า stub เข้าไปใน executable code
- เมื่อมีการเรียกใช้ routine ที่เป็น stub OS จะเช็คว่ามี routine นี้ใน (virtual) memory ของ process หรือไม่ ถ้ามีก็จะแทน stub ด้วย (virtual) memory address ของ routine แต่ถ้าไม่มี ก็จะเพิ่ม routine นั้นใน virtual memory ของ process
- OS เป็นผู้จัดการ load และ link dynamic linking library ให้

Dynamic Linking

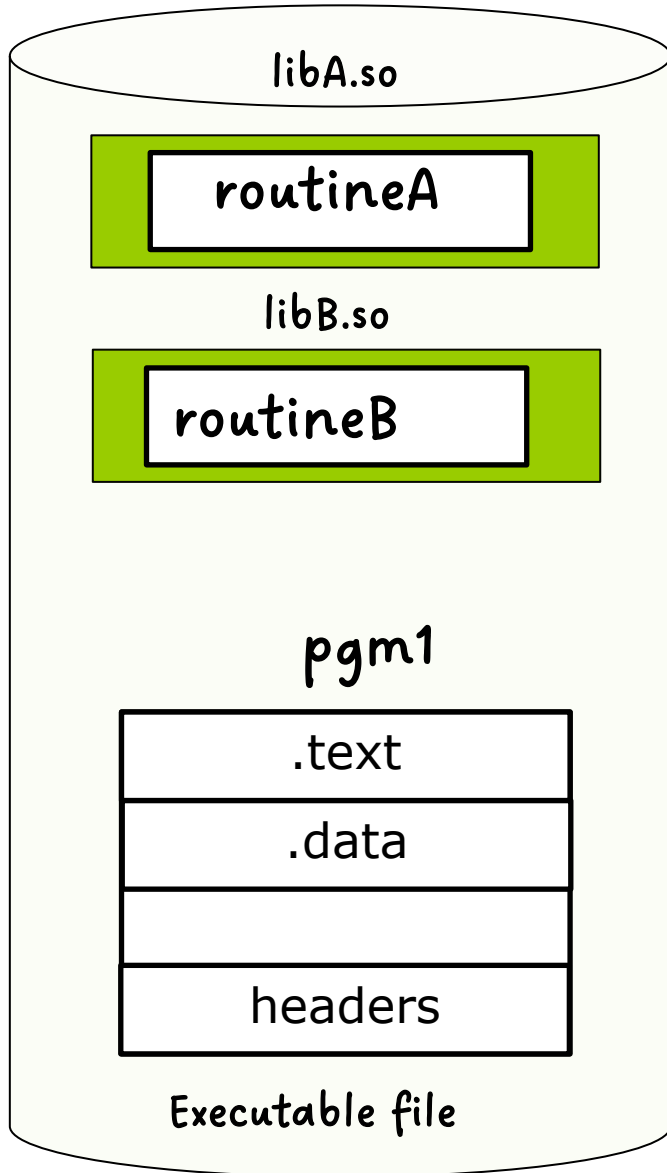
- Dynamic linking มีประโยชน์มากเพราะช่วยประหยัดพื้นที่ของ executable
- library มักจะเป็น shared library
- Shared library เป็น dynamic library ที่ Process หลาย Process สามารถแชร์ร่วมกันได้ โดยโอเอสจะเป็นคนจัดการๆ share ให้
- ถ้า Process เรียกใช้ routine แล้ว OS พบว่า Process นั้นไม่มี routine นั้นใน process memory space OS ก็จะเช็คหา routine นี้มีอยู่ใน memory space ของ Process อื่นหรือไม่ ถ้ามีมันจะขอแชร์ใช้ด้วย แต่ถ้าไม่มี OS จะโหลด routine นั้นให้ Process นั้น
- ต้องมีการระบุเวอร์ชันของ shared library ที่ process ด้วยเพื่อความถูกต้องในการประมวลผล

ตัวอย่าง static linking



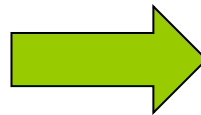
ตัวอย่าง dynamic linking

Disk

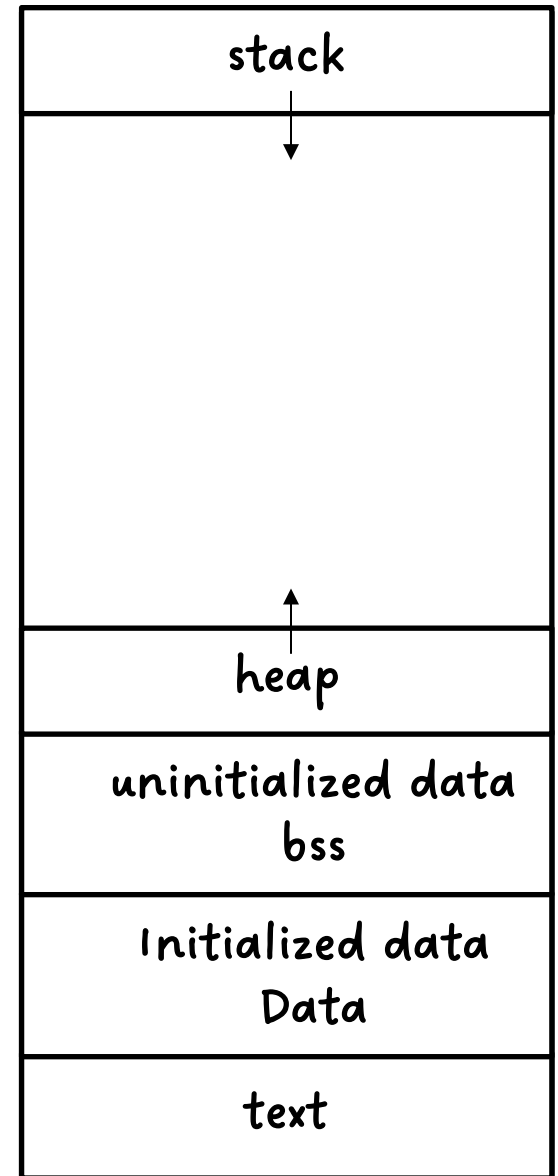


Virtual
Address
space

...
Call routineA (stub)

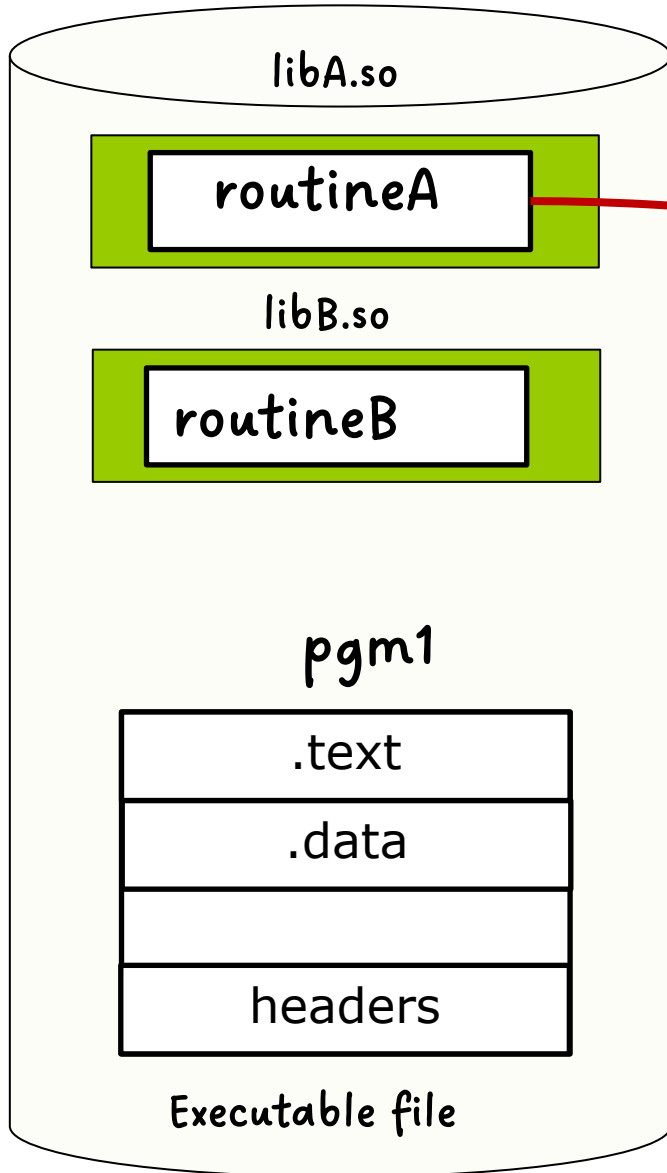


pc →
0000



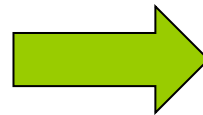
ตัวอย่าง dynamic linking

Disk



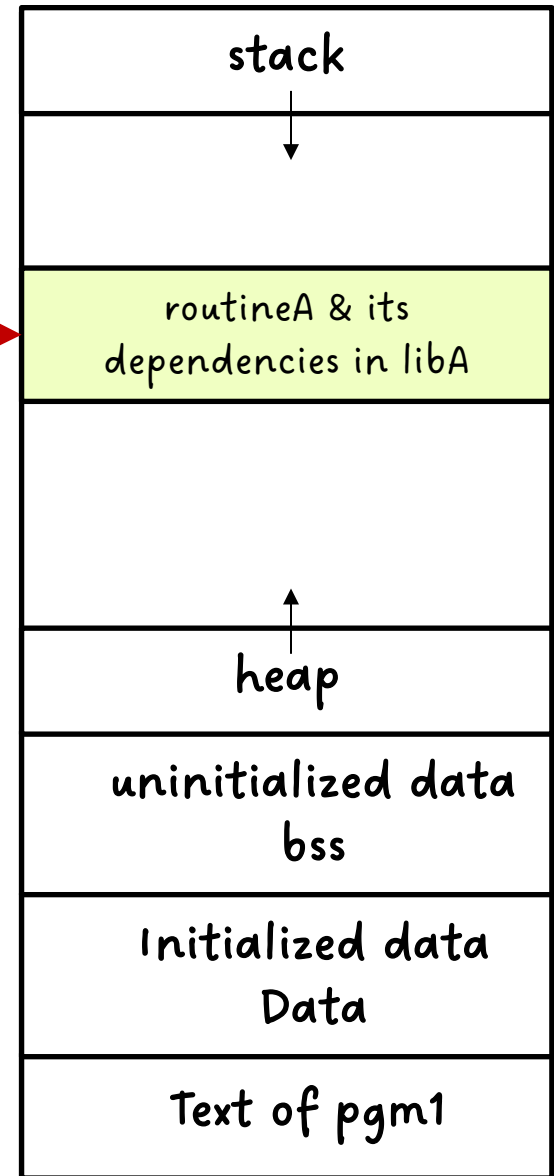
Virtual
Address
space

OS พบว่าไม่มีใน
process และ OS เช็คว่าไม่มี Process
อื่นอันไหนมี
OS จะโหลดเข้ามาให้



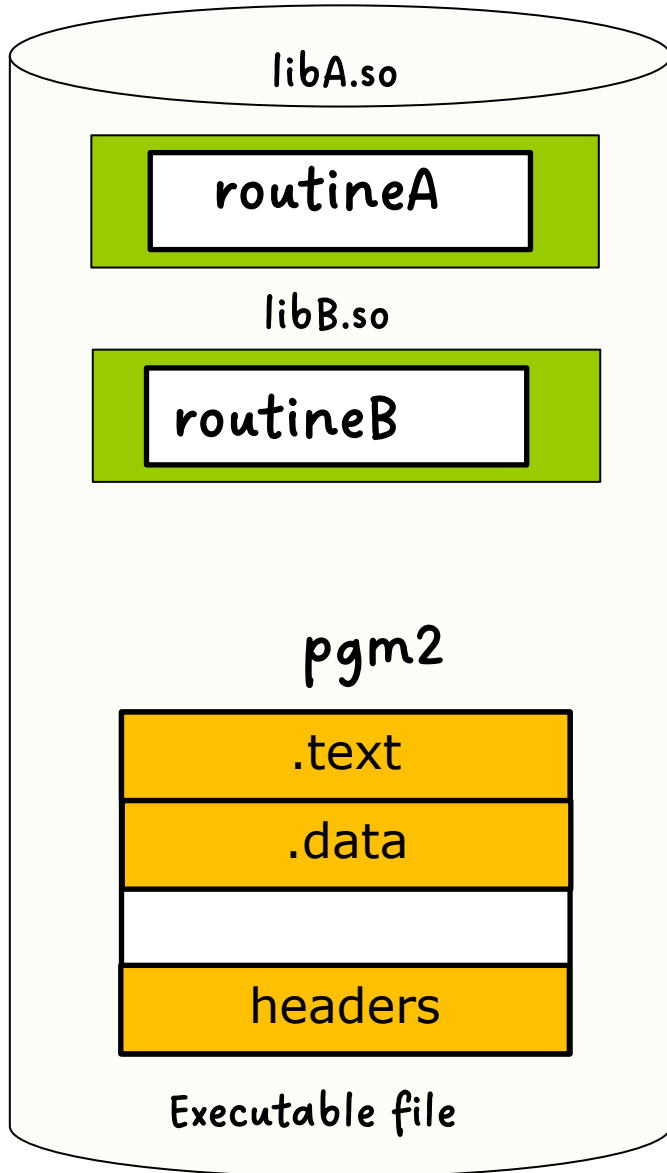
Call routineA

pc →
0000



ตัวอย่าง dynamic linking

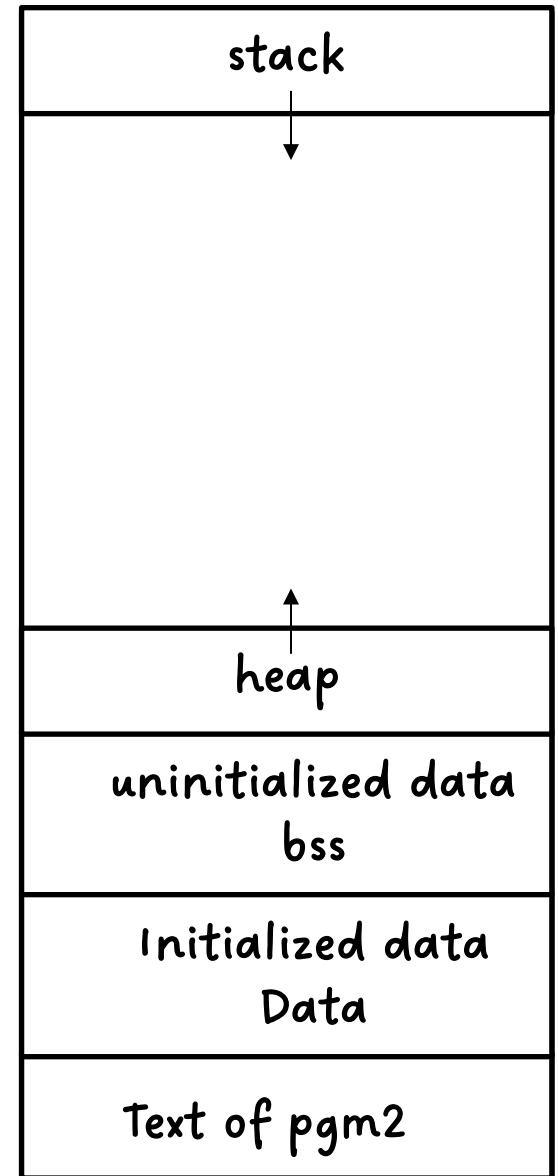
Disk



Virtual
Address
space

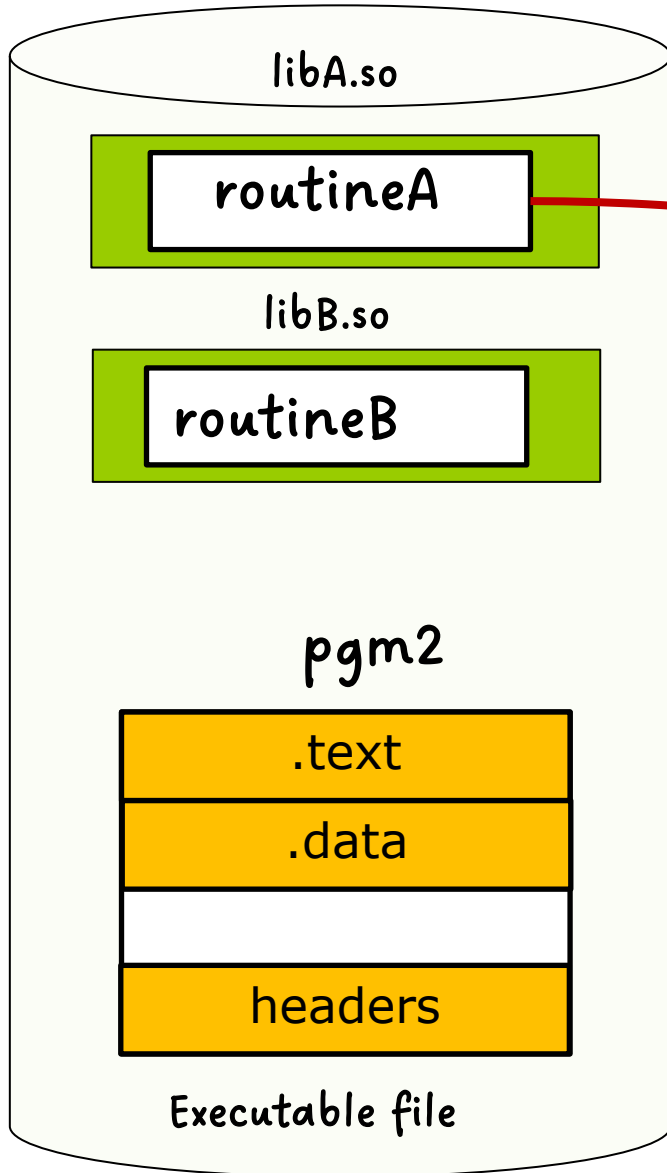
Call routineA

pc →
0000



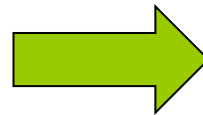
ตัวอย่าง dynamic linking

Disk



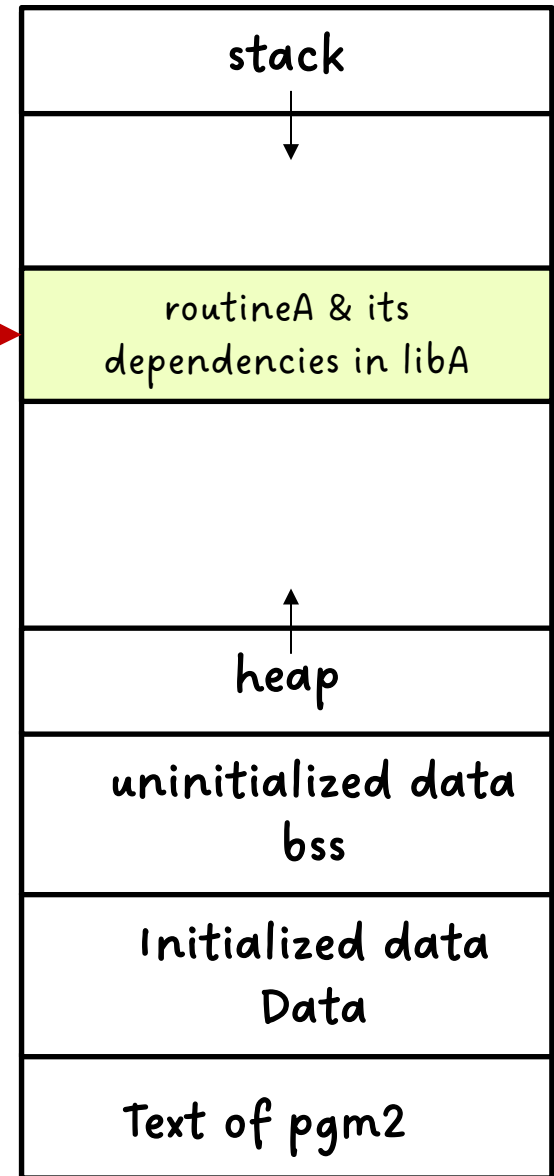
Virtual
Address
space

OS พบว่าไม่มีใน
process และ OS เช็ค
แล้วพบว่า มี Process
Pgm1 ใช้อยู่
OS จะแชร์ให้



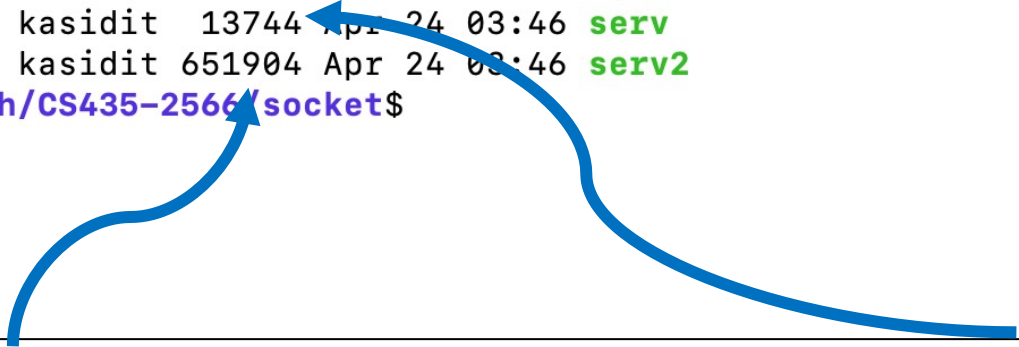
Call routineA

pc →
0000



ตัวอย่าง static vs dynamic linking

```
[kasidit@vm:~/Research/CS435-2566/socket$ gcc -o serv pgm2-echo-server.c
[kasidit@vm:~/Research/CS435-2566/socket$ 
[kasidit@vm:~/Research/CS435-2566/socket$ 
[kasidit@vm:~/Research/CS435-2566/socket$ gcc -o serv2 -static pgm2-echo-server.c
[kasidit@vm:~/Research/CS435-2566/socket$ 
[kasidit@vm:~/Research/CS435-2566/socket$ 
[kasidit@vm:~/Research/CS435-2566/socket$ ls -l
total 672
-rw-rw-r-- 1 kasidit kasidit 2208 Apr 24 03:46 pgm10-2-echo-server-select.c
-rw-rw-r-- 1 kasidit kasidit 1279 Apr 24 03:46 pgm1-echo-client.c
-rw-rw-r-- 1 kasidit kasidit 1302 Apr 24 03:46 pgm2-echo-server.c
-rw-rw-r-- 1 kasidit kasidit 2169 Apr 24 03:46 pgm8-2-echo-server-concur-thread.c
-rwxrwxr-x 1 kasidit kasidit 13744 Apr 24 03:46 serv
-rwxrwxr-x 1 kasidit kasidit 651904 Apr 24 03:46 serv2
[kasidit@vm:~/Research/CS435-2566/socket$
```



ขนาดของ statically link executable file จะใหญ่กว่าของ dynamically linked file

ตัวอย่าง static vs dynamic linking

```
kasidit@vm:~/Research/CS435-2566/socket$ size serv
text    data    bss      dec       hex filename
3297    776      32      4105     1009 serv
kasidit@vm:~/Research/CS435-2566/socket$ size serv2
text    data    bss      dec       hex filename
515573  22520   22272   560365   88ced serv2
kasidit@vm:~/Research/CS435-2566/socket$ ldd serv
linux-vdso.so.1 (0x0000ffff9945c000)
libc.so.6 => /lib/aarch64-linux-gnu/libc.so.6 (0x0000ffff99250000)
/lib/ld-linux-aarch64.so.1 (0x0000ffff99423000)
kasidit@vm:~/Research/CS435-2566/socket$ ldd serv2
not a dynamic executable
```



1. ขนาดของ **statically link executable file** จะใหญ่กว่าของ **dynamically linked file**
2. คำสั่ง **ldd** แสดงรายการ **dynamically linked libraries**
→ ชื่อ **.so** ไฟล์และ **version** ของ library
3. **ldd** จะไม่มีอะไรแสดงสำหรับ **statically link executable**

ตัวอย่าง static vs dynamic linking

1. ใช้ /proc ดูข้อมูลของ dynamically linked process

```
kasidit@enterprise:~$ vi hello.c
kasidit@enterprise:~$ gcc -o hello-ddl hello.c
kasidit@enterprise:~$ ./hello-ddl &
[1] 2519820
kasidit@enterprise:~$ hello world
```

```
kasidit@enterprise:~$ cat /proc/2519820/maps
596bd1978000-596bd1979000 r--p 00000000 103:04 15210843
596bd1979000-596bd197a000 r-xp 00001000 103:04 15210843
596bd197a000-596bd197b000 r--p 00002000 103:04 15210843
596bd197b000-596bd197c000 r--p 00002000 103:04 15210843
596bd197c000-596bd197d000 rw-p 00003000 103:04 15210843
596c071ba000-596c071db000 rw-p 00000000 00:00 0
73a810e00000-73a810e28000 r--p 00000000 103:04 31348591
73a810e28000-73a810fb0000 r-xp 00028000 103:04 31348591
73a810fb0000-73a810fff000 r--p 001b0000 103:04 31348591
73a810fff000-73a811003000 r--p 001fe000 103:04 31348591
73a811003000-73a811005000 rw-p 00202000 103:04 31348591
73a811005000-73a811012000 rw-p 00000000 00:00 0
73a811048000-73a81104b000 rw-p 00000000 00:00 0
73a811066000-73a811068000 rw-p 00000000 00:00 0
73a811068000-73a811069000 r--p 00000000 103:04 31348585
73a811069000-73a811094000 r-xp 00001000 103:04 31348585
73a811094000-73a81109e000 r--p 0002c000 103:04 31348585
73a81109e000-73a8110a0000 r--p 00036000 103:04 31348585
73a8110a0000-73a8110a2000 rw-p 00038000 103:04 31348585
7ffd3f680000-7ffd3f6a1000 rw-p 00000000 00:00 0
7ffd3f741000-7ffd3f745000 r--p 00000000 00:00 0
7ffd3f745000-7ffd3f747000 r-xp 00000000 00:00 0
fffffffff60000-fffffffff601000 --xp 00000000 00:00 0
```

```
/home/kasidit/hello-ddl
/home/kasidit/hello-ddl
/home/kasidit/hello-ddl
/home/kasidit/hello-ddl
/home/kasidit/hello-ddl
[heap]
/usr/lib/x86_64-linux-gnu/libc.so.6
/usr/lib/x86_64-linux-gnu/libc.so.6
/usr/lib/x86_64-linux-gnu/libc.so.6
/usr/lib/x86_64-linux-gnu/libc.so.6
/usr/lib/x86_64-linux-gnu/libc.so.6
```

```
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
/usr/lib/x86_64-linux-gnu/ld-linux-x86-64.so.2
[stack]
[vvar]
[vdso]
[vsyscall]
```

2. ใช้ /proc ดูข้อมูลของ statically linked process

```
kasidit@enterprise:~$ gcc -o hello-sta -static hello.c
kasidit@enterprise:~$ ./hello-sta &
[2] 2519929
[1] Done
kasidit@enterprise:~$ hello world
```

```
kasidit@enterprise:~$ cat /proc/2519929/maps
00400000-00401000 r--p 00000000 103:04 15250169
00401000-0047f000 r-xp 00001000 103:04 15250169
0047f000-004a5000 r--p 0007f000 103:04 15250169
004a5000-004aa000 r--p 000a4000 103:04 15250169
004aa000-004ac000 rw-p 000a9000 103:04 15250169
004ac000-004b2000 rw-p 00000000 00:00 0
27fa1000-27fc3000 rw-p 00000000 00:00 0
7ffd1bd84000-7ffd1bda5000 rw-p 00000000 00:00 0
7ffd1bdd8000-7ffd1bddc000 r--p 00000000 00:00 0
7ffd1bddc000-7ffd1bdde000 r-xp 00000000 00:00 0
fffffffff60000-fffffffff601000 --xp 00000000 00:00 0
```

```
/home/kasidit/hello-sta
/home/kasidit/hello-sta
/home/kasidit/hello-sta
/home/kasidit/hello-sta
/home/kasidit/hello-sta
```

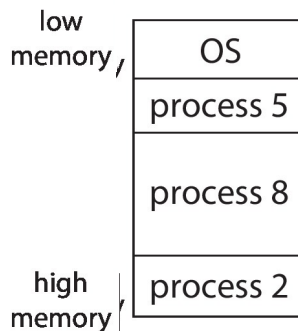
```
[heap]
[stack]
[vvar]
[vdso]
[vsyscall]
```


การจัดการเนื้อหาโปรแกรมใน **Memory**

- **Contiguous Allocation**
- **Paging**

Contiguous Allocation

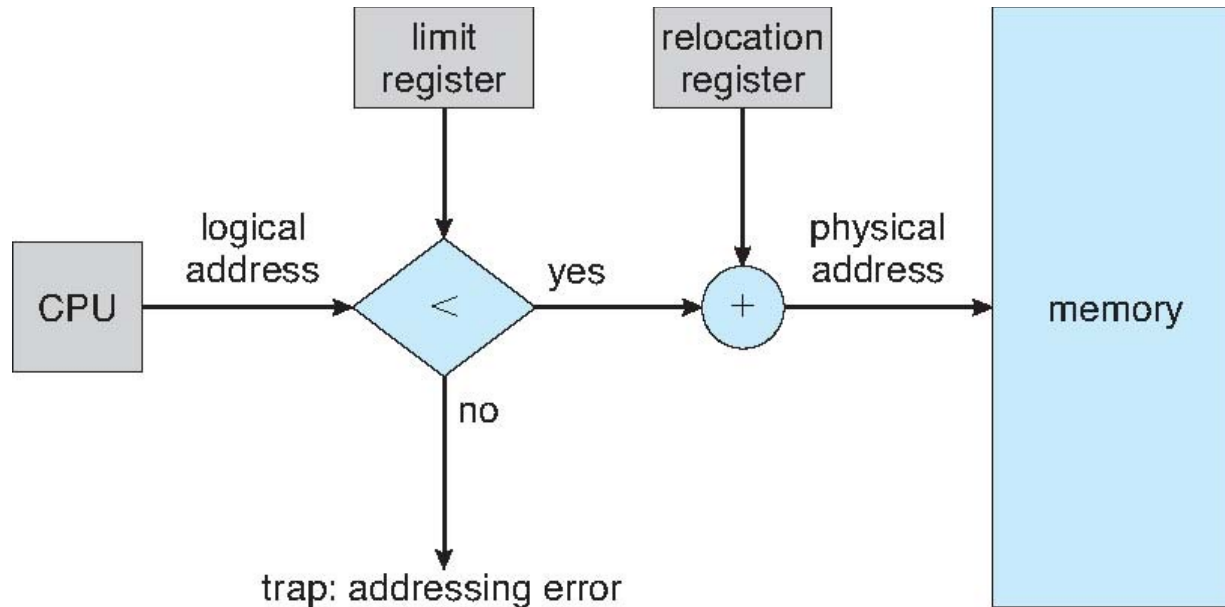
- Main memory must support both OS and user processes
- Limited resource, must allocate efficiently
- **Contiguous allocation is one early method**
- Main memory usually into two **partitions**:
 - Resident operating system, usually held in low memory with interrupt vector
 - ▶ ตำแหน่ง high/low memory แตกต่างจากของ Linux ในปัจจุบัน
 - User processes then held in high memory
 - Each process contained in single contiguous section of memory



Contiguous Allocation (Cont.)

- Relocation registers used to protect user processes from each other, and from changing operating-system code and data
 - Base register contains value of smallest physical address
 - Limit register contains range of logical addresses – each logical address must be less than the limit register
 - MMU maps logical address *dynamically*
 - Can then allow actions such as kernel code being **transient** and kernel changing size
 - ▶ สามารถเปลี่ยนตำแหน่งของ user processes เพื่อขยายขนาด kernel

Hardware Support for Relocation and Limit Registers



ตย. Relocation = 100040 และ limit = 74600

Logical Address = 100 ผลคือ (yes/no) ถ้า yes physical address คือ?

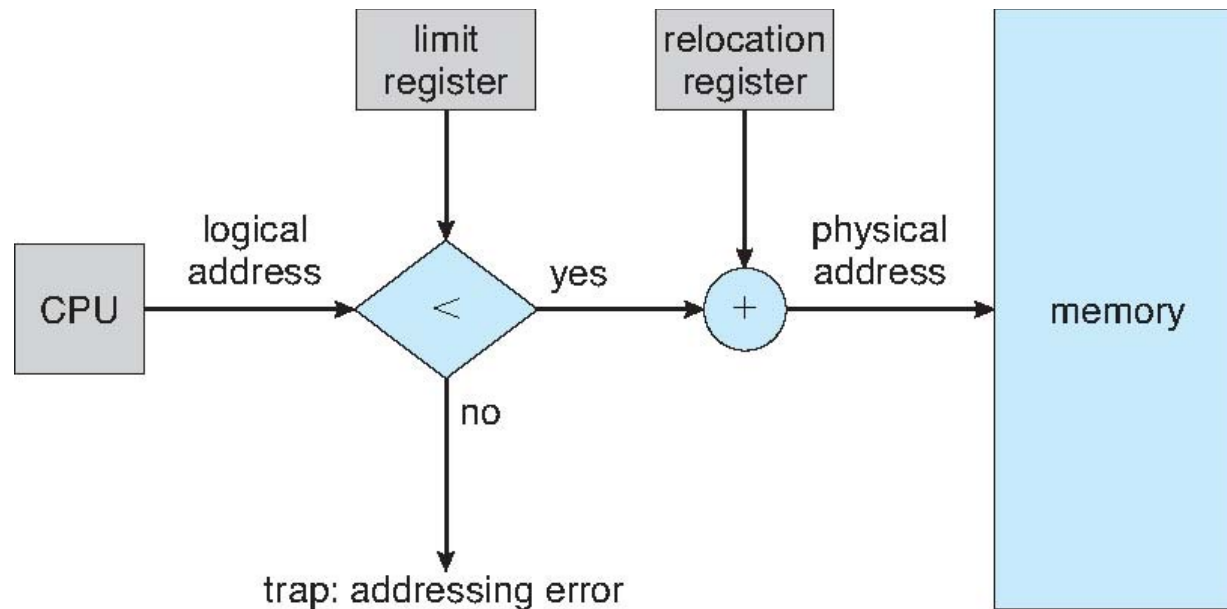
Logical Address = 0 ผลคือ

Logical Address = 100040 ผลคือ

Logical Address = 74599 ผลคือ

Logical Address = 100039 ผลคือ

Hardware Support for Relocation and Limit Registers



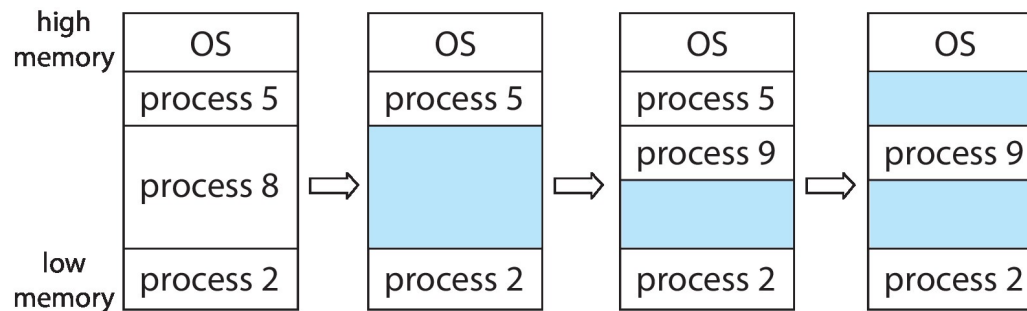
ตย. Relocation = 100040 และ limit = 74600

Logical Address Space ของ Process คือ: __ ถึง __

Physical Address Space ของ Process คือ: __ ถึง __

Variable Partition

- Multiple-partition allocation
 - Degree of multiprogramming limited by number of partitions
 - **Variable-partition** sizes for efficiency (sized to a given process' needs)
 - **Hole** – block of available memory; holes of various size are scattered throughout memory
 - When a process arrives, it is allocated memory from a hole large enough to accommodate it
 - Process exiting frees its partition, adjacent free partitions combined
 - Operating system maintains information about:
a) **allocated partitions** b) **free partitions (hole)**



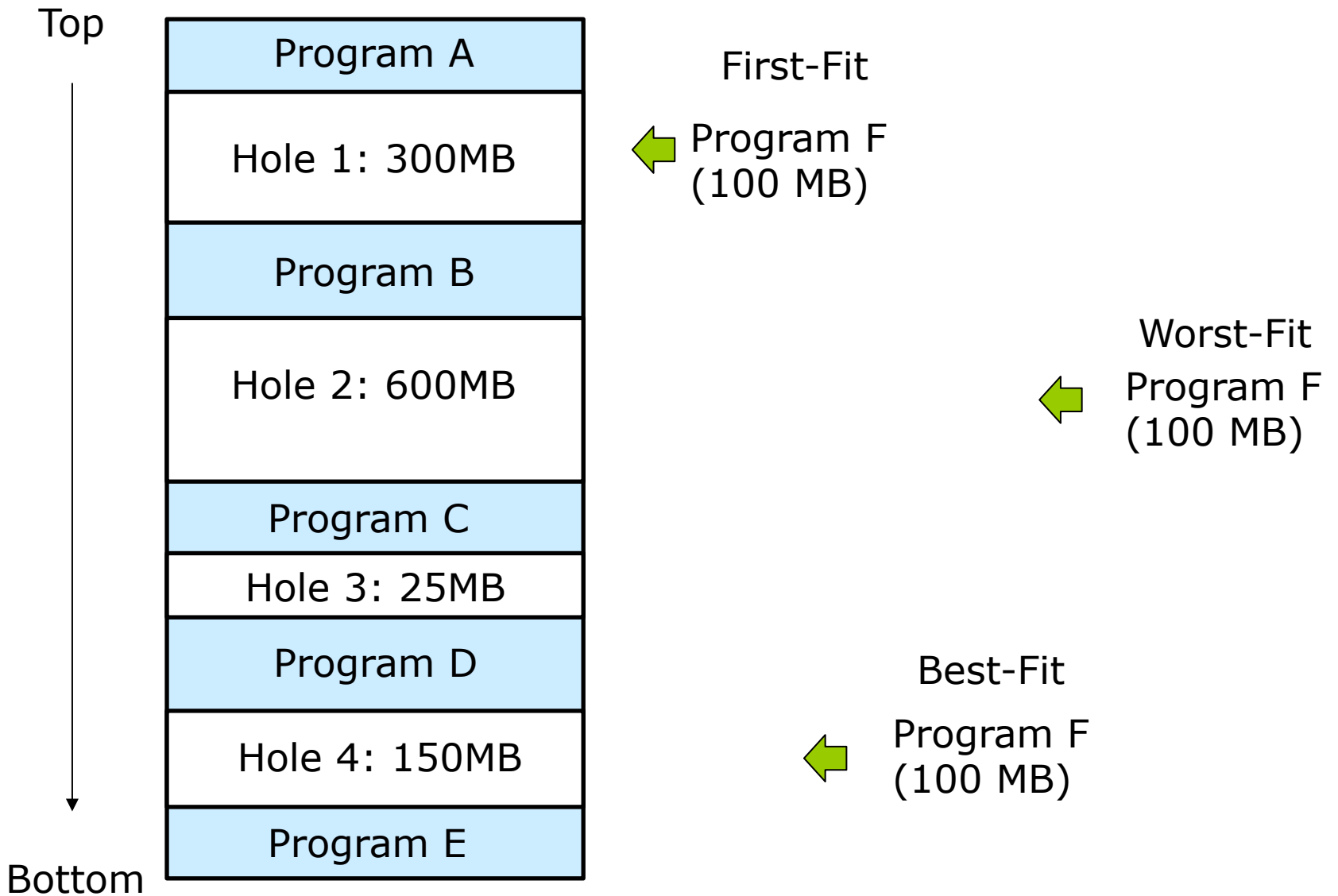
Dynamic Storage-Allocation Problem

How to satisfy a request of size n from a list of free holes?

- **First-fit**: Allocate the **first** hole that is big enough
- **Best-fit**: Allocate the **smallest** hole that is big enough; must search entire list, unless ordered by size
 - Produces the smallest leftover hole
- **Worst-fit**: Allocate the **largest** hole; must also search entire list
 - Produces the largest leftover hole

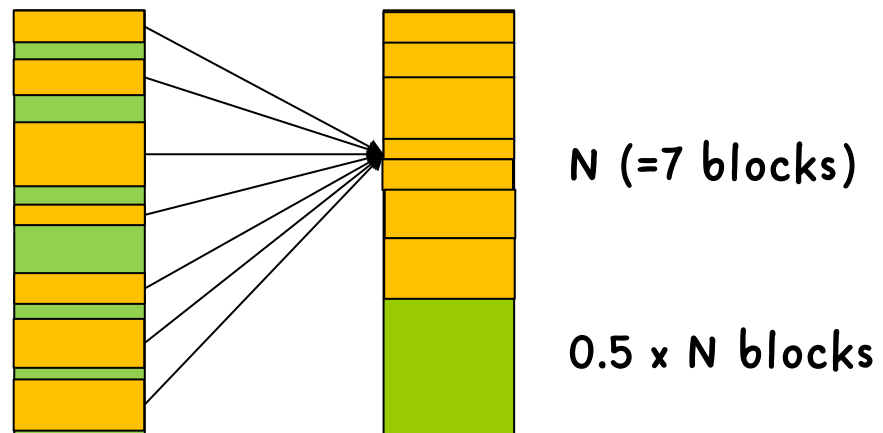
First-fit and best-fit better than worst-fit in terms of speed and storage utilization

- First fit เร็วที่สุด
- Best fit ลดจำนวนช่องที่เสียเปล่า
- Worst fit ลดจำนวน ช่องใหญ่ๆ ทำให้มีช่องที่ไม่พอเยอะ ช้ากว่าด้วยเพราะต้องหาช่องทั้ง mem



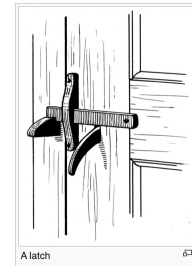
Fragmentation

- **External Fragmentation** – total memory space exists to satisfy a request, but it is not contiguous
- **Internal Fragmentation** – allocated memory may be slightly larger than requested memory; this size difference is memory internal to a partition, but not being used
- First fit analysis reveals that given N blocks allocated, $0.5 N$ blocks lost to fragmentation
 - $1/3$ may be unusable -> **50-percent rule**
 - ตย. ถ้าใช้หน่วยความจำไป **150 blocks** (สมมติว่า **1 block = 1 KB**) จะเสียพื้นที่ให้กับ **fragmentation 50 blocks**



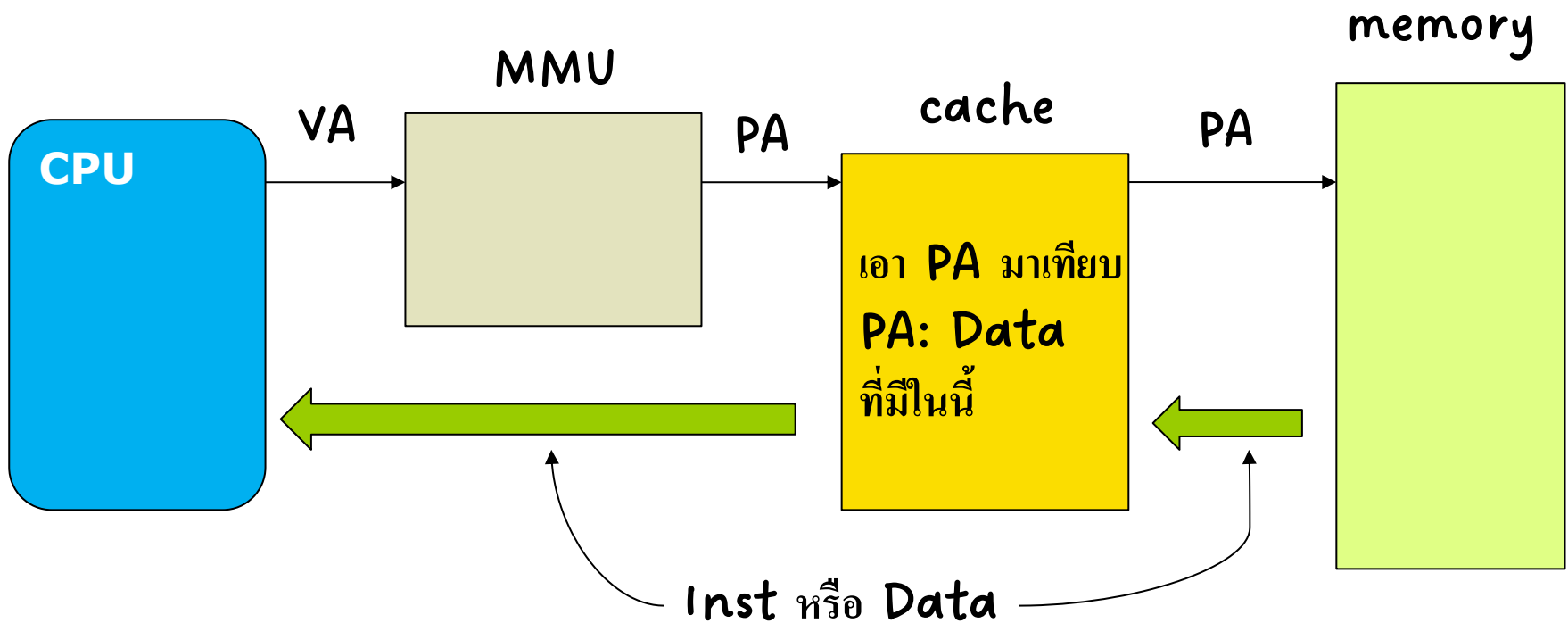
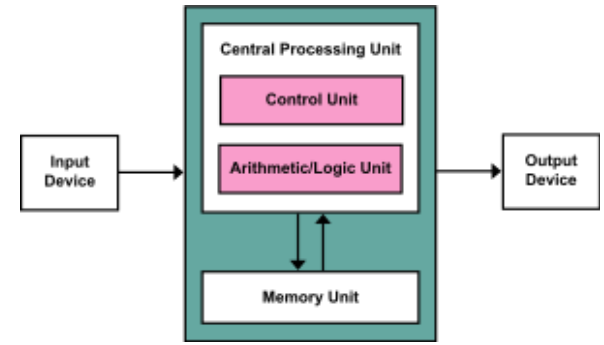
Fragmentation (Cont.)

- Reduce external fragmentation by **compaction**
 - Shuffle memory contents to place all free memory together in one large block
 - Compaction is possible only if relocation is dynamic, and is done at execution time
 - I/O problem
 - ▶ Latch job in memory while it is involved in I/O
 - ▶ Do I/O only into OS buffers
- Now consider that backing store (storage) has same fragmentation problems

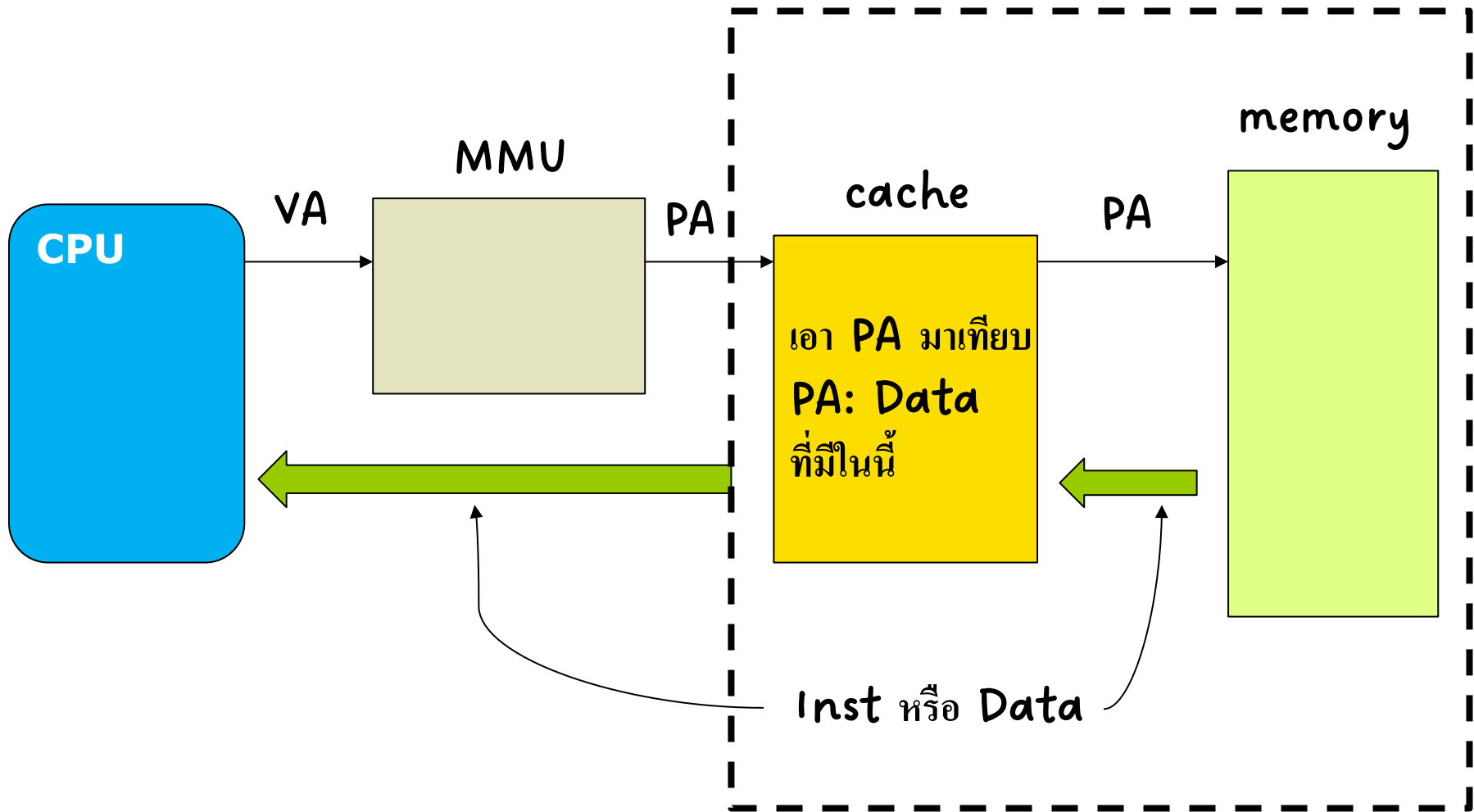


บทบาทการทำงานของ CPU และ Memory

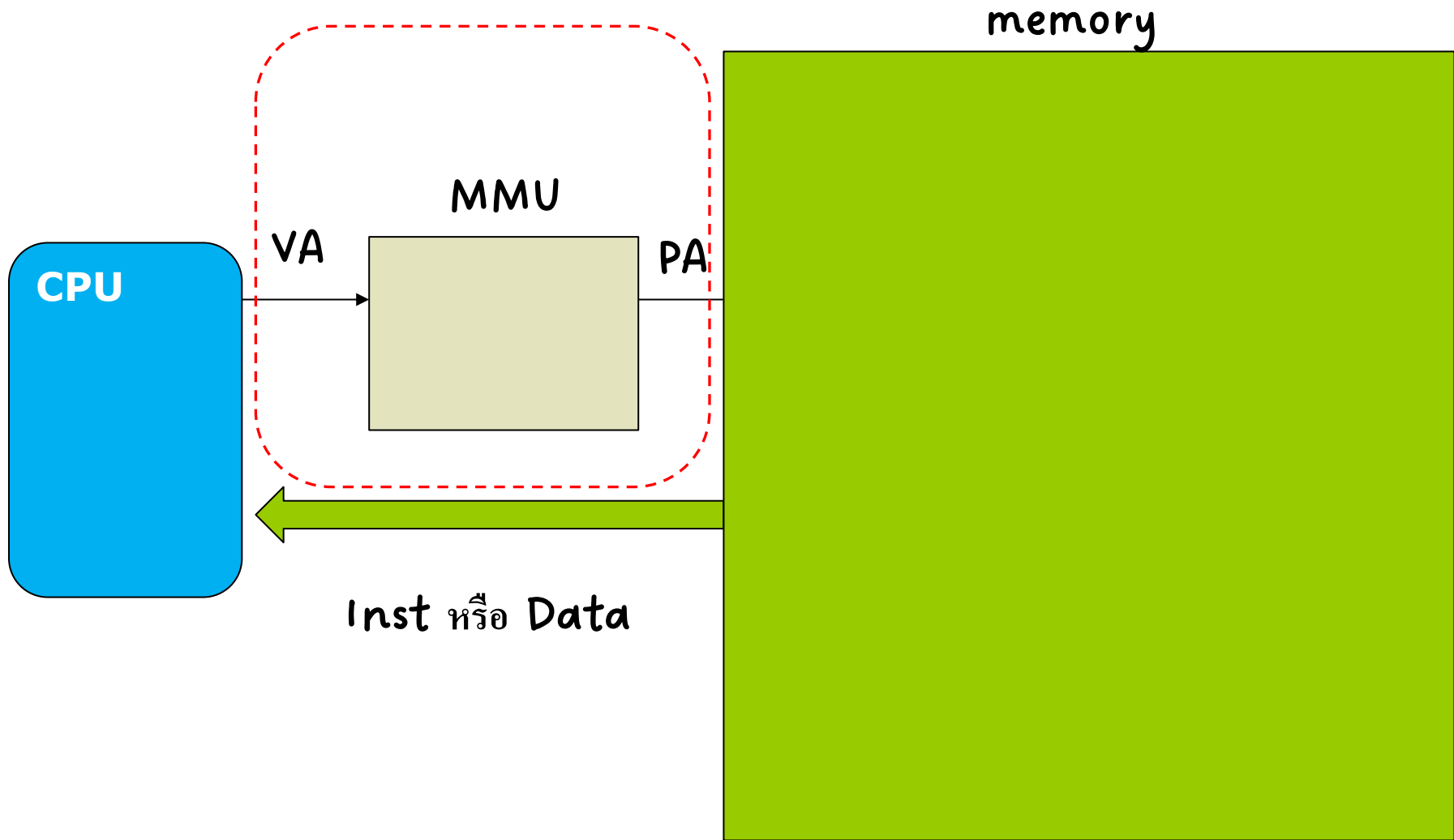
- เริ่มต้นจาก von Neumann Architecture
ที่กล่าวถึง คอมพิวเตอร์ที่เก็บโปรแกรมใน memory
- CPU ต้องการชุดคำสั่งหรือข้อมูลก็จะส่งคำขอไปให้ Memory
- Memory รับคำขอและส่งข้อมูลกลับ



การทำงานของ CPU และ Memory



การทำงานของ CPU และ Memory

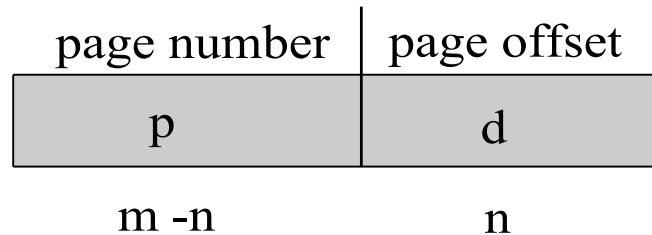


Paging

- Physical address space of a process can be noncontiguous; process is allocated physical memory whenever the latter is available
 - Avoids external fragmentation
 - Avoids problem of varying sized memory chunks
- Divide physical memory into fixed-sized blocks called **frames**
 - Size is power of 2, between 512 bytes and 16 Mbytes
- Divide logical memory into blocks of same size called **pages**
- Keep track of all free frames
- To run a program of size **N** pages, need to find **N** free frames and load program
- Set up a **page table** to translate logical to physical addresses
- Backing store likewise split into pages
- Still have Internal fragmentation

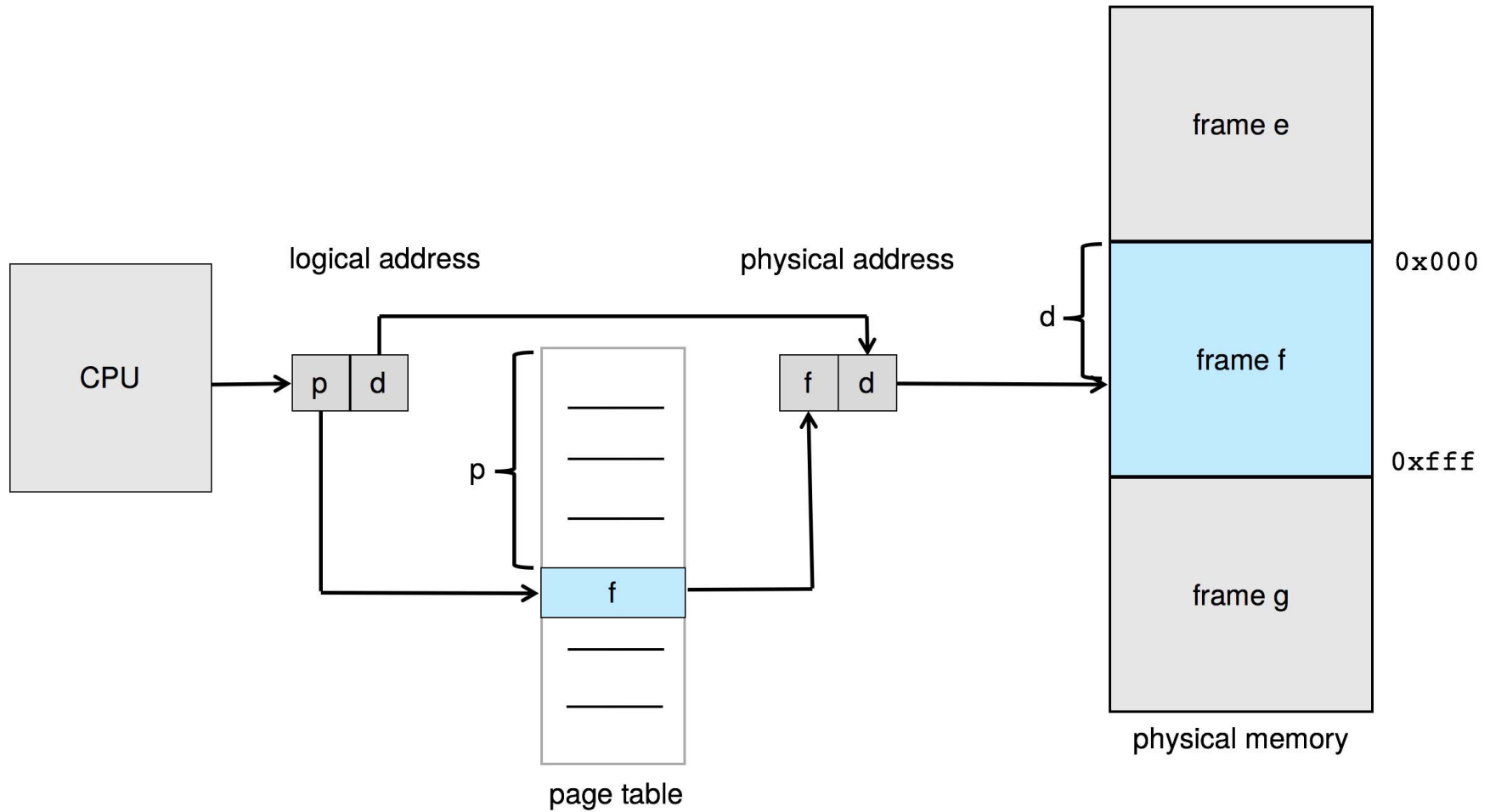
Address Translation Scheme

- Address generated by CPU is divided into:
 - **Page number** (p) – used as an index into a **page table** which contains base address of each page in physical memory
 - **Page offset** (d) – combined with base address to define the physical memory address that is sent to the memory unit

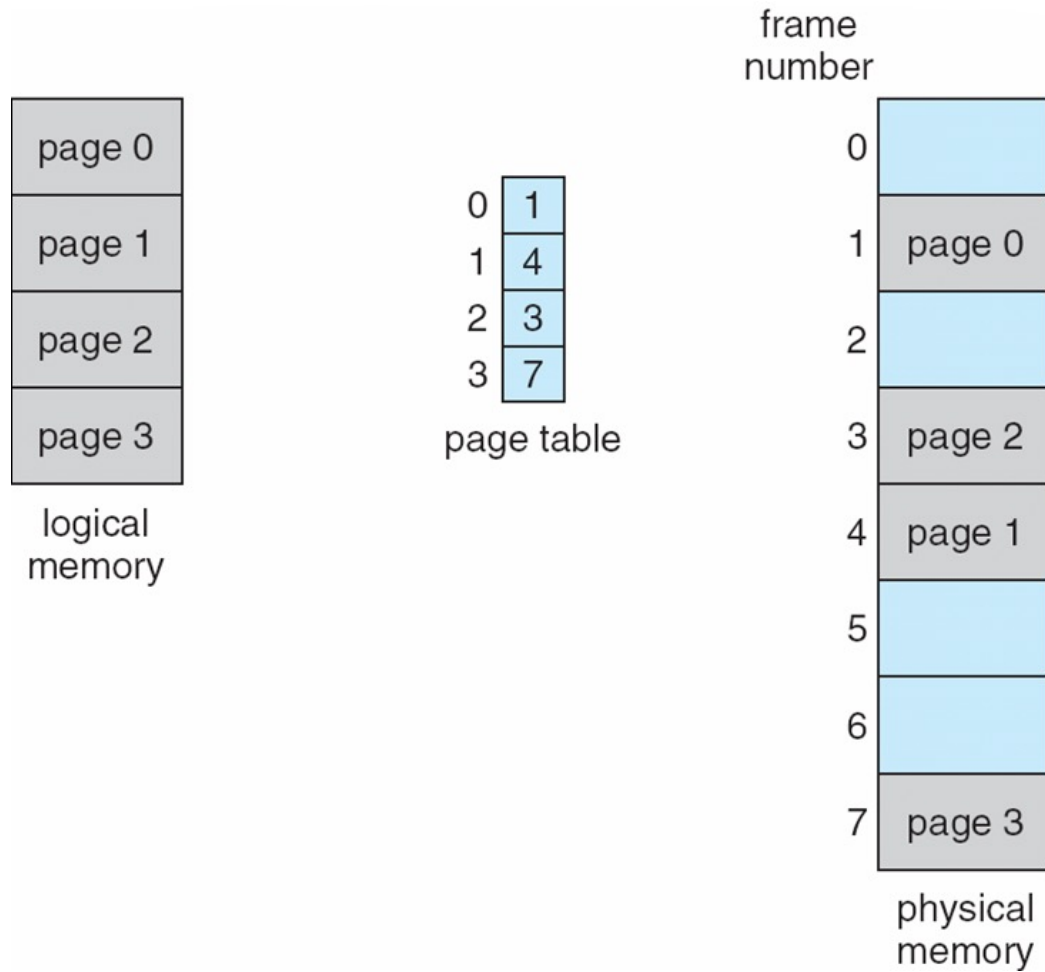


- For given logical address space 2^m and page size 2^n
- Memory จะเป็นแบบ Byte Addressable หมายความว่า Address 1 คำ เป็น Address สำหรับ 1 byte ในหน่วยความจำ

Paging Hardware

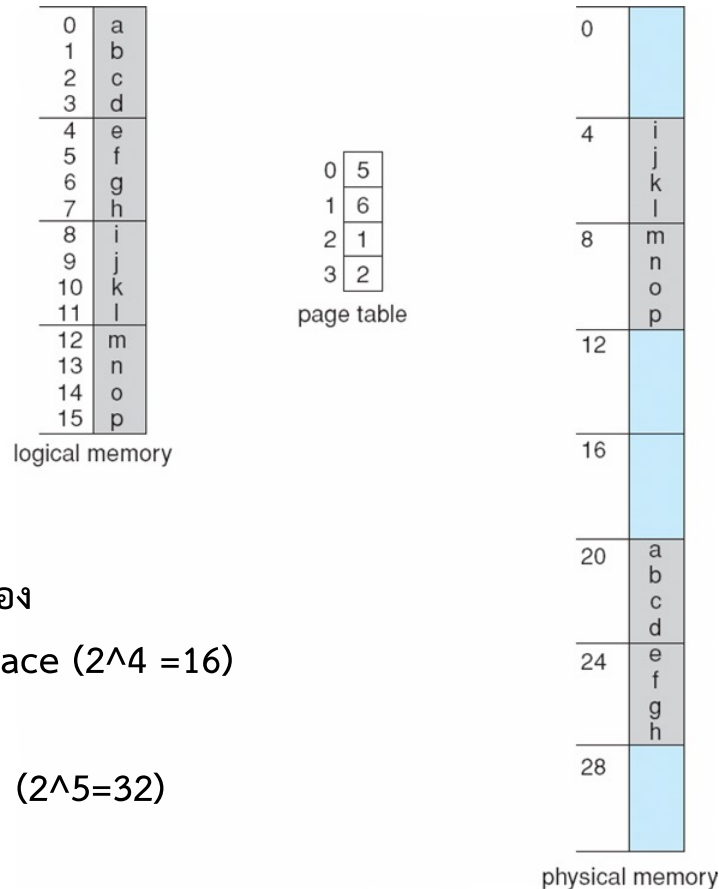


Paging Model of Logical and Physical Memory



Paging Example

- Logical address: $n = 2$ and $m = 4$. Using a page size of 4 bytes and a physical memory of 32 bytes (8 pages)



ในตัวอย่างนี้ ขนาดของ

logical address space ($2^4 = 16$)

น้อยกว่าขนาดของ

physical memory ($2^5 = 32$)

- Logical address: $n = 2$ and $m = 5$. Using a page size of $2^2 = 4$ bytes and a physical memory of 32 bytes (8 pages)

000	00	a
000	01	b
000	10	c
000	11	d
001	00	e
001	01	f
001	10	g
001	11	h
010	00	i
010	01	j
010	10	k
010	11	l
011	00	m
011	01	n
011	10	o
011	11	p

Logical memory

Page Table

p	f
000	101
001	110
010	001
011	010

ในตัวอย่างนี้ ขนาดของ

logical address space ($2^5 = 32$)

เท่ากับขนาดของ

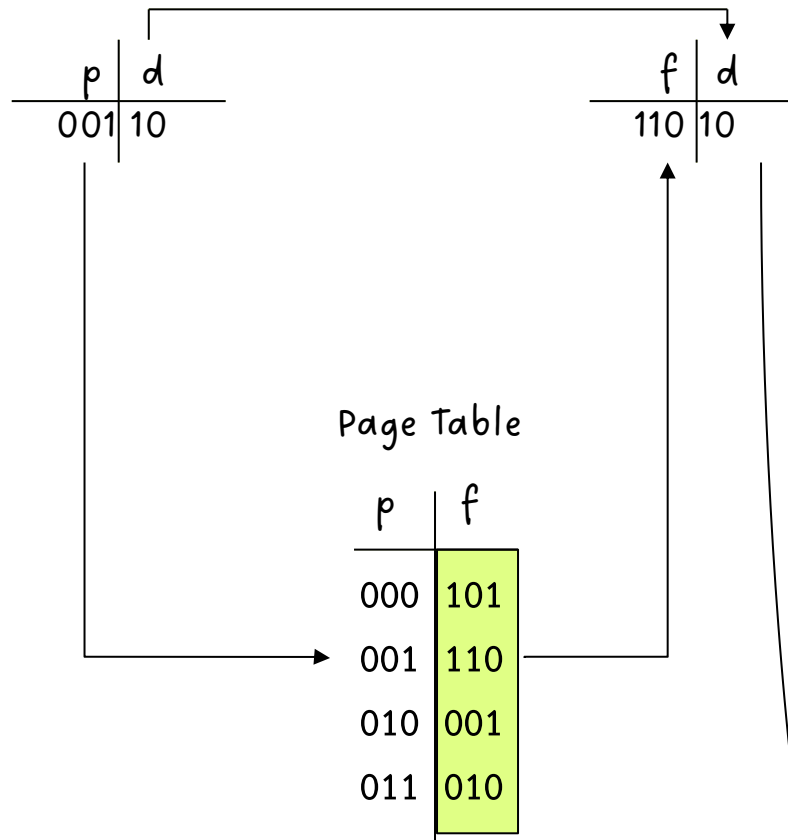
physical memory ($2^5 = 32$)

000	00	
000	01	
000	10	
000	11	
001	00	i
001	01	j
001	10	k
001	11	l
010	00	m
010	01	n
010	10	o
010	11	p
011	00	
011	01	
011	10	
011	11	
100	00	
100	01	
100	10	
100	11	
101	00	a
101	01	b
101	10	c
101	11	d
110	00	e
110	01	f
110	10	g
110	11	h
111	00	
111	01	
111	10	
111	11	

physical memory

000	00	a
000	01	b
000	10	c
000	11	d
001	00	e
001	01	f
001	10	g
001	11	h
010	00	i
010	01	j
010	10	k
010	11	l
011	00	m
011	01	n
011	10	o
011	11	p

Logical memory



000	00	
000	01	
000	10	
000	11	
001	00	i
001	01	j
001	10	k
001	11	l
010	00	m
010	01	n
010	10	o
010	11	p
011	00	
011	01	
011	10	
011	11	
100	00	
100	01	
100	10	
100	11	
101	00	a
101	01	b
101	10	c
101	11	d
110	00	e
110	01	f
110	10	g
110	11	h
111	00	
111	01	
111	10	
111	11	

physical memory

- Logical address: $n = 3$ and $m = 5$. Using a page size of $2^3 = 8$ bytes and a physical memory of 32 bytes (4 pages)

00	000	a
00	001	b
00	010	c
00	011	d
00	100	e
00	101	f
00	110	g
00	111	h
01	000	i
01	001	j
01	010	k
01	011	l
01	100	m
01	101	n
01	110	o
01	111	p

Logical memory

Page Table

p	f
00	11
01	10

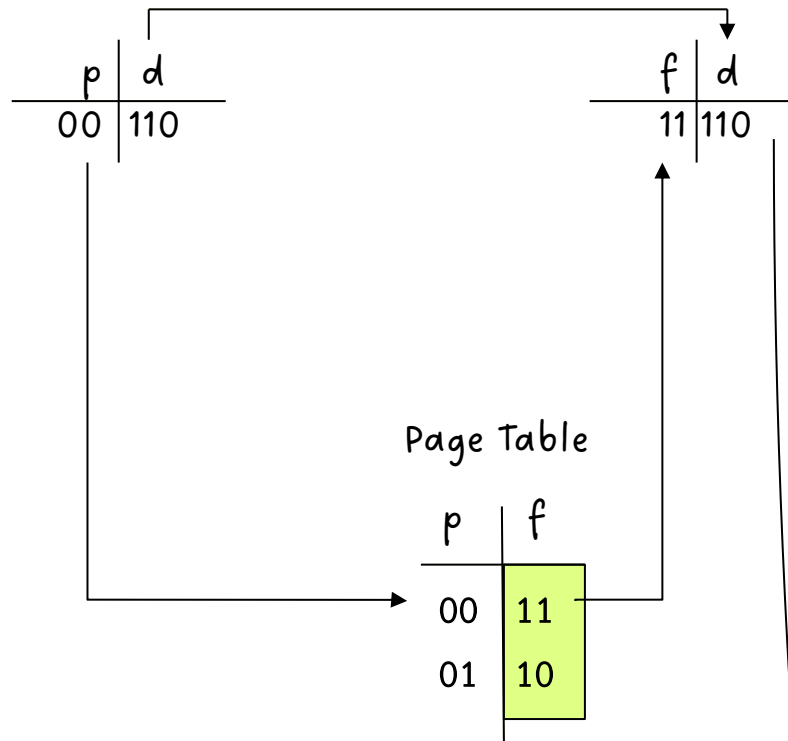
00. 000
00. 001
00. 010
00. 011
00. 100
00. 101
00. 110
00. 111
01. 000
01. 001
01. 010
01. 011
01. 100
01. 101
01. 110
01. 111
10. 000
10. 001
10. 010
10. 011
10. 100
10. 101
10. 110
10. 111
11. 000
11. 001
11. 010
11. 011
11. 100
11. 101
11. 110
11. 111

i
j
k
l
m
n
o
p
a
b
c
d
e
f
g
h

physical memory

00	000	a
00	001	b
00	010	c
00	011	d
00	100	e
00	101	f
00	110	g
00	111	h
01	000	i
01	001	j
01	010	k
01	011	l
01	100	m
01	101	n
01	110	o
01	111	p

Logical memory

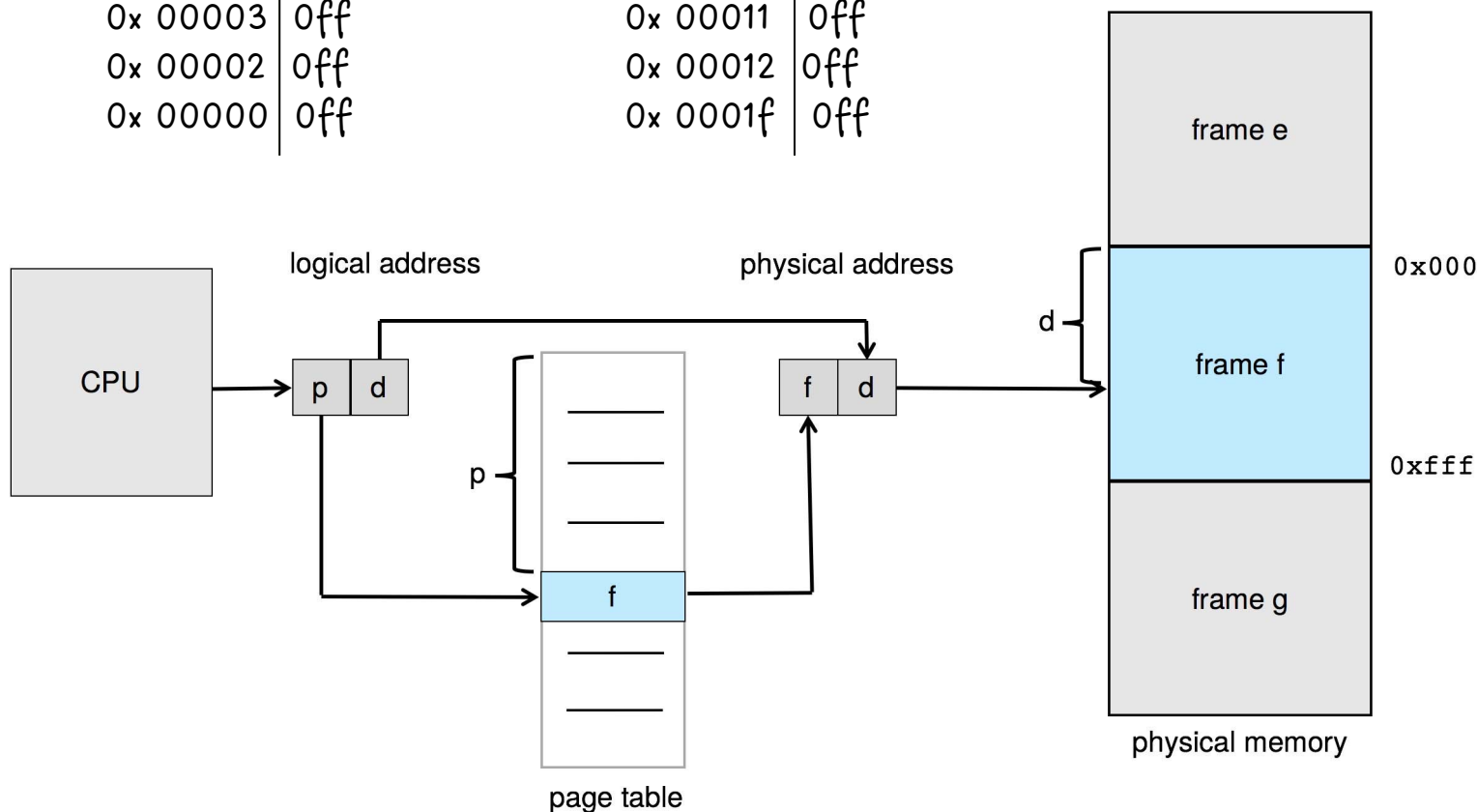


00.	000	
00.	001	
00.	010	
00.	011	
00.	100	
00.	101	
00.	110	
00.	111	
01.	000	
01.	001	
01.	010	
01.	011	
01.	100	
01.	101	
01.	110	
01.	111	
10.	000	i
10.	001	j
10.	010	k
10.	011	l
10.	100	m
10.	101	n
10.	110	o
10.	111	p
11.	000	a
11.	001	b
11.	010	c
11.	011	d
11.	100	e
11.	101	f
11.	110	g
11.	111	h

physical memory

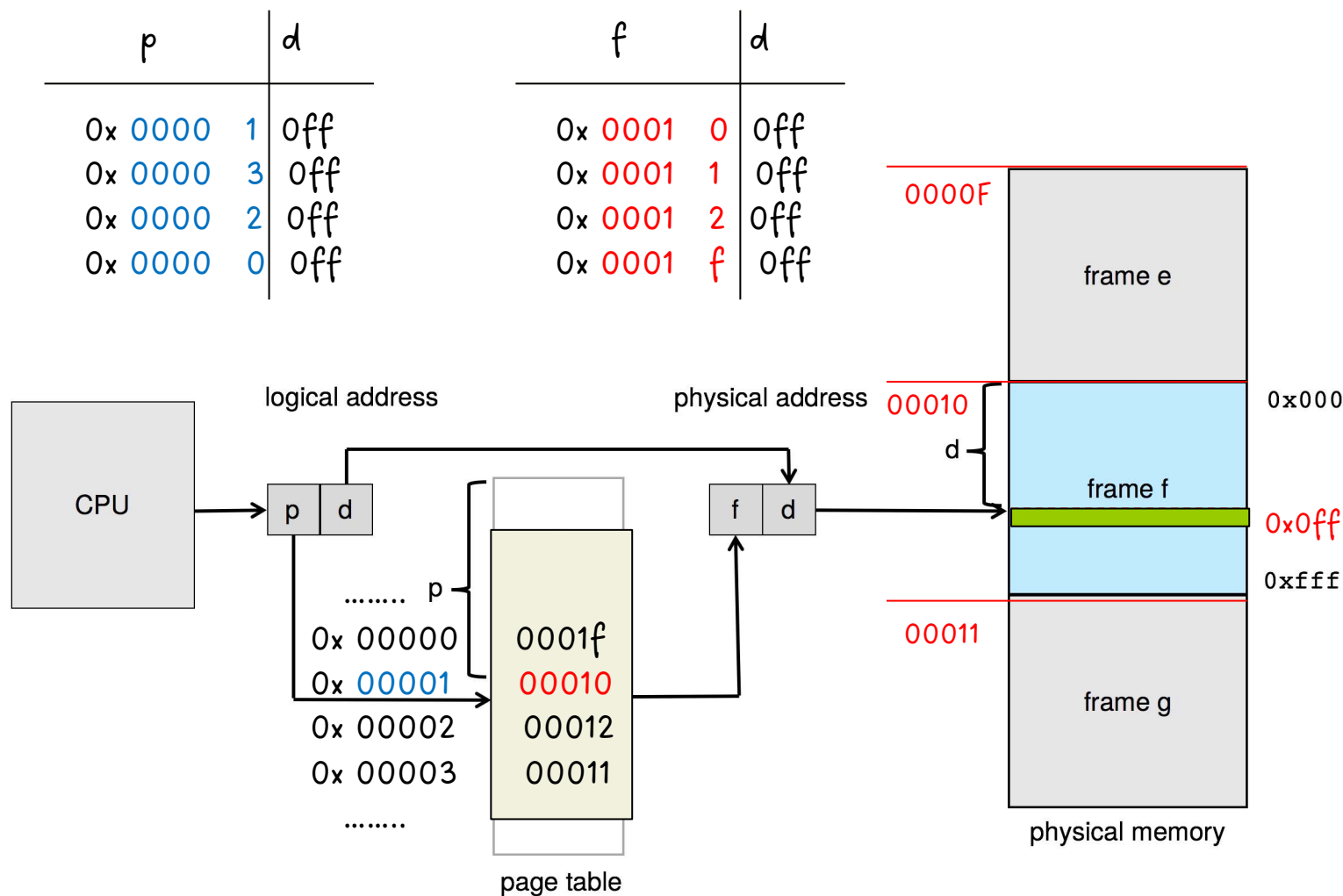
- Logical address: $n = 12$ and $m = 32$. Using a page size of $2^{12} = 4\text{KB}$ bytes and a physical memory of $2^{32} = 4\text{GB}$ ($4 \cdot 1024^3 / 4 \cdot 1024^1 = 1024^2 = 1048576$ pages)

p	d	f	d
0x 00001	off	0x 00010	off
0x 00003	off	0x 00011	off
0x 00002	off	0x 00012	off
0x 00000	off	0x 0001f	off



- ให้ ตัวอย่างของการแปลง **logical addr** ไปเป็น **physical addr** แล้วให้ นศ สร้าง **page table**

- Logical address: $n = 12$ and $m = 32$. Using a page size of $2^{12} = 4\text{KB}$ bytes and a physical memory of $2^{32} = 4\text{GB}$ ($4 \cdot 1024^3 / 4 \cdot 1024^1 = 1024^2 = 1048576$ pages)

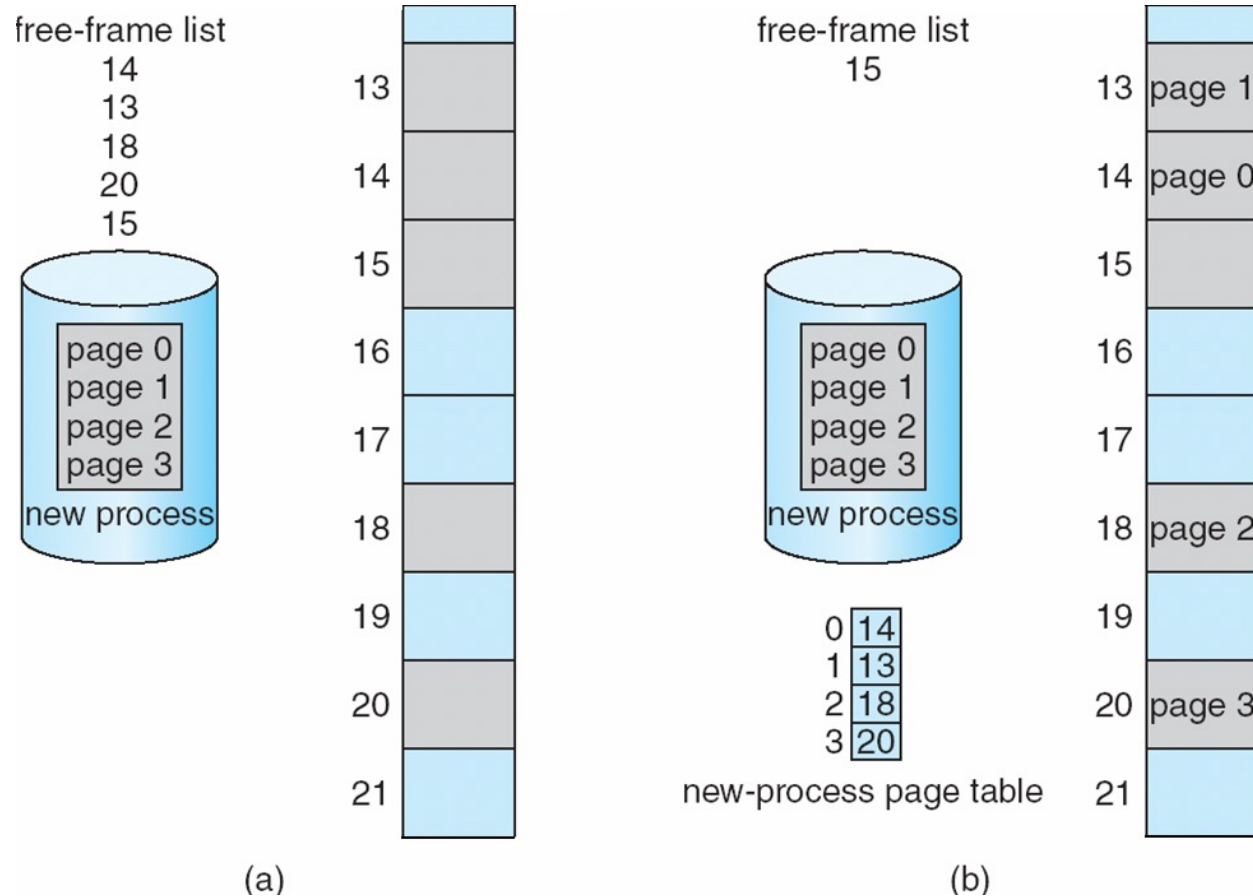


- โจทย์ ให้ ตัวอย่างของการแปลง **logical addr** ไปเป็น **physical addr** แล้วให้ นศ สร้าง **page table**

Paging -- Calculating internal fragmentation

- Page size = 2,048 bytes
- Process size = 72,766 bytes
- 35 pages + 1,086 bytes
- Internal fragmentation of $2,048 - 1,086 = 962$ bytes
- Worst case fragmentation = 1 frame – 1 byte
- On average fragmentation = 1 / 2 frame size
- So small frame sizes desirable?
- But each page table entry takes memory to track
- Page sizes growing over time
 - Solaris supports two page sizes – 8 KB and 4 MB
 - Windows 10 supports two page sizes – 4 KB and 2MB
 - Linux usual page size is 4KB
 - ▶ Linux support huge page – 2 MB to 256 MB

Free Frames



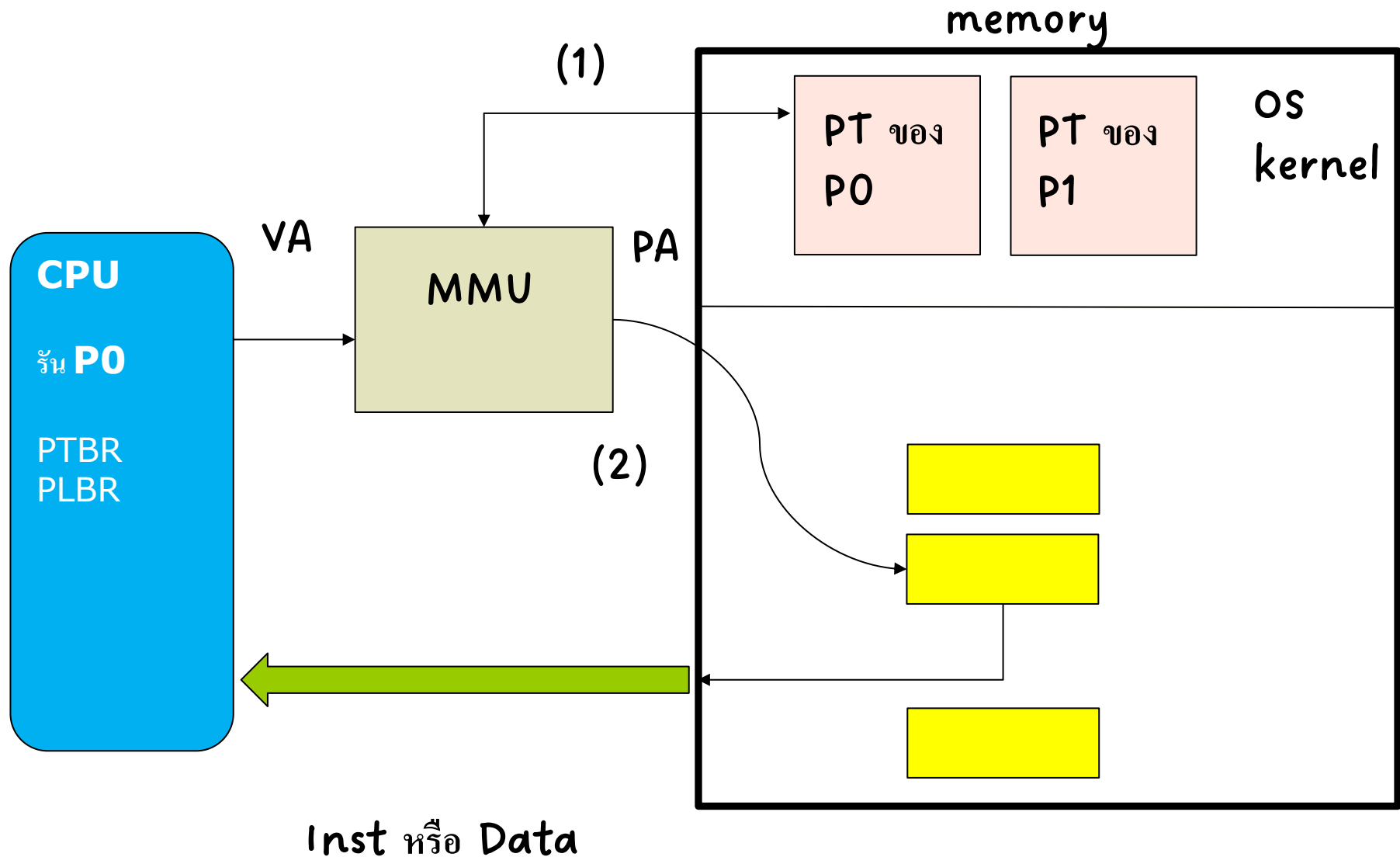
Before allocation

After allocation

Implementation of Page Table

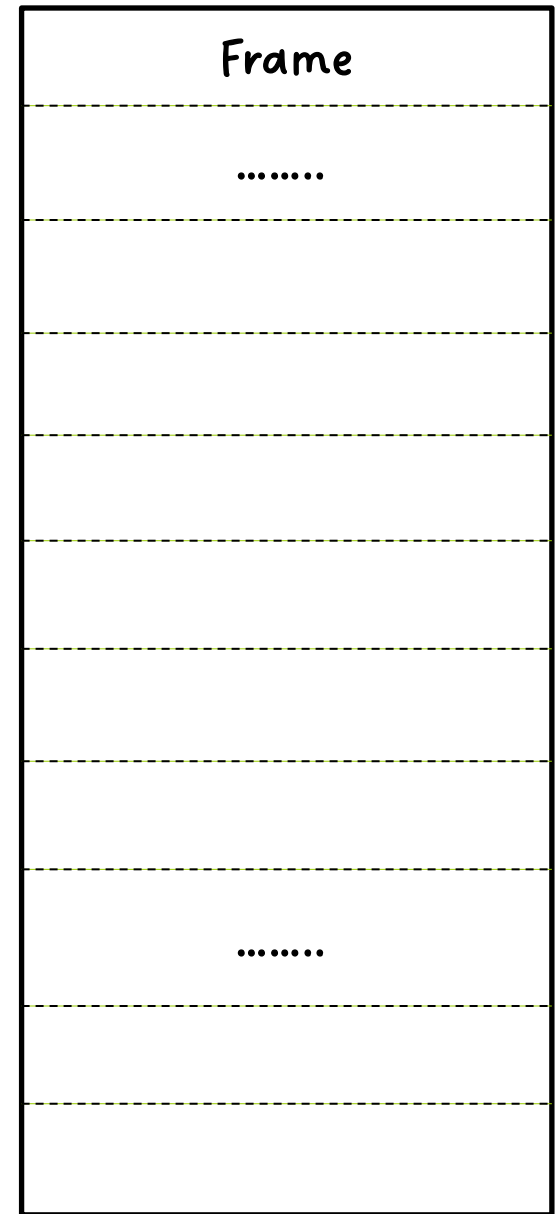
- Page table is kept in main memory
 - **Page-table base register (PTBR)** points to the page table
 - ▶ ใน x86_64 จะใช้ CR3 register
 - **Page-table length register (PTLR)** indicates size of the page table
- In this scheme every data/instruction access requires **two memory accesses**
 - One for the page table and one for the data / instruction
- The two-memory access problem can be solved by the use of a special fast-lookup hardware cache called **translation look-aside buffers (TLBs)** (also called **associative memory**).

Page Table อยู่ที่ไหน



0x0000

- Physical Memory Space ถูกแบ่งเป็น Frames
- Logical Memory Space ถูกแบ่งเป็น Pages
- ขนาดของ Page กับ Frame เท่ากัน

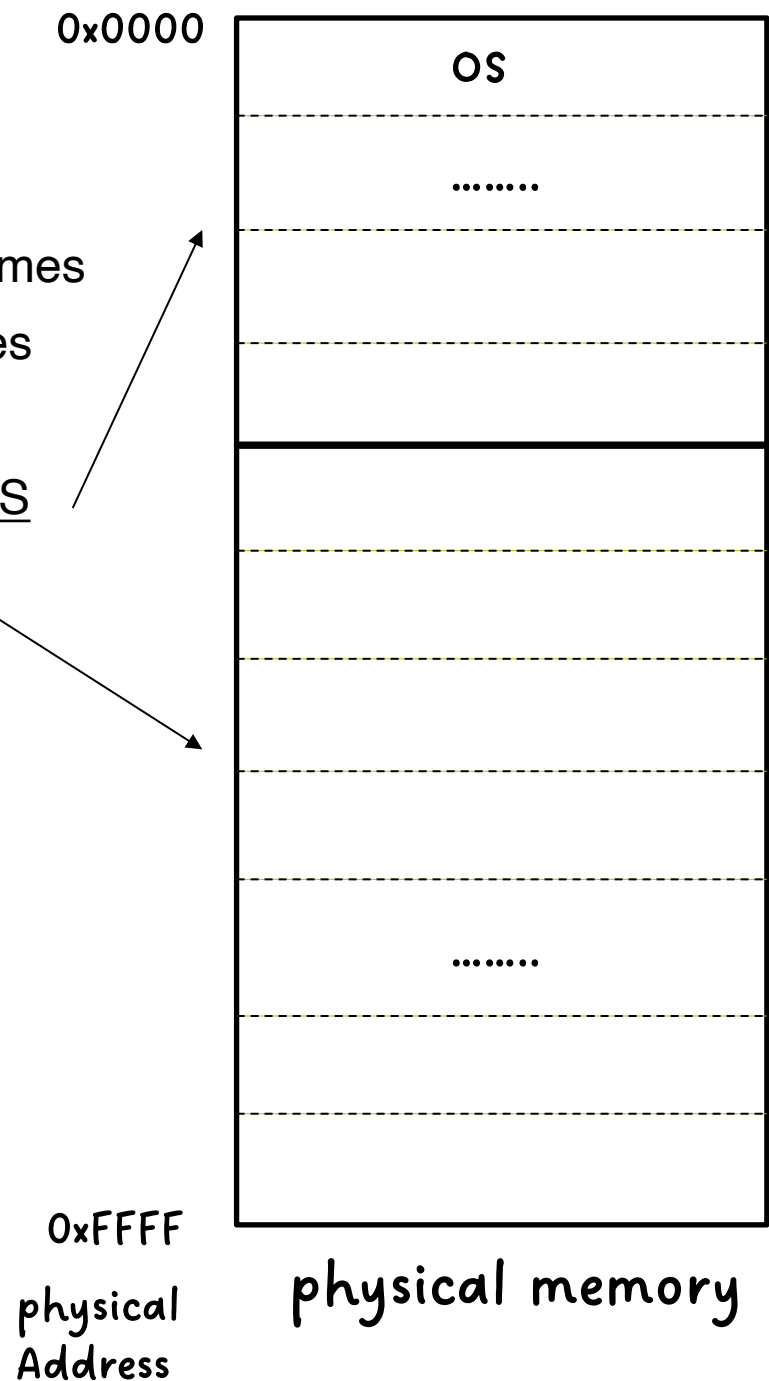


0xFFFF

physical
Address

physical memory

- Physical Memory Space ถูกแบ่งเป็น Frames
- Logical Memory Space ถูกแบ่งเป็น Pages
- ขนาดของ Page กับ Frame เท่ากัน
- Frames จำนวนหนึ่งจะถูกจองไว้ใช้ตลอดเวลาโดย OS
- ส่วนที่เหลือใช้สำหรับเก็บข้อมูลของ Process



0x0000

OS

- Physical Memory Space ถูกแบ่งเป็น Frames
- Logical Memory Space ถูกแบ่งเป็น Pages
- ขนาดของ Page กับ Frame เท่ากัน
- Frames จำนวนหนึ่งจะถูกจองไว้ใช้ตลอดเวลาโดย OS
- ส่วนที่เหลือใช้สำหรับเก็บข้อมูลของ Process
- Page Table ของแต่ละ Process จะอยู่ใน memory ของ OS
- เวลาที่แต่ละ Process ประมวลผล OS จะกำหนดค่า Page-table base register (PTBR) และ Page-table length register (PTLR)

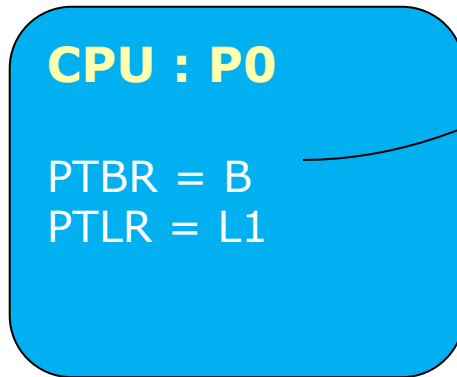
.....

0xFFFF

physical
Address

physical memory

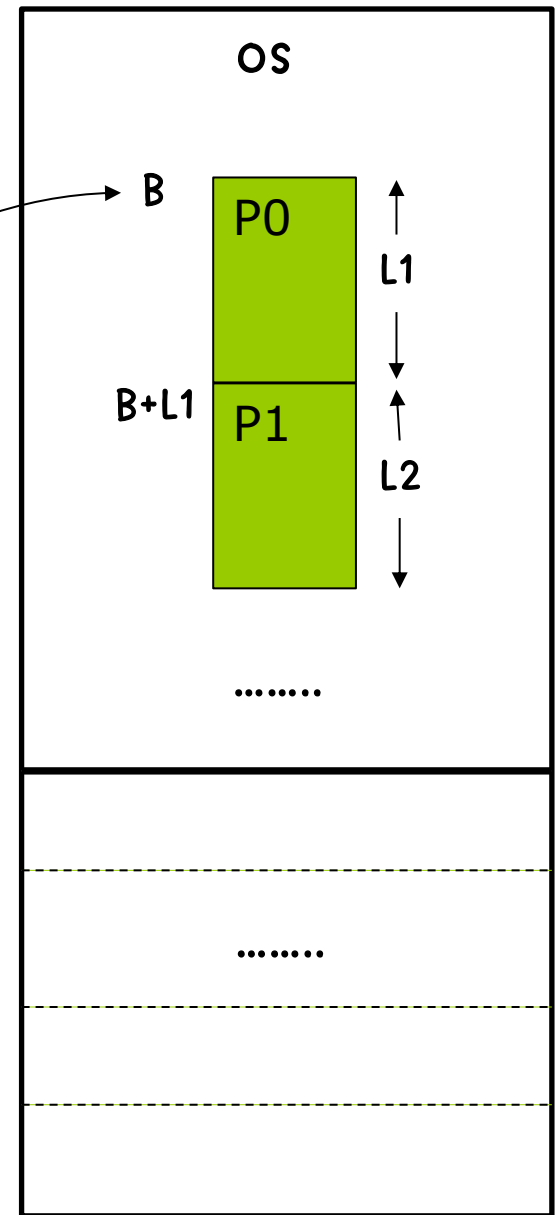
- เวลาที่แต่ละ Process P0 ประมวลผล OS จะกำหนดค่า Page-table base register (PTBR) และ Page-table length register (PTLR)



- เรามี Page-table length register (PTLR) เพราะ Process ไม่ได้ใช้ Logical Address Space ทั้งหมด และเพื่อป้องกันไม่ให้เสียพื้นที่ page table โดยเปล่าประโยชน์

0x0000

0xFFFF
physical
Address

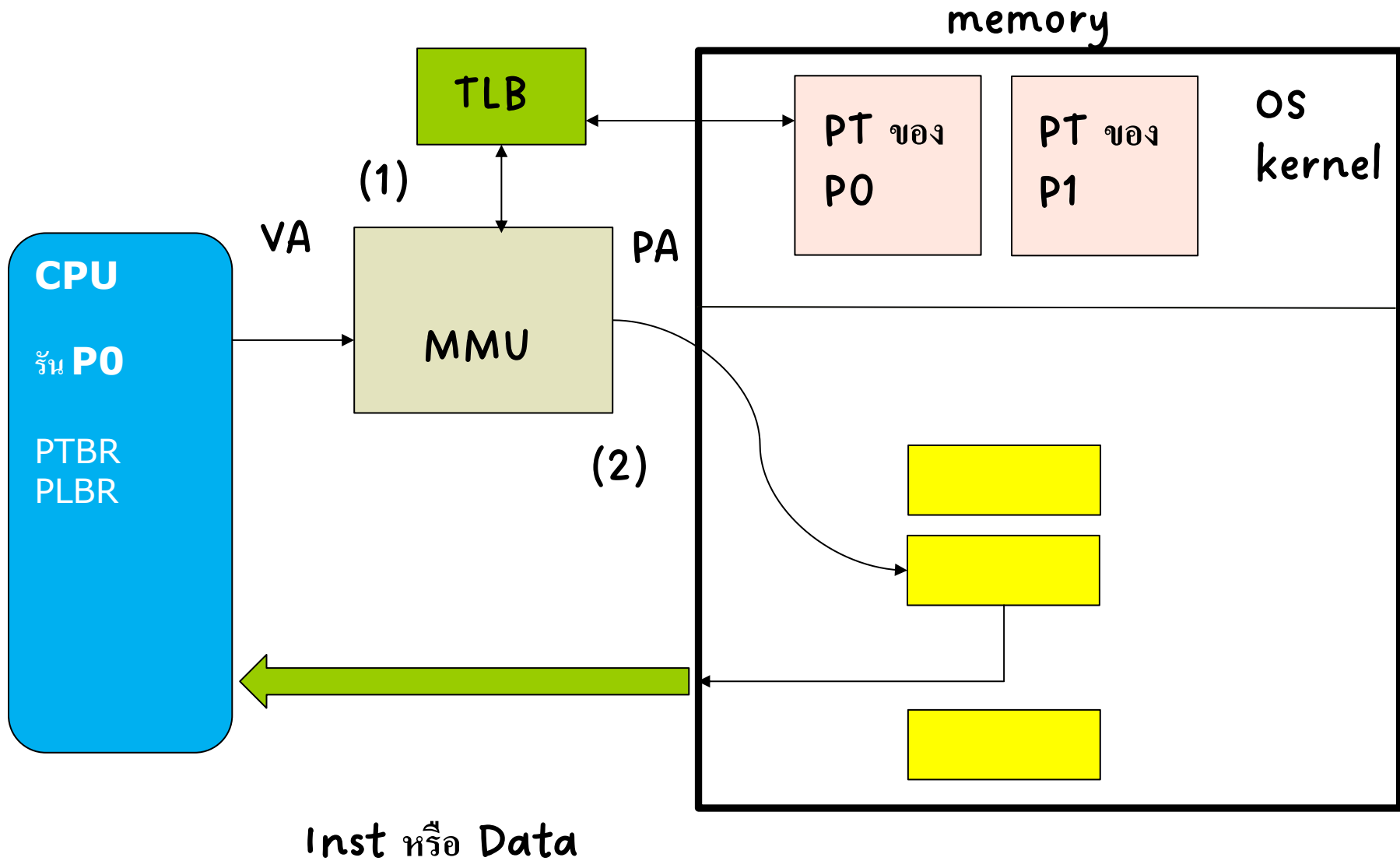


physical memory

Translation Look-Aside Buffer

- ปัญหาของ Page Table คือ มันทำให้การเข้าถึง Memory ครั้งหนึ่ง ต้องเข้าถึง memory ถึง 2 ครั้ง
 - **Translation Look-Aside Buffer** คือ หน่วยความจำความเร็วสูง ที่ MMU ใช้เก็บข้อมูลการ **mapping** ของ **page table** ที่ใช้บ่อย (เหมือน **Data** หรือ **Instruction cache**)
 - Some TLBs store **address-space identifiers (ASIDs)** in each TLB entry – uniquely identifies each process to provide address-space protection for that process
 - Otherwise need to flush at every context switch
 - TLBs typically small (64 to 1,024 entries)
 - On a TLB miss, value is loaded into the TLB for faster access next time
 - Replacement policies must be considered
 - Some entries can be **wired down** for permanent fast access
- บาง address entry สามารถถูกเก็บถาวรใน TLB เช่นตำแหน่งของ kernel code

TLB อยู่ที่ไหน



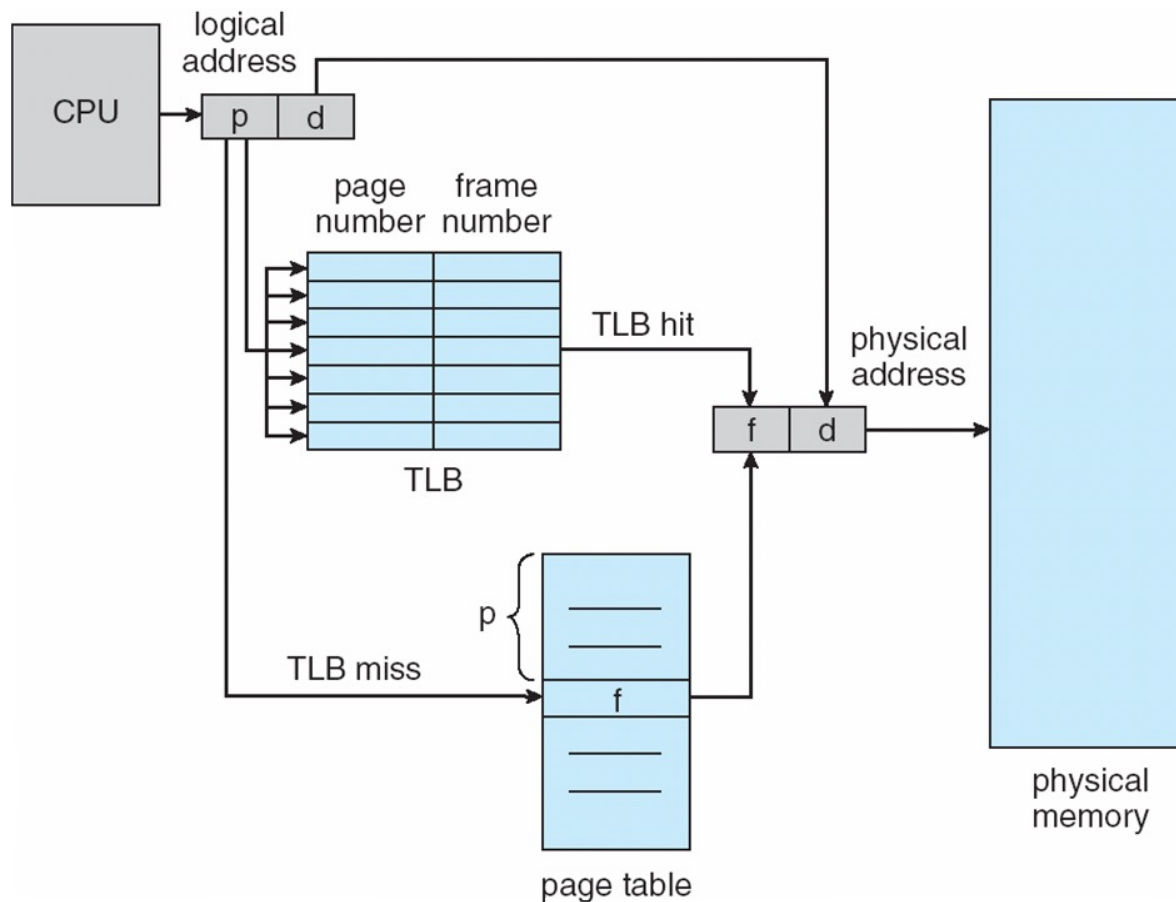
Hardware

- Associative memory – parallel search

Page #	Frame #

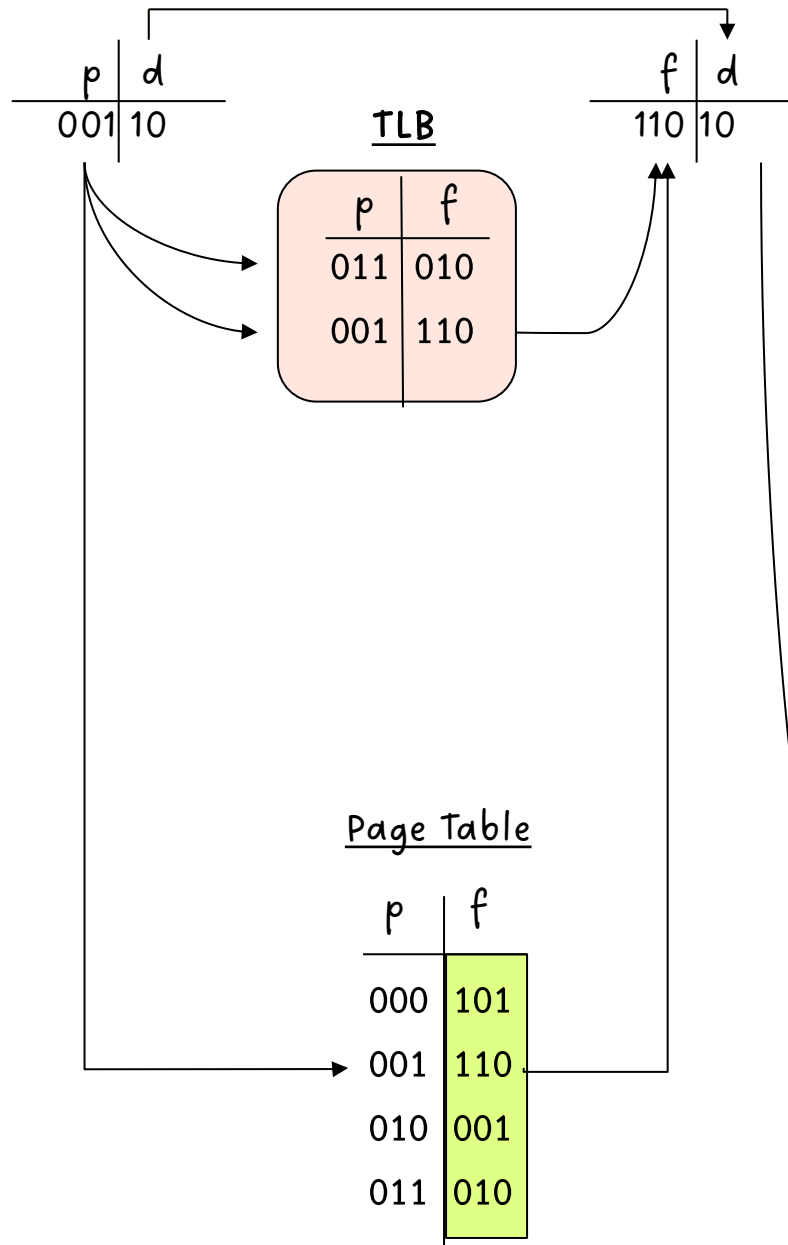
- Address translation (p, d)
 - If p is in associative register, get frame # out
 - Otherwise get frame # from page table in memory
- ใน สถาปัตยกรรม 64 bit
 - Page # คือ bit ที่ 12 ถึง 47

Paging Hardware With TLB



000	00	a
000	01	b
000	10	c
000	11	d
001	00	e
001	01	f
001	10	g
001	11	h
010	00	i
010	01	j
010	10	k
010	11	l
011	00	m
011	01	n
011	10	o
011	11	p

Logical memory



000	00	
000	01	
000	10	
000	11	
001	00	i
001	01	j
001	10	k
001	11	l
010	00	m
010	01	n
010	10	o
010	11	p
011	00	
011	01	
011	10	
011	11	
100	00	
100	01	
100	10	
100	11	
101	00	a
101	01	b
101	10	c
101	11	d
110	00	e
110	01	f
110	10	g
110	11	h
111	00	
111	01	
111	10	
111	11	

physical memory

Effective Access Time

- Hit ratio – percentage of times that a page number is found in the TLB
- An 80% hit ratio means that we find the desired page number in the TLB 80% of the time.
- Suppose that 10 nanoseconds to access memory.
 - If we find the desired page in TLB then a mapped-memory access take 10 ns
 - Otherwise we need two memory access so it is 20 ns
- **Effective Access Time (EAT)**
$$\text{EAT} = 0.80 \times 10 + 0.20 \times 20 = 12 \text{ nanoseconds}$$
implying 20% slowdown in access time
- Consider amore realistic hit ratio of 99%,
$$\text{EAT} = 0.99 \times 10 + 0.01 \times 20 = 10.1\text{ns}$$
implying only 1% slowdown in access time.
- Intel Core i7 မှီ 128 entries L1 Instruction TLB နှင့် 64 entries L1 data TLB
နှင့် 512 entries L2 TLB

TLB: ASID

- Address Space Identifier (ASID) เป็นตัวเลขที่ OS กำหนดให้แต่ละ Process
- ทำให้สามารถใช้ TLB เก็บ mapping page# → frame # ของหลาย Process ได้
- Associative memory – parallel search

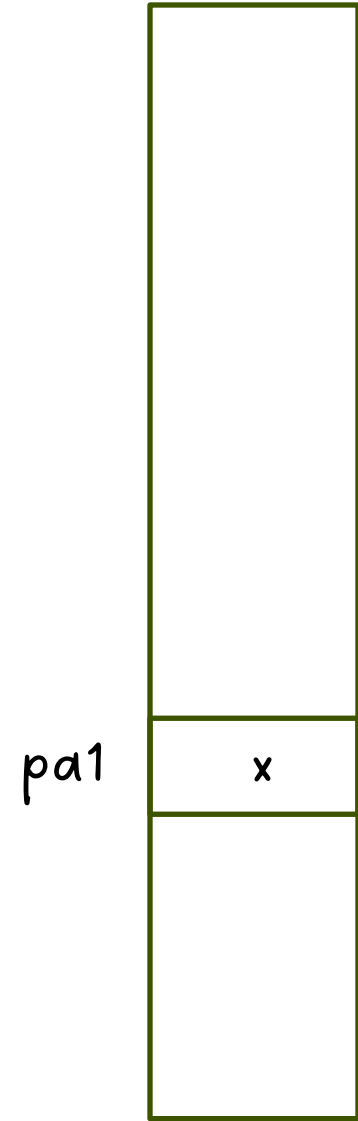
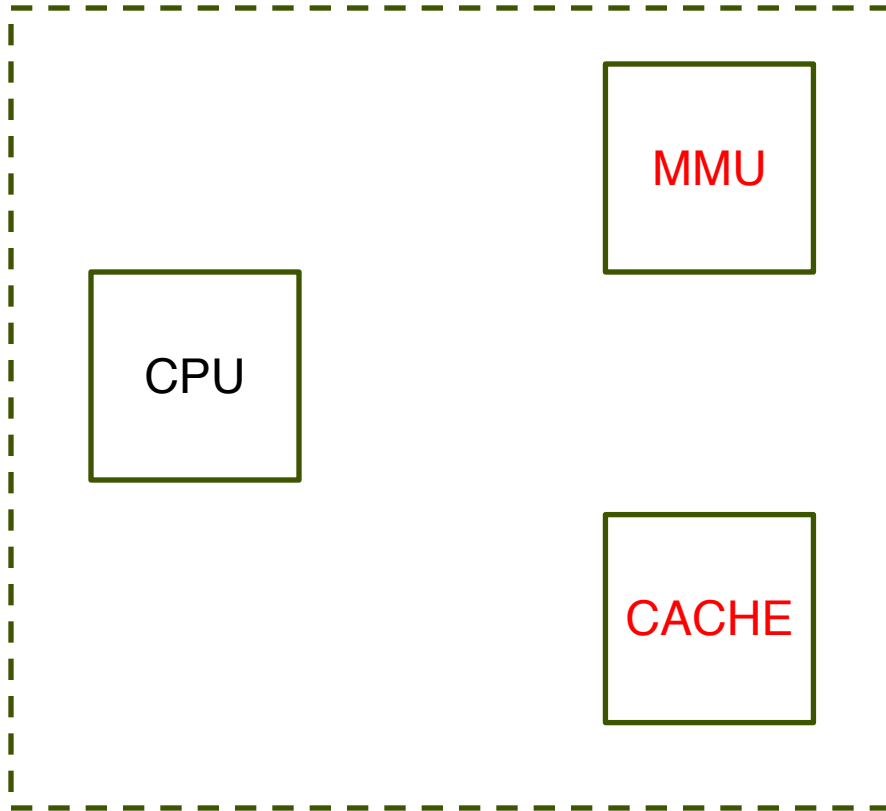
Page #	ASID #	Frame #

- Address translation (p, a, d)
 - If p is in associative register and a เป็น id ของ Process ที่ใช้งาน CPU อยู่,
get frame # out
 - Otherwise get frame # from page table in memory
- ASID ช่วยให้ OS ไม่ต้อง Flush TLB ทุกครั้งที่มีการ context switching process ใหม่
เข้ามาใช้งาน CPU

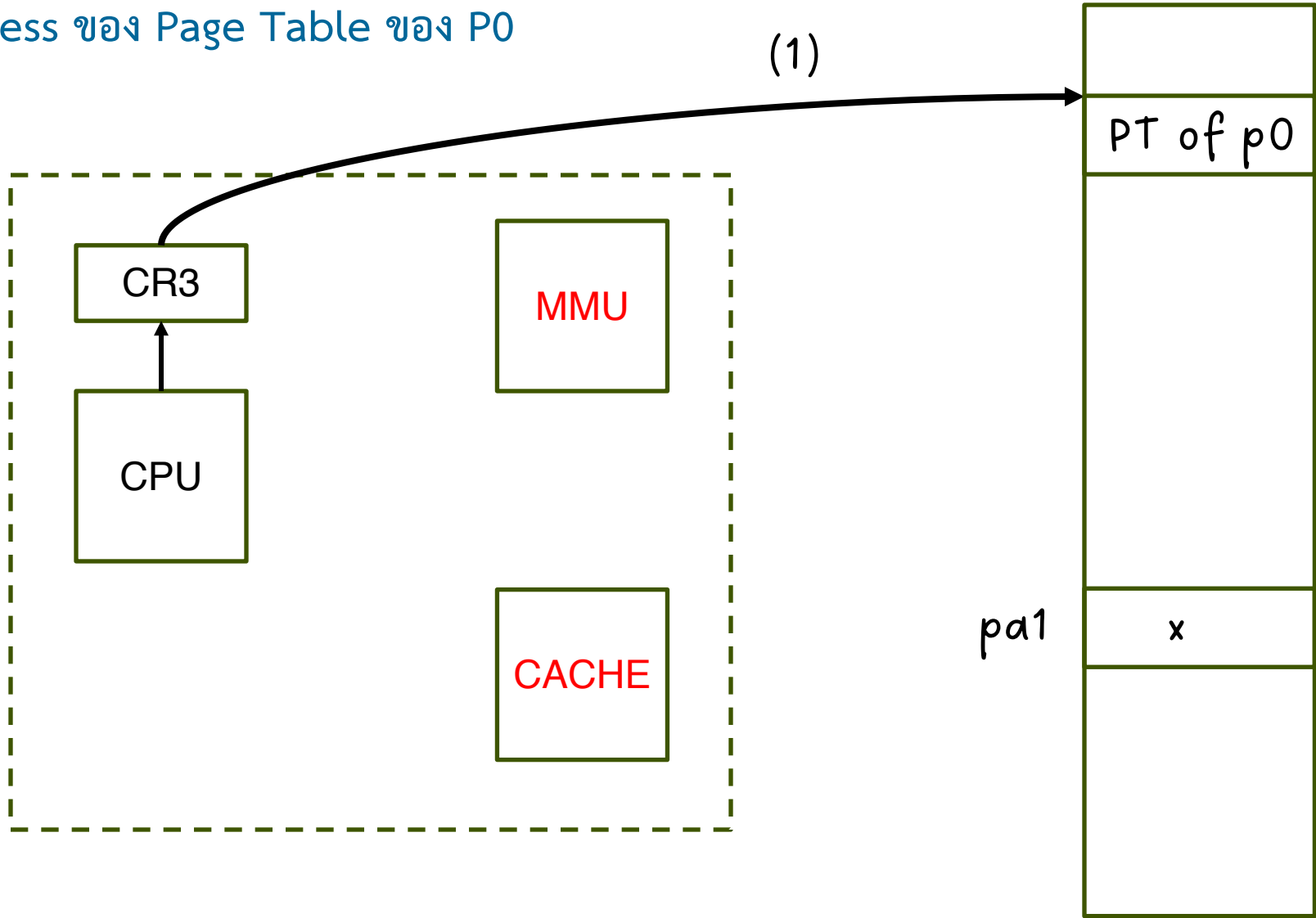


ภาพรวมการเข้าถึงหน่วยความจำของ **CPU**

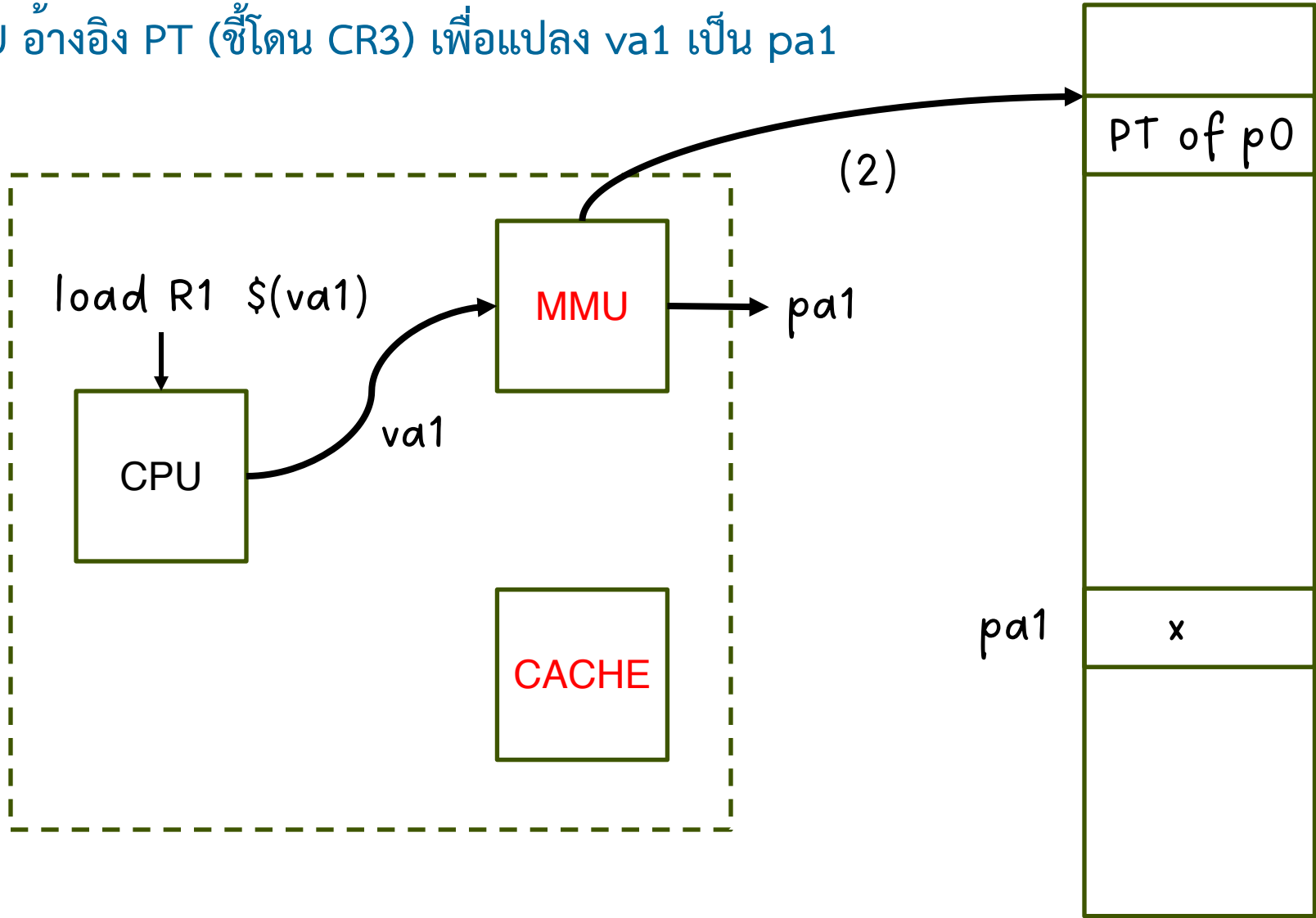
การเข้าถึงข้อมูลที่ virtual addr = va1
สมมติว่า va1 คือ physical addr = pa1
ซึ่งเก็บข้อมูลคือ x

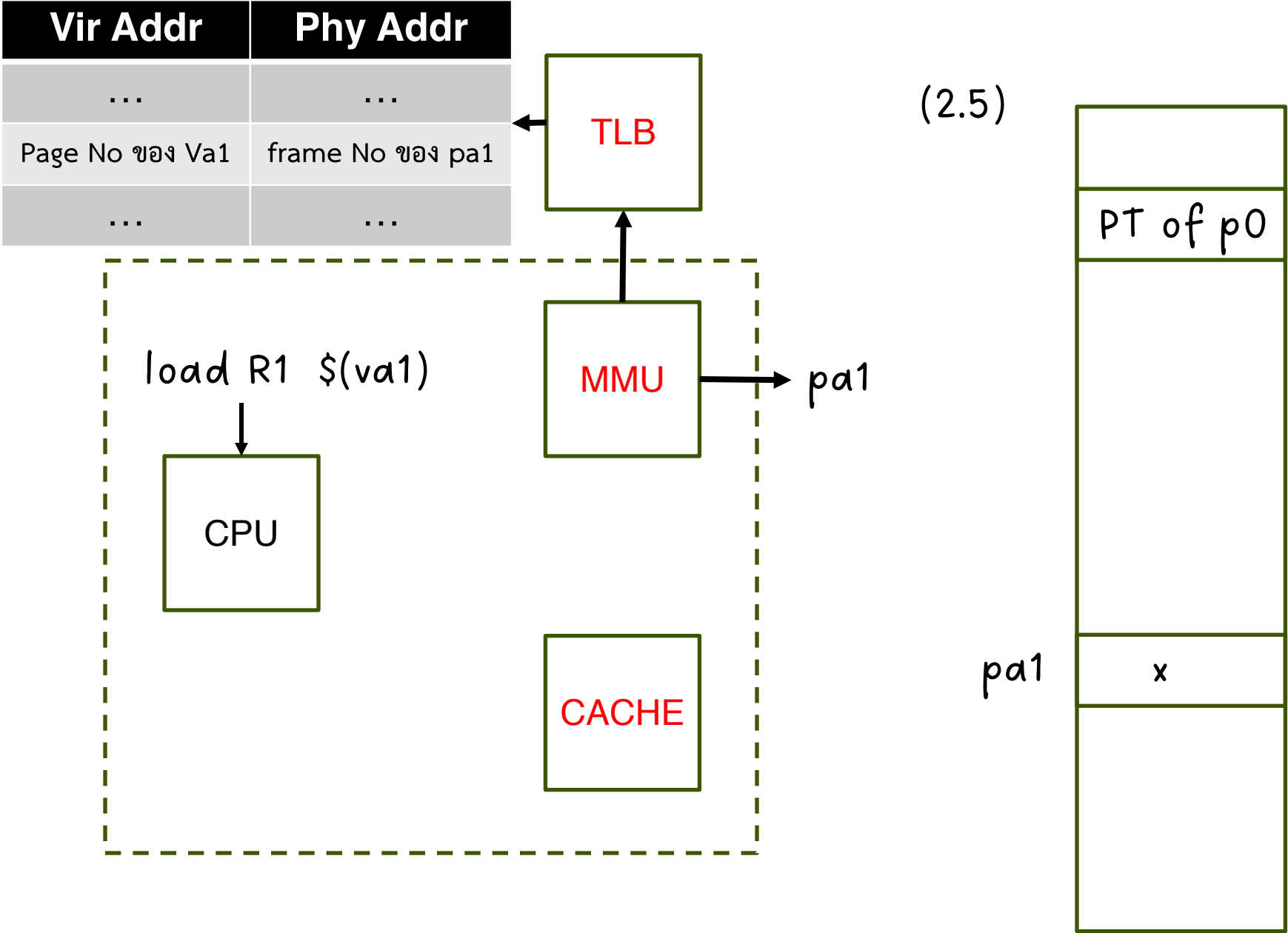


(1) กำหนดให้ CR3ชี้ที่ physical memory address ของ Page Table ของ P0



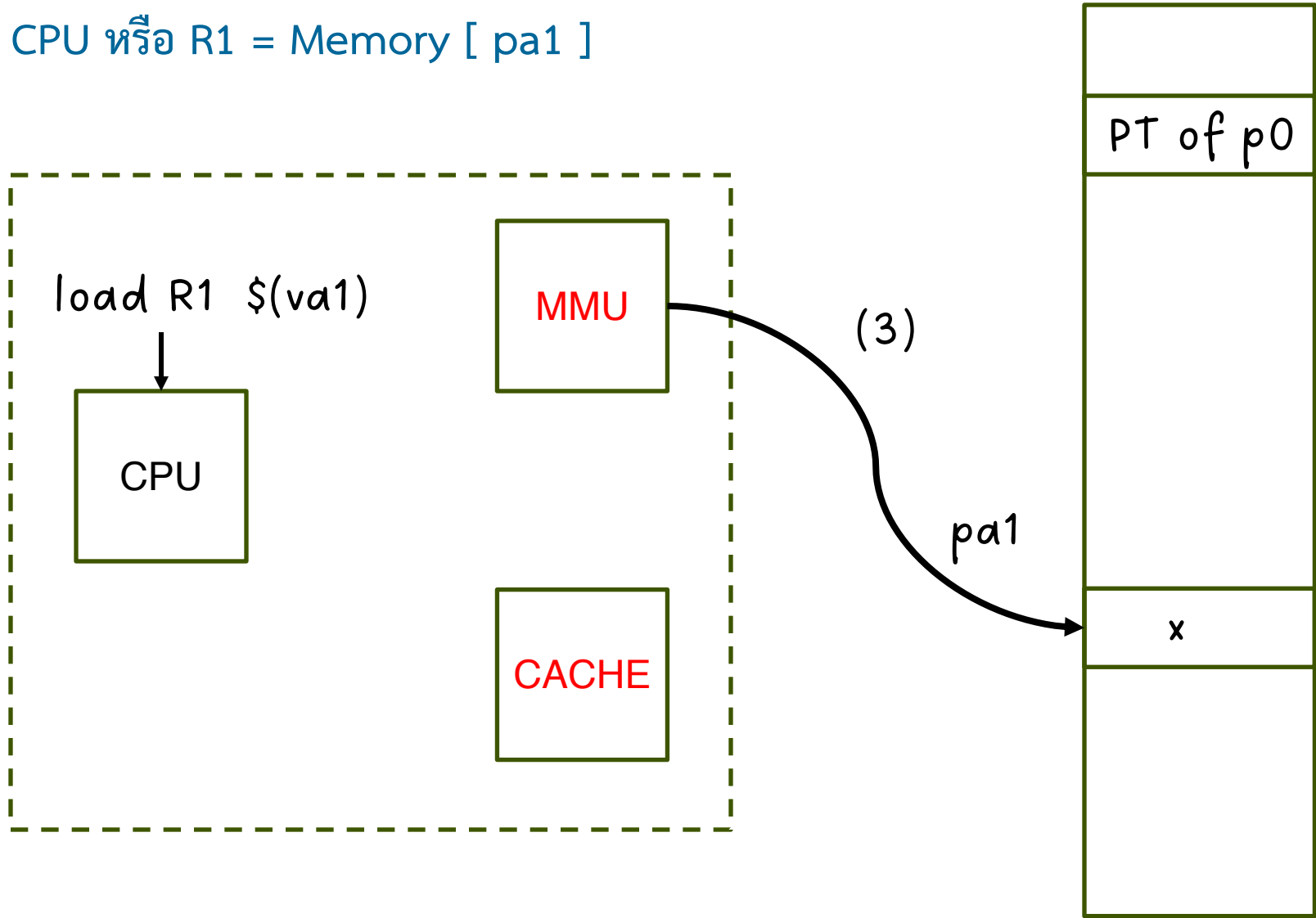
(2) cpu รับคำสั่งอ่านค่าจาก va1 มายัง register R1
MMU อ้างอิง PT (ชี้โดน CR3) เพื่อแปลง va1 เป็น pa1



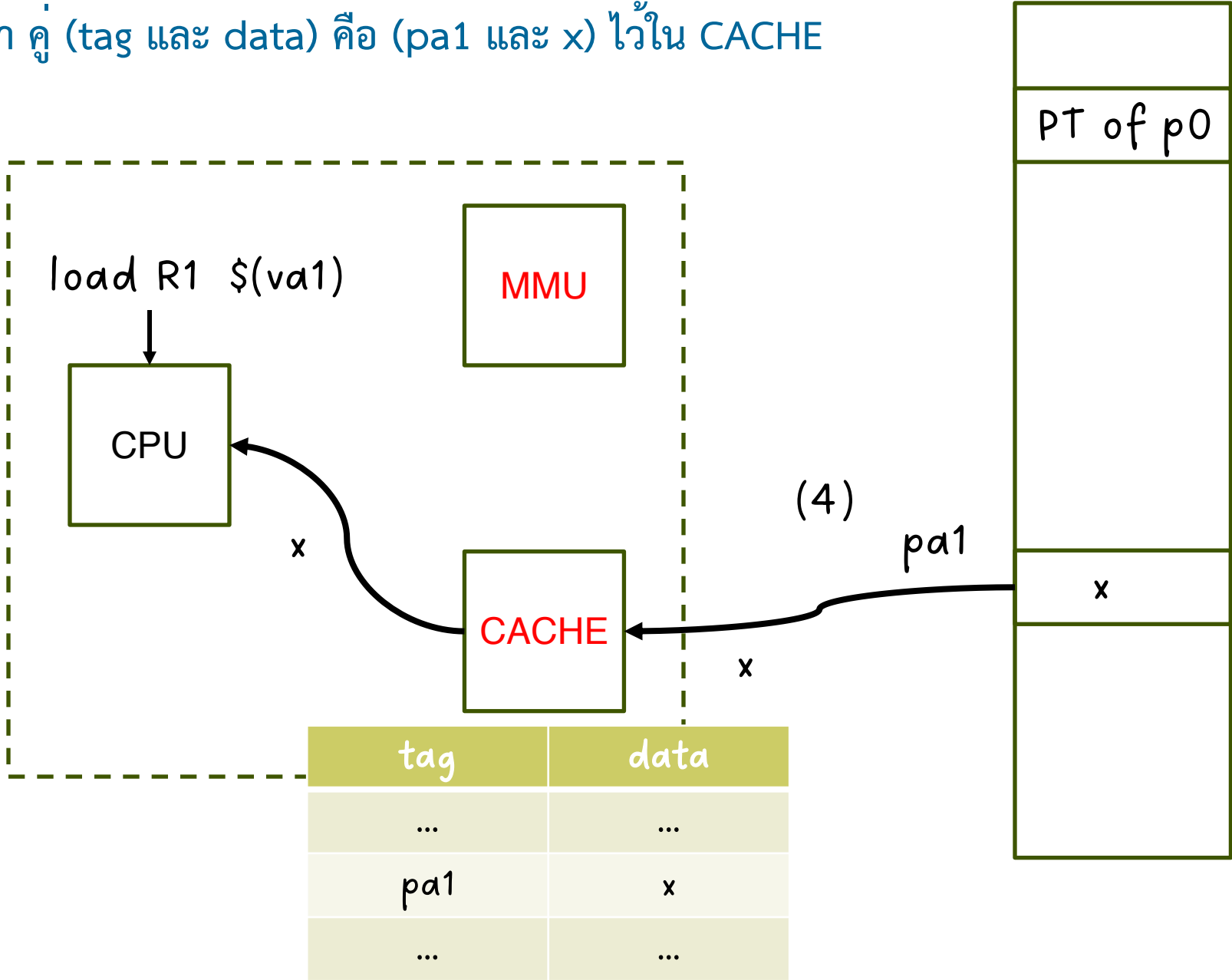


(2.5) MMU นำค่า va1 และ pa1 ไปเก็บใน TLB (เรียกอีกอย่างว่า CACHE ของ Memory Address Translation

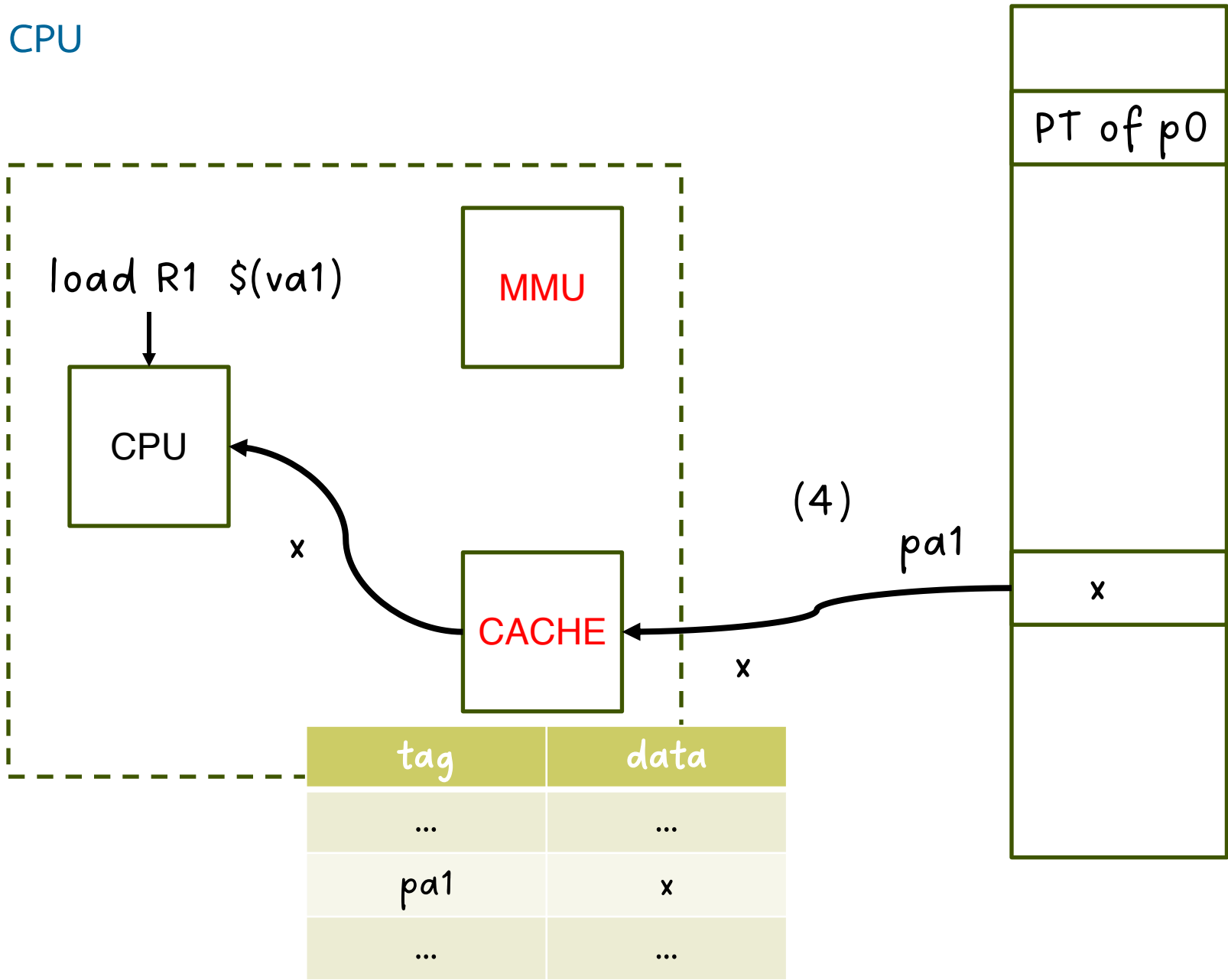
(3) MMU ส่งคำขอที่ memory เพื่อ ขอให้ส่งค่าที่เก็บใน pa1
มายัง CPU หรือ $R1 = \text{Memory} [\text{pa1}]$



(4) Memory ส่งค่า x มาให้กับ CPU ผ่านระบบ CACHE ซึ่งเก็บค่า คู่ (tag และ data) คือ (pa1 และ x) ไว้ใน CACHE

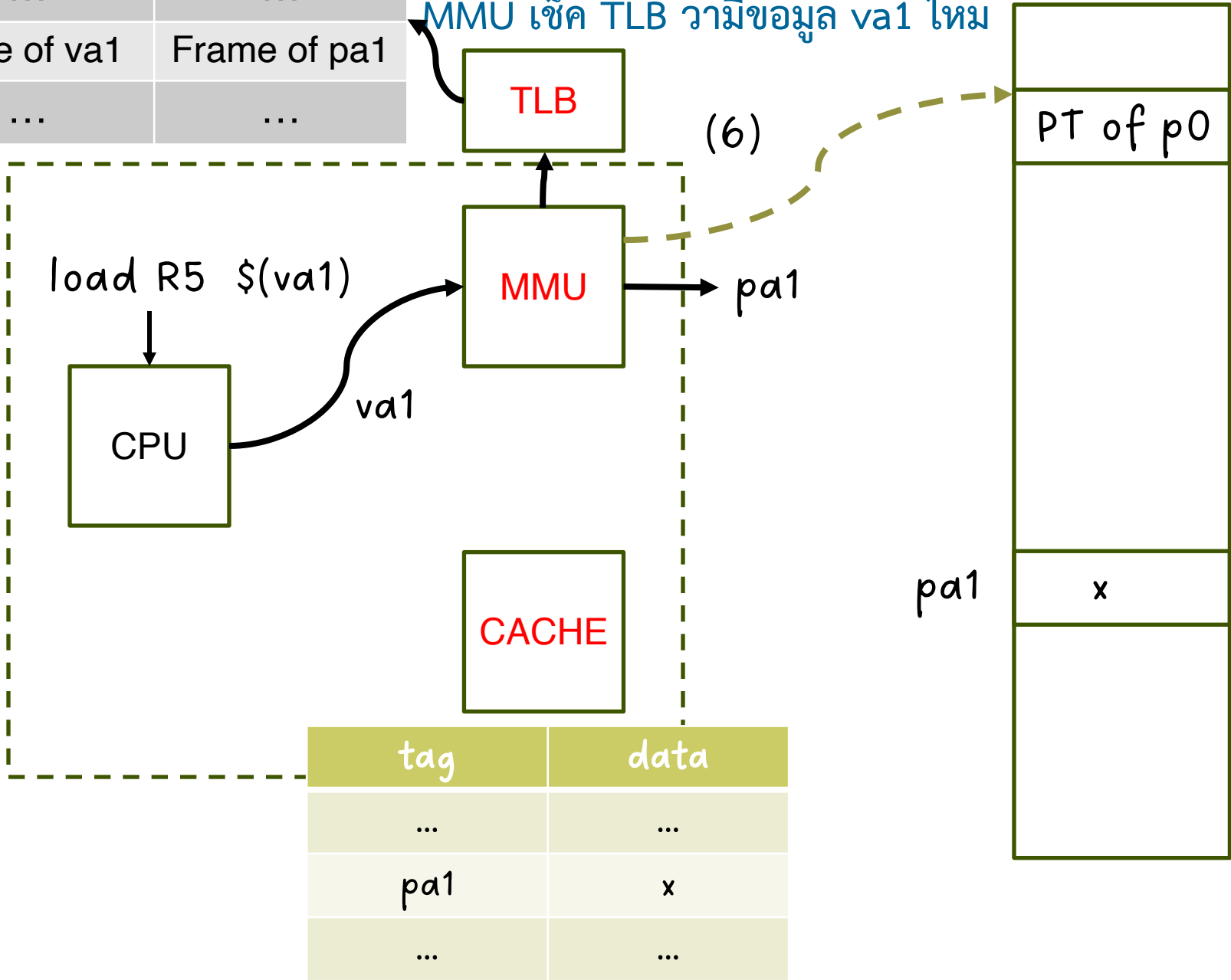


(5) MMU ส่งคำขอที่ memory เพื่อ ขอให้ส่งค่าที่เก็บใน pa1
มายัง CPU

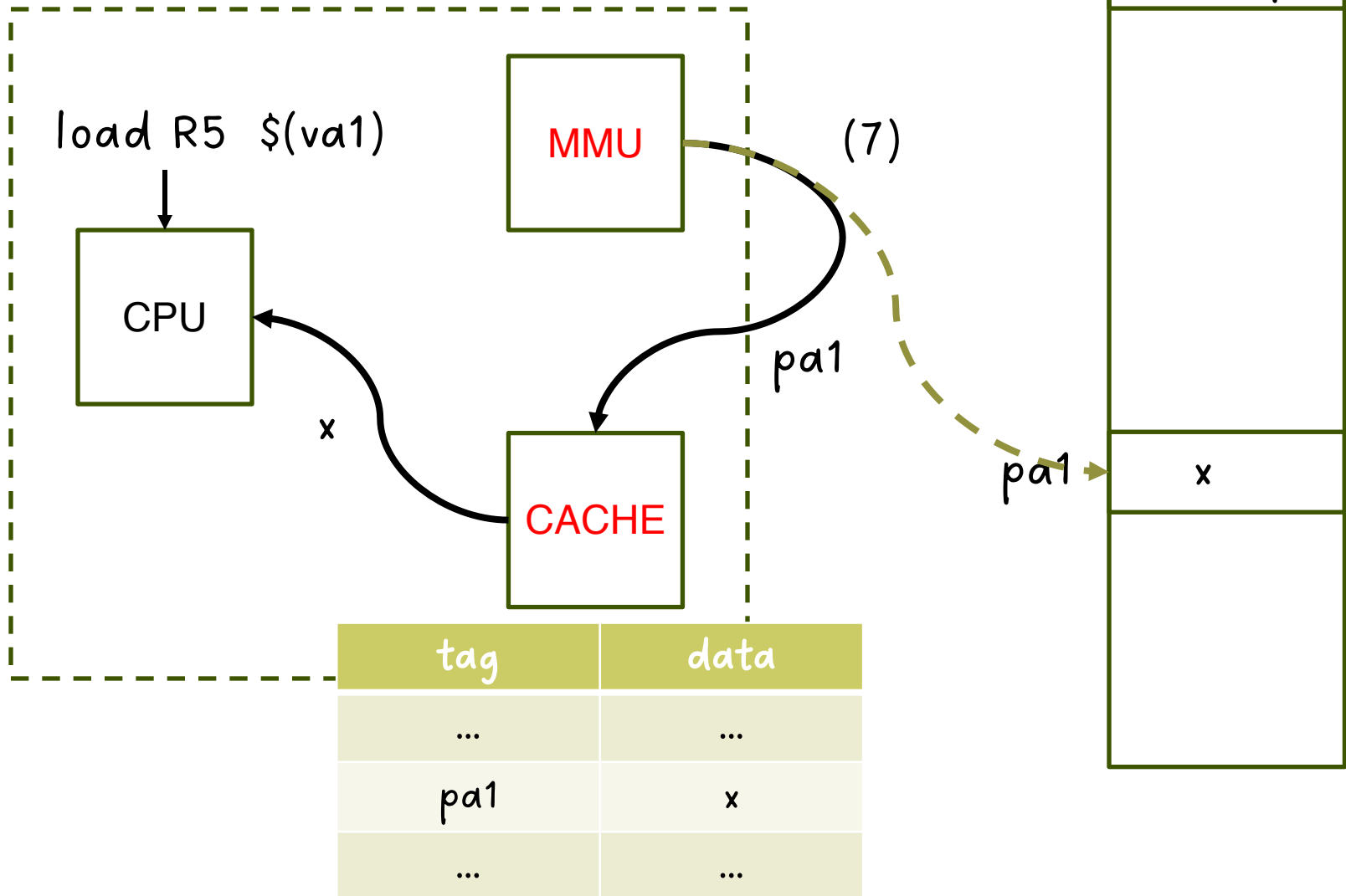


Vir Addr	Phy Addr
...	...
page of va1	Frame of pa1
...	...

(6) cpu รับคำสั่ง(ใหม่)เพื่ออ่านค่าจาก va1 มายัง register
MMU เช็ค TLB ว่ามีข้อมูล va1 ไหม



(7) MMU จะส่ง physical address pa1 ไปให้ระบบ CACHE
เช็คก่อนว่า เก็บข้อมูลไว้หรือเปล่า ถ้ามีก็เอาข้อมูลส่งให้ CPU ถ้า
ไม่มีถึงจะส่ง pa1 ไปให้ Memory

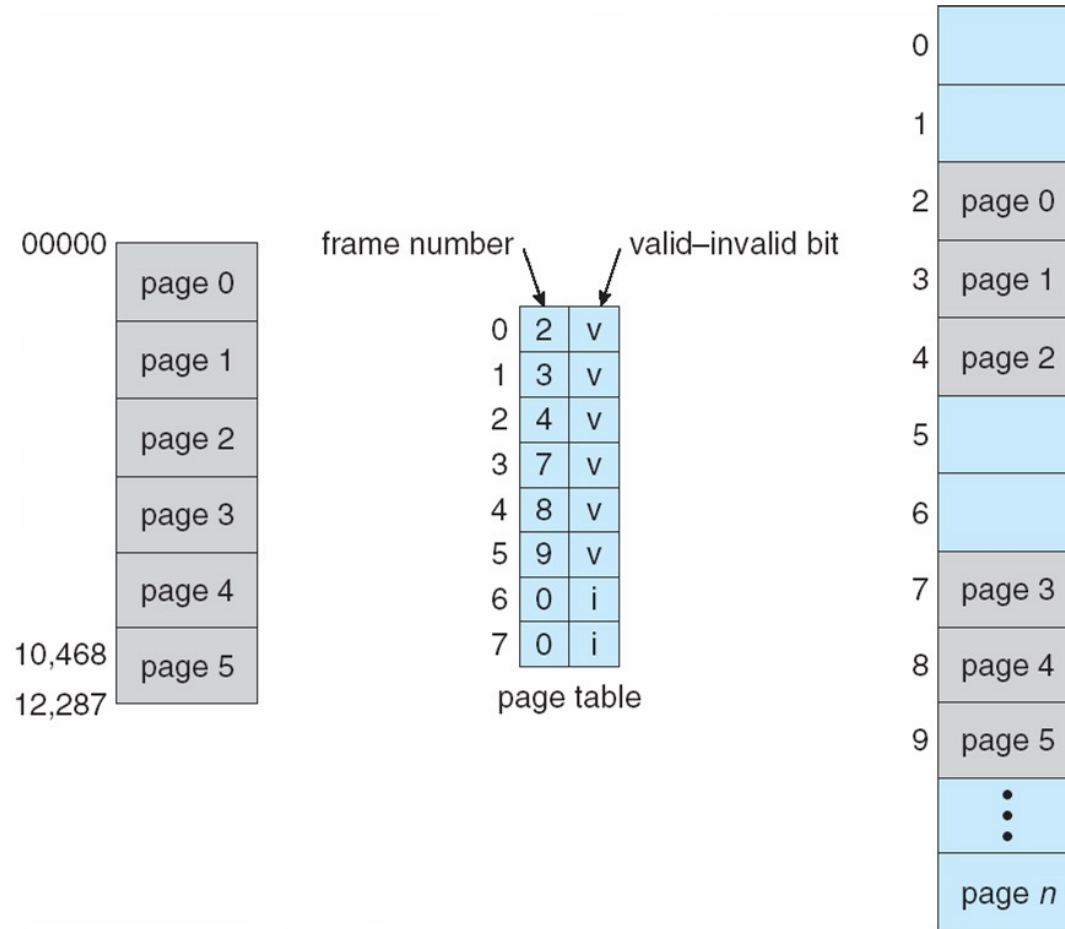


Memory Protection & Page Sharing

Memory Protection

- Memory protection implemented by associating protection bit with each frame to indicate if read-only or read-write access is allowed
 - Can also add more bits to indicate page execute-only, and so on
- **Valid-invalid** bit attached to each entry in the page table:
 - “valid” indicates that the associated page is in the process’ logical address space, and is thus a legal page
 - “invalid” indicates that the page is not in the process’ logical address space
 - Or use **page-table length register (PTLR)**
- Any violations result in a trap to the kernel

Valid (v) or Invalid (i) Bit In A Page Table



Shared Pages

■ Shared code

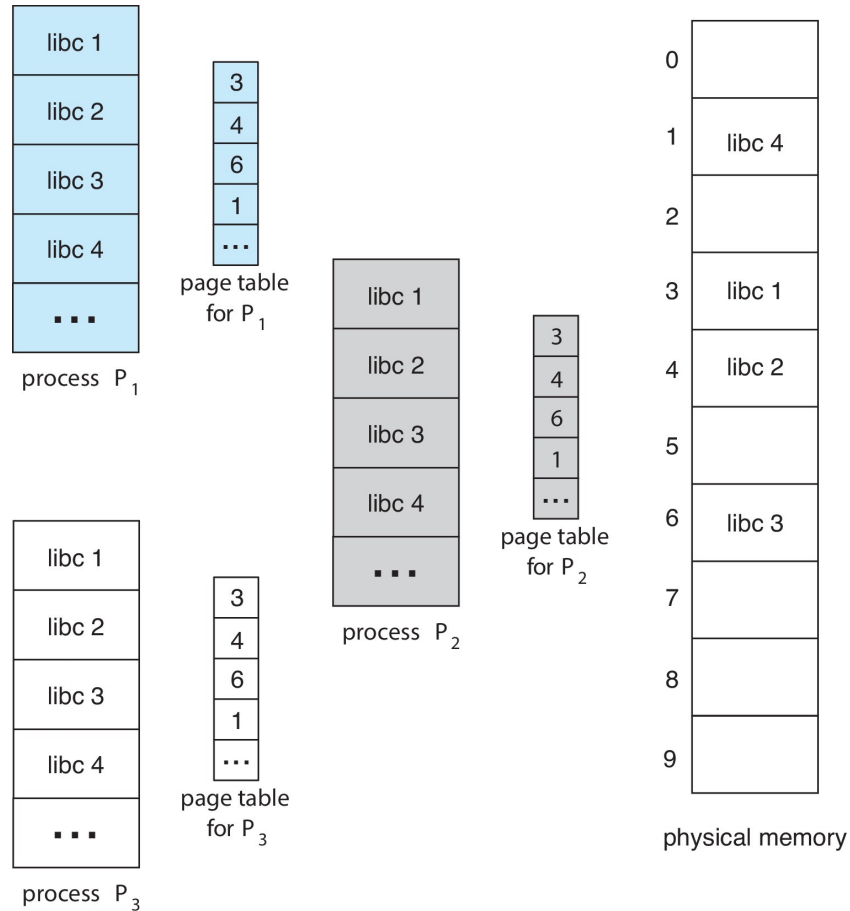
- One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
- *Reentrant code is non-self-modifying code: it never changes during execution.*
- Similar to multiple threads sharing the same process space
- Also useful for interprocess communication if sharing of read-write pages is allowed

■ Private code and data

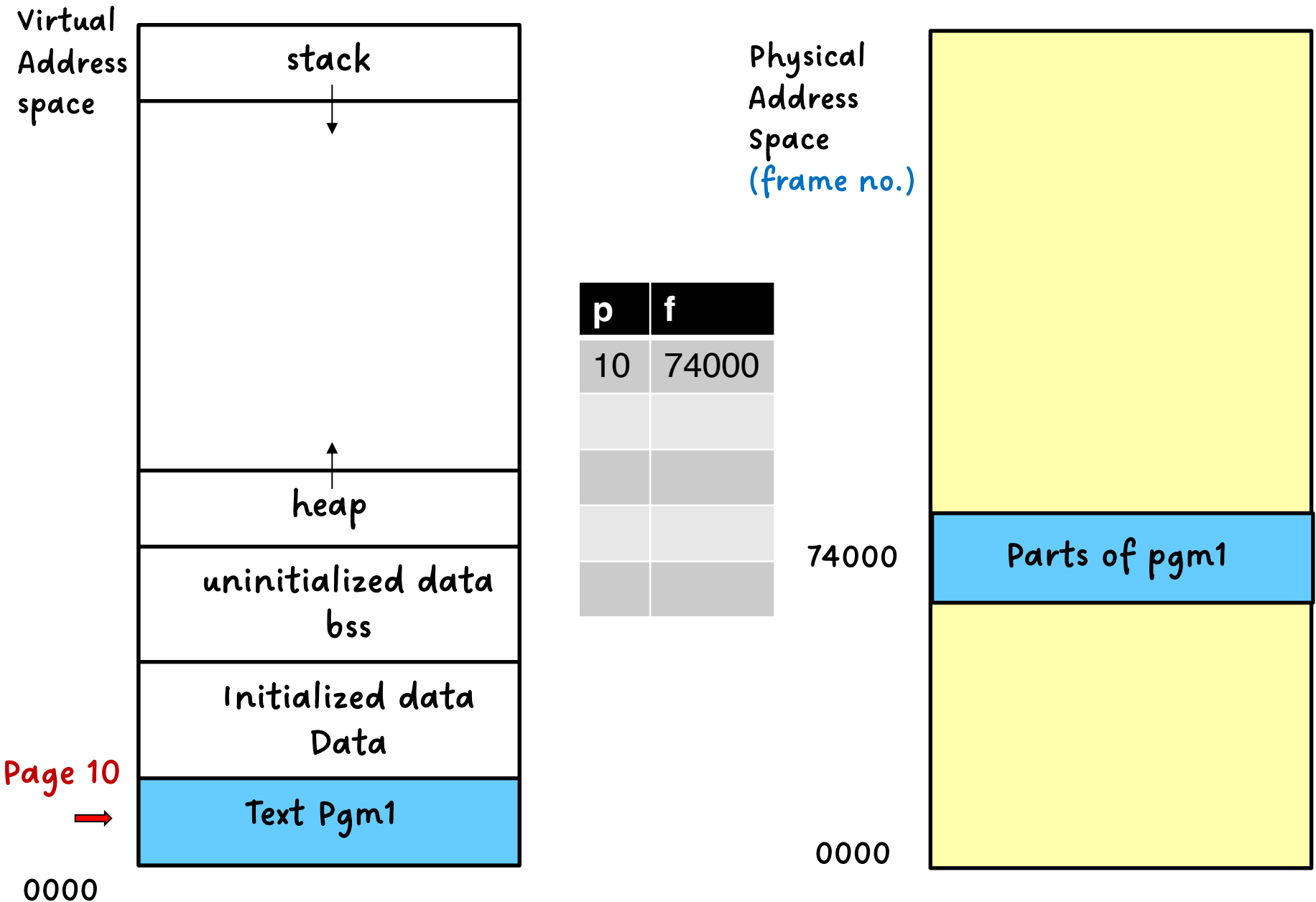
- Each process keeps a separate copy of the code and data
- The pages for the private code and data can appear anywhere in the logical address space

If a system has 40 user processes, and the libc library is 2 MB, this would require 80 MB of memory.

Shared Pages Example



1. Process Pgm1 ประมวลผล Physical memory



2. OS โหลด routineA ให้ Pgm1

Disk

libA.so

routineA

libB.so

routineB

pgm1

.text

.data

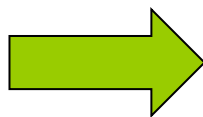
headers

Executable file

Virtual
Address
space

Page 97

OS พบว่าไม่มีใน
process และ OS เช็ค
ด้วยว่าไม่มี Process
อื่นอันไหนมี
OS จะโหลดเข้ามาให้



Call routineA

pc → Page 10

0000

stack



routineA & its
dependencies in libA



heap

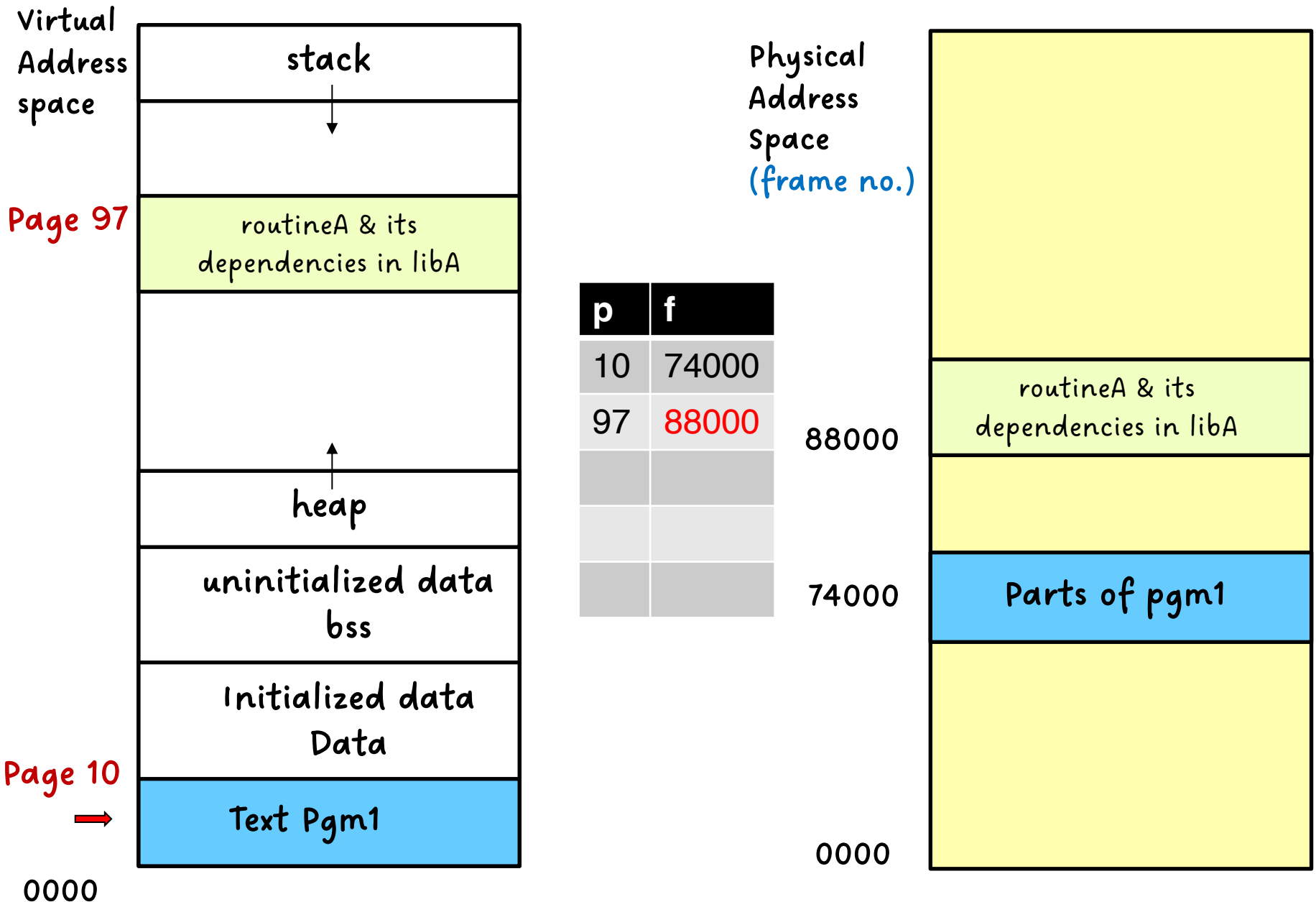
uninitialized data
bss

Initialized data
Data

Text of pgm1

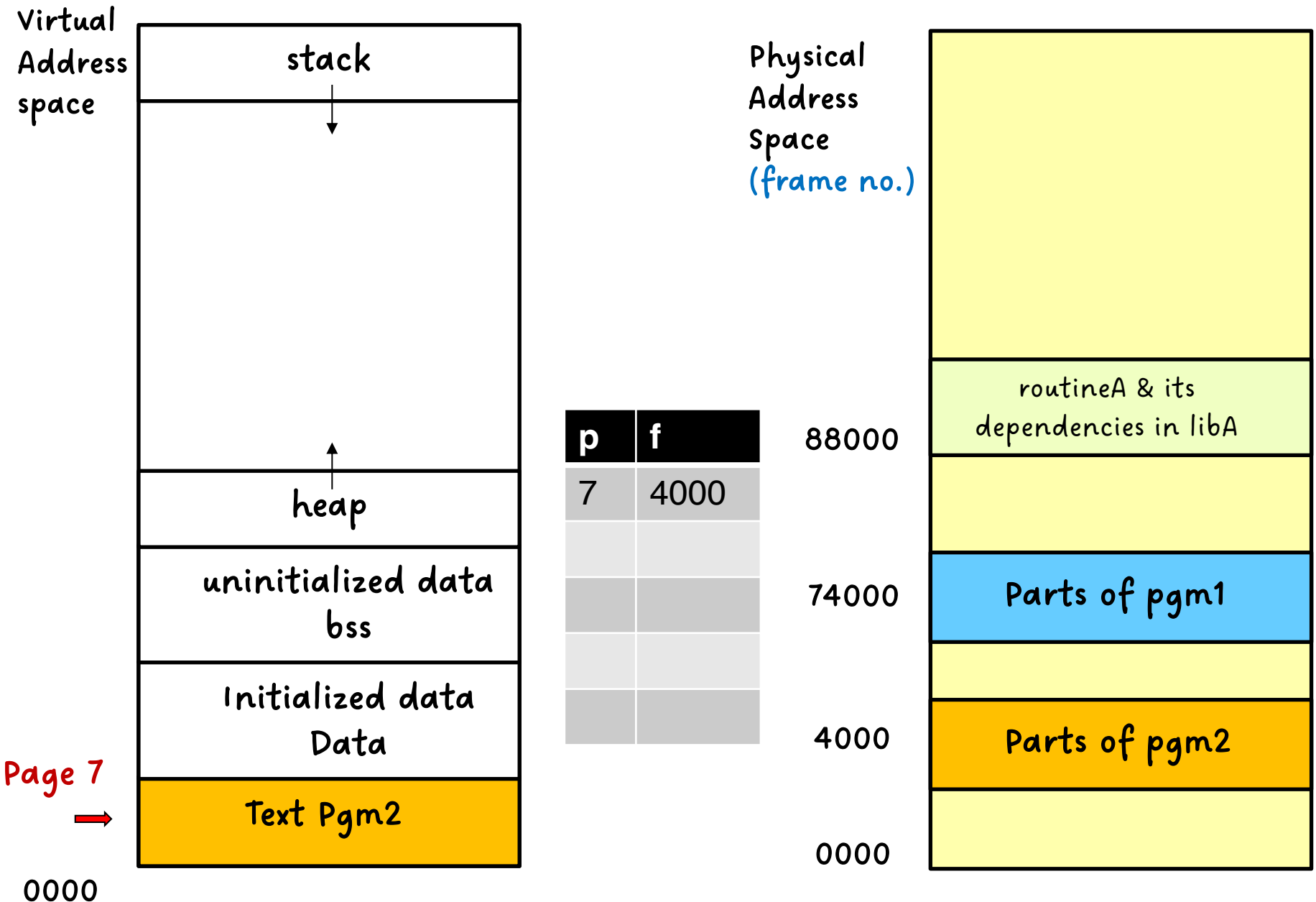
3. OS ปรับ PT ของ Pgm1

Physical memory



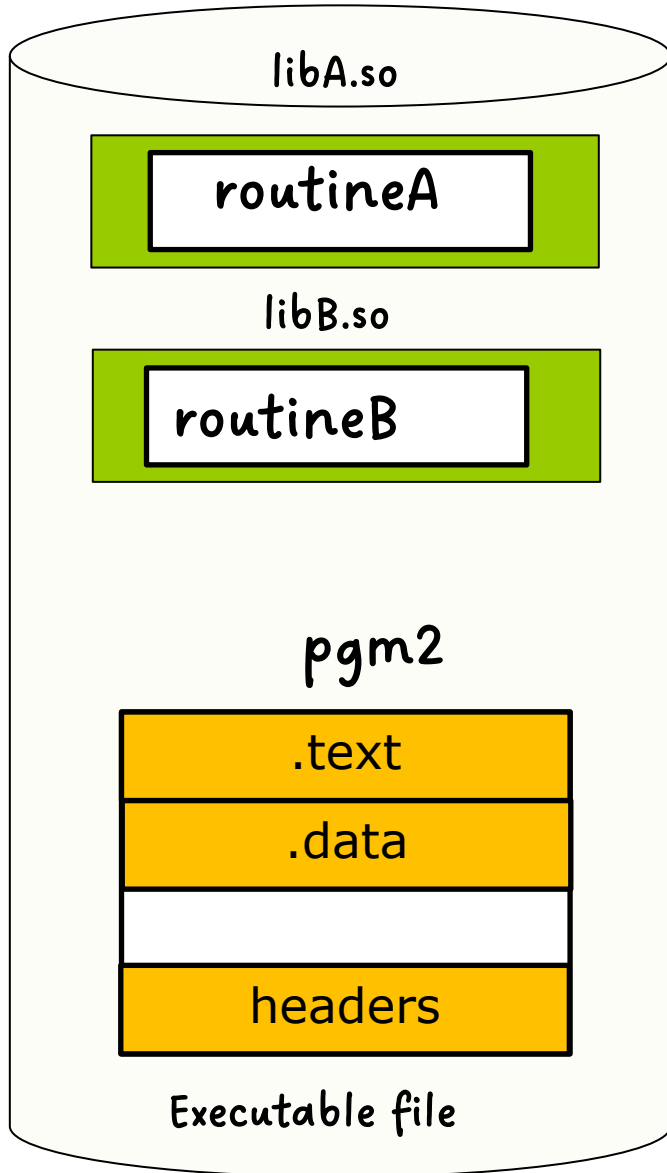
4. Pgm2 ประมวลผล

Physical memory



5. Pgm2 เรียกใช้ routineA

Disk



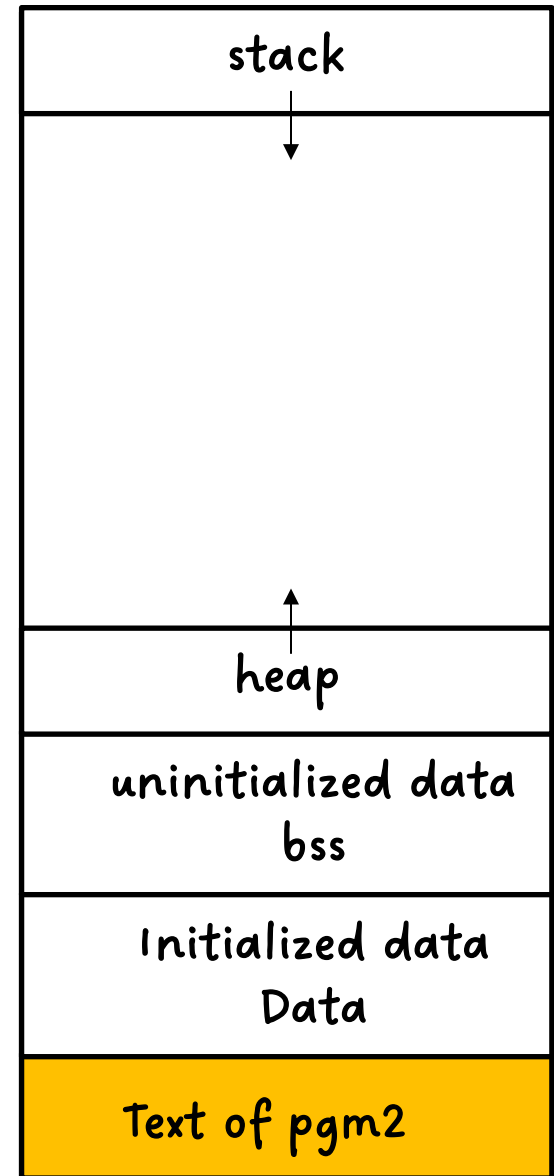
Virtual
Address
space

OS พบว่าไม่มีใน
process และ OS เช็ค
แล้วพบว่ามี **Process**
Pgm1 ใช้อยู่
OS จะแชร์ให้



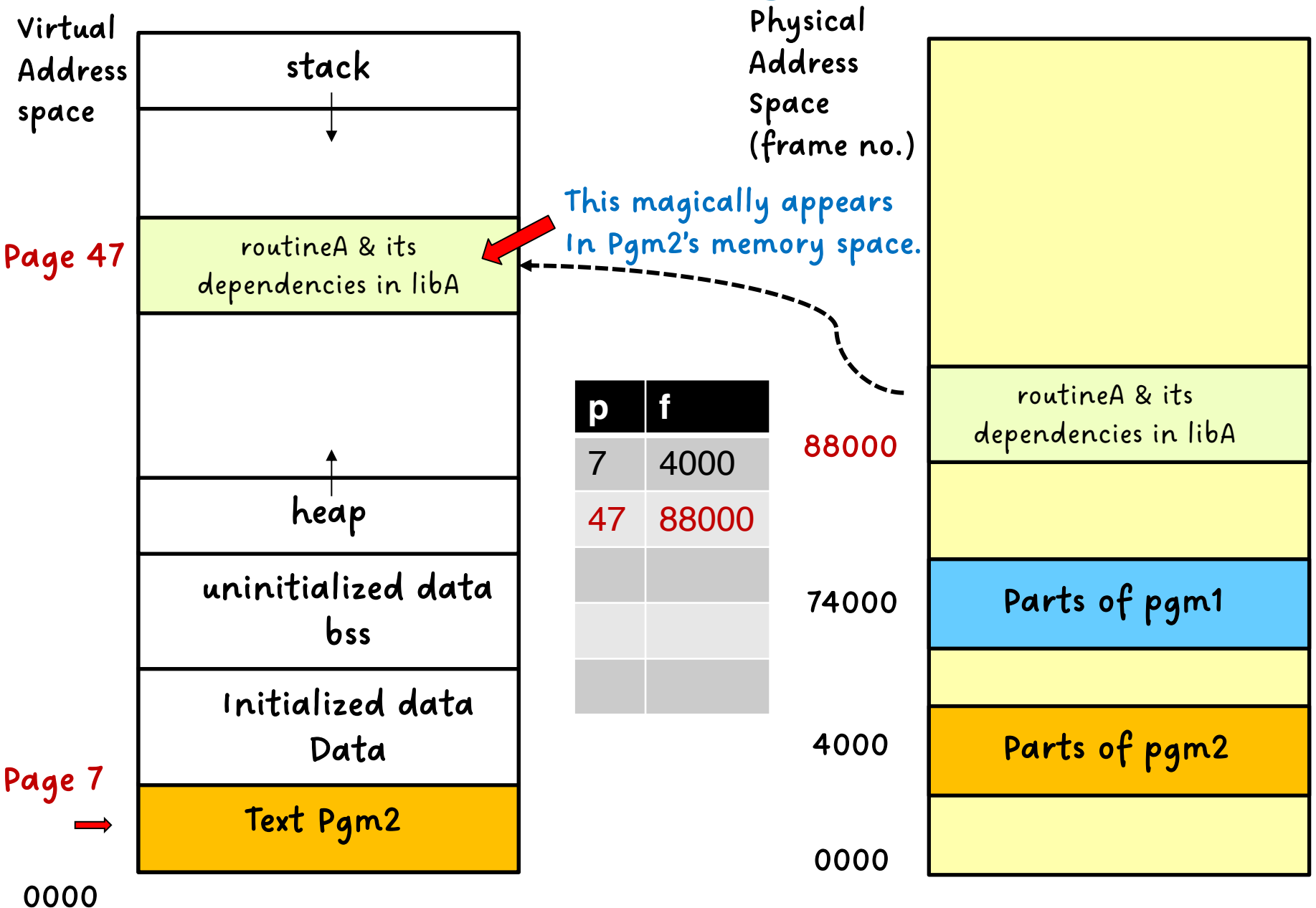
Call routineA (stub)

pc → **Page 7**
0000



6. OS เพิ่มค่าใน Pgm2's PT

Physical memory



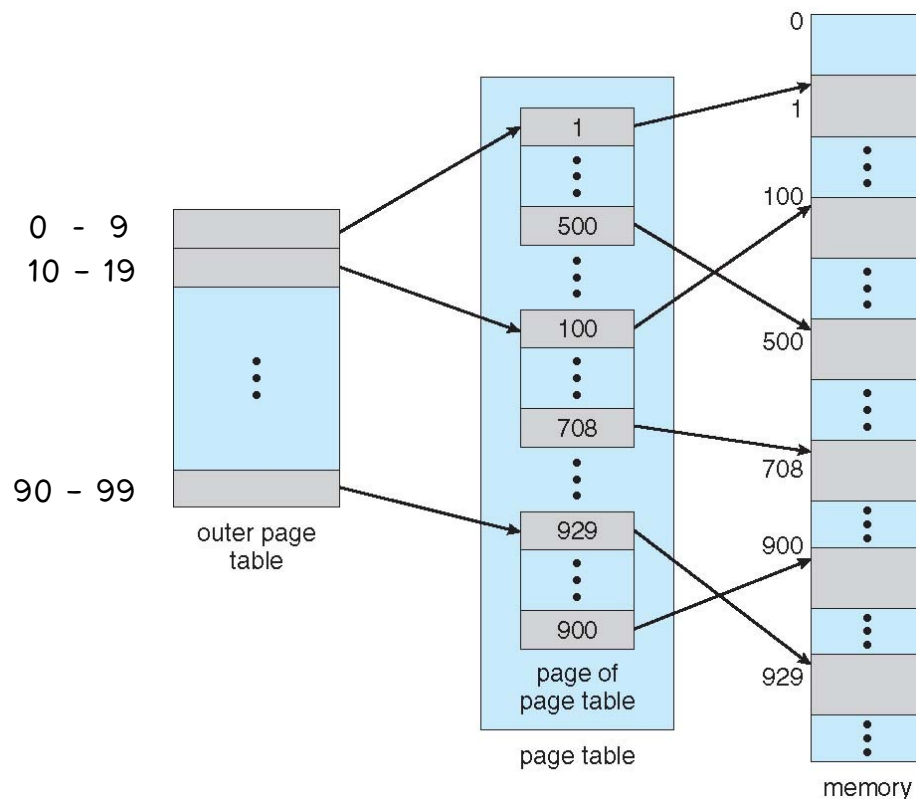
Page Table Structure & Hierarchical Page Table

Structure of the Page Table

- Memory structures for paging can get huge using straight-forward methods
 - Consider a 32-bit logical address space as on modern computers
 - Page size of 4 KB (2^{12})
 - Page table would have 1 million entries ($2^{32} / 2^{12}$)
 - If each entry is 4 bytes → each process 4 MB of physical address space for the page table alone
 - ▶ Don't want to allocate that contiguously in main memory
 - One simple solution is to divide the page table into smaller units
 - ▶ Hierarchical Paging
 - ▶ Hashed Page Tables
 - ▶ Inverted Page Tables

Hierarchical Page Tables

- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table



การใช้งาน Hierarchical Page Tables

■ แบบ Page Table (PT) ใหญ่ 1 ระดับ

- ข้อดี: MMU เข้าถึง Memory เพื่ออ่าน PT ครั้งเดียว (กรณี PT ก้อนเดียว) เพื่อแปลงค่า Process Virtual Address (PVA) ให้เป็น Host Physical memory Address (HPA)
- ข้อเสีย: ใช้พื้นที่หน่วยความจำสำหรับสร้าง PT มาก

■ แบบ Page Table เล็ก หลาย ระดับ

- ข้อดี: ใช้พื้นที่หน่วยความจำสำหรับสร้าง PT ตามปริมาณการใช้จริง (น้อยเนื่องจาก locality และไม่มีโปรแกรมไหนใหญ่เท่า logical memory space)
- ข้อเสีย: MMU ต้องเข้าถึง PT หลายครั้ง เพื่อแปลงค่า PVA ให้เป็น HPA

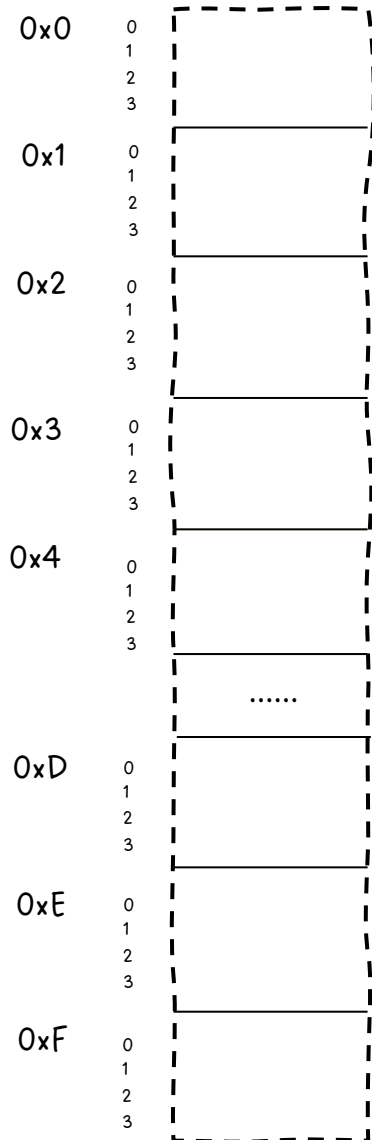
■ Page Table N ระดับ

- MMU ต้องเข้าถึง Memory N ครั้ง เพื่อแปลงค่า PVA ให้เป็น HPA
- ขึ้นอยู่กับสถาปัตยกรรม 32bits 48bits 64bits etc.

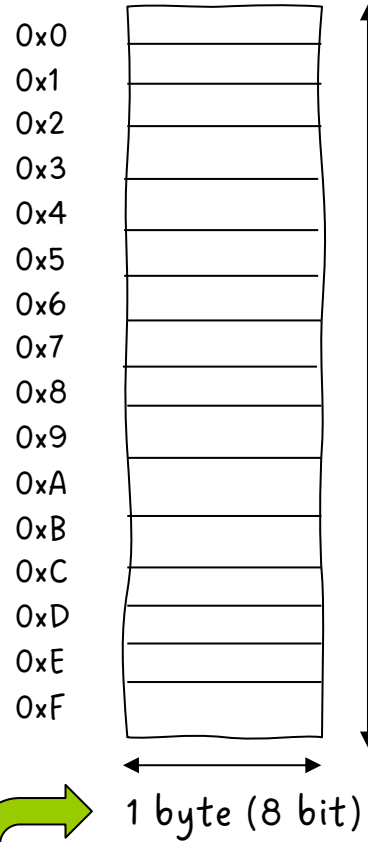
Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits

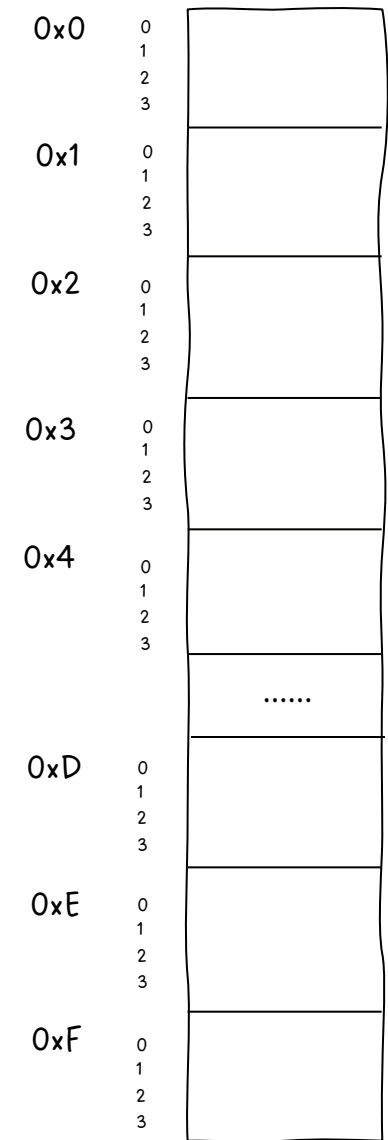


Page id (p)



Frame id (f) : offset (d)

4 bits : 2 bits

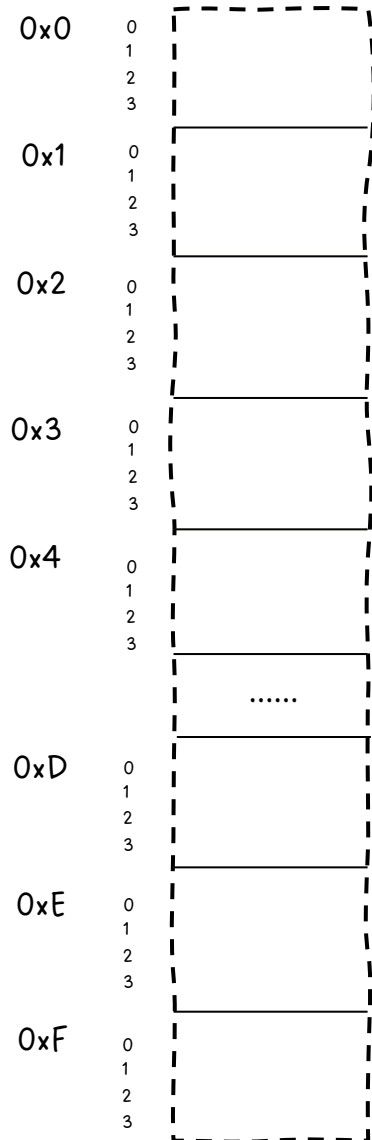


เนื่องจากขนาดของ memory address คือ $m = 6$ bits แต่ memory unit คือ 1 byte (8 bit) ดังนั้นเราจึงต้องใช้พื้นที่ 1 bytes เพื่อเก็บค่า 6 bits

Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

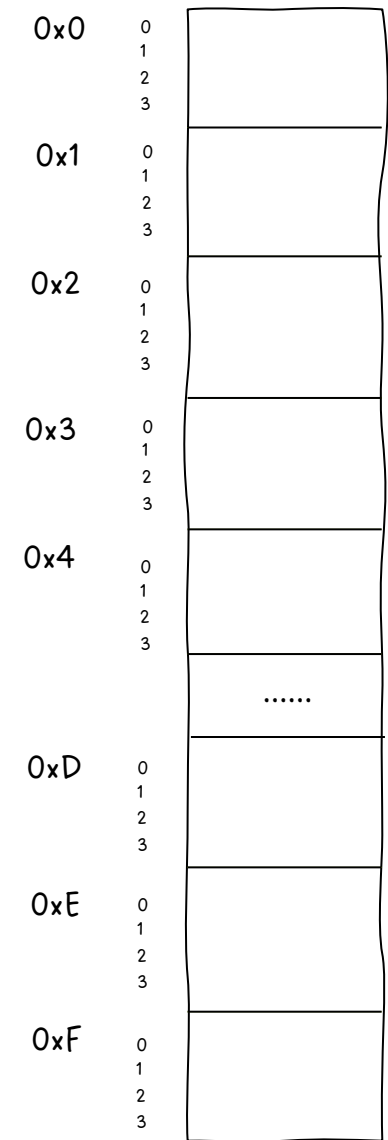
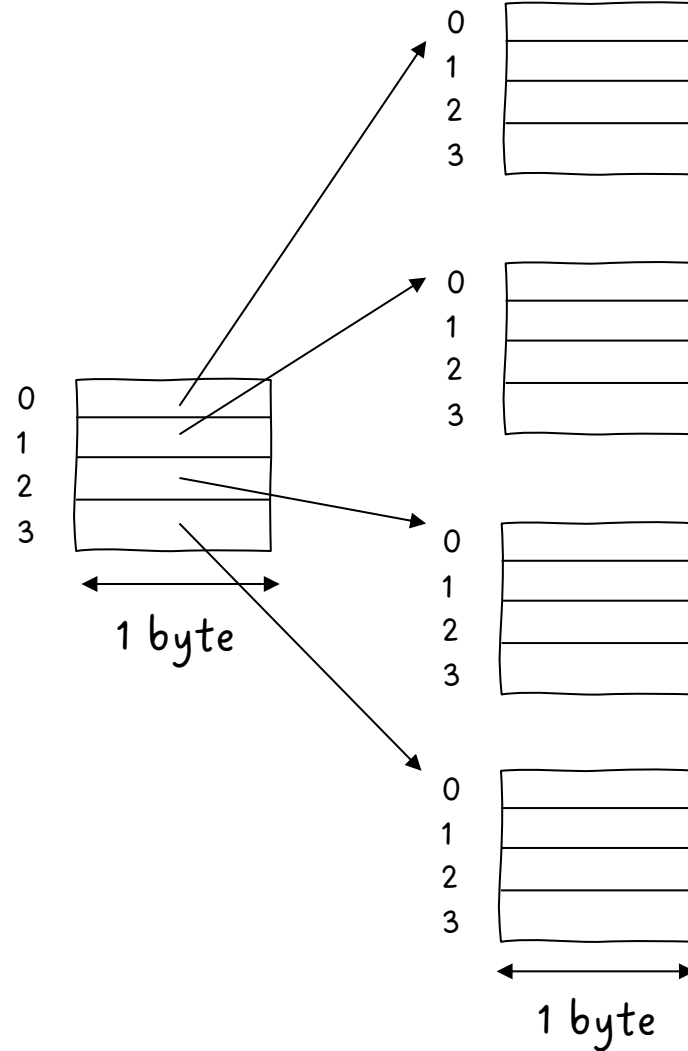
Page id (p) : offset (d)

4 bits : 2 bits



Frame id (f) : offset (d)

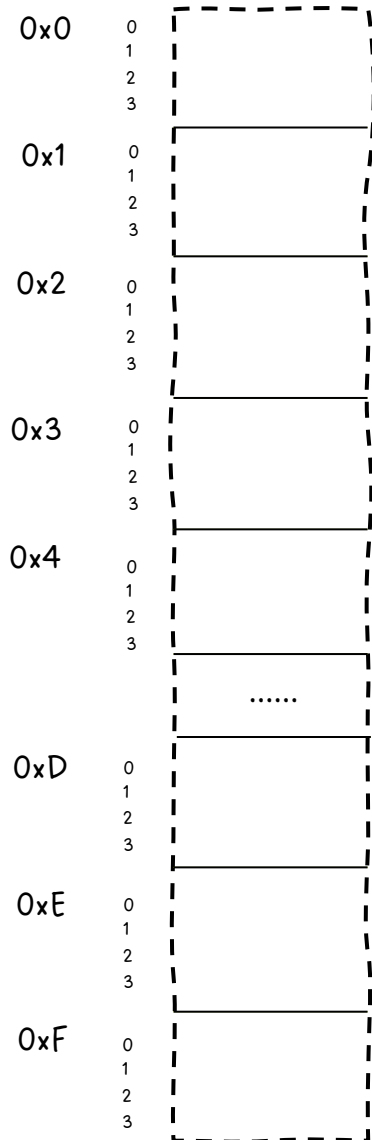
4 bits : 2 bits



Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits



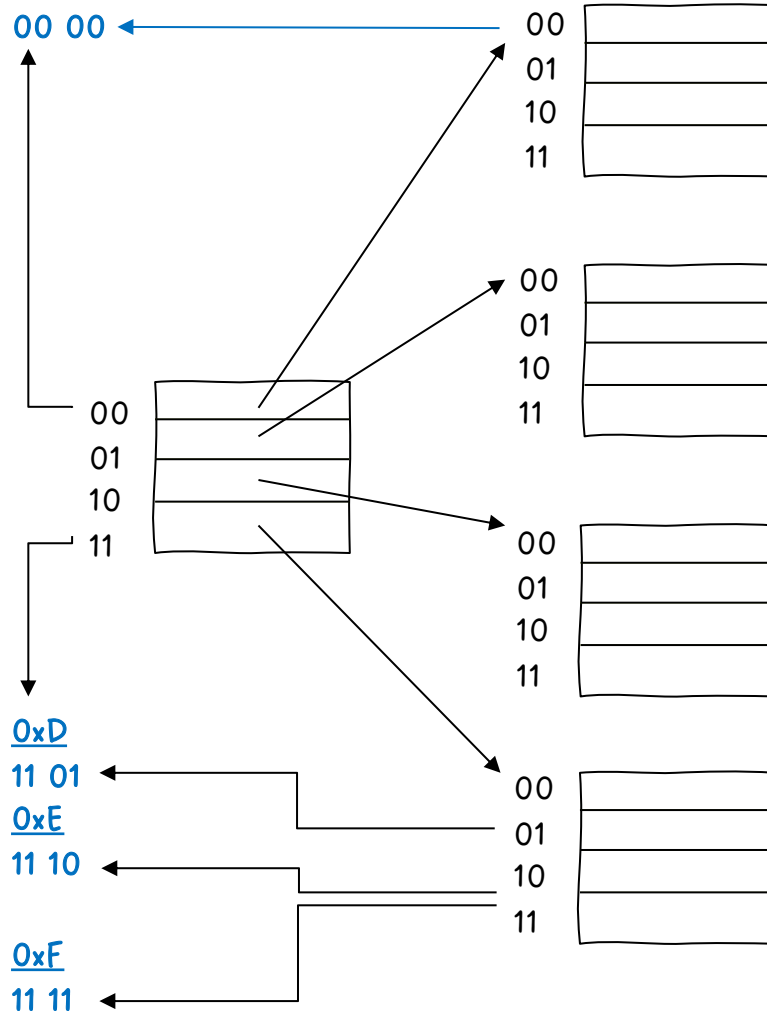
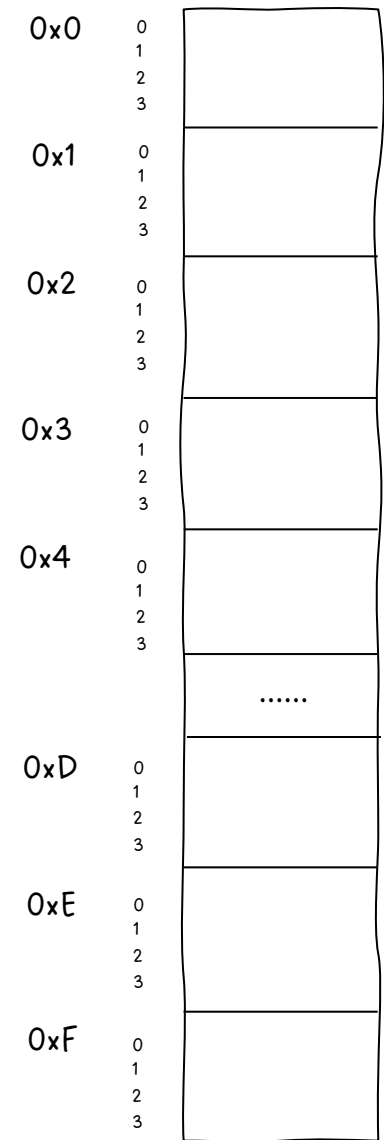
Page id (p)

0x0
00 00

0xD
11 01
0xE
11 10
0xF
11 11

Frame id (f) : offset (d)

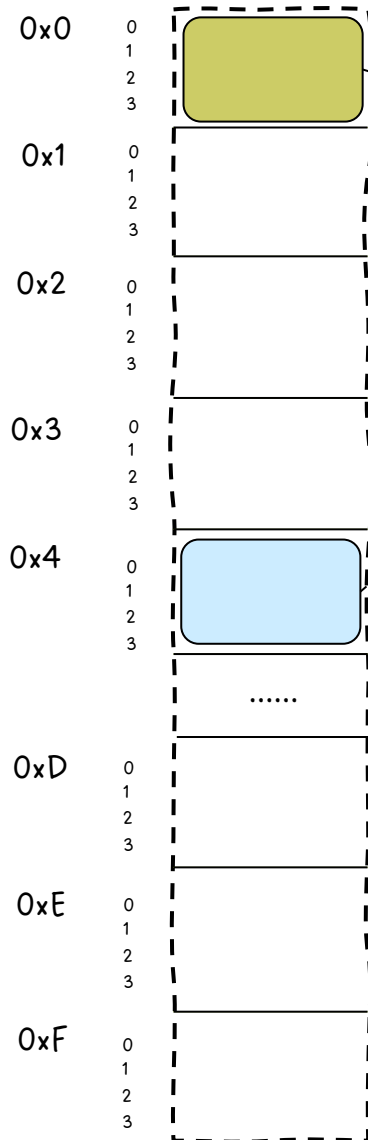
4 bits : 2 bits



Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits



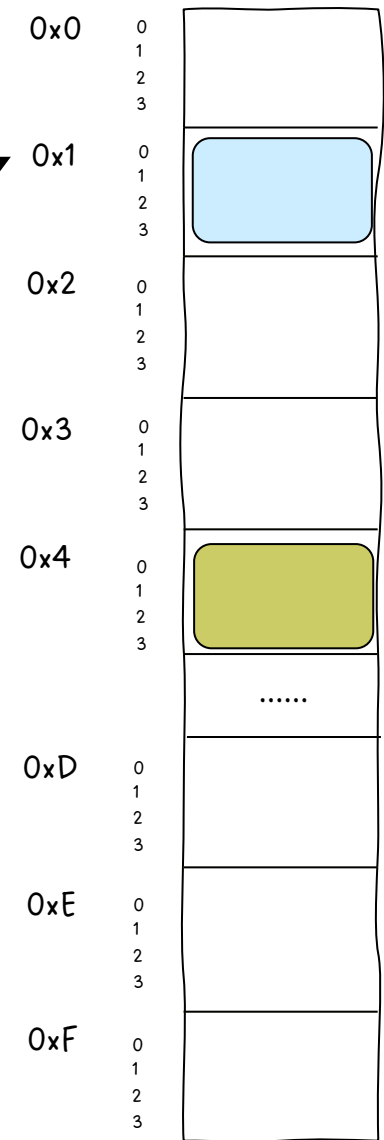
Page id (p)

0x0
0x1
0x2
0x3
0x4
0x5
0x6
0x7
0x8
0x9
0xA
0xB
0xC
0xD
0xE
0xF



Frame id (f) : offset (d)

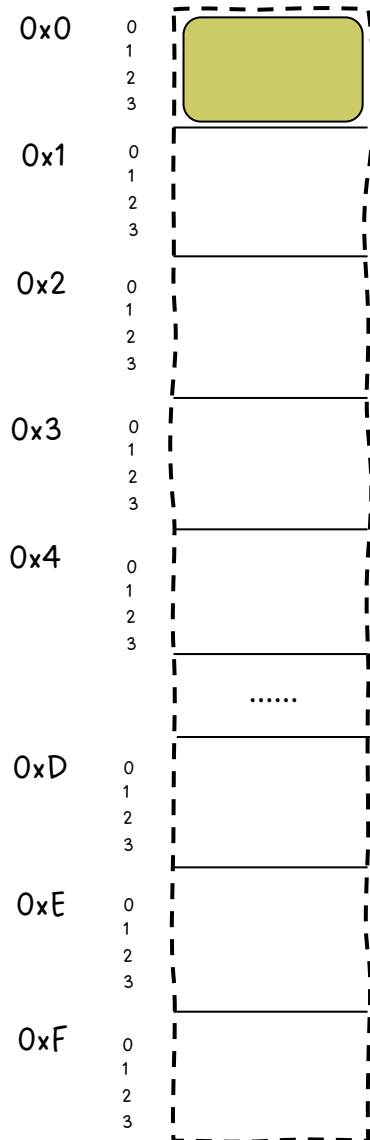
4 bits : 2 bits



Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

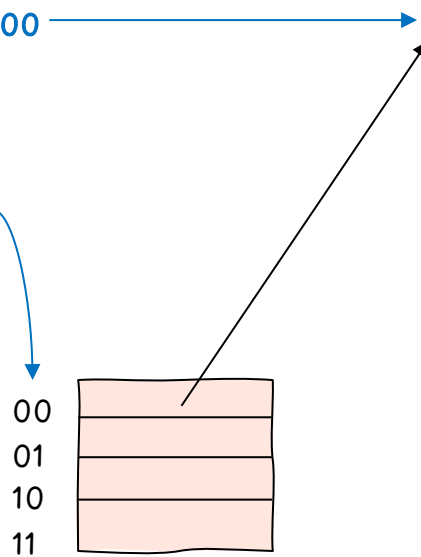
Page id (p) : offset (d)

4 bits : 2 bits



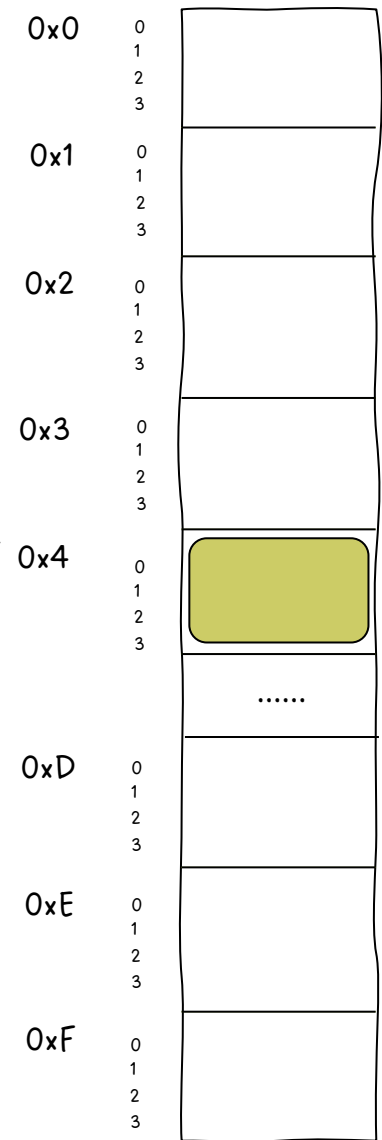
Page id (p)

0x0
00 00



Frame id (f) : offset (d)

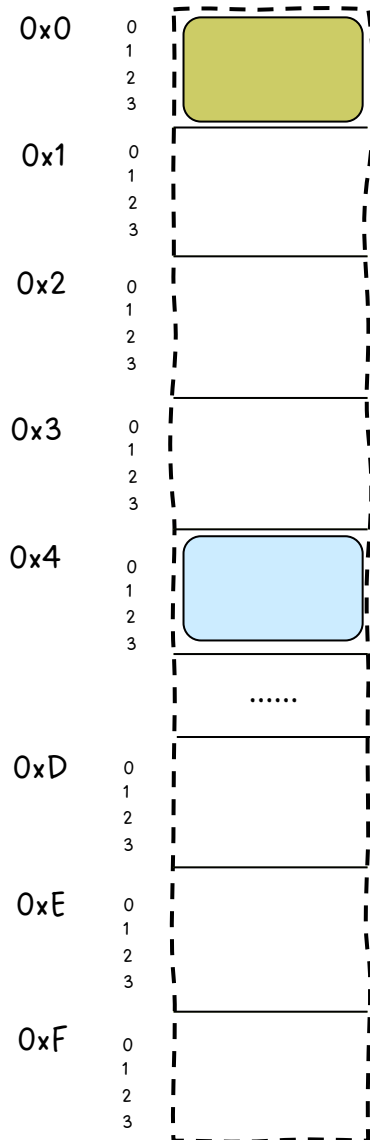
4 bits : 2 bits



Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

Page id (p) : offset (d)

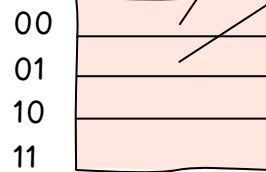
4 bits : 2 bits



Page id (p)

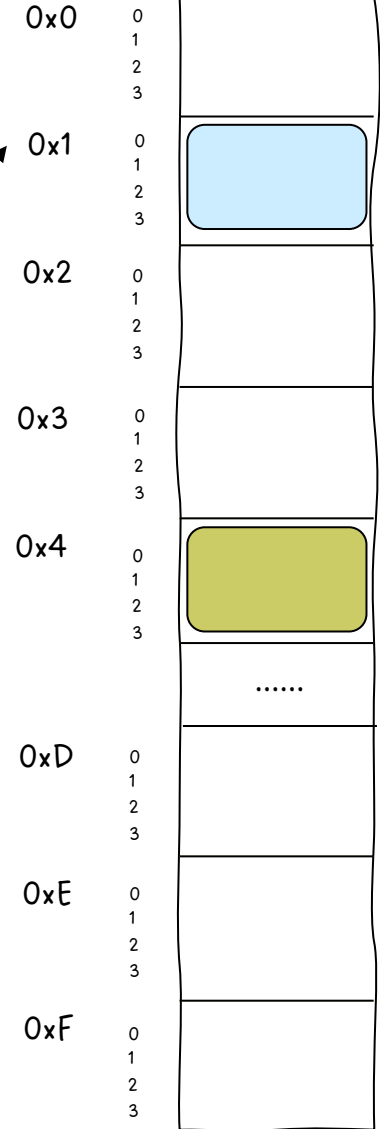
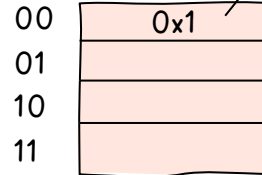
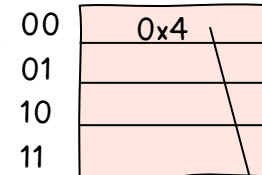
0x0
01 00

01 00



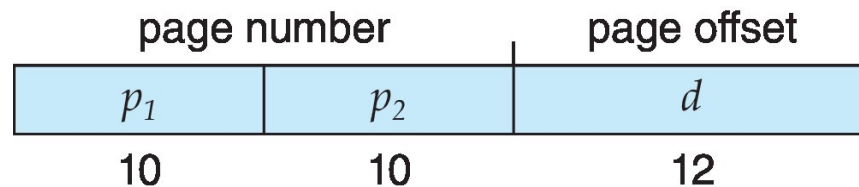
Frame id (f) : offset (d)

4 bits : 2 bits



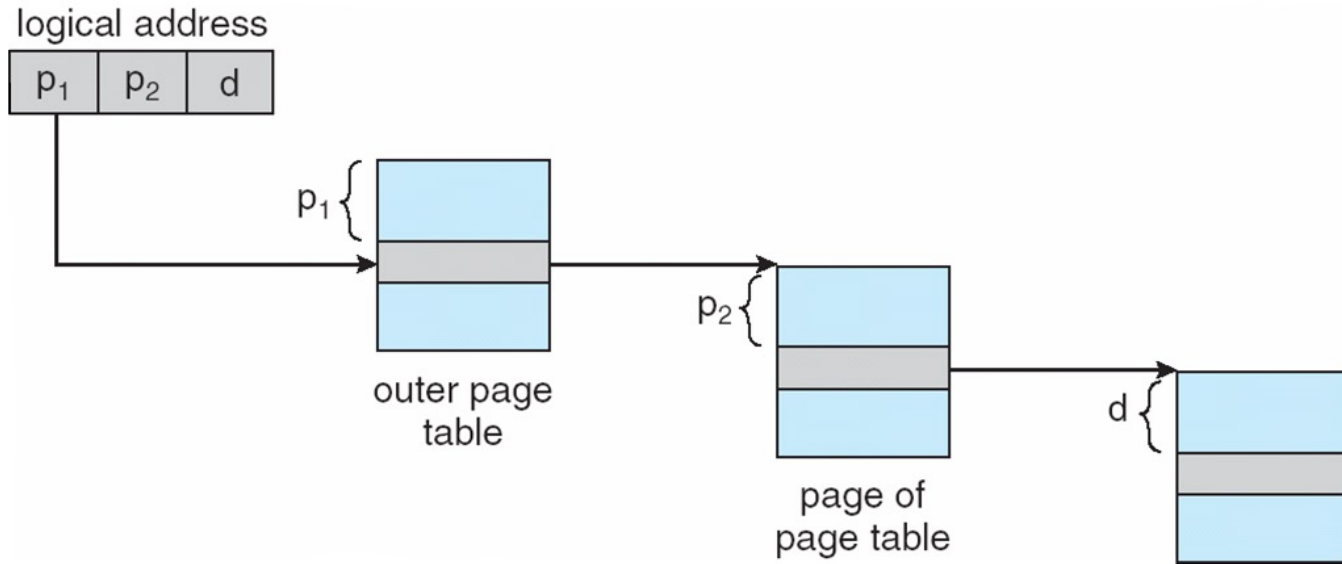
Two-Level Paging Example

- A logical address (on 32-bit machine with 4K page size) is divided into:
 - a page number consisting of 20 bits
 - a page offset consisting of 12 bits
- Since the page table is paged, the page number is further divided into:
 - a 10-bit page number
 - a 10-bit page offset
- Thus, a logical address is as follows:



- where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table
- Known as **forward-mapped page table**

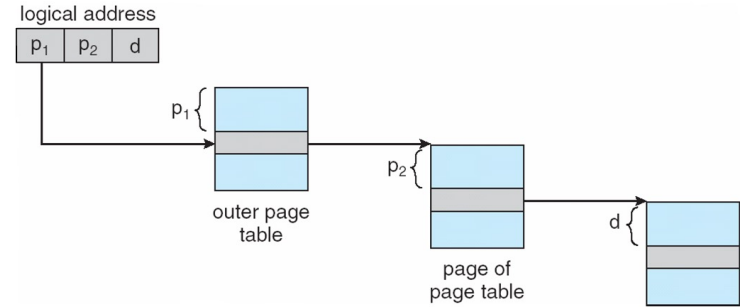
Address-Translation Scheme



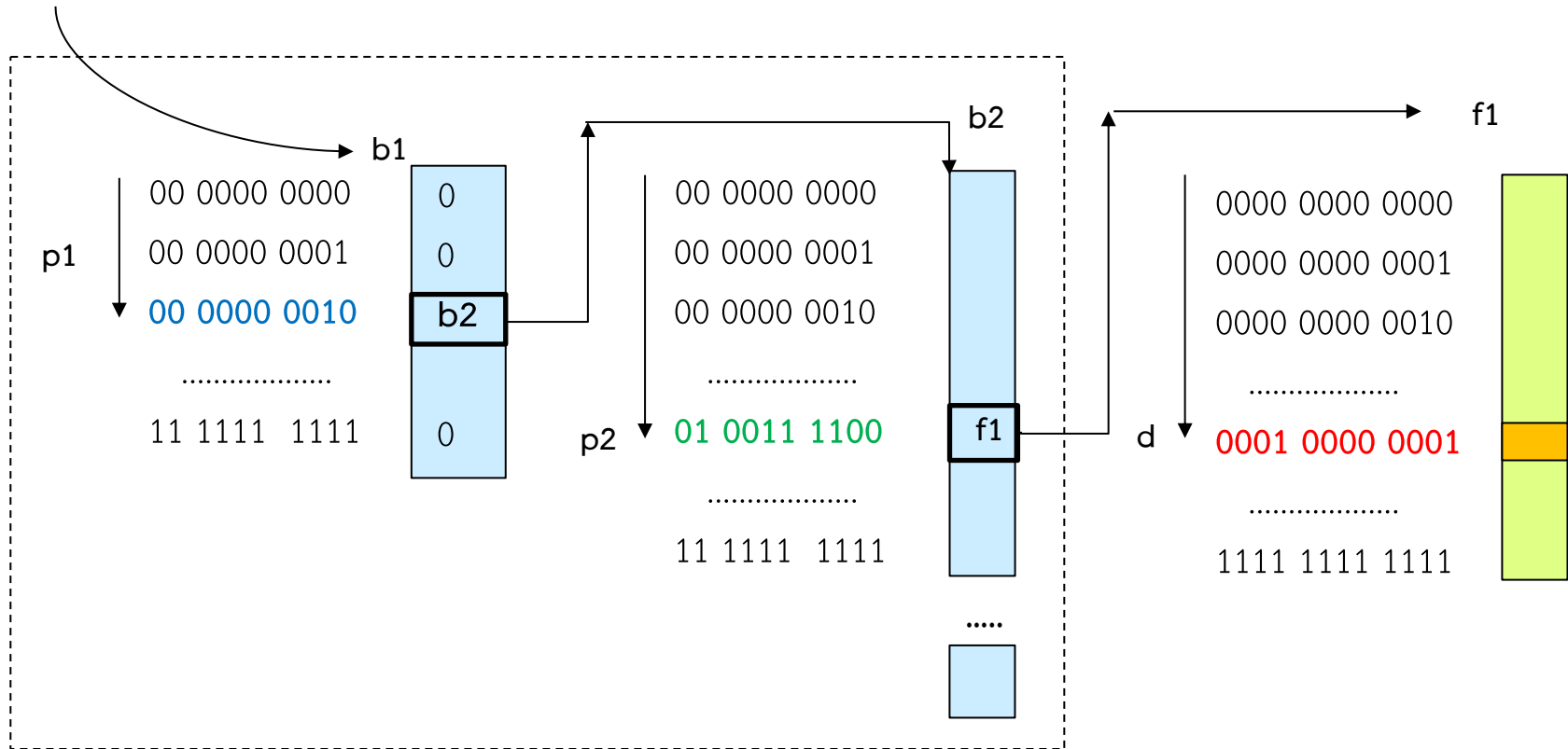
Address-Translation Scheme 32 bits

A logical address (32)

p1 (10)	p2 (10)	d (12)
00 0000 0010	01 0011 1100	0001 0000 0001



PTBR



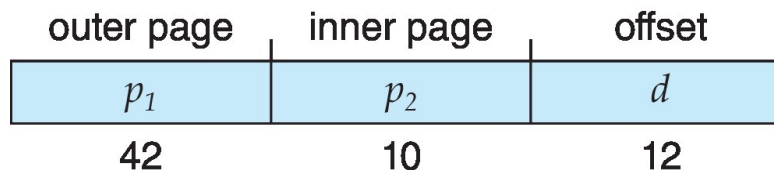
64-bit Logical Address Space

UNIT	ABBREVIATION	STORAGE
Bit	B	Binary Digit, Single 1 or 0
Nibble	-	4 bits
Byte/Octet	B	8 bits
Kilobyte	KB	1024 bytes
Megabyte	MB	1024 KB
Gigabyte	GB	1024 MB
Terabyte	TB	1024 GB
Petabyte	PB	1024 TB
Exabyte	EB	1024 PB
Zettabyte	ZB	1024 EB
Yottabyte	YB	1024 ZB

Storage units (www.byte-notes.com)

https://lecture481.rssing.com/chan-14834568/all_p1.html

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
 - Then page table has 2^{52} entries.
 - ▶ $(4,503,599,627,370,496) = 4K\ T$ หรือ $4P$ entries
 - ใช้พื้นที่ **Memory Space** = $2^{52} \times 2^3 = 2^{55} = \underline{32\ Peta\ Bytes}$
 - If two level scheme, inner page tables could be 2^{10} 4-byte entries
 - Address would look like



- Outer page table has 2^{42} entries or 2^{44} bytes = 17,592,186,044,416 bytes
- One solution is to add a 2^{nd} outer page table
- But in the following example the 2^{nd} outer page table is still 2^{34} bytes in size
 - ▶ And possibly 4 memory access to get to one physical memory location

Three-level Paging Scheme

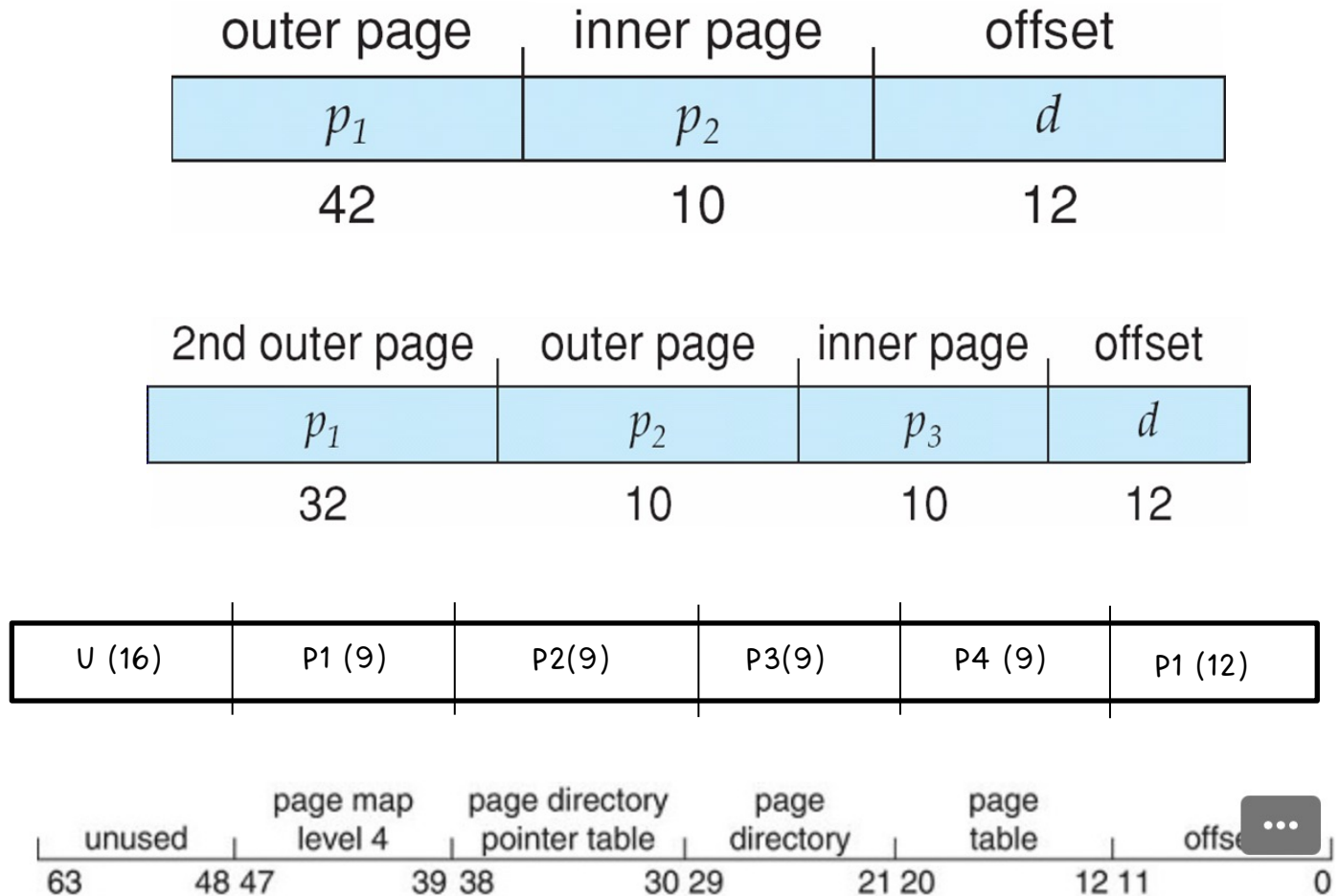


Figure 9.25 x86-64 linear address.

Page Global Directory

Root ของทุก **Page Tables**

Page Table ถูกเก็บไว้ที่ไหน

- OS kernel สร้างพื้นที่ Page Table Directory เป็น Data Structure หนึ่ง ใน Virtual Memory Space ของมัน
 - CPU เข้าถึงหน่วยความจำด้วย Virtual Memory Address เสมอ ถึงแม้ว่าจะรัน OS Kernel
- OS kernel จะเก็บค่า Physical Memory Address ของ Page Table ของ โพรเซสที่มันสร้างขึ้นใน Page Global Directory
 - OS kernel มีวิธีแปลงค่า Virtual memory address ของมันเองเป็น Physical memory address (ใช้ macro ย้อน direct mapping)
- OS kernel จะแปลงค่า Page Table ที่มันเก็บ ใน Page Global Directory ให้เป็น raw physical address ก่อน assign ให้ CR3 register ตอนมันจะรัน Process นั้น

OS

Page Global Directory

Process	Virtual Address of PT
P0	pa0
P1	pa1
P2	pa2
P3	pa3

Virtual
Memory
of Kernel

Page Global Dir

Root Page (PML4)

Page Table P0

Page Table P1

Page Table P2

Page Table P3

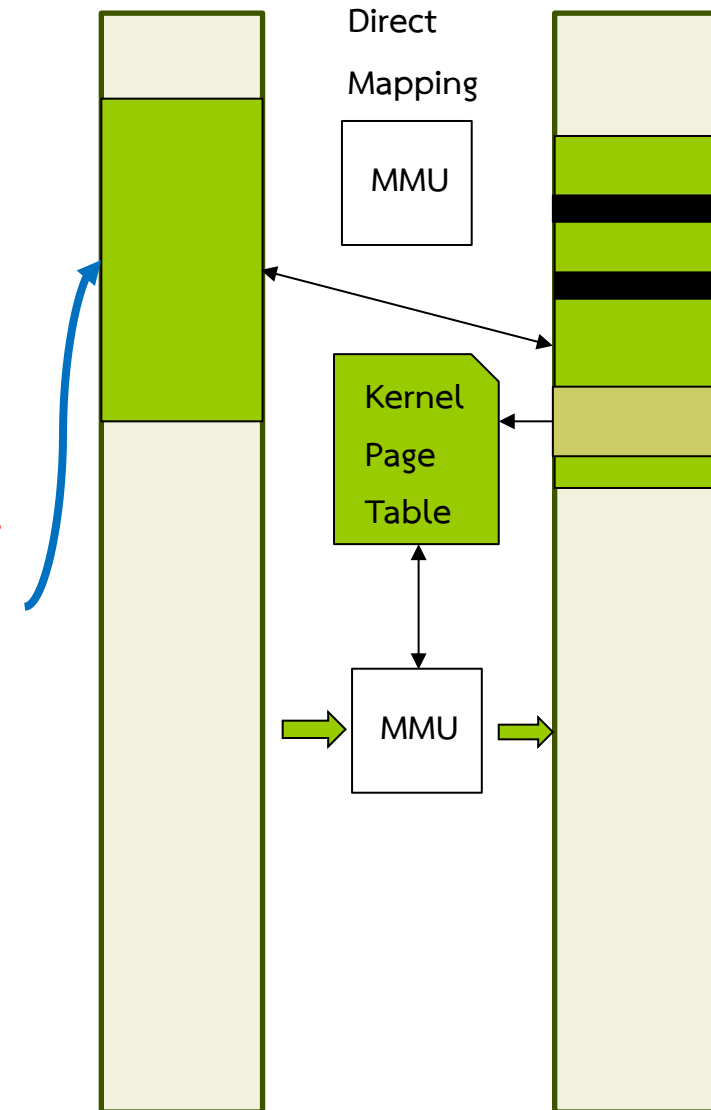
1. เมื่อ OS รัน Process P0 มันจะโหลด P0 เข้า Memory และสร้าง page table ใน kernel memory และเก็บค่า Virtual Memory addr ของ P0 ใน Page Global Directory
2. OS kernel มีวิธีแปลงค่าของมันเอง ที่ทำให้คำนวณหาค่า physical addr ได้อย่างรวดเร็ว
3. เมื่อ OS จะให้ CPU รัน P0 มันจะ assign ค่า physical addr นี้ใน CR3 register

(รายละเอียด) Kernel Virtual Memory Space

- OS kernel เข้าถึง memory ด้วย virtual memory เหมือน Process อื่น
- แต่มันมีวิธีการ แปลงค่า virtual address เป็น physical address 2 แบบ
- แบบ1: Direct Mapping เป็นการแปลงโดยใช้สูตร

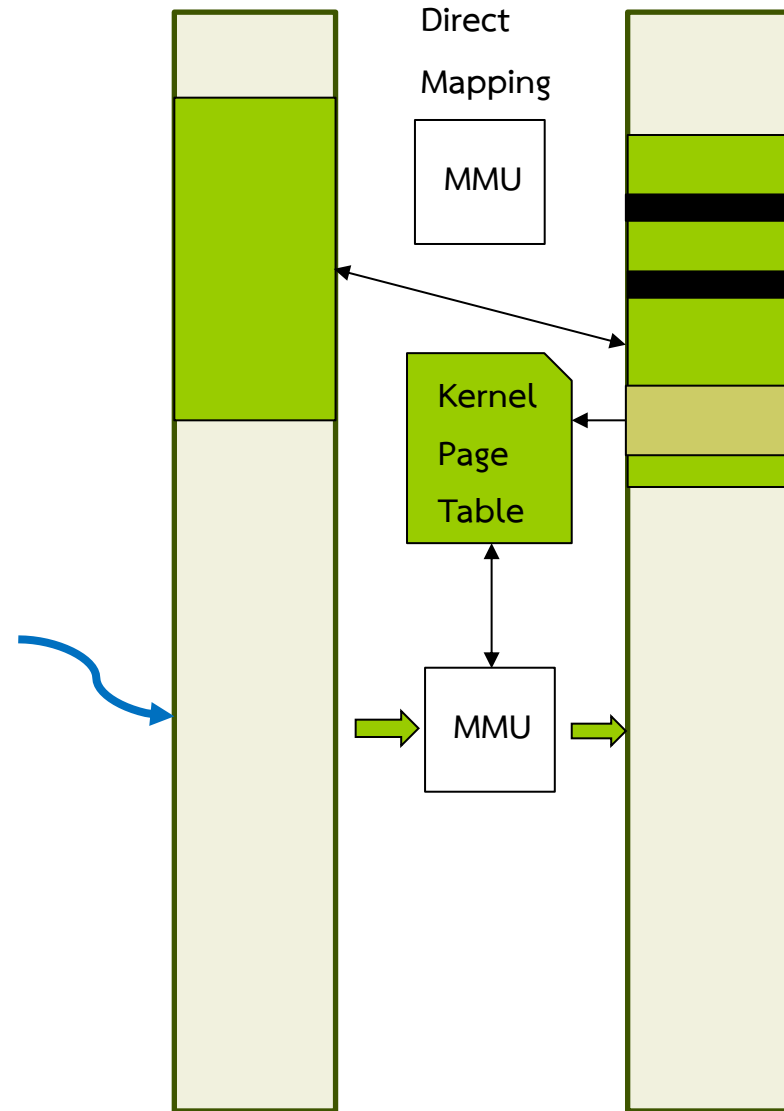
$\text{kernel virtual addr} = \text{physical addr} + \text{PAGE_OFFSET}$

- ใช้สำหรับ data structures ที่ต้องการให้ CPU เข้าถึง mem อย่างรวดเร็ว เช่น
 - Page Tables
 - Slab Allocator
 - kmalloc()



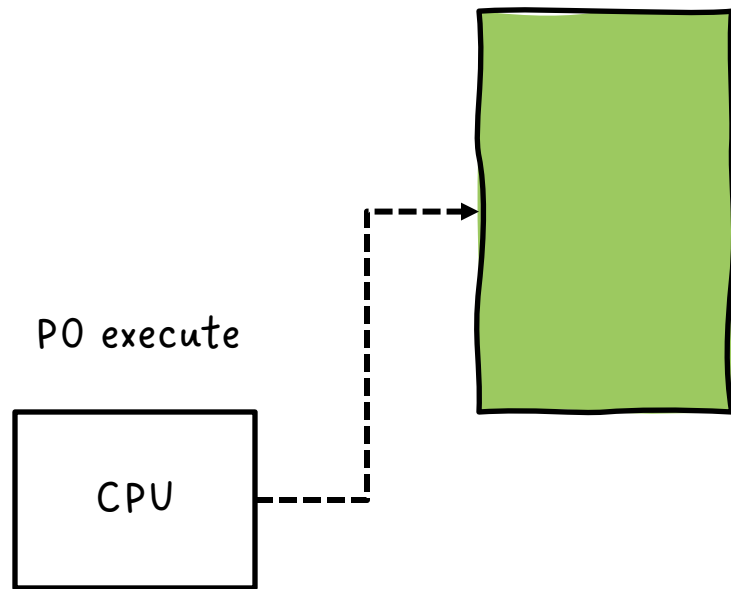
(รายละเอียด) Kernel Virtual Memory Space

- แบบ2: ใช้ Kernel Page Table เหมือน process อื่นๆ
- ใช้สำหรับเข้าถึง system call และ interrupt handler codes และ data structure ทั่วไป



พิจารณา **64bits Hierarchical Page Table**

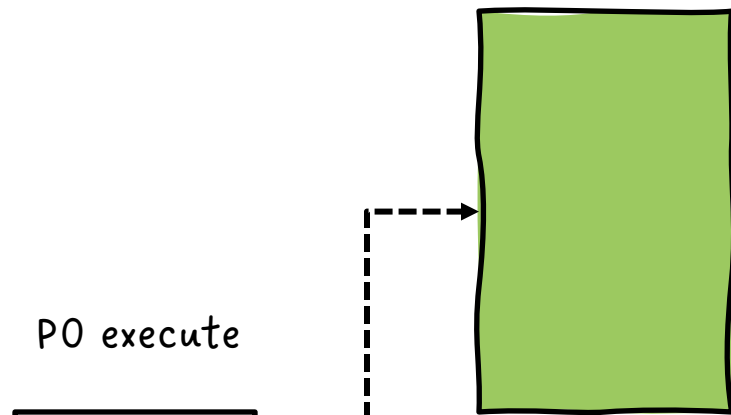
Virtual memory
space of P0



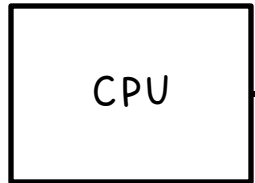
- CPU เข้าถึง(อ่าน/เขียน)
Process Virtual Memory
(PVA) เท่านั้น

Process

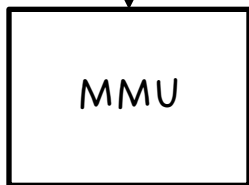
Virtual memory
space of P0



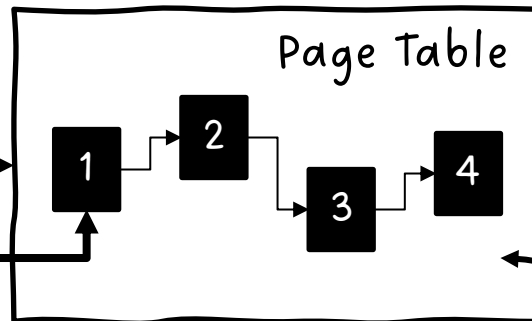
P0 execute



CPU



MMU



Page Table

1

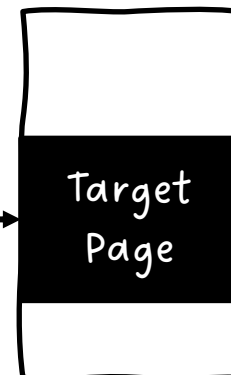
2

3

4

CR3

Physical Memory



Target
Page

Process

Physical

- MMU translate PVA เป็น Host Physical Address (HPA)
- Access Page Table 4 ครั้ง
- Access ข้อมูล 1 ครั้ง

OS กำหนดให้ CR3 เก็บค่า Physical Address ของ Page Table ของ P0

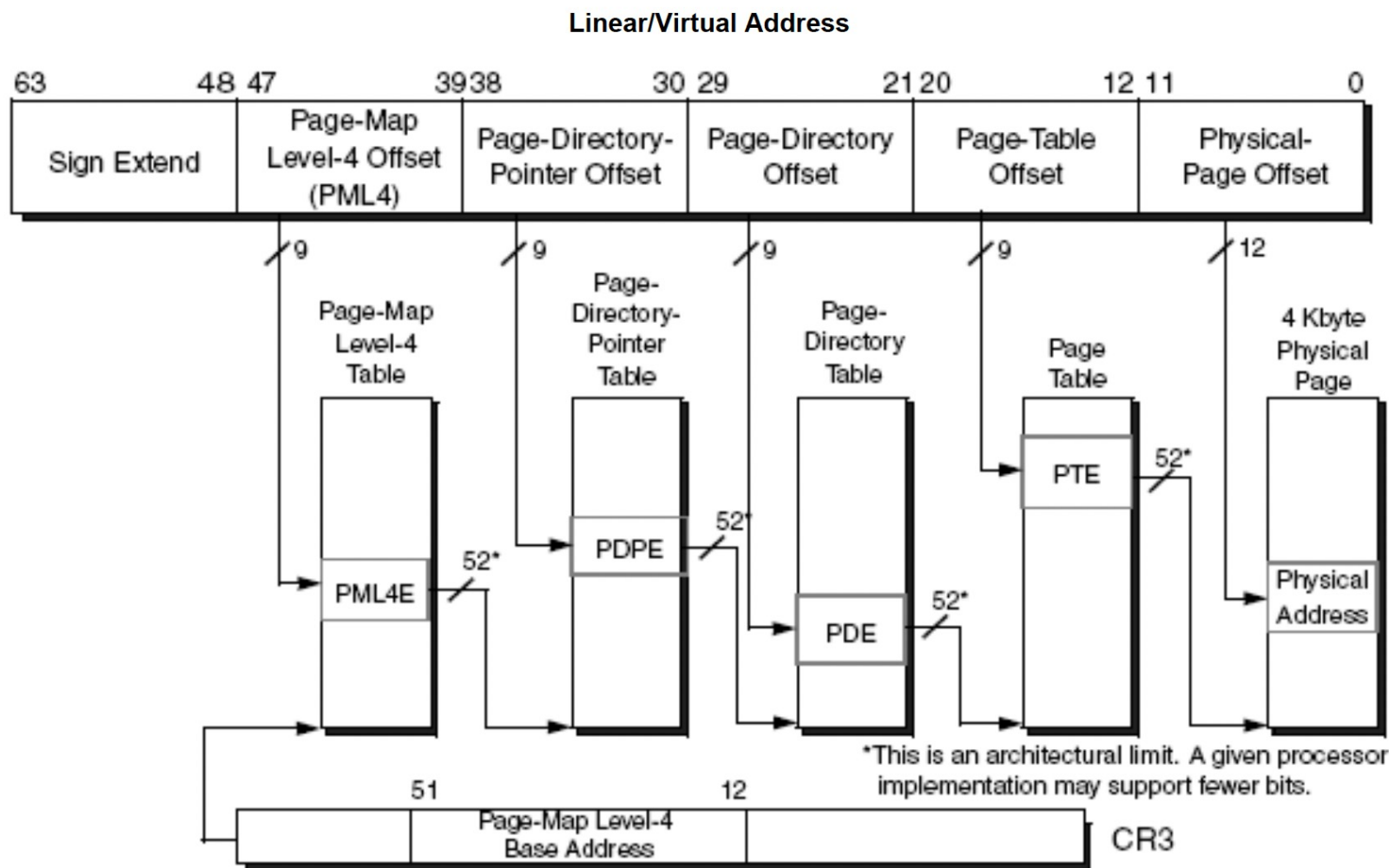
Physical Memory Pages

Hierarchical PT สำหรับ 64 bit Arch

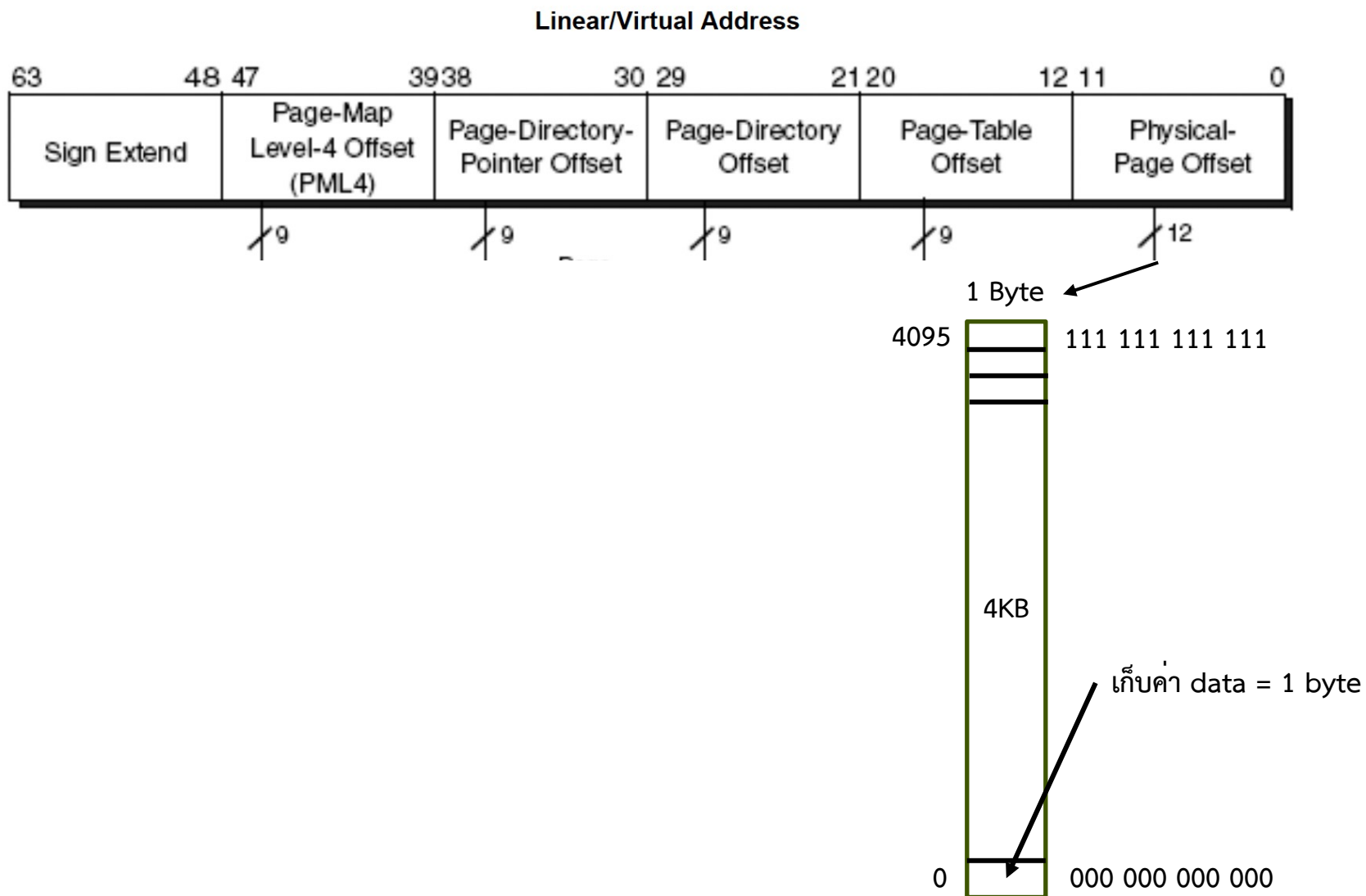
- MMU ต้องเข้าถึง Memory 4 ครั้งเพื่อแปลง Process Virtual Address (PVA) เป็น Host Physical Address (HPA)
- ส่วนประกอบของ Intel 64bits Architecture's Page Table ได้แก่

Layer	Page Table Structure	index bits
L4	Page Map Level 4 (PML4) Table	47:39
L3	Page Directory Pointer (PDP) Table	38:30
L2	Page Directory (PD) Table	29:21
L1	Page Table (PT)	20:12

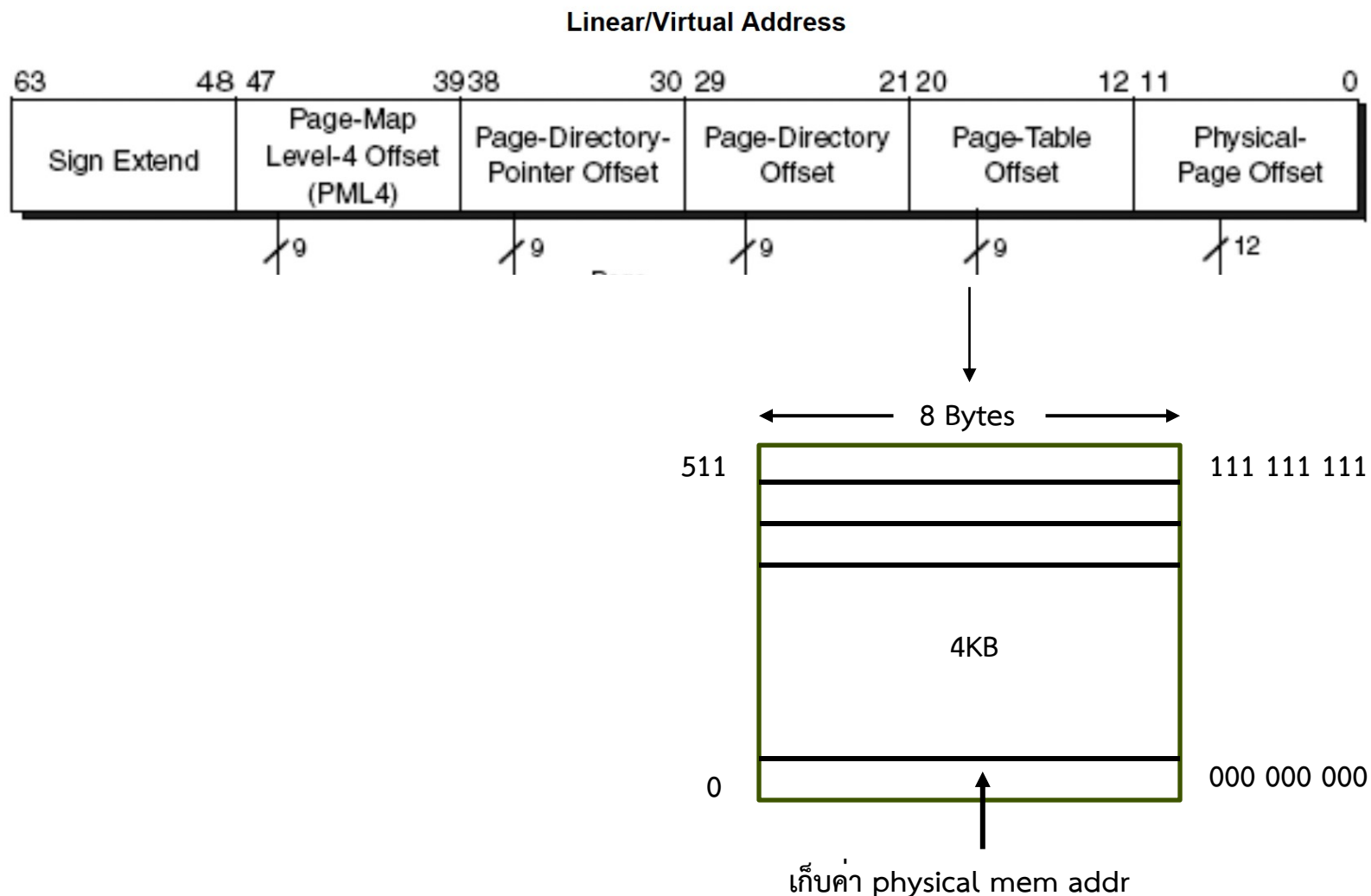
Hierarchical Page table



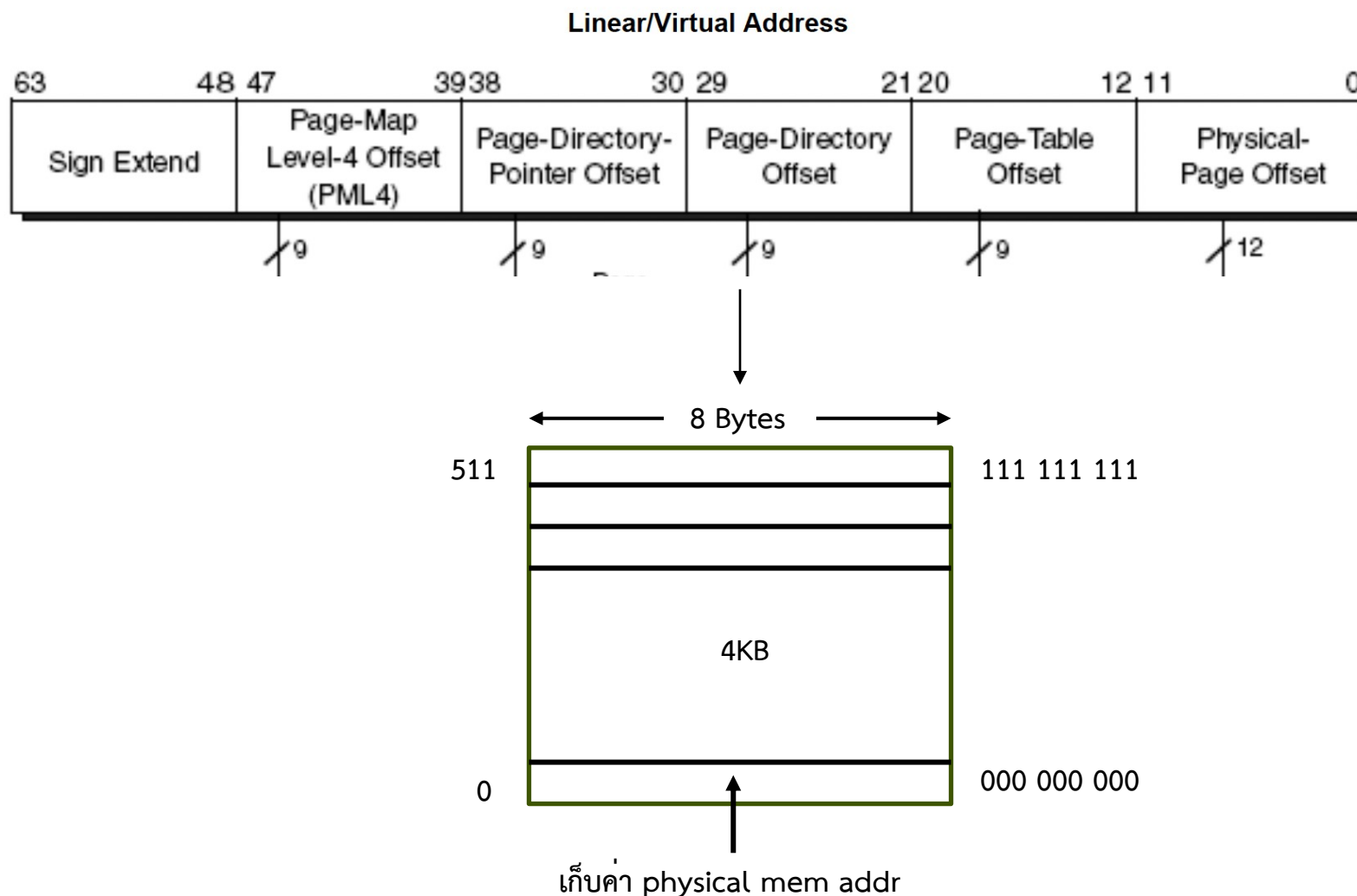
Hierarchical Page table



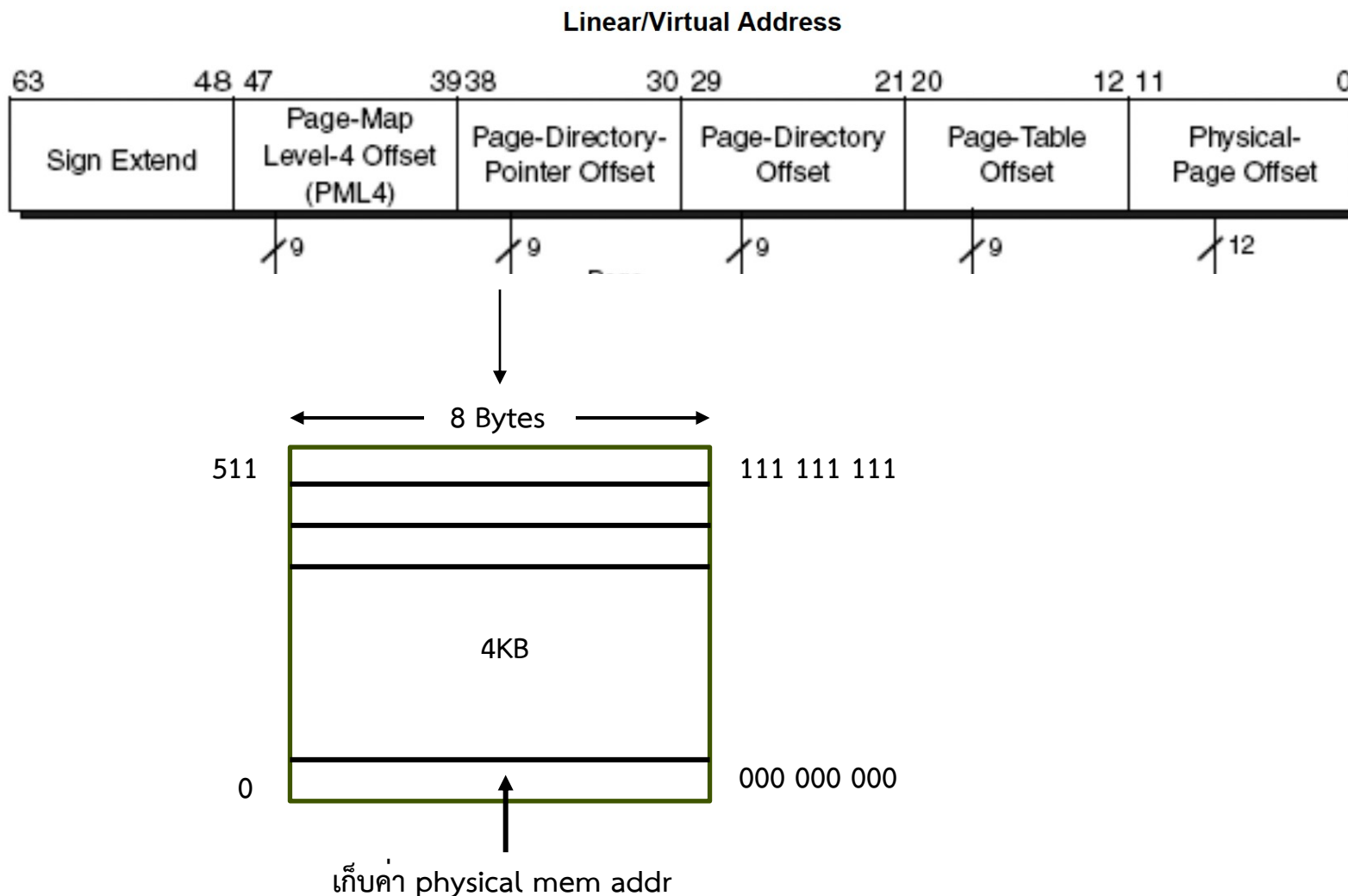
Hierarchical Page table



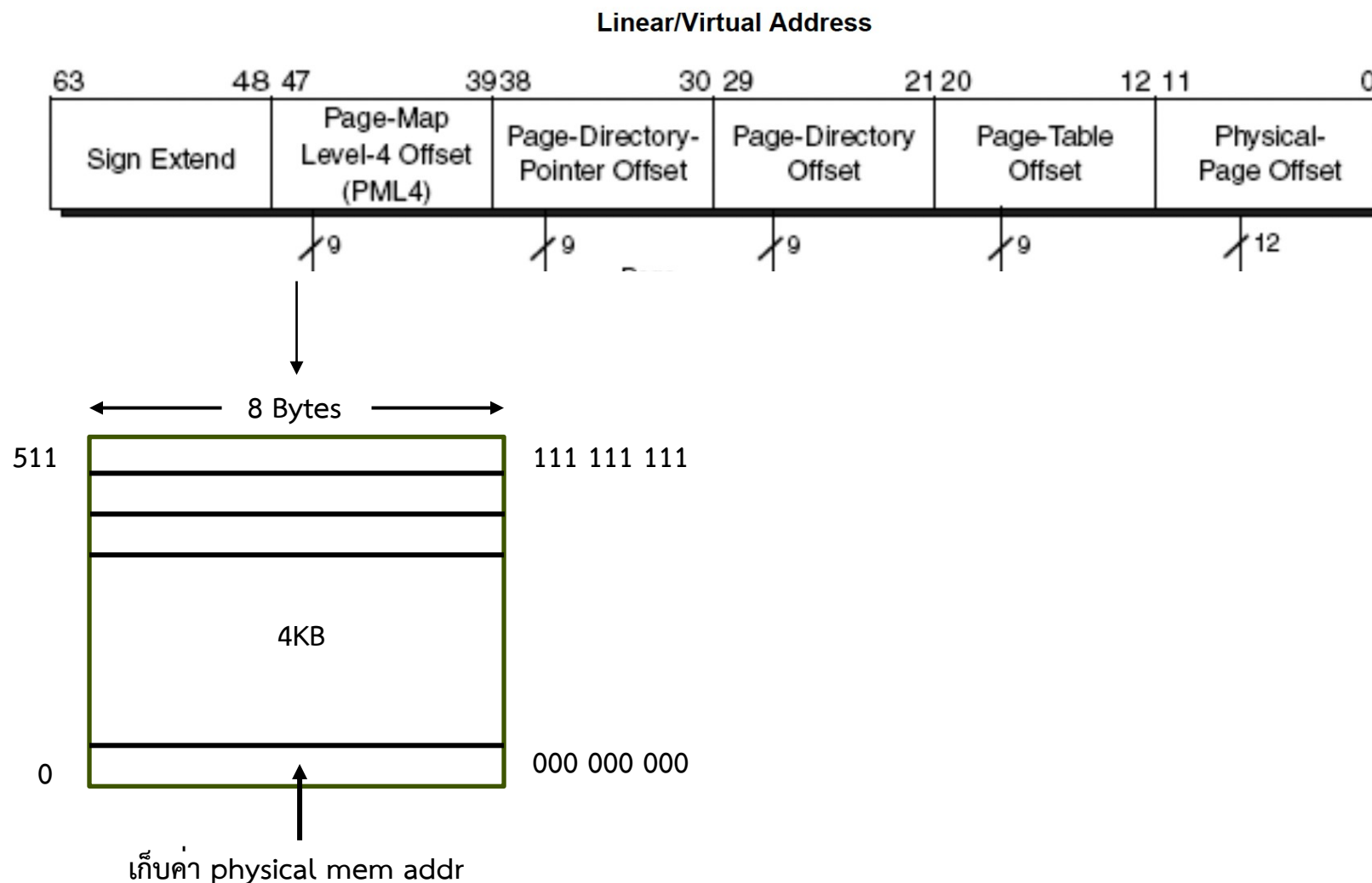
Hierarchical Page table



Hierarchical Page table



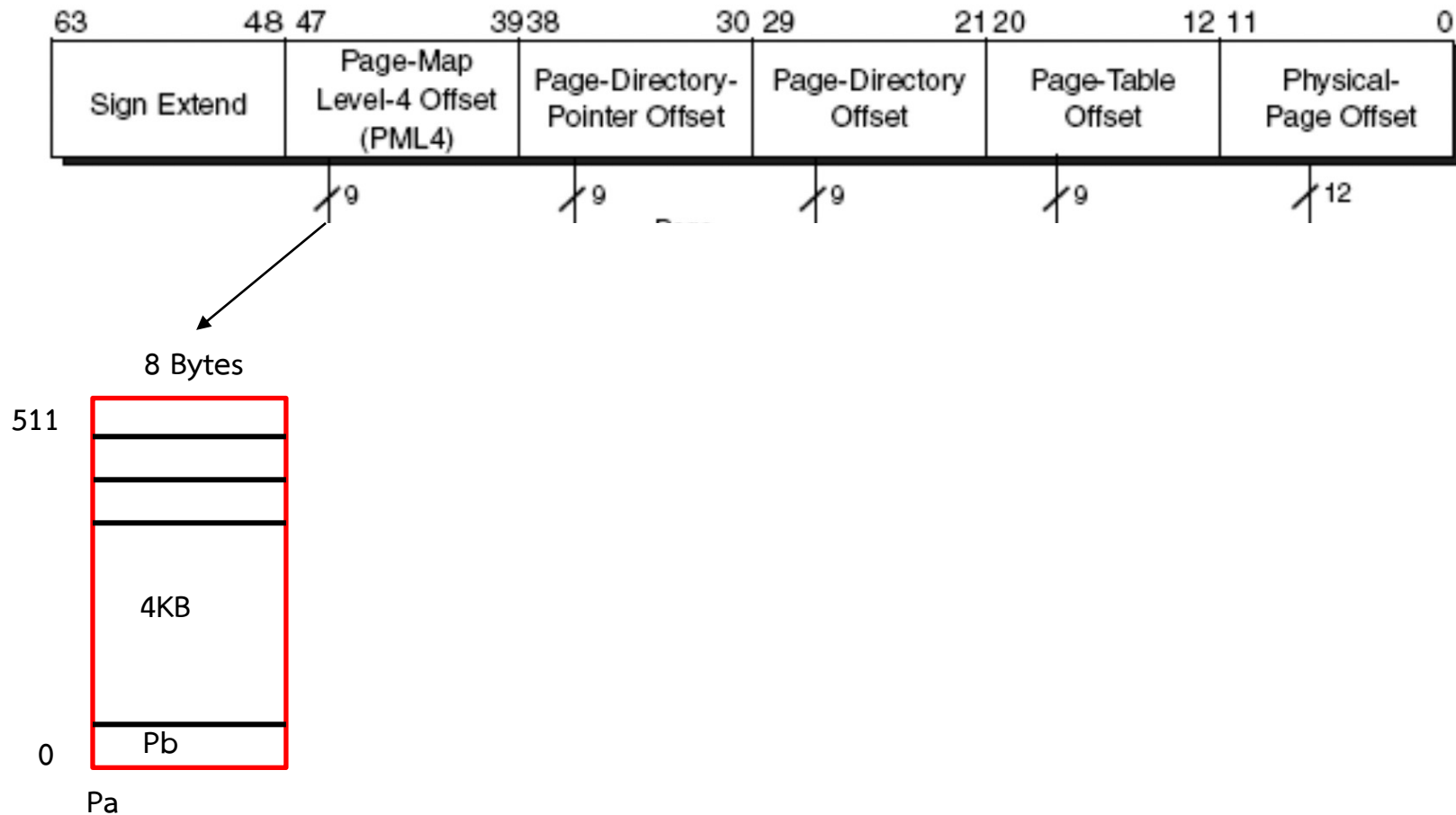
Hierarchical Page table



$$VA = 0x\ 10 = 0b\ 0001\ 0000$$

000_8 000_8 000_8 000_8 0020_8

Linear/Virtual Address



CR3 = P_a

VA = 0x 10 = 0b 0001 0000

000₈

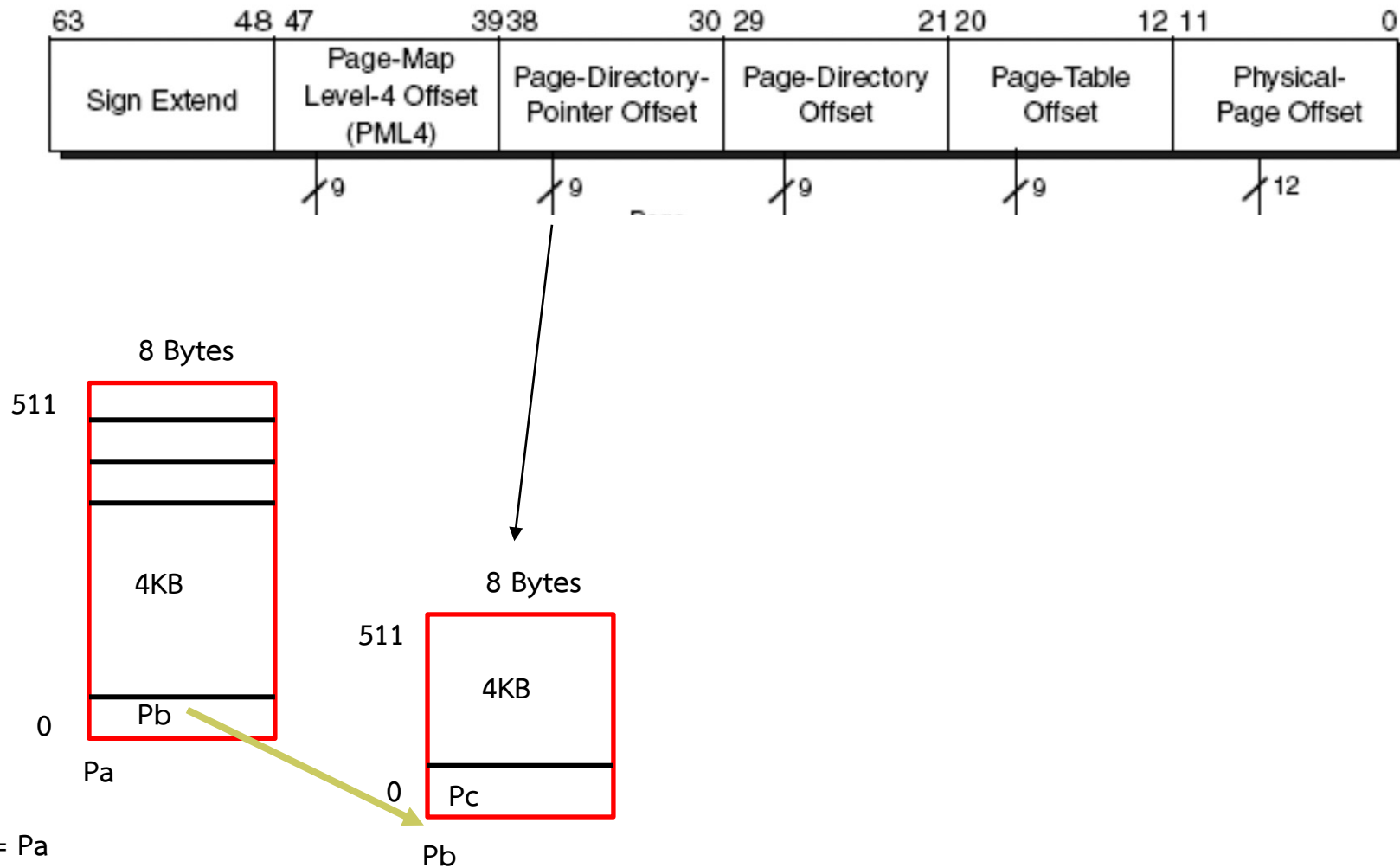
000₈

000₈

000₈

0020₈

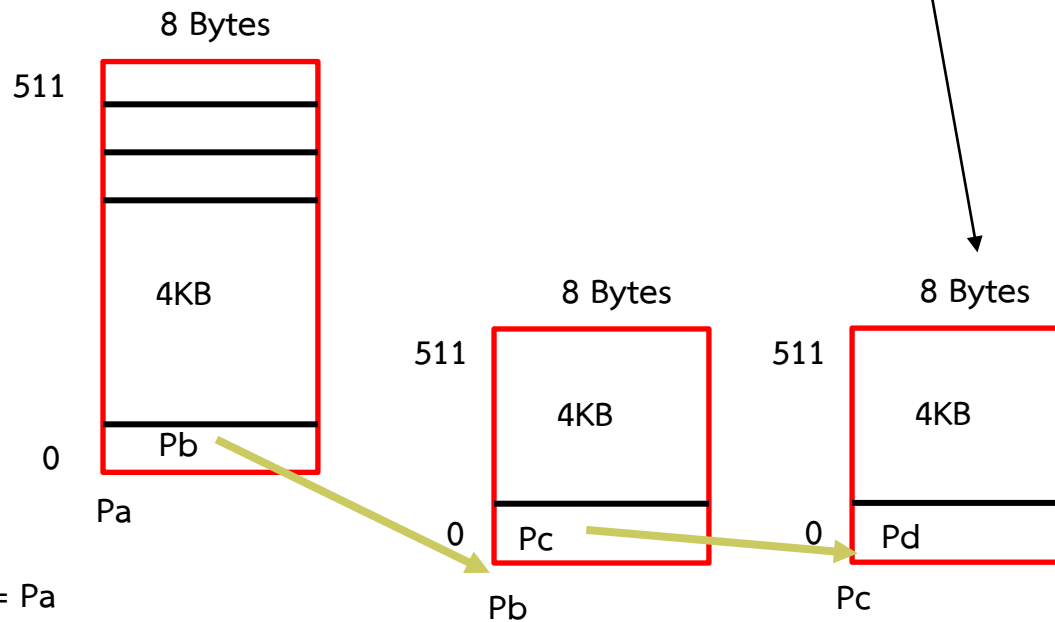
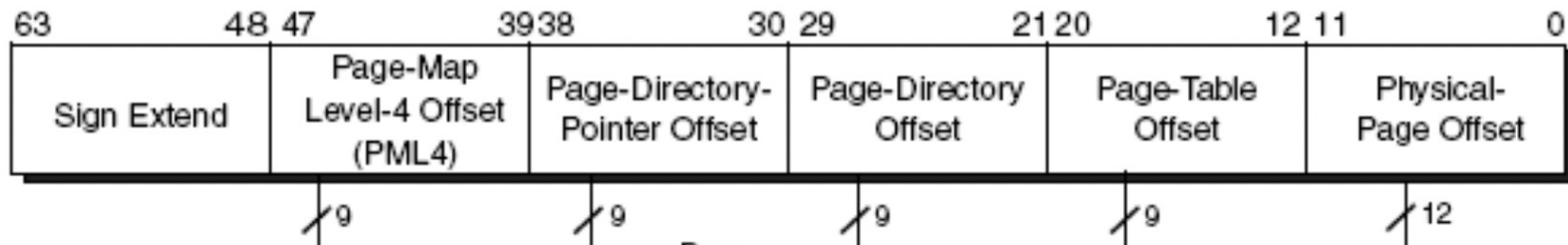
Linear/Virtual Address



$$VA = 0x\ 10 = 0b\ 0001\ 0000 = 0o\ 20$$

000_8 000_8 000_8 000_8 0020_8

Linear/Virtual Address

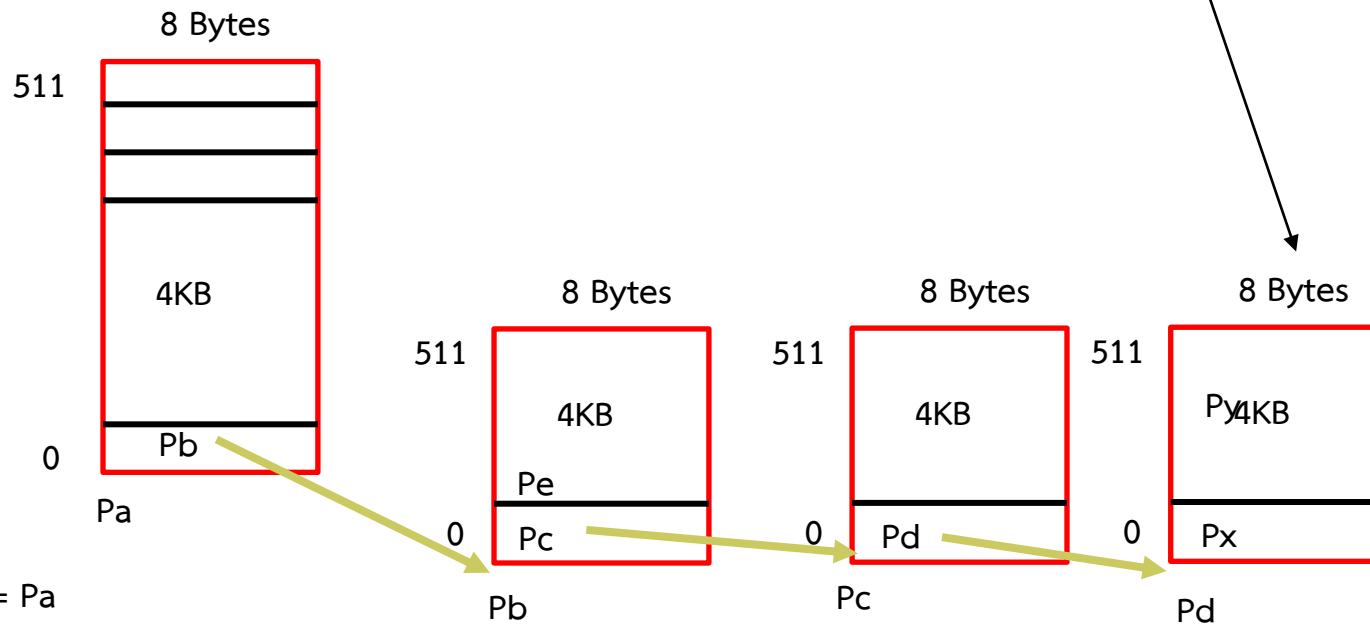
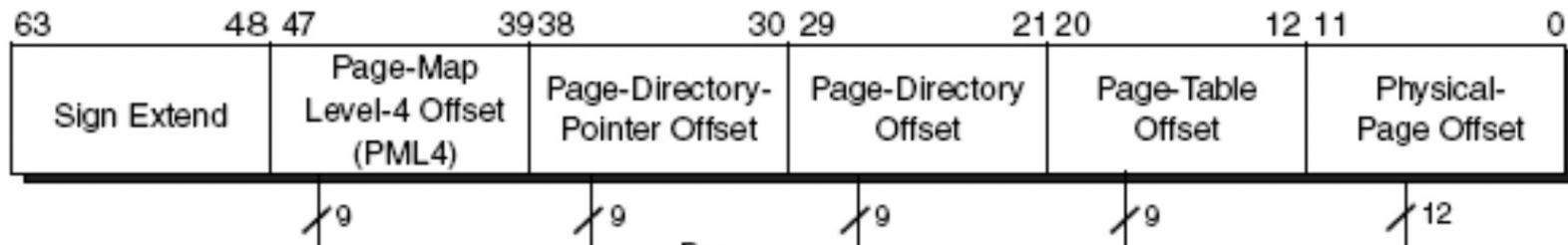


CR3 = Pa

$$VA = 0x\ 10 = 0b\ 0001\ 0000 = 0o\ 20$$

000₈ 000₈ 000₈ 000₈ 0020₈

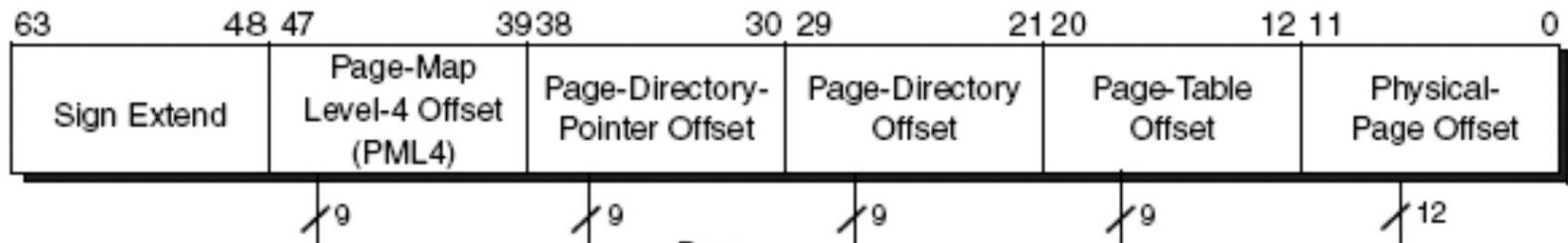
Linear/Virtual Address



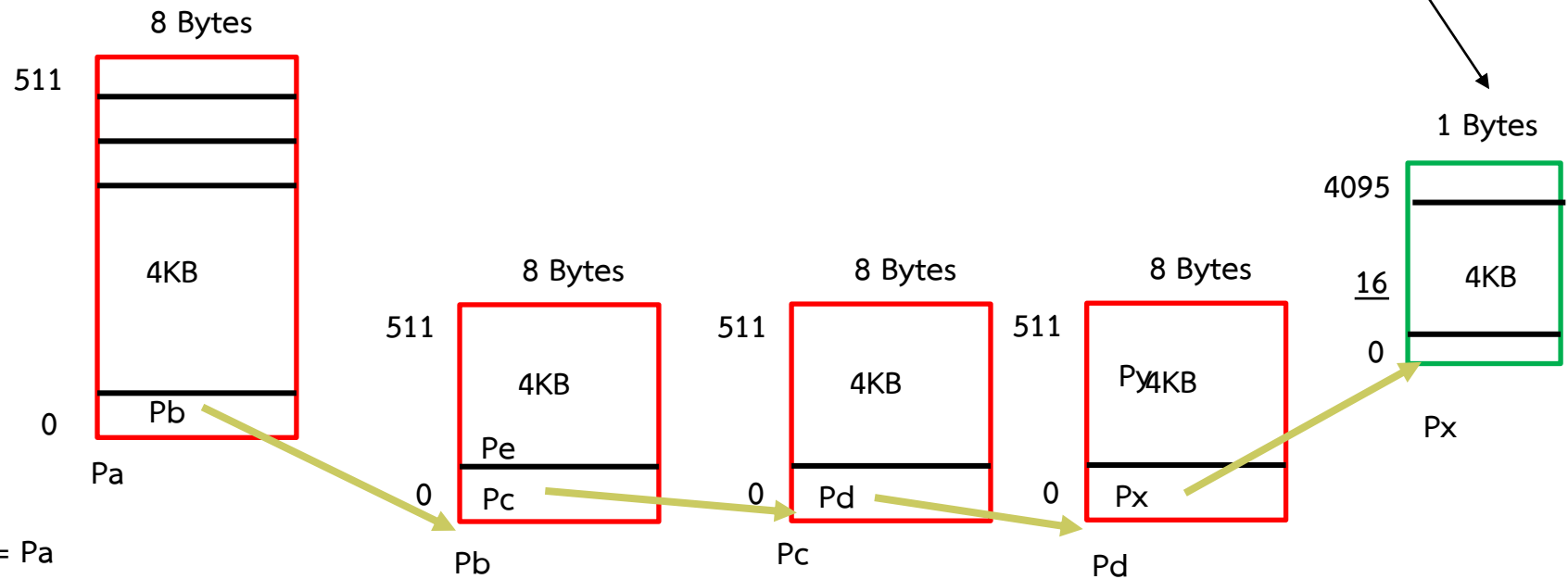
$$VA = 0x\ 10 = 0b\ 0001\ 0000 = 0o\ 20$$

000_8 000_8 000_8 000_8 0020_8

Linear/Virtual Address



User Process Memory



CR3 = Pa

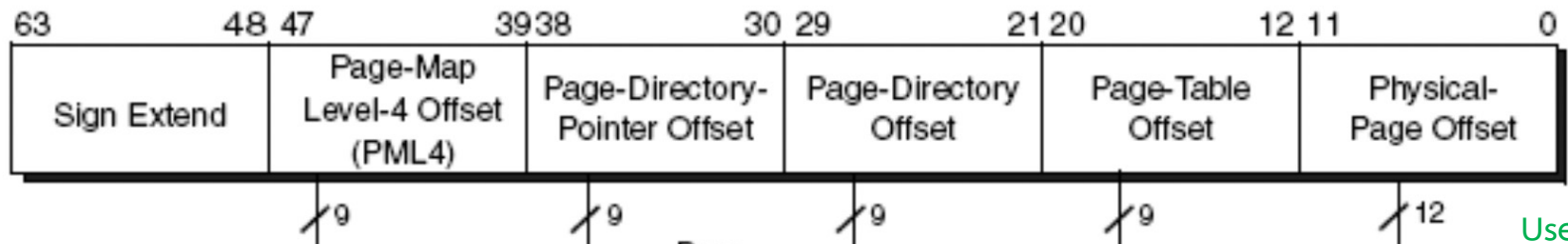
อ้างอิง AMD, AMD-V Nested Paging, 2008

Kernel Memory

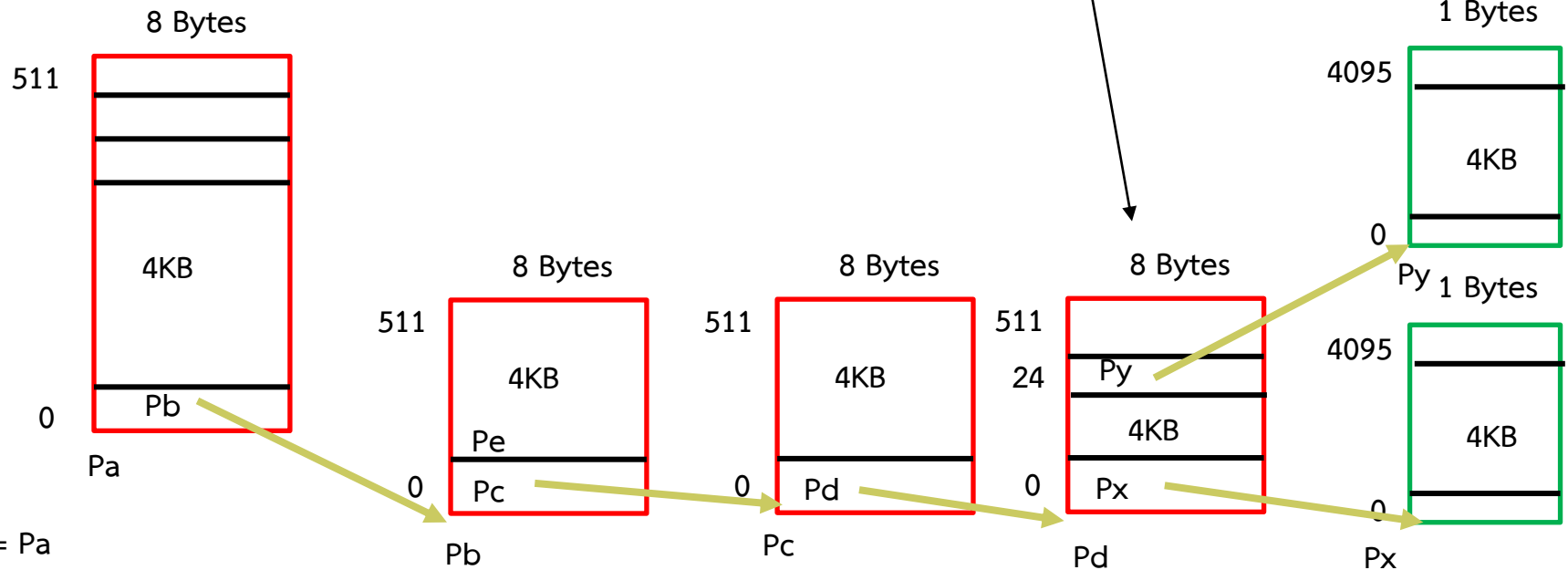
$$VA = 0x\ 3000 = 0b\ 0011\ 0000\ 0000\ 0000 = 0o\ 30000$$

000_8 000_8 000_8 003_8 0000_8

Linear/Virtual Address



User Process Memory



CR3 = Pa

อ้างอิง AMD, AMD-V Nested Paging, 2008

Kernel Memory

New VA

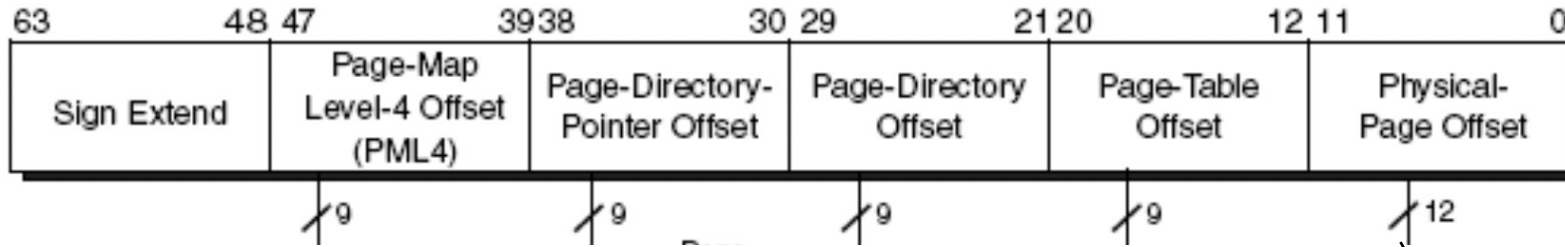
000₈

001₈

000₈

000₈

0000₈



8 Bytes

511

4KB

Pb

Pa

CR3 = Pa

อ้างอิง AMD, AMD-V Nested Paging, 2008

8 Bytes

511

4KB

Pe

Pc

Pb

8 Bytes

511

4KB

Pd

Pc

8 Bytes

511

Py

Px

Pd

1 Bytes

4095

4KB

0

Py 1 Bytes

4095

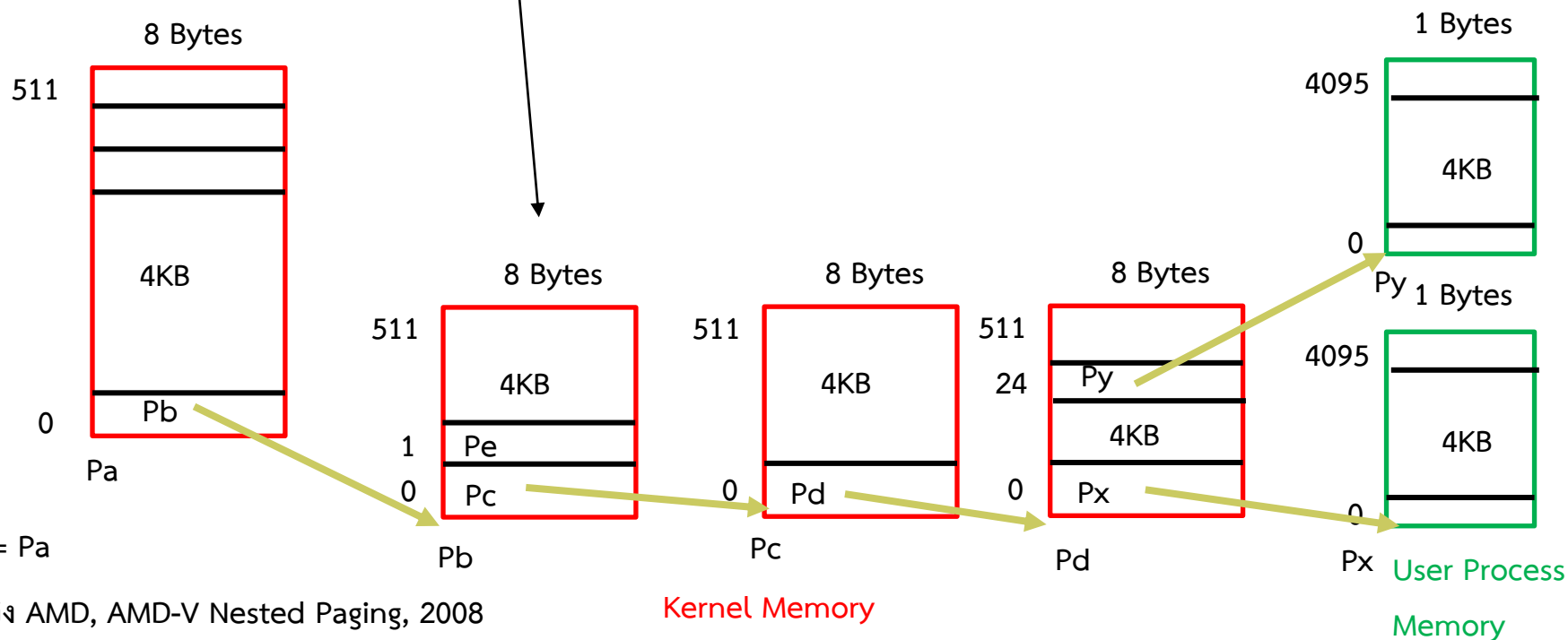
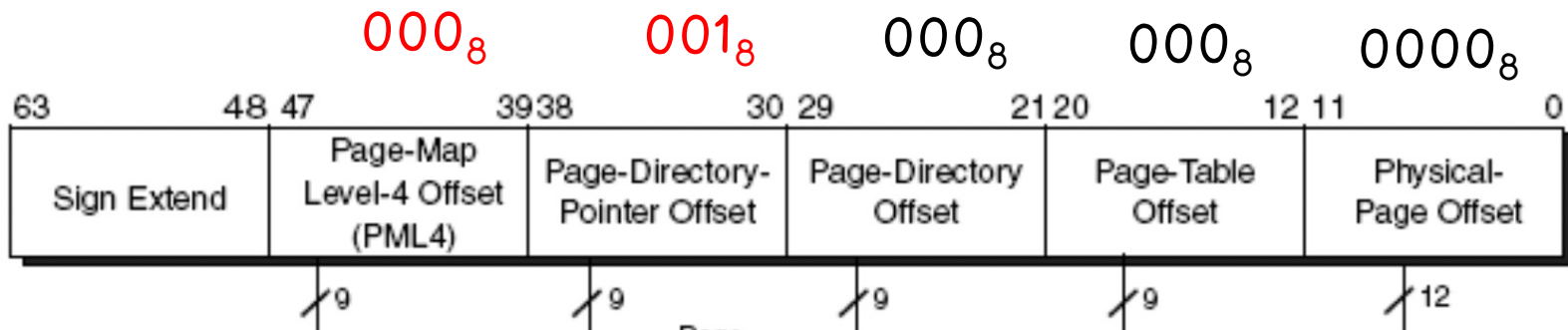
4KB

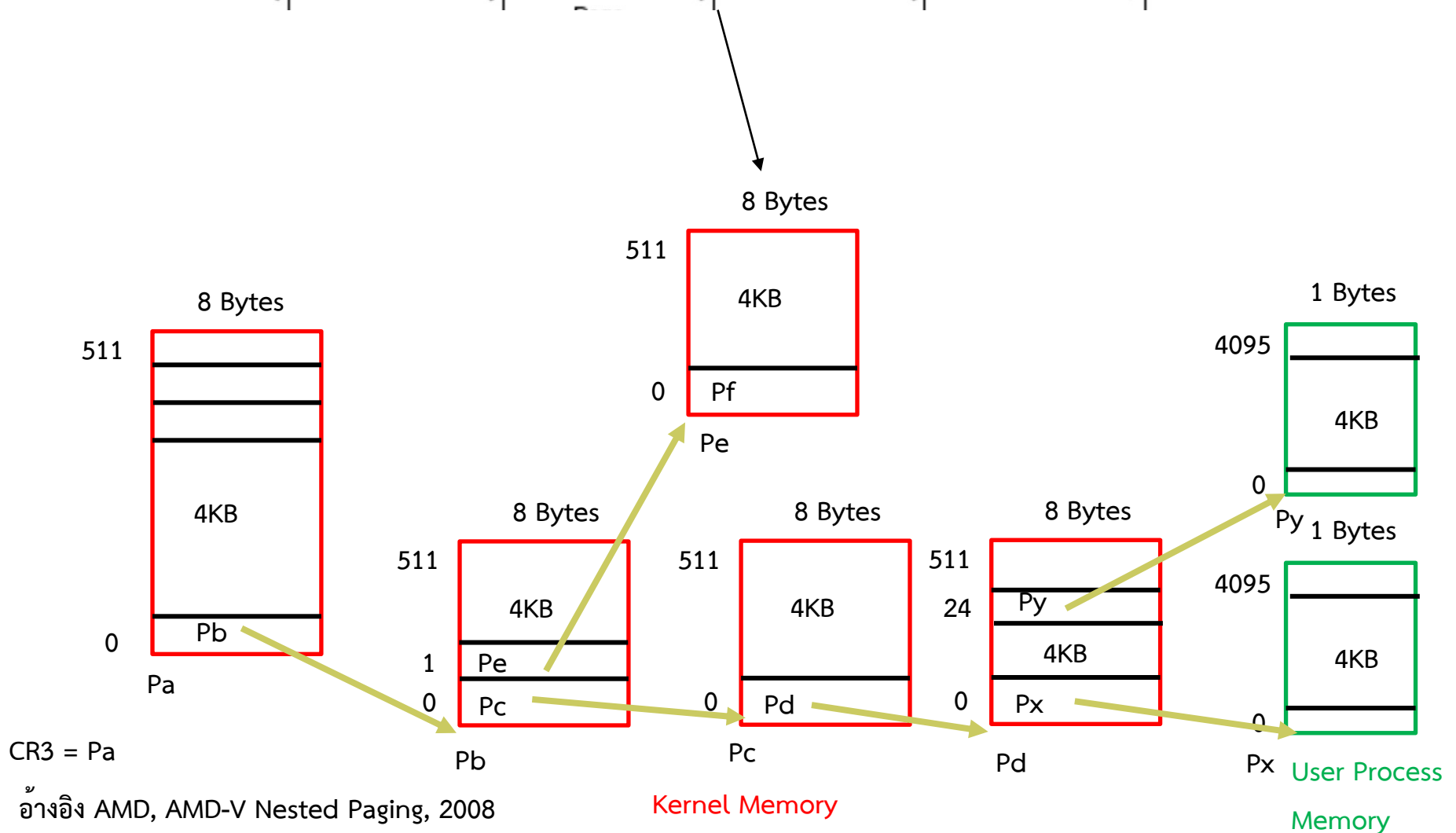
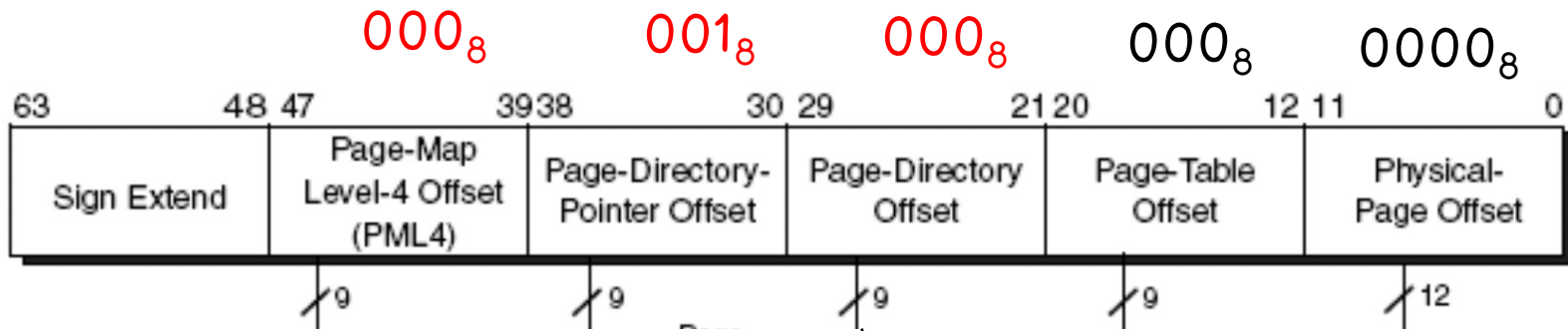
0

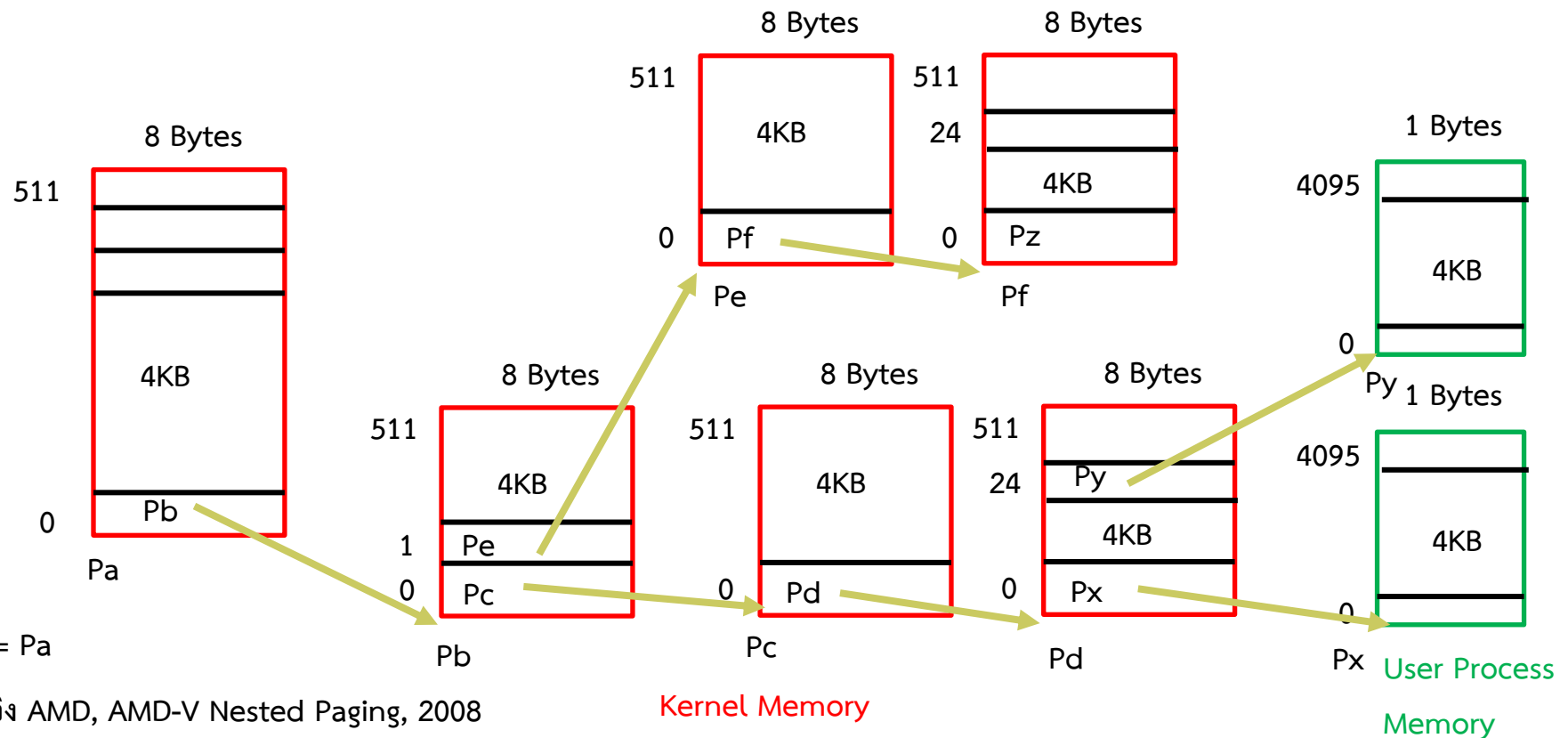
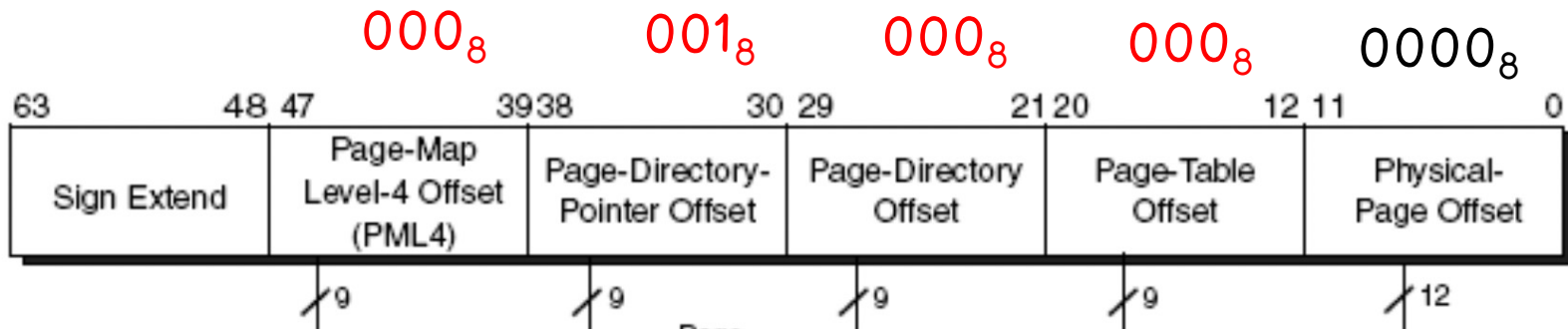
Px

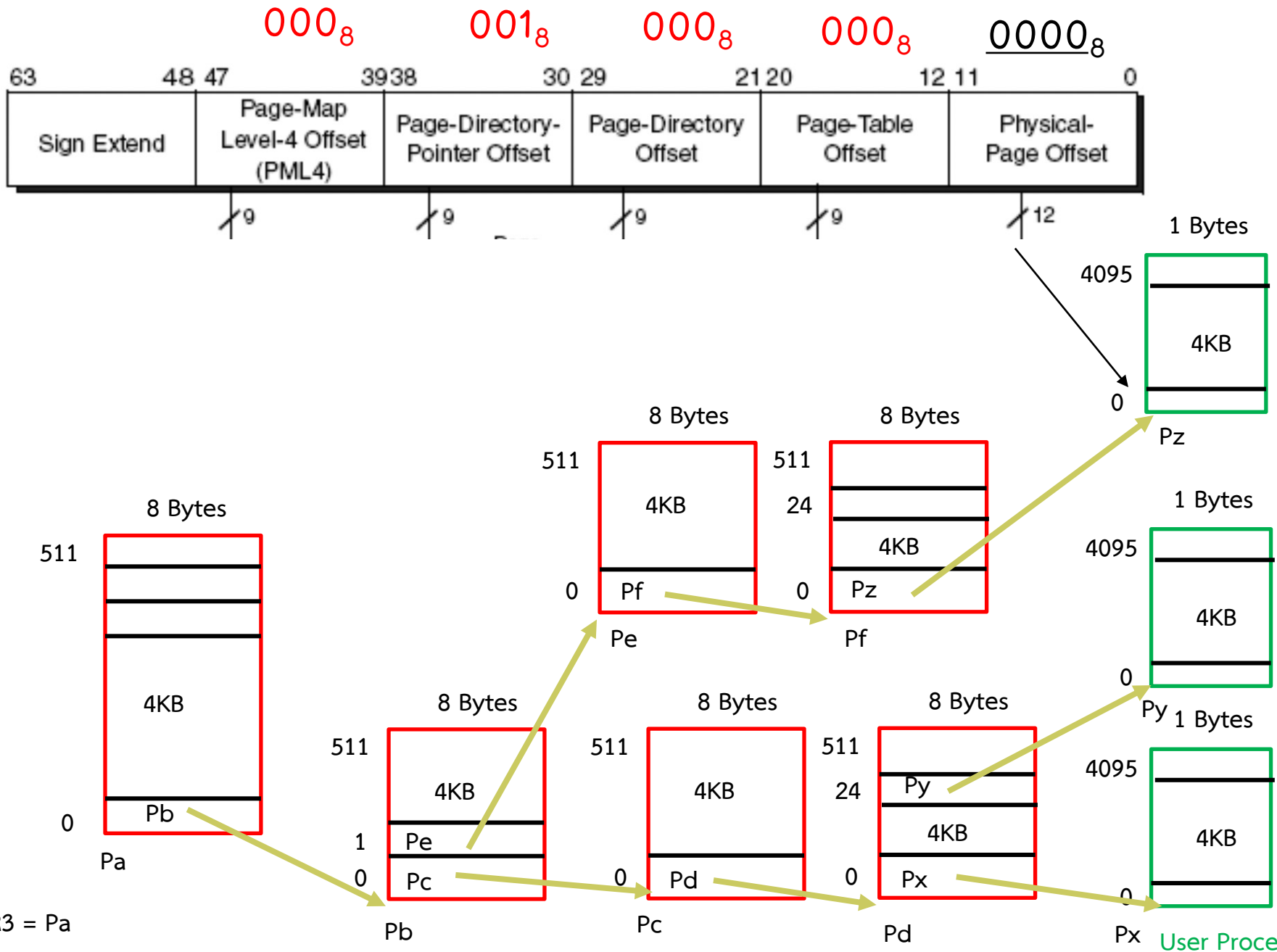
User Process
Memory

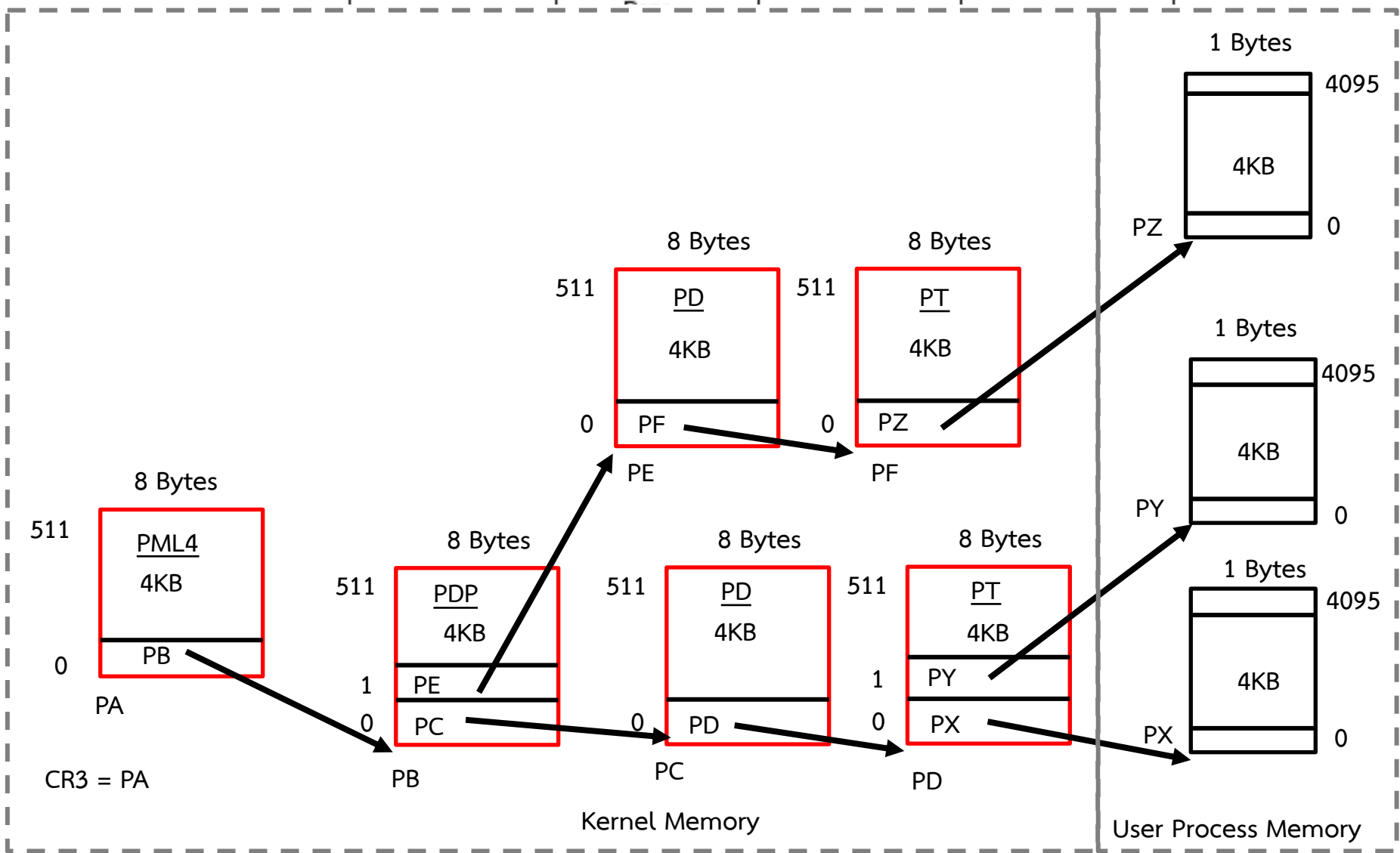
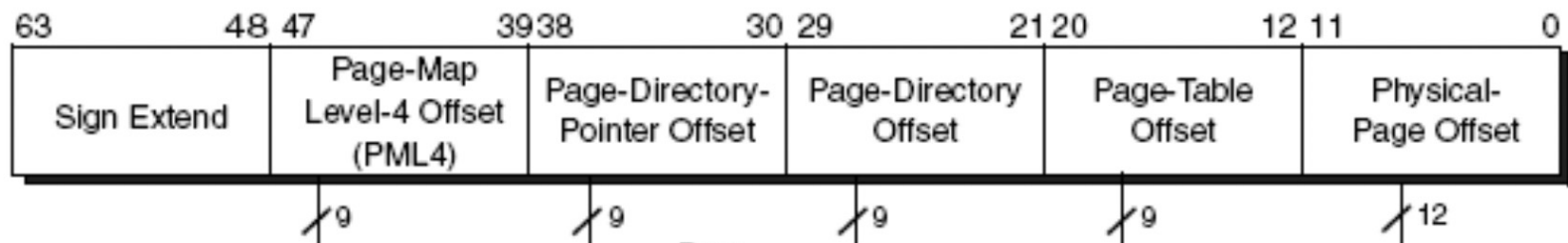
Kernel Memory











ปัญหาและวิธีแก้ปัญหของ Hierarchical Page Table

- **ปัญหา:** MMU ต้องเข้าถึง Memory 4 ครั้งเพื่อแปลงค่า PVA to HPA
- **วิธีการแก้ปัญหา:**
 1. ใช้ TLB เก็บค่า PVA to HPA mapping ถ้า TLB hit สูงก็จะใช้ MMU น้อย
 2. TLB มีหลายชั้น เช่น L1 TLB และ L2 TLB (เหมือน Cache มี L1 L2...) เพื่อเพิ่มเปอร์เซ็นต์ TLB hit แต่ละระดับมีความเร็วต่างกัน แต่ยังดีกว่าการใช้ Page Table มาก

Tatar, A., et al. (2022). DR: Enhancing TLB-based attacks with TLB desynchronized reverse engineering. *USENIX Security Symposium*. https://www.usenix.org/system/files/sec22fall_tatar.pdf

3. **เมื่อเกิด TLB miss** ก็ลดจำนวนการเข้าถึง memory ในระหว่าง walk โดยใช้ **Paging Structure Cache**

Section 4.10,

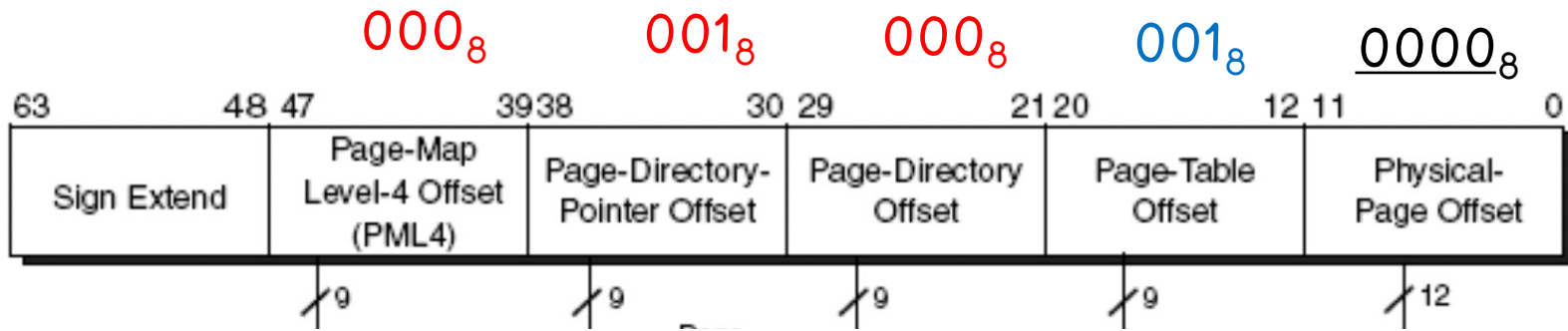
<https://www.intel.com/content/dam/www/public/us/en/documents/manuals/64-ia-32-architectures-software-developer-vol-3a-part-1-manual.pdf>

Paging Structure Cache

- เป็น Cache ที่เก็บข้อมูล Tag และค่า Physical Memory Address ดังนี้

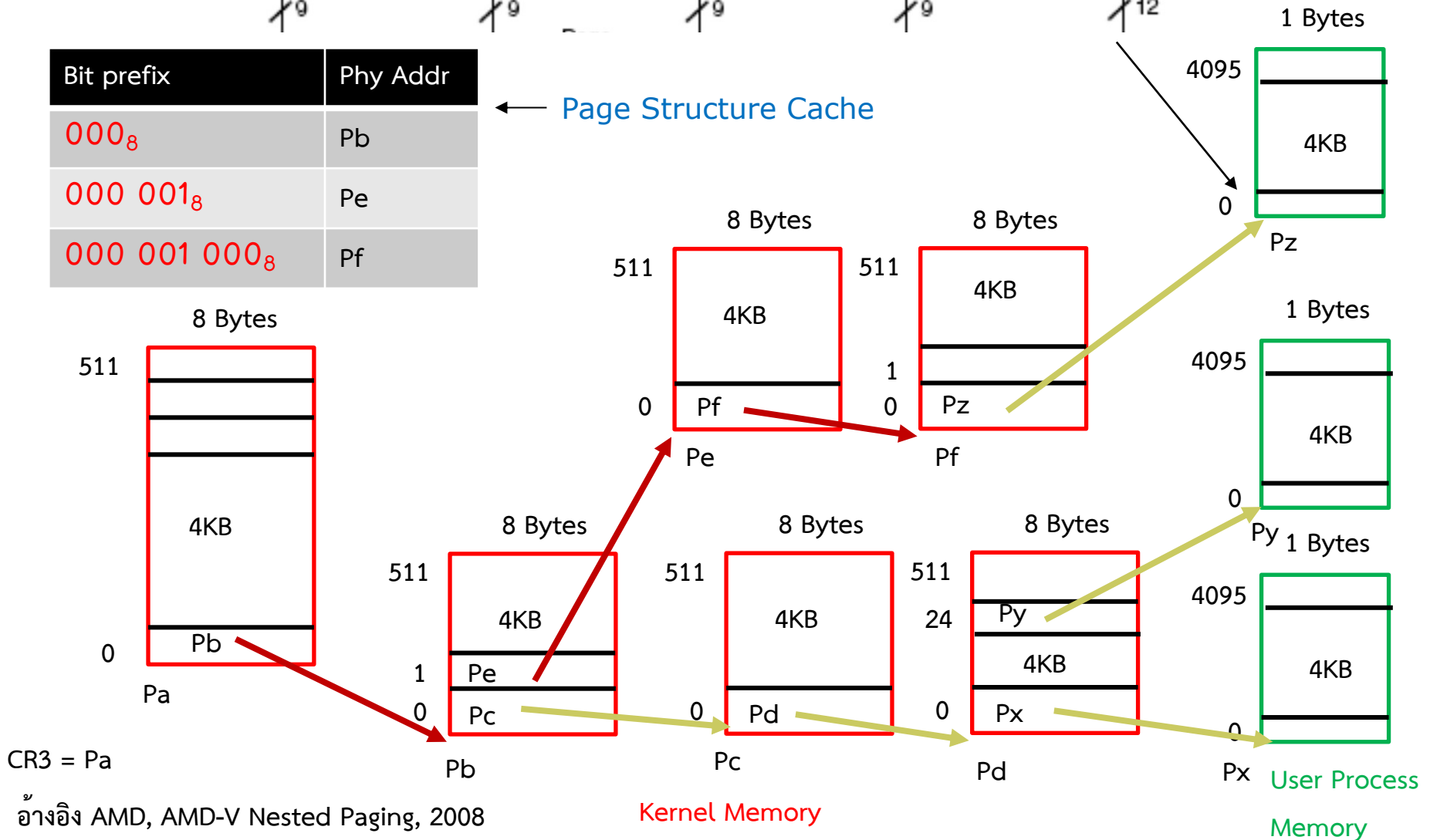
Cache	Cache Tag เก็บค่า bit ตำแหน่งสูง ของ PVA	Cache Text เก็บค่า Physical Address (PA) ของ Page Table ระดับต่างๆ
Page Structure Cache	Bit [47 : 39]	PA ของ PDP Table
	Bit [47 : 30]	PA ของ PD Table
	Bit [47 : 21]	PA ของ PT
TLB	Bit [47 : 12]	Frame No ของ Target Page

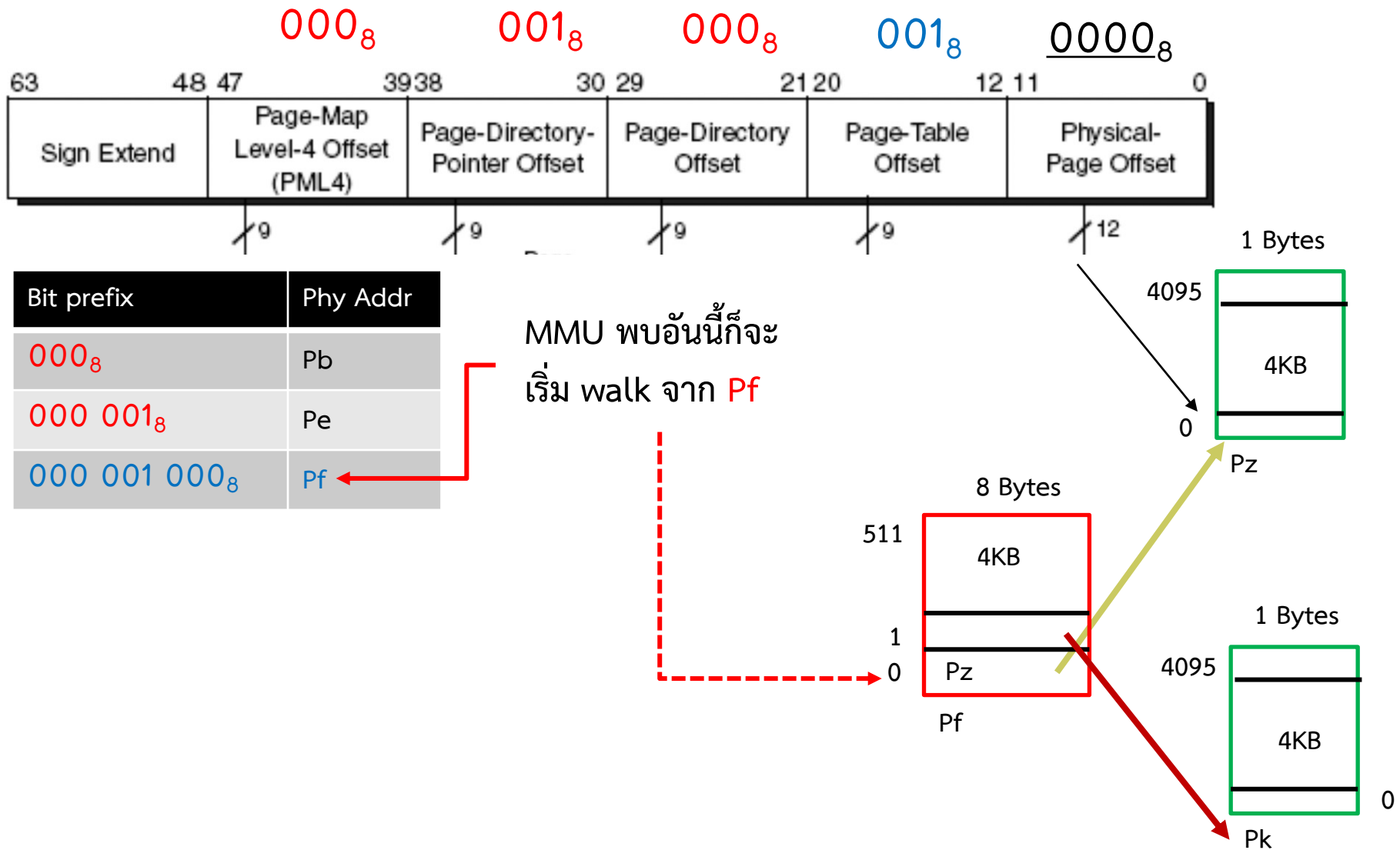
Layer	Page Table Structure	index bits
L4	Page Map Level 4 (PML4) Table	47:39
L3	Page Directory Pointer (PDP) Table	38:30
L2	Page Directory (PD) Table	29:21
L1	Page Table (PT)	20:12



Bit prefix	Phy Addr
000_8	Pb
$000\ 001_8$	Pe
$000\ 001\ 000_8$	Pf

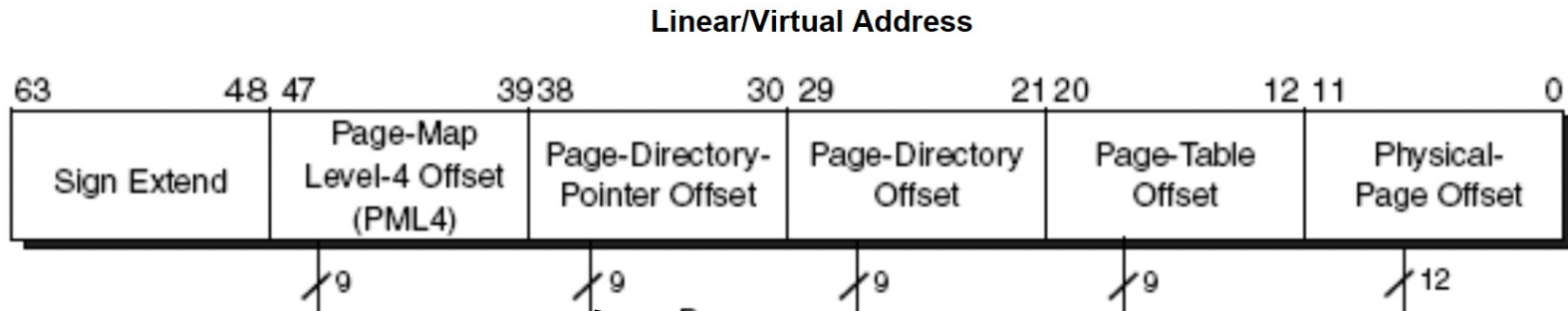
← Page Structure Cache





Let's walk Page Table

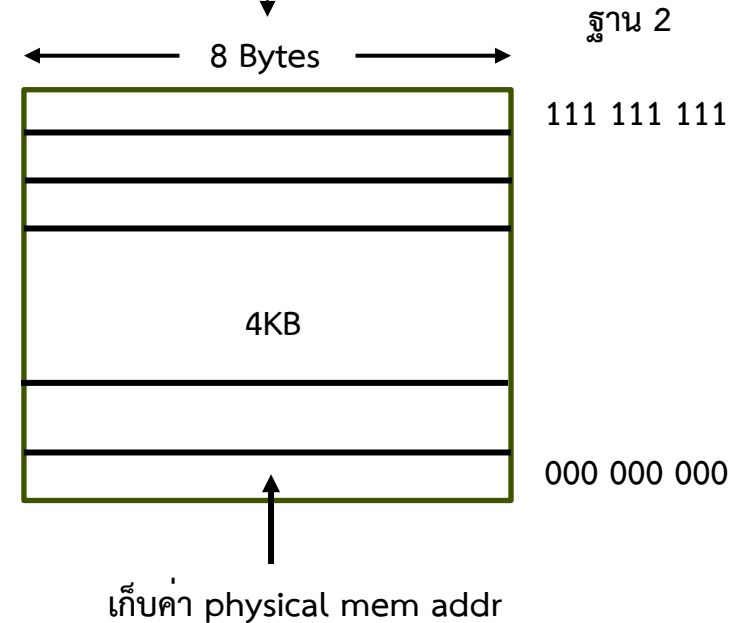
Hierarchical Page table



$$F(x) = \text{index} * 8 + \text{Base}$$

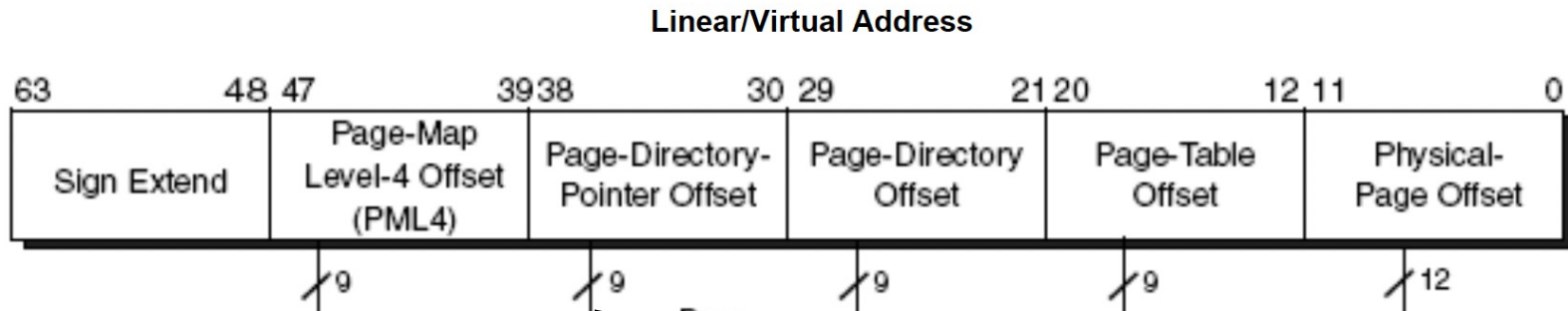
$$\begin{aligned}
 F(000_8) &= 000_8 * 8 + \text{Base} \\
 &= 000_8 * 2^3 + \text{Base} \\
 &= (000_8 \ll 3) + \text{Base} \\
 &= 0000_8 + \text{Base} \\
 &= \text{Base}
 \end{aligned}$$

index	
ฐาน 8	ฐาน 10
777 ₈	511
...	...
010 ₈	8
007 ₈	7
...	...
001 ₈	1
000 ₈	0



Base

Hierarchical Page table

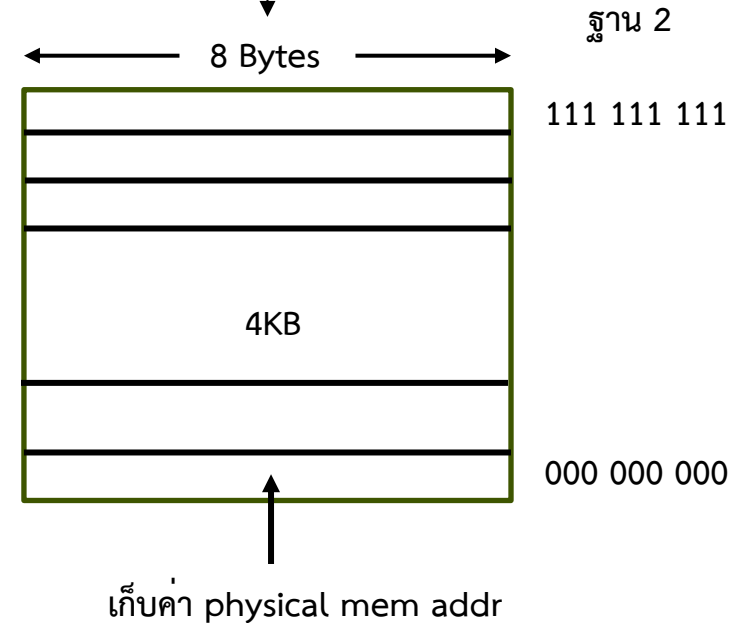


$$F(x) = \text{index} * 8 + \text{Base}$$

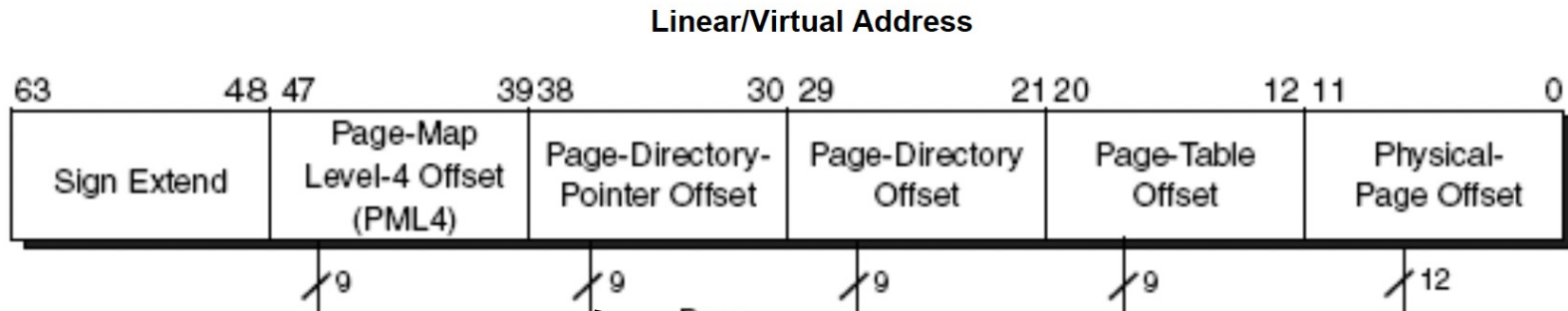
$$\begin{aligned}
 F(000_8) &= 001_8 * 8 + \text{Base} \\
 &= 001_8 * 2^3 + \text{Base} \\
 &= (001_8 \ll 3) + \text{Base} \\
 &= 0010_8 + \text{Base}
 \end{aligned}$$

index	
ฐาน 8	ฐาน 10
777 ₈	511
...	...
010 ₈	8
007 ₈	7
...	...
001 ₈	1
000 ₈	0

Base



Hierarchical Page table

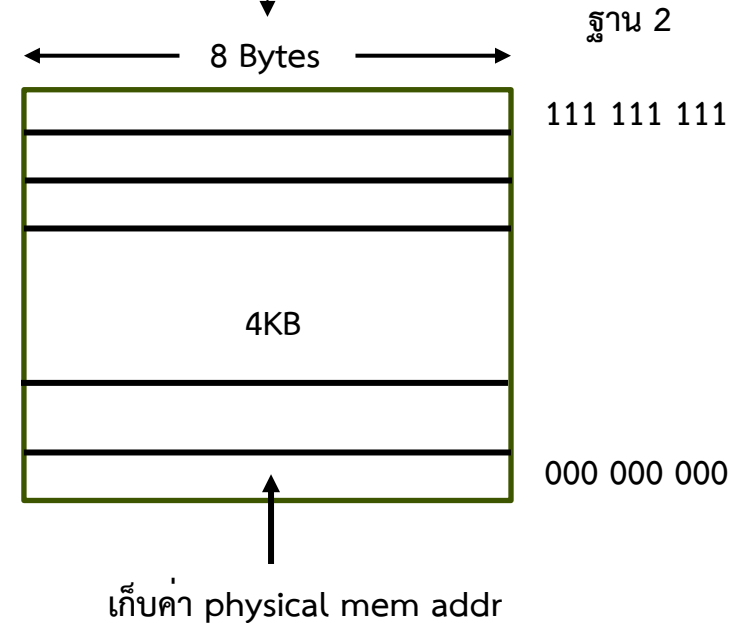


$$F(x) = \text{index} * 8 + \text{Base}$$

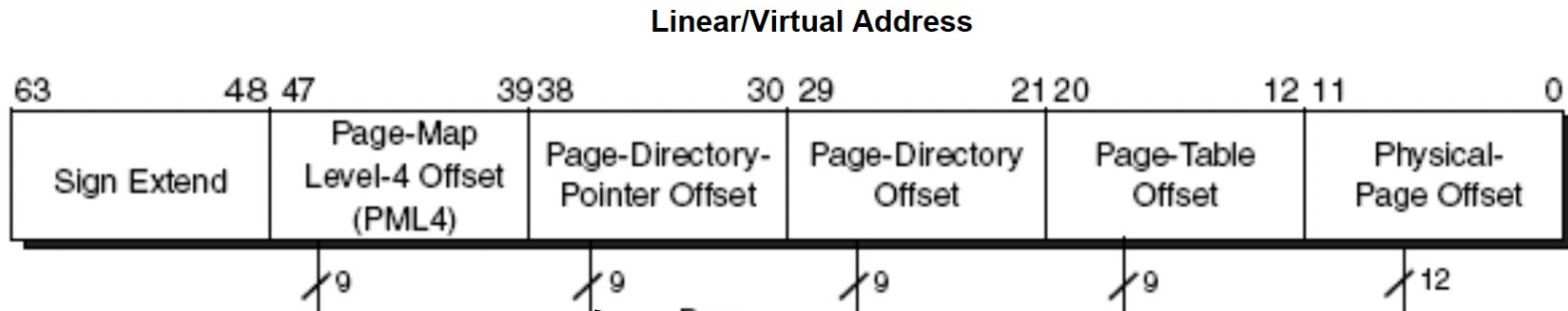
$$\begin{aligned}
 F(000_8) &= 007_8 * 8 + \text{Base} \\
 &= 007_8 * 2^3 + \text{Base} \\
 &= (007_8 \ll 3) + \text{Base} \\
 &= 0070_8 + \text{Base}
 \end{aligned}$$

index	
ฐาน 8	ฐาน 10
777 ₈	511
...	...
010 ₈	8
007 ₈	7
...	...
001 ₈	1
000 ₈	0

Base



Hierarchical Page table

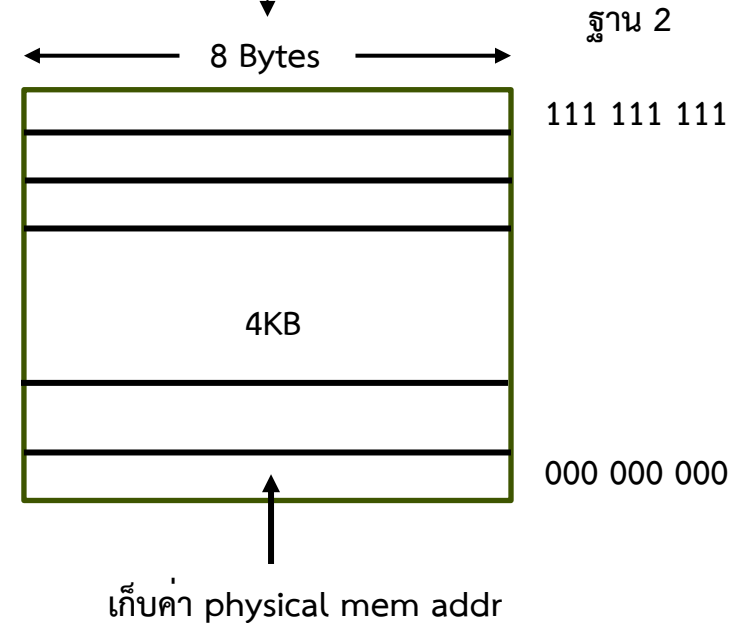


$$F(x) = \text{index} * 8 + \text{Base}$$

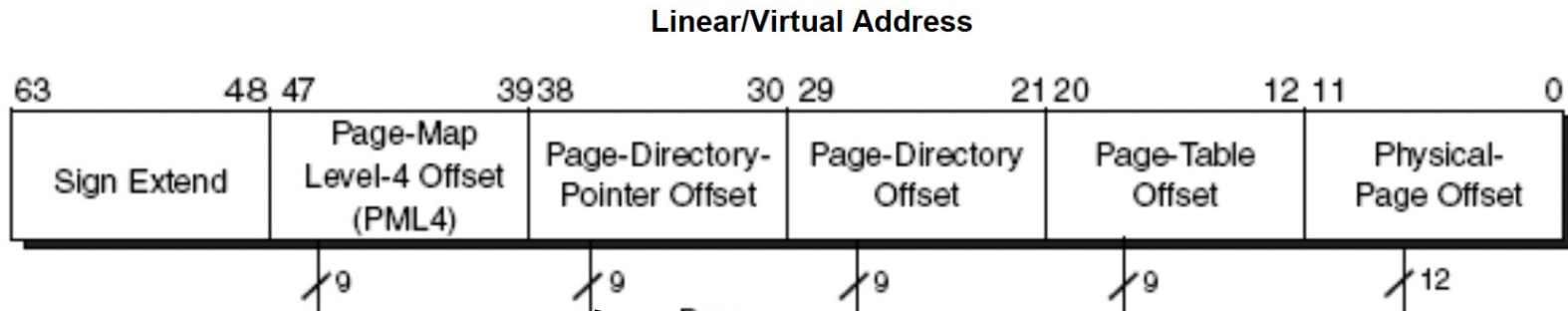
$$\begin{aligned}
 F(000_8) &= 010_8 * 8 + \text{Base} \\
 &= 010_8 * 2^3 + \text{Base} \\
 &= (010_8 \ll 3) + \text{Base} \\
 &= 0100_8 + \text{Base}
 \end{aligned}$$

index	
ฐาน 8	ฐาน 10
777 ₈	511
...	...
010 ₈	8
007 ₈	7
...	...
001 ₈	1
000 ₈	0

Base



Hierarchical Page table

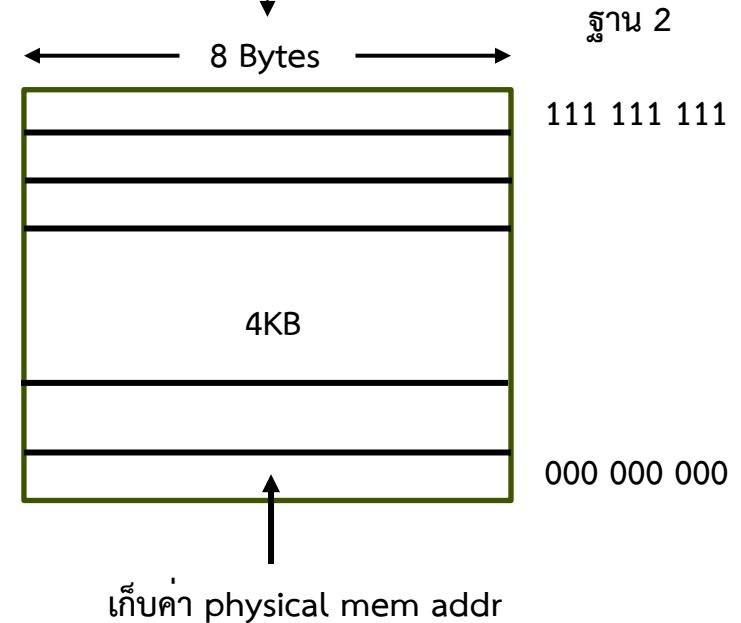


$$F(x) = \text{index} * 8 + \text{Base}$$

$$\begin{aligned}
 F(000_8) &= 777_8 * 8 + \text{Base} \\
 &= 777_8 * 2^3 + \text{Base} \\
 &= (777_8 \ll 3) + \text{Base} \\
 &= 7770_8 + \text{Base}
 \end{aligned}$$

index	
ฐาน 8	ฐาน 10
777 ₈	511
...	...
010 ₈	8
007 ₈	7
...	...
001 ₈	1
000 ₈	0

Base



P0

VA= 111 222 333 444 1765₈

47

39

38

30

29

21

20

12

11

0

111₈

222₈

333₈

444₈

1765₈

P0

VA= 111 222 333 444 1765₈

47 39

111₈

38 30

222₈

29 21

333₈

20 12 11

444₈

1765₈

0

64 bits

F(777₈)

F(111₈)

15000₈

F(000₈)

140000₈

CR3

$$F(x) = \text{index} * 8 + \text{Base}$$

$$\begin{aligned} F(111_8) &= 111_8 * 8 + \text{Base} \\ &= 111_8 * 2^3 + \text{Base} \\ &= (111_8 \ll 3) + \text{Base} \\ &= 1110_8 + \text{Base} \\ &= 1110_8 + 140000_8 \\ &= 14\underline{111}0_8 \end{aligned}$$

P0

VA= 111 222 333 444 1765₈

47 39

111₈

38 30

222₈

29 21

333₈

20 12 11

444₈

1765₈

0

64 bits

147770₈

141110₈

150000₈

140000₈

140000₈

CR3

$$F(x) = \text{index} * 8 + \text{Base}$$

$$\begin{aligned} F(111_8) &= 111_8 * 8 + \text{Base} \\ &= 111_8 * 2^3 + \text{Base} \\ &= (111_8 \ll 3) + \text{Base} \\ &= 1110_8 + \text{Base} \\ &= 1110_8 + 140000_8 \\ &= 141110_8 \end{aligned}$$

P0

VA= 111 222 333 444 1765₈

47 39

111₈

38 30

222₈

29 21

333₈

20 12 11

444₈

1765₈

0

64 bits

147770₈

157770₈

152220₈

160000₈

141110₈

150000₈

140000₈

150000₈

140000₈

CR3

$$\begin{aligned}
 F(222_8) &= 222_8 * 8 + \text{Base} \\
 &= 222_8 * 2^3 + \text{Base} \\
 &= (222_8 \ll 3) + \text{Base} \\
 &= 2220_8 + \text{Base} \\
 &= 2220_8 + 150000_8 \\
 &= 15\underline{222}0_8
 \end{aligned}$$

P0

VA= 111 222 333 444 1765₈

47 39

38 30

29 21

20 12 11

0

111₈

222₈

333₈

444₈

1765₈

64 bits

147770₈

157770₈

167770₈

163330₈

170000₈

152220₈

160000₈

150000₈

141110₈

140000₈

150000₈

160000₈

140000₈

CR3

$$\begin{aligned}
 F(333_8) &= \textcolor{blue}{333}_8 * 8 + \textcolor{red}{Base} \\
 &= \textcolor{blue}{333}_8 * 2^3 + \textcolor{red}{Base} \\
 &= (\textcolor{blue}{333}_8 \ll 3) + \textcolor{red}{Base} \\
 &= \textcolor{blue}{3330}_8 + \textcolor{red}{Base} \\
 &= \textcolor{blue}{3330}_8 + \textcolor{red}{160000}_8 \\
 &= \textcolor{red}{16}\textcolor{blue}{333}0_8
 \end{aligned}$$

P0

VA= 111 222 333 444 1765₈

47 39

38 30

29 21

20 12 11

0

111₈

222₈

333₈

444₈

1765₈

64 bits

147770₈

157770₈

167770₈

177770₈

141110₈

150000₈

140000₈

152220₈

160000₈

163330₈

170000₈

174440₈

700000₈

150000₈

160000₈

170000₈

140000₈

CR3

$$\begin{aligned}
 F(444_8) &= 444_8 * 8 + \text{Base} \\
 &= 444_8 * 2^3 + \text{Base} \\
 &= (444_8 \ll 3) + \text{Base} \\
 &= 4440_8 + \text{Base} \\
 &= 4440_8 + 170000_8 \\
 &= 174440_8
 \end{aligned}$$

P0

VA= 111 222 333 444 1765₈

47 39

38 30

29 21

20 12 11

0

111₈

222₈

333₈

444₈

1765₈

Offset

64 bits

147770₈

157770₈

167770₈

177770₈

141110₈

150000₈

152220₈

160000₈

163330₈

170000₈

174440₈

700000₈

701765₈

22₈

140000₈

150000₈

160000₈

170000₈

700000₈

140000₈

CR3

Page's Virtual Address: 111 222 333 444 0000₈

MMU Walk Page Table และได้ค่า

Page's Physical Address: 000 000 000 070 0000₈

Offset

ดังนั้น Virtual Address: 111 222 333 444 1765₈

จะได้ Physical Address: 000 000 000 070 0000₈ + 1765₈ = **70 1765₈**

P0

VA= 111 222 333 444 1765₈

47 39

38 30

29 21

20 12 11

0

111₈

222₈

333₈

444₈

1765₈

64 bits

147770₈

157770₈

167770₈

177770₈

174440₈

701765₈

22₈

141110₈

150000₈

160000₈

170000₈

700000₈

140000₈

150000₈

160000₈

170000₈

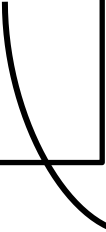
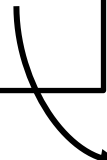
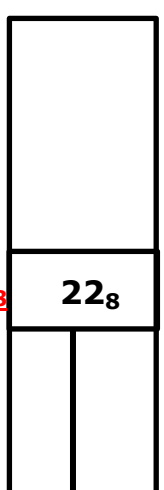
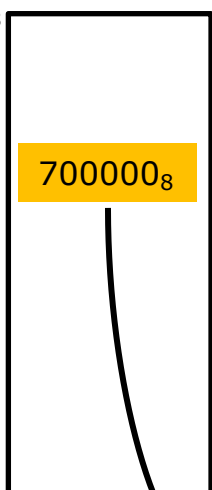
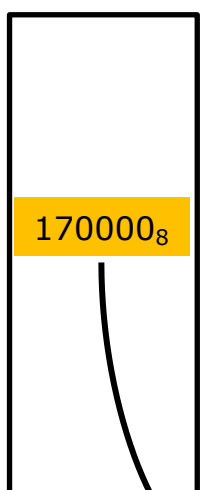
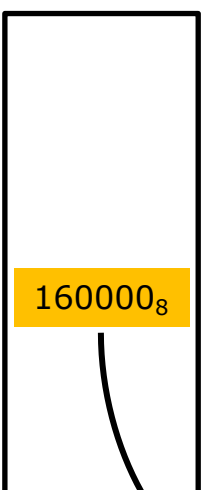
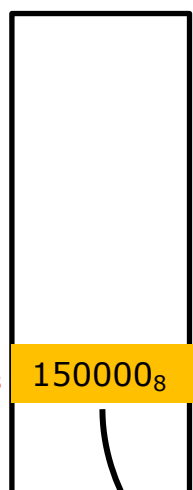
700000₈

140000₈

CR3

22₈

00 010 010₈



P0

VA= aaa bbb ccc ddd eeee₈

47 39

aaa₈

38 30

bbb₈

29 21

ccc₈

20 12 11

ddd₈

eeee₈

0

64 bits

147770₈

157770₈

167770₈

177770₈

141310₈

150000₈

154220₈

160000₈

163730₈

170000₈

174540₈

700000₈

701755₈

22₈

140000₈

150000₈

160000₈

170000₈

700000₈

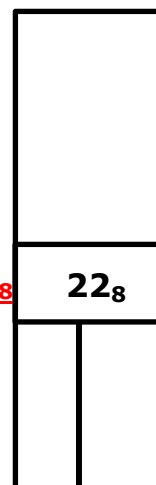
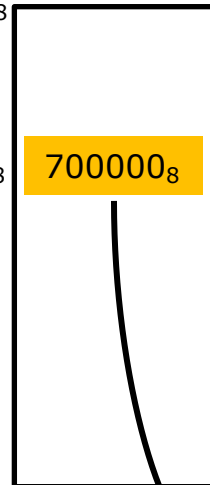
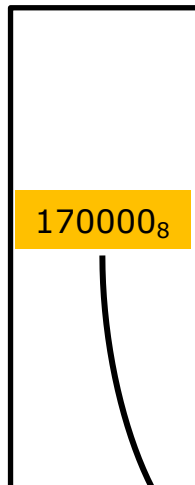
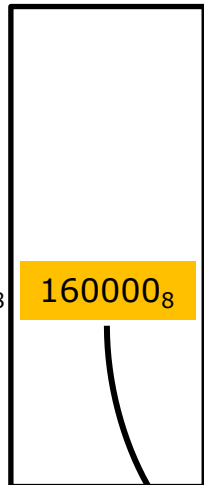
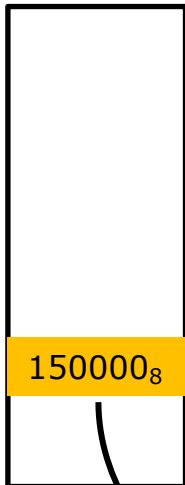
140000₈

CR3

$$F(x) = x * 8 + B$$

22₈

00 010 010₈



P0

VA= aaa bbb ccc ddd eeee₈

47 39

aaa₈

38 30

bbb₈

29 21

ccc₈

20 12 11

ddd₈

11 0

eeee₈

64 bits

147770₈

157770₈

167770₈

177770₈

141110₈

154120₈

163130₈

171540₈

701005₈

140000₈

150000₈

160000₈

170000₈

700000₈

22₈

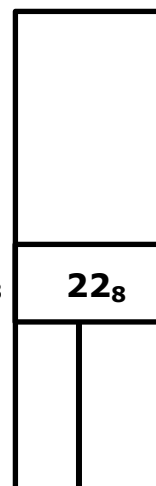
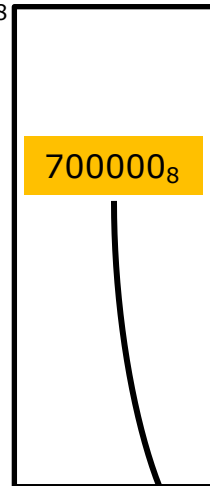
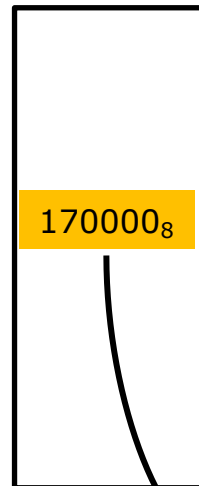
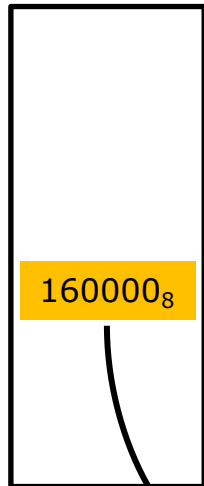
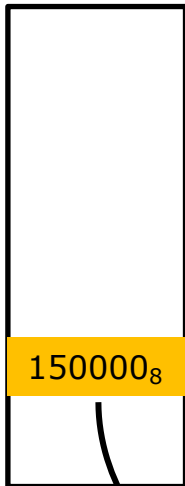
140000₈

CR3

$$F(x) = x * 8 + B$$

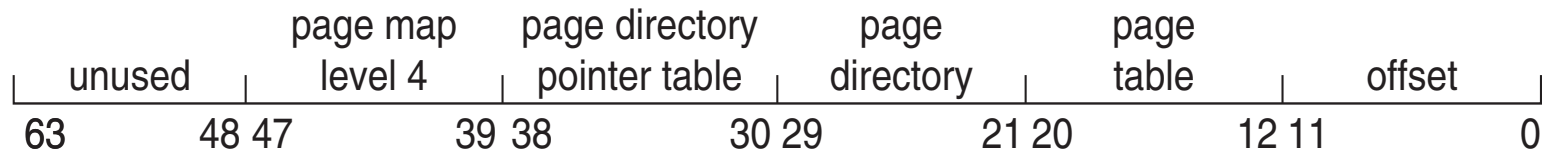
22₈

00 010 010₈

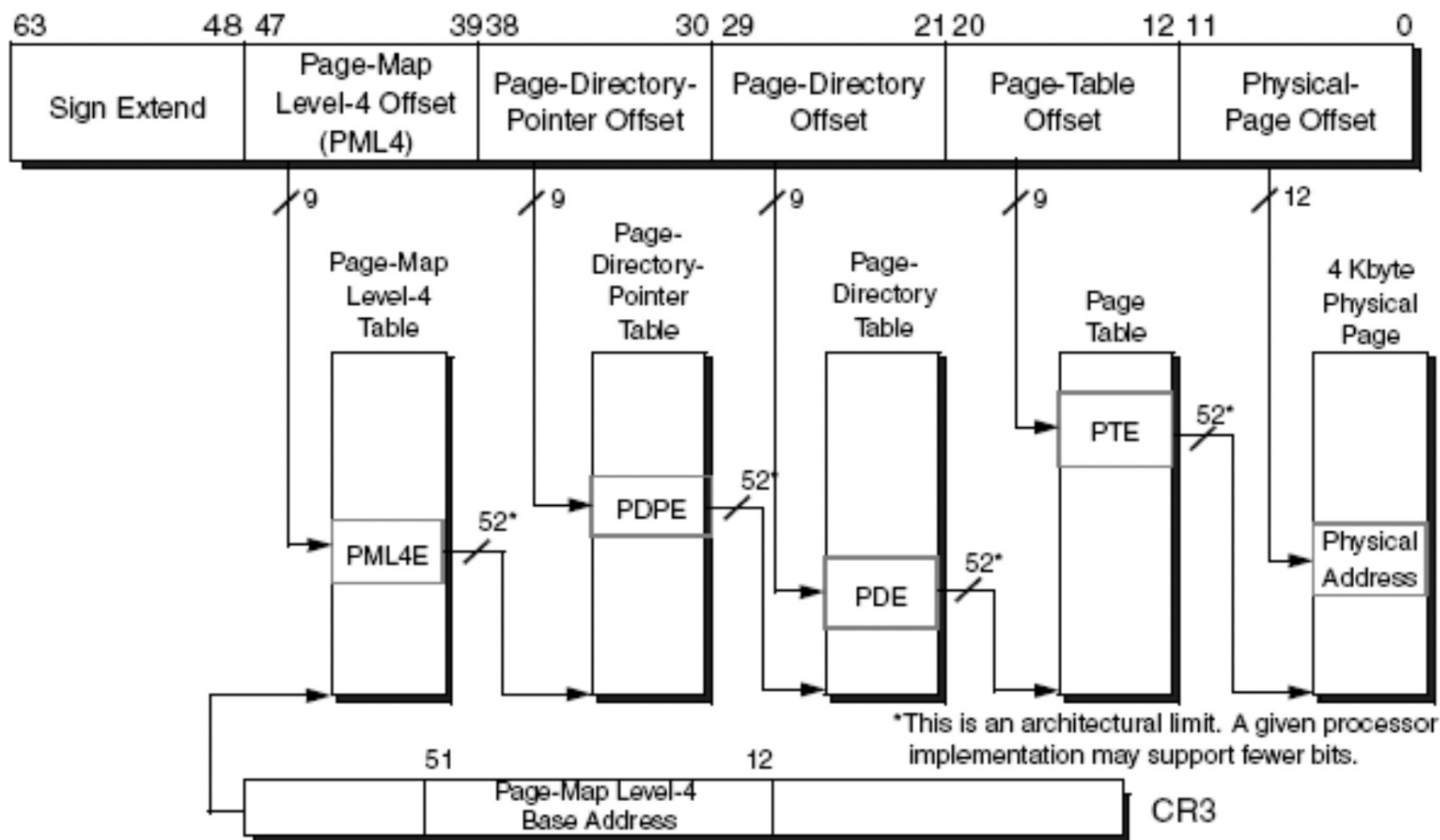


Intel x86-64

- Current generation Intel x86 architecture
- 64 bits is ginormous (> 16 exabytes)
- In practice only implement 48 bit addressing
 - Page sizes of 4 KB, 2 MB, 1 GB
 - Four levels of paging hierarchy
- Can also use PAE so virtual addresses are 48 bits and physical addresses are 52 bits (เราไม่พิจารณาเรื่อง PAE ในวิชานี้)

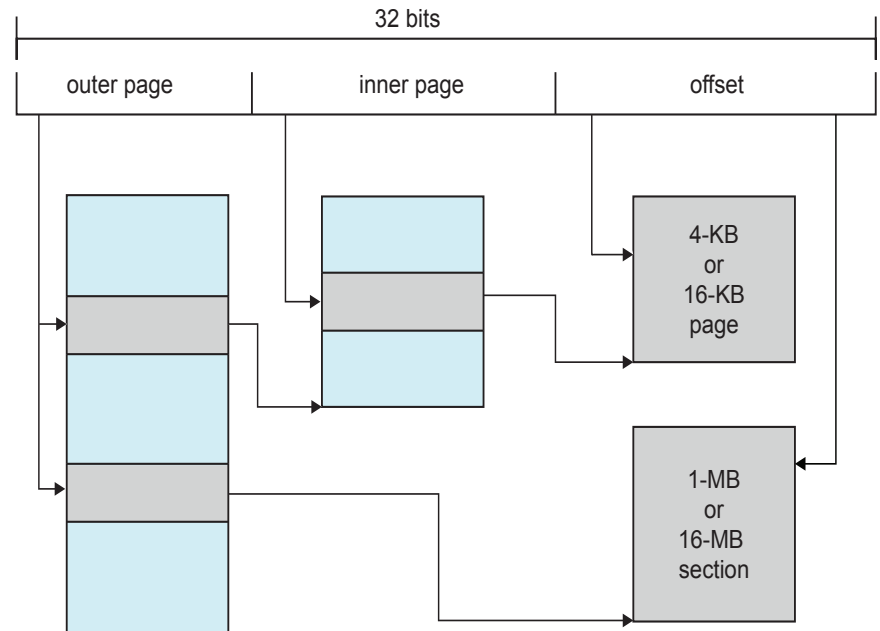


Linear/Virtual Address



Example: ARM Architecture

- Dominant mobile platform chip (Apple iOS and Google Android devices for example)
- Modern, energy efficient, 32-bit CPU
- 4 KB and 16 KB pages
- 1 MB and 16 MB pages (termed **sections**)
- One-level paging for sections, two-level for smaller pages
- Two levels of TLBs
 - Outer level has two micro TLBs (one data, one instruction)
 - Inner is single main TLB
 - First inner is checked, on miss outers are checked, and on miss page table walk performed by CPU



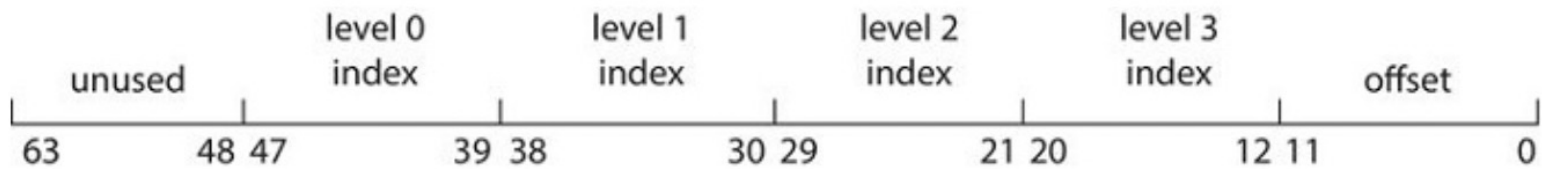


Figure 9.26 ARM 4-KB translation granule.

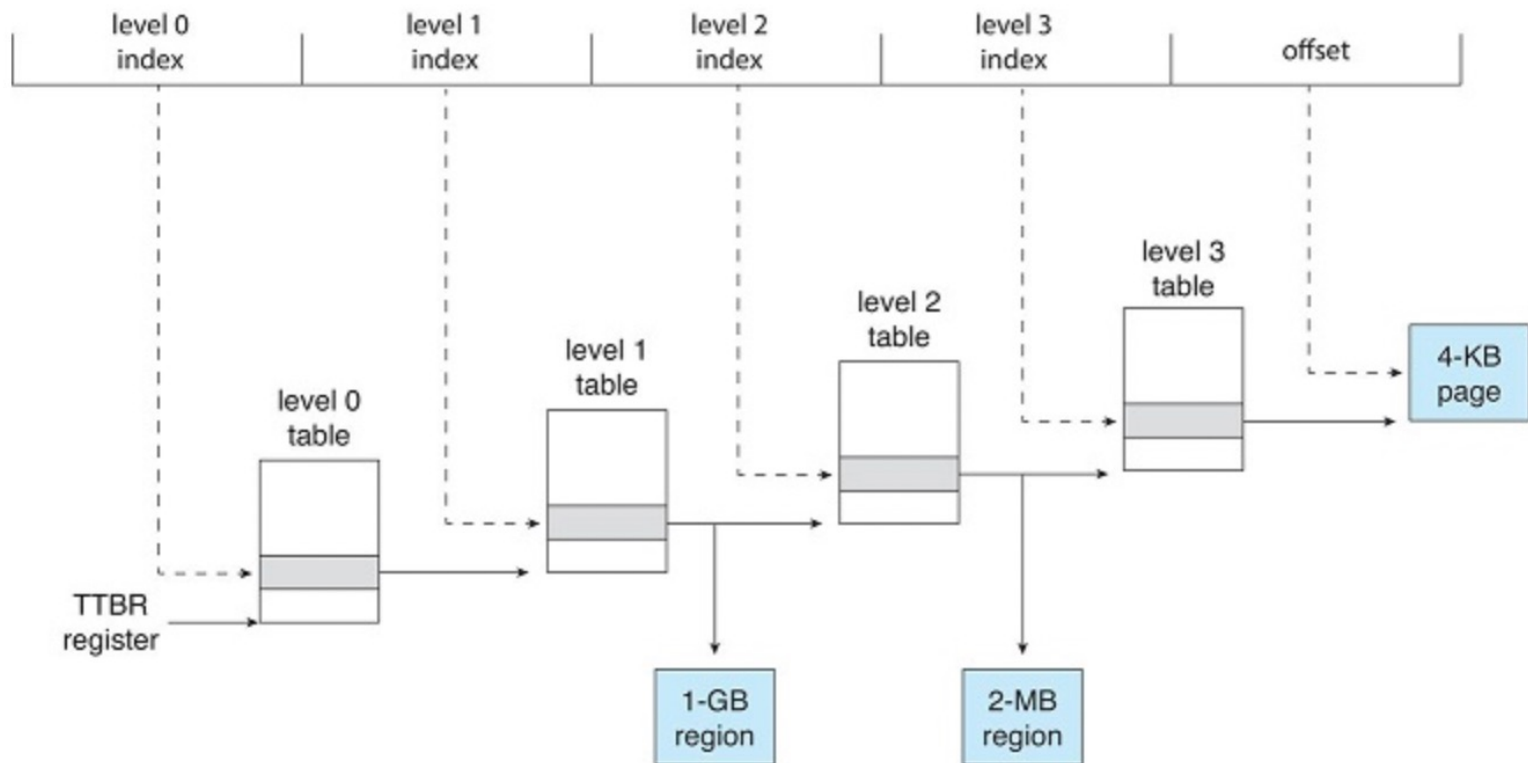
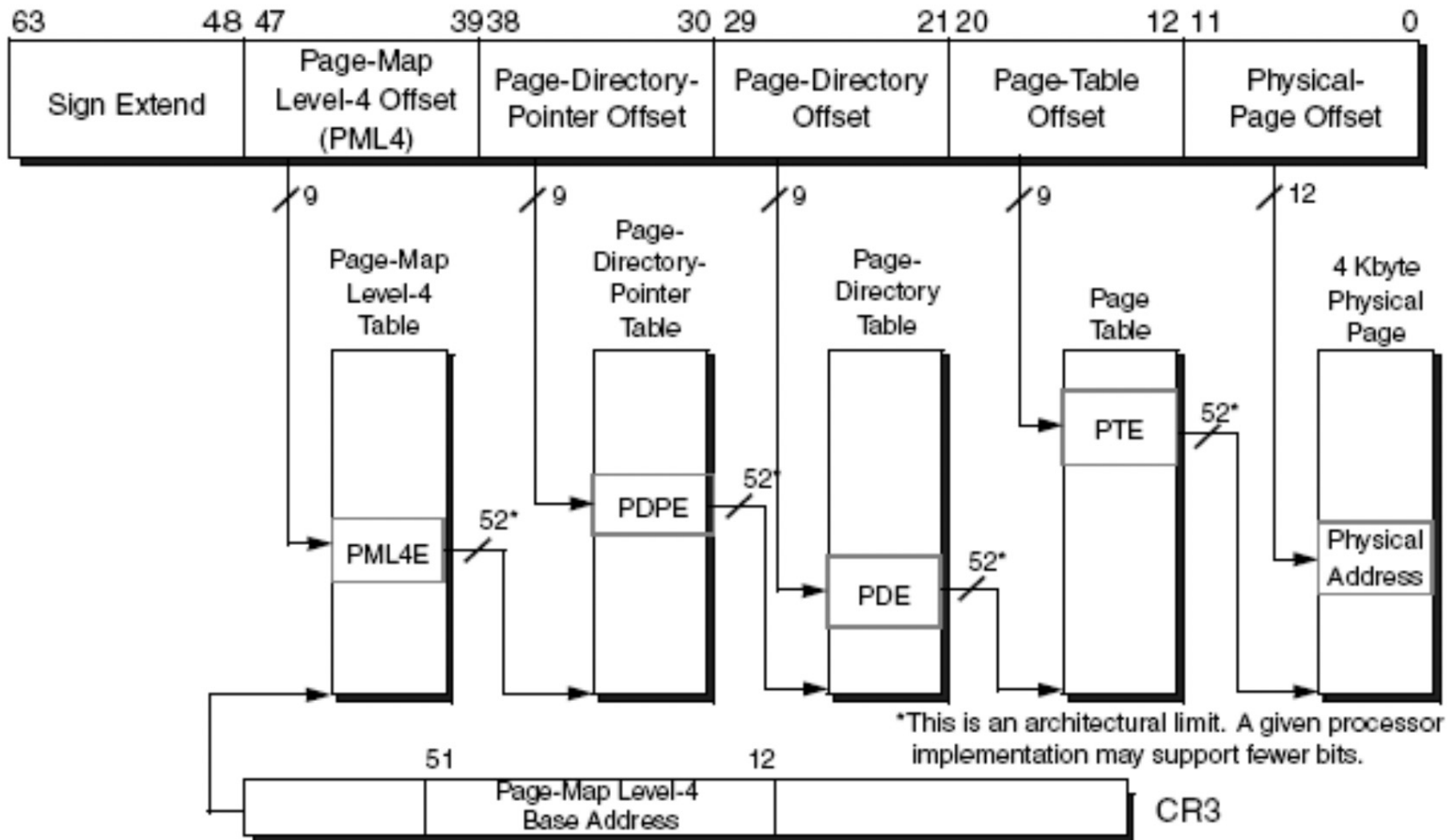


Figure 9.27 ARM four-level hierarchical paging.

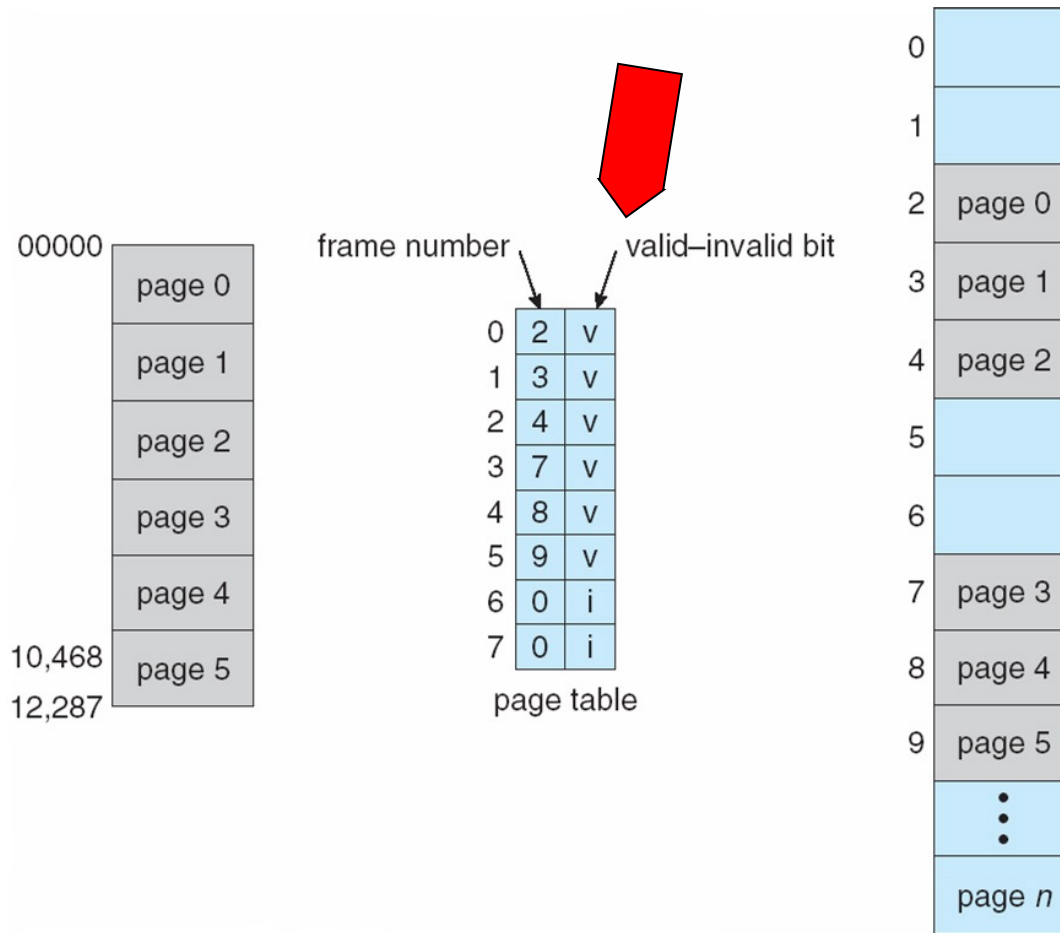
Memory Protection in 64bits Hierarchical Page Table

Revisit: x86_64 Architecture

Linear/Virtual Address

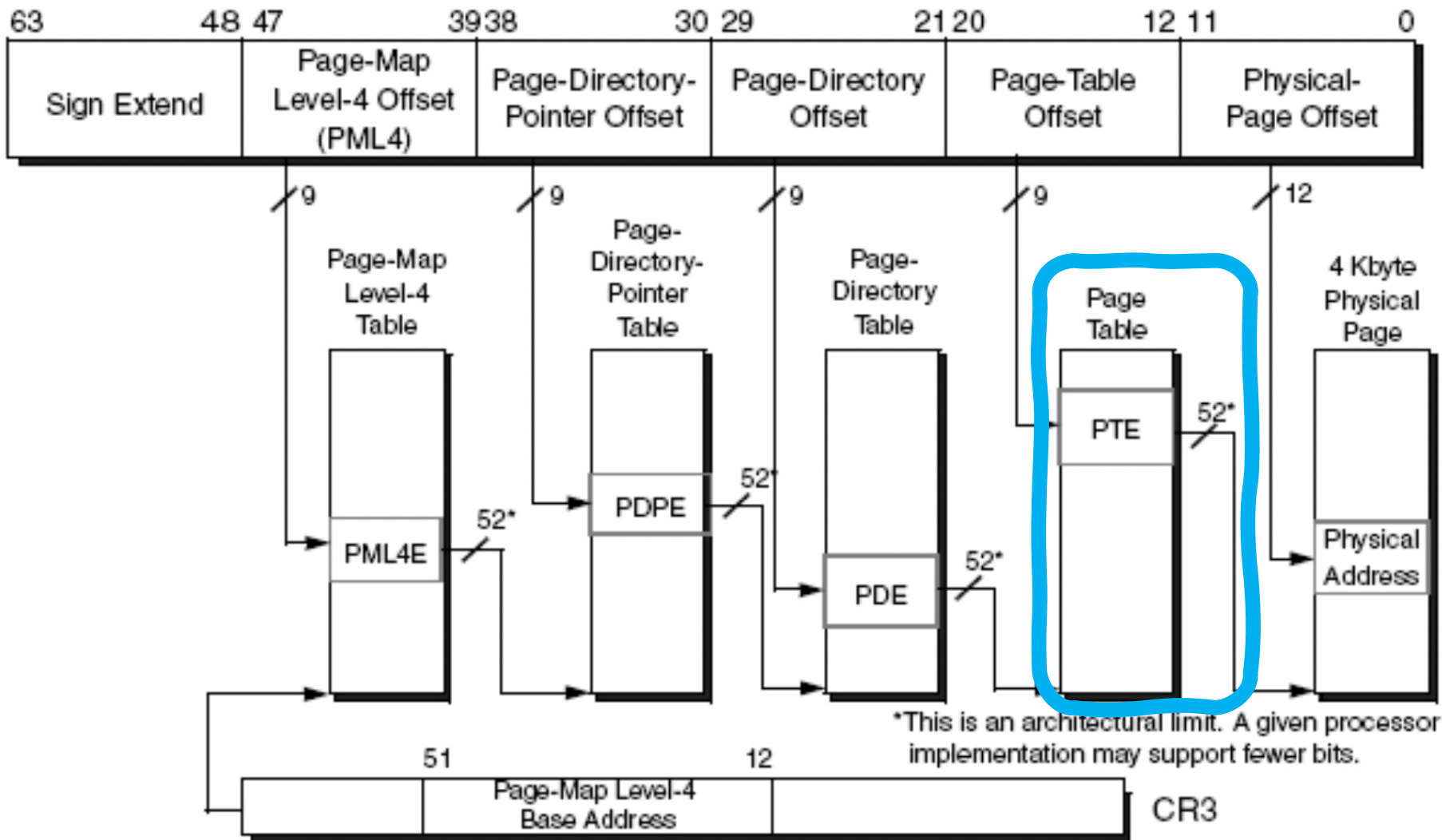


So, where Linux store those Valid (v) or Invalid (i) Bit In A Page Table?

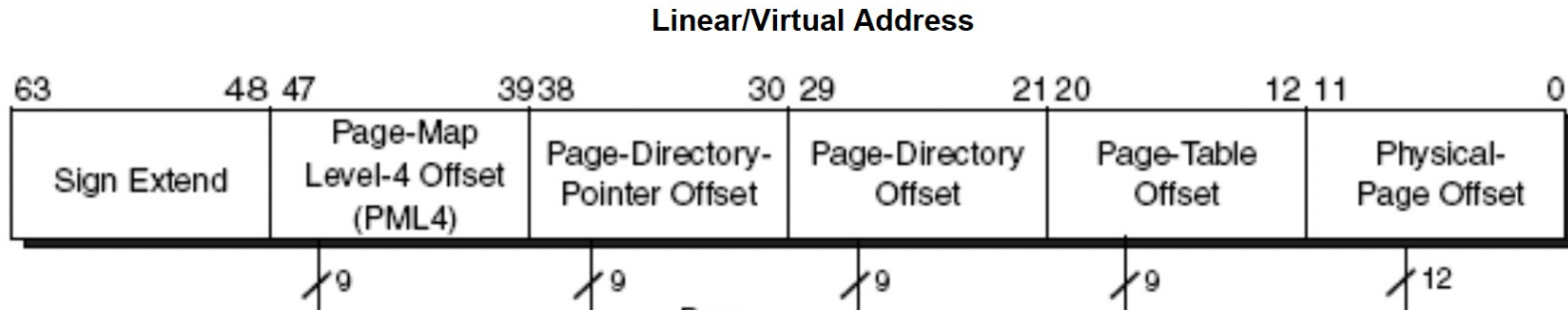


มันอยู่ใน Page Table Entry (PTE)

Linear/Virtual Address



Page table Entry



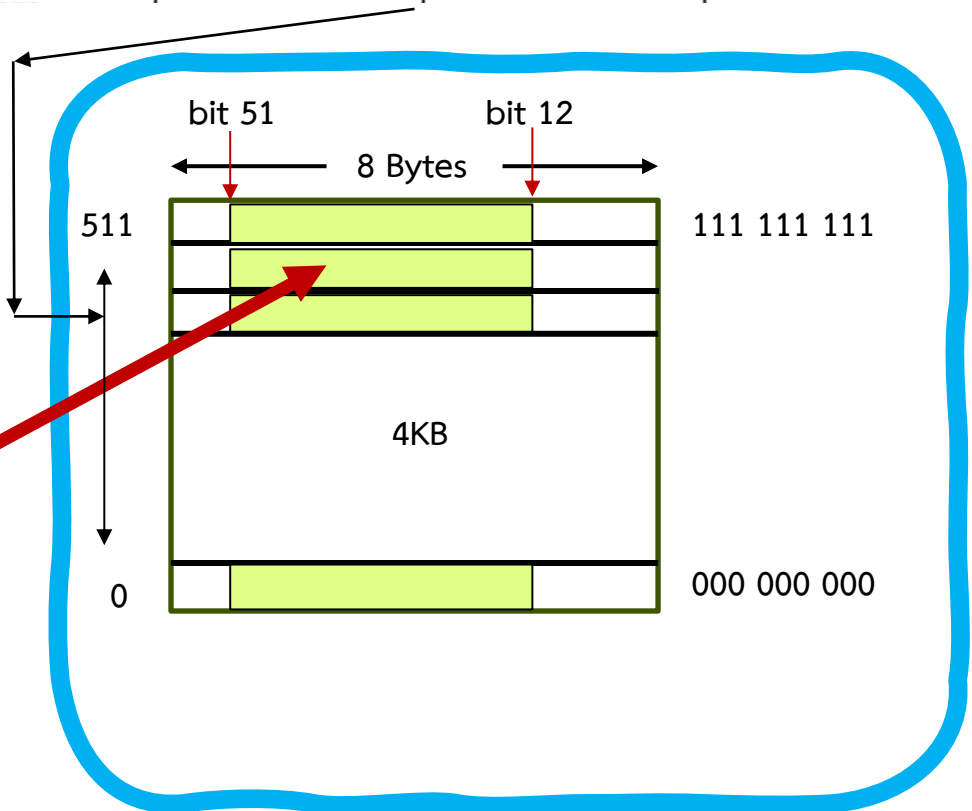
- PTE มีขนาดรวม 4KB

- เป็น Array ของ Element 512 Element

- แต่ละ Element มีขนาด 64 bits (หรือ 8 bytes)

- แต่ละ Element ใช้เพียงแค่ bit ที่ 12 ถึง 51 เพื่อเก็บ physical memory address ที่ชี้ไปยัง target physical mem frame

- Bit ที่เหลือเอาไว้เก็บค่า status



x86_64 Page table Entry Format

666655555555		3210987654321		M ¹ M-1		3332222222221111111111		210987654321098765432109876543210																						
X D	Prot. Key ⁴	Ignored	Rsvd.	Address of 4KB page frame																Ign.	G	P A T	D	A	P C D	W T	U S	R / W	1	PTE: 4KB page
				Ignored																									0	PTE: not present

Bit Position(s)	Contents
0 (P)	Present; must be 1 to map a 4-KByte page
1 (R/W)	Read/write; if 0, writes may not be allowed to the 4-KByte page referenced by this entry (see Section 4.6)
2 (U/S)	User/supervisor; if 0, user-mode accesses are not allowed to the 4-KByte page referenced by this entry (see Section 4.6)
3 (PWT)	Page-level write-through; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2)
4 (PCD)	Page-level cache disable; indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2)
5 (A)	Accessed; indicates whether software has accessed the 4-KByte page referenced by this entry (see Section 4.8)
6 (D)	Dirty; indicates whether software has written to the 4-KByte page referenced by this entry (see Section 4.8)
7 (PAT)	Indirectly determines the memory type used to access the 4-KByte page referenced by this entry (see Section 4.9.2)
8 (G)	Global; if CR4.PGE = 1, determines whether the translation is global (see Section 4.10); ignored otherwise
11:9	Ignored
(M-1):12	Physical address of the 4-KByte page referenced by this entry
51:M	Reserved (must be 0)
58:52	Ignored
62:59	Protection key; if CR4.PKE = 1, determines the protection key of the page (see Section 4.6.2); ignored otherwise
63 (XD)	If IA32_EFER.NXE = 1, execute-disable (if 1, instruction fetches are not allowed from the 4-KByte page controlled by this entry; see Section 4.6); otherwise, reserved (must be 0)

x86_64 Page table Entry Format

(อ้างอิง Perplexity AI)

Bits	Field/Usage
0	Present (P)
1	Read/Write (R/W)
2	User/Supervisor (U/S)
3	Page-level Write-Through
4	Page-level Cache Disable
5	Accessed (Reference)
6	Dirty
7	Page Size (for higher levels)
8	Global
9-11	Available to OS
12-51	Physical page frame address
52-62	Available to OS
63	No-execute (NX)

← Valid Bit (bit 0)

← Reference Bit (bit 5)

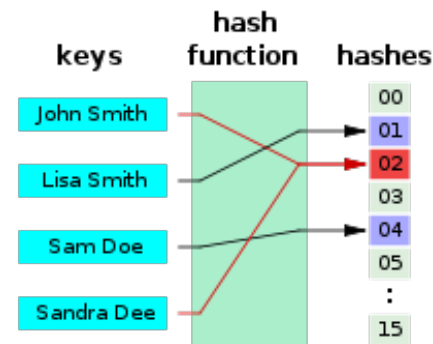
← Dirty Bit (bit 6)

← Physical Address

↑
เอา bit ที่ 12 ถึง 51 มาใช้ และเติมค่า 0 ใน bit 0 ถึง 11

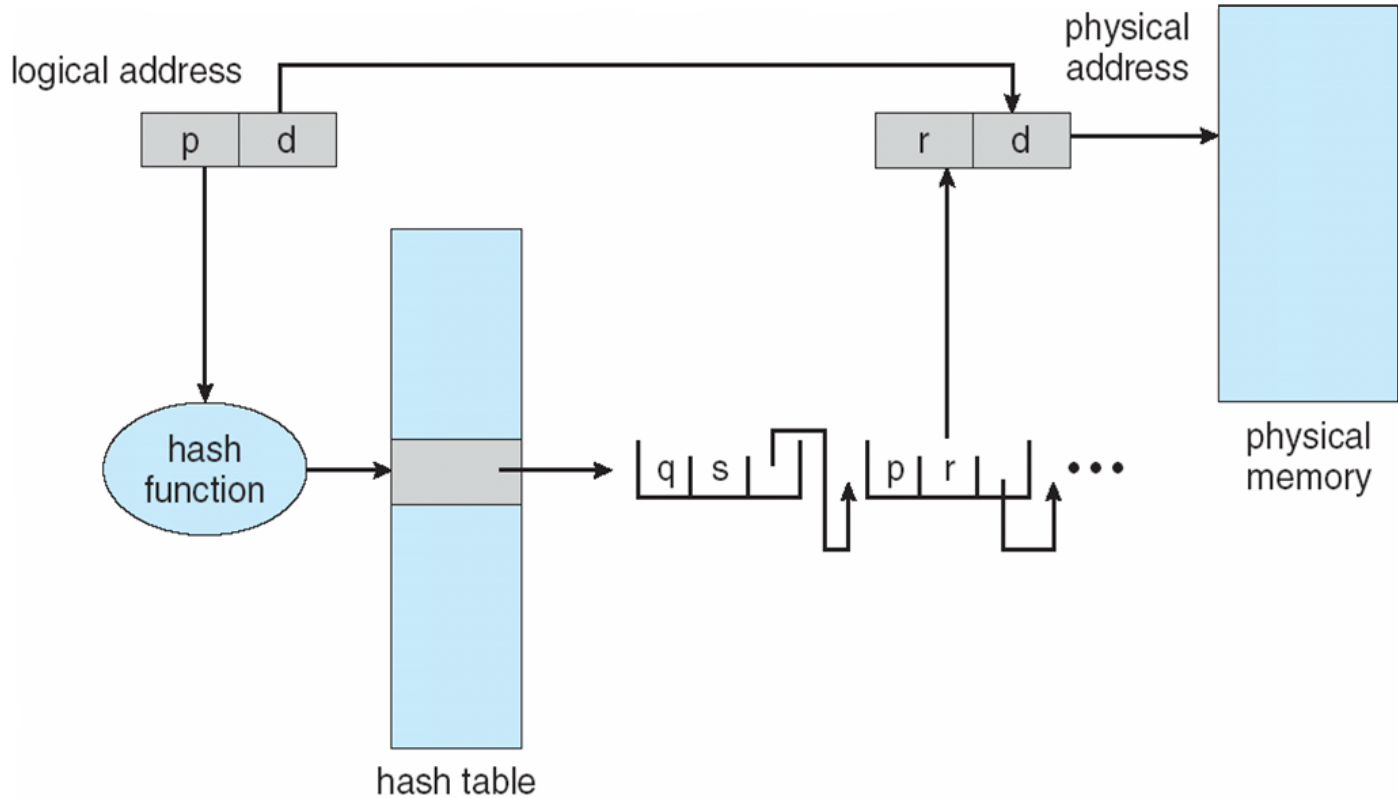
Hash Page Table

Hashed Page Tables



- hash function คือ function ที่ map ค่าข้อมูล ให้เป็นค่าที่มีขนาดจำกัด
 - ยกตัวอย่าง เช่น $f(x) = x \bmod 10$
 - ถ้าได้ค่า $f(x)$ เดียวกัน จากค่า x ต่างกัน เรียกว่าเกิด collision
 - <https://opensa-server.cs.vt.edu/ODSA/Books/CS3/html/HashFuncExamp.html>
 - https://en.wikipedia.org/wiki/Hash_function
- Common in address spaces > 32 bits
- The virtual page number is hashed into a page table
 - This page table contains a chain of elements hashing to the same location
- Each element contains (1) the virtual page number (2) the value of the mapped page frame (3) a pointer to the next element
- Virtual page numbers are compared in this chain searching for a match
 - If a match is found, the corresponding physical frame is extracted

Hashed Page Table



ในตัวอย่างนี้ $\text{hash}(p) = \text{hash}(q) = x$

จะต้องมี **list** ของ **page table entries** สำหรับแก้ปัญหา **collision**

Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits

0x0	0	
	1	
	2	
	3	
0x1	0	
	1	
	2	
	3	
0x2	0	
	1	
	2	
	3	
0x3	0	
	1	
	2	
	3	
0x4	0	
	1	
	2	
	3	
		
0xD	0	
	1	
	2	
	3	
0xE	0	
	1	
	2	
	3	
0xF	0	
	1	
	2	
	3	

Page id (p)

0x0	
0x1	
0x2	
0x3	
0x4	
0x5	
0x6	
0x7	
0x8	
0x9	
0xA	
0xB	
0xC	
0xD	
0xE	
0xF	

Frame id (f) : offset (d)

4 bits : 2 bits

0x0	0	
	1	
	2	
	3	
0x1	0	
	1	
	2	
	3	
0x2	0	
	1	
	2	
	3	
0x3	0	
	1	
	2	
	3	
0x4	0	
	1	
	2	
	3	
		
0xD	0	
	1	
	2	
	3	
0xE	0	
	1	
	2	
	3	
0xF	0	
	1	
	2	
	3	

Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits

0x0	0	
	1	
	2	
	3	
0x1	0	
	1	
	2	
	3	
0x2	0	
	1	
	2	
	3	
0x3	0	
	1	
	2	
	3	
0x4	0	
	1	
	2	
	3	
		
0xD	0	
	1	
	2	
	3	
0xE	0	
	1	
	2	
	3	
0xF	0	
	1	
	2	
	3	

$$X = \text{hash}(p) = p \% 2^3$$

$$= p \% 8$$

$$= p \% 0b1000$$

$$0x0 = 0\ 000$$

$$0x1 = 0\ 001$$

$$0x2 = 0\ 010$$

$$0x3 = 0\ 011$$

$$0x4 = 0\ 100$$

$$0x5 = 0\ 101$$

$$0x6 = 0\ 110$$

$$0x7 = 0\ 111$$

$$0x8 = 1\ 000$$

$$0x9 = 1\ 001$$

$$0xA = 1\ 010$$

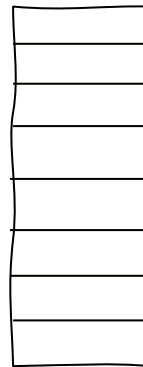
$$0xB = 1\ 011$$

$$0xC = 1\ 100$$

$$0xD = 1\ 101$$

$$0xE = 1\ 110$$

$$0xF = 1\ 111$$



$$0x0 = 000$$

$$0x1 = 001$$

$$0x2 = 010$$

$$0x3 = 011$$

$$0x4 = 100$$

$$0x5 = 101$$

$$0x6 = 110$$

$$0x7 = 111$$

Frame id (f) : offset (d)

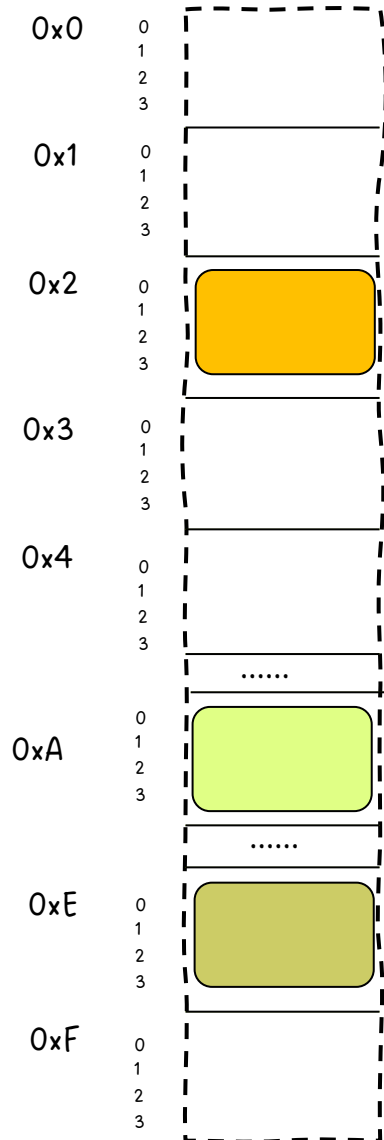
4 bits : 2 bits

0x0	0	
	1	
	2	
	3	
0x1	0	
	1	
	2	
	3	
0x2	0	
	1	
	2	
	3	
0x3	0	
	1	
	2	
	3	
0x4	0	
	1	
	2	
	3	
		
0xD	0	
	1	
	2	
	3	
0xE	0	
	1	
	2	
	3	
0xF	0	
	1	
	2	
	3	

Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 32 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits



$$\begin{aligned}
 X &= \text{hash}(p) = p \% 2^3 \\
 &= p \% 8 \\
 &= p \% 0b1000
 \end{aligned}$$

0x0 = 0 000
 0x1 = 0 001
 0x2 = 0 010
 0x3 = 0 011
 0x4 = 0 100
 0x5 = 0 101
 0x6 = 0 110
 0x7 = 0 111
 0x8 = 1 000
 0x9 = 1 001
 0xA = 1 010
 0xB = 1 011
 0xC = 1 100
 0xD = 1 101
 0xE = 1 110
 0xF = 1 111

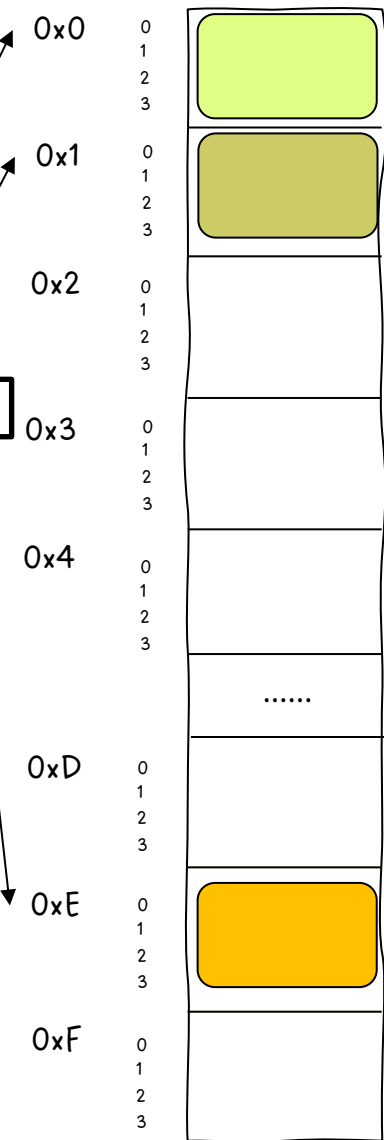
0x0 = 000
 0x1 = 001
 0x2 = 010
 0x3 = 011
 0x4 = 100
 0x5 = 101
 0x6 = 110
 0x7 = 111

A, 0x0

E, 0x1

Frame id (f) : offset (d)

4 bits : 2 bits



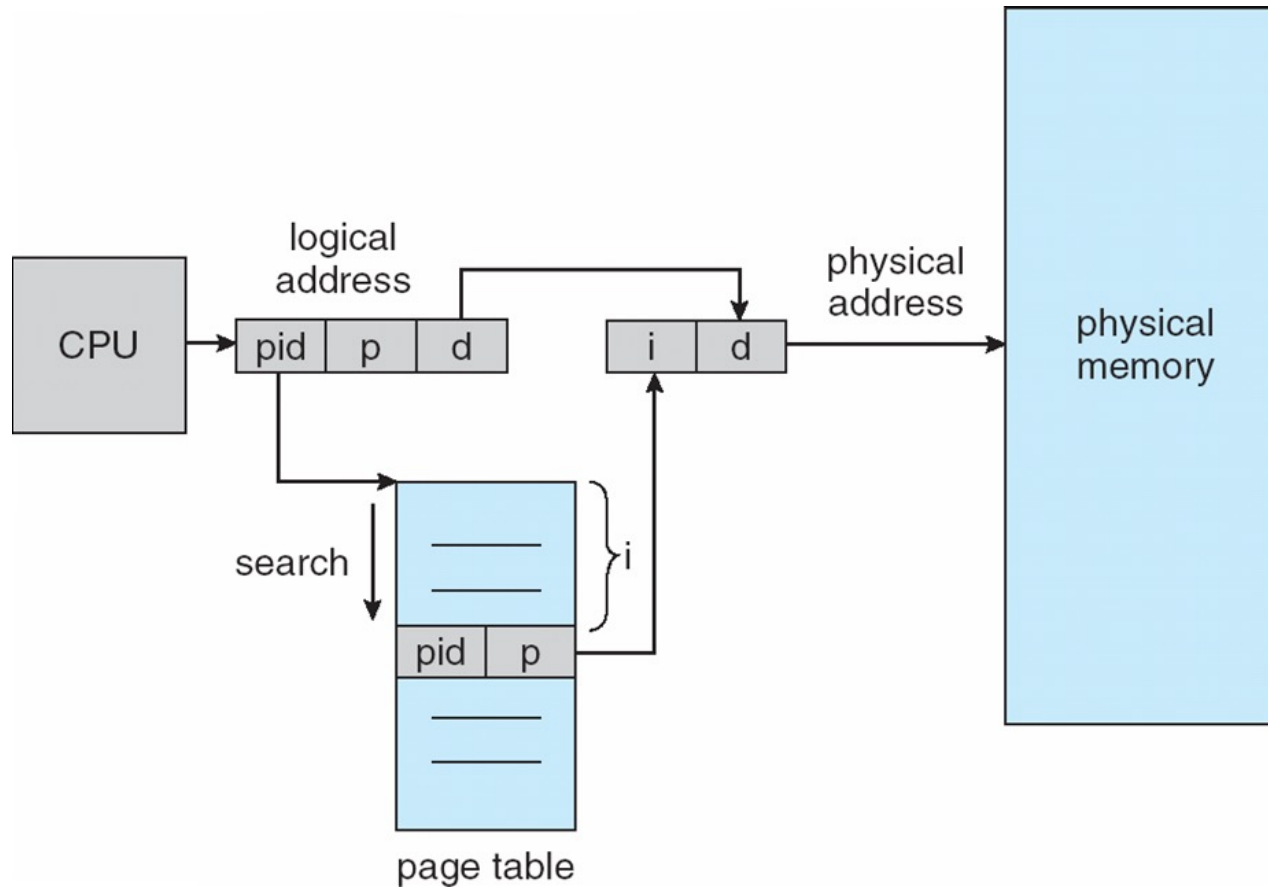
$p = E$, $\text{hash}(6) = 6$
 $P = A$, $\text{hash}(A) = 2$
 $P = 2$, $\text{hash}(2) = 2$ (collision)

Inverted Page Table

Inverted Page Table

- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- **Decreases memory** needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
 - TLB can accelerate access
- But how to implement shared memory?
 - One mapping of a virtual address to the shared physical address
 - ในช่อง **frame** หนึ่งช่อง เก็บค่า **page id** ได้ค่าเดียว

Inverted Page Table Architecture



Page Table แต่ละช่องเป็นตัวแทน frame

Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 32 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits

0x0	0 1 2 3	
0x1	0 1 2 3	
0x2	0 1 2 3	
0x3	0 1 2 3	
0x4	0 1 2 3	
		
0xD	0 1 2 3	
0xE	0 1 2 3	
0xF	0 1 2 3	

Inverted Page table

Frame id	PID	Page id
0x0	P1	0x11
0x1	P2	0x3C
0x2	P2	0x06
0x3	----	----
0x4	----	----
0x5	P1	0x10
0x6	----	----
0x7	P2	0x2E
0x8	P1	0x0F
0x9	----	----
0xA	P1	0x1B
0xB	P2	0x34
0xC	----	----
0xD	P1	0x2A
0xE	P1	0x14
0xF	----	----

Frame id (f) : offset (d)

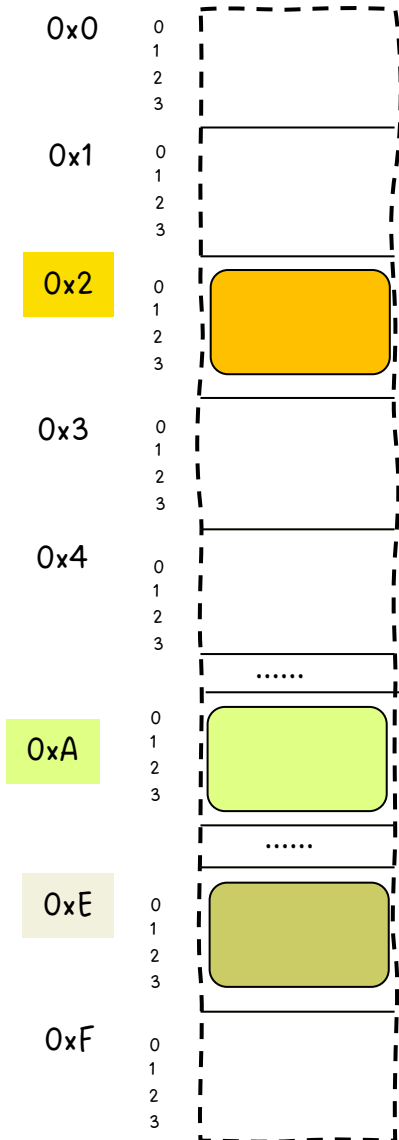
4 bits : 2 bits

0x0	0 1 2 3	
0x1	0 1 2 3	
0x2	0 1 2 3	
0x3	0 1 2 3	
0x4	0 1 2 3	
		
0xD	0 1 2 3	
0xE	0 1 2 3	
0xF	0 1 2 3	

Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 32 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits



Inverted Page Table
(IVT)

Frame id : Page id

0x0	0xA	p0
0x1	0xE	p0
0x2		
0x3	0xA	p1
0x4		
0x5		
0x6		
0x7		
0x8		
0x9	0x0	p1
0xA		
0xB		
0xC		
0xD		
0xE	0x2	p0
0xF		

When p0 runs:

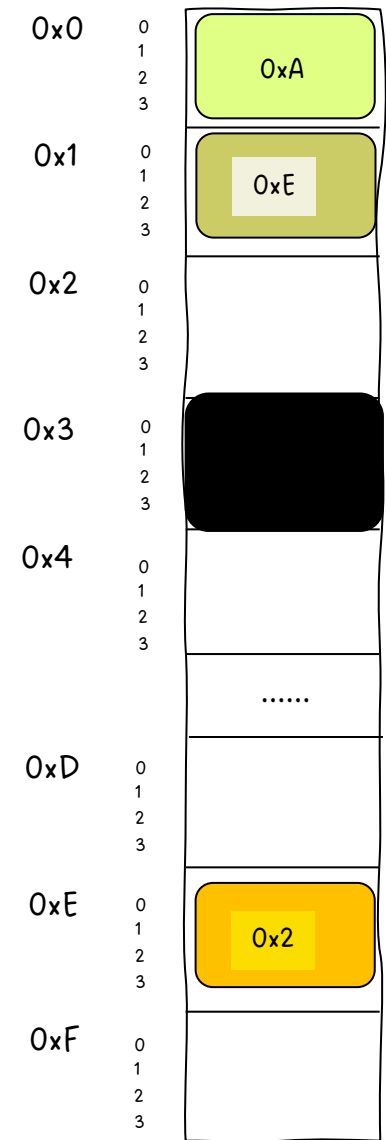
Pg = 0xE, Search entire IVT for ("0xE", p0) = 0x1

Pg = 0xA, Search entire IVT for ("0xA", p0) = 0x0

Pg = 0x2, Search entire IVT for ("0x2", p0) = 0xE

Frame id (f) : offset (d)

4 bits : 2 bits



Logical address: $n = 2$ and $m = 6$. Using a page size of $2^2 = 4$ bytes and a physical memory of 64 bytes (16 pages)

Page id (p) : offset (d)

4 bits : 2 bits

0x0	0	
	1	
	2	
	3	
0x1	0	
	1	
	2	
	3	
0x2	0	
	1	
	2	
	3	
0x3	0	
	1	
	2	
	3	
0x4	0	
	1	
	2	
	3	
		
0xD	0	
	1	
	2	
	3	
0xE	0	
	1	
	2	
	3	
0xF	0	
	1	
	2	
	3	

frame id Page table contents

0x0	0x11
0x1	0x3C
0x2	0x06
0x3	----
0x4	----
0x5	0x10
0x6	----
0x7	0x2E
0x8	0x0F
0x9	----
0xA	0x1B
0xB	0x34
0xC	----
0xD	0x2A
0xE	0x14
0xF	----

Frame id (f) : offset (d)

4 bits : 2 bits

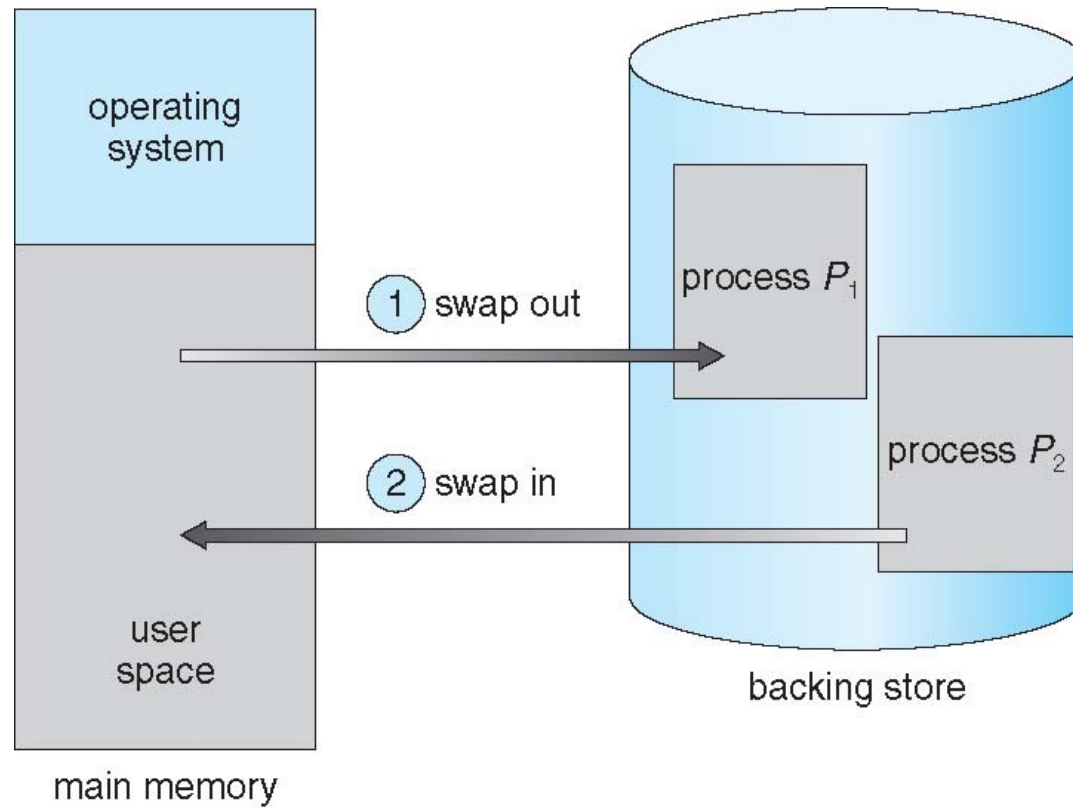
0x0	0	
	1	
	2	
	3	
0x1	0	
	1	
	2	
	3	
0x2	0	
	1	
	2	
	3	
0x3	0	
	1	
	2	
	3	
0x4	0	
	1	
	2	
	3	
		
0xD	0	
	1	
	2	
	3	
0xE	0	
	1	
	2	
	3	
0xF	0	
	1	
	2	
	3	

- โจทย์ ให้ ตัวอย่างเนื้อหาใน Physical Mem
แล้วให้ นศ สร้าง page table

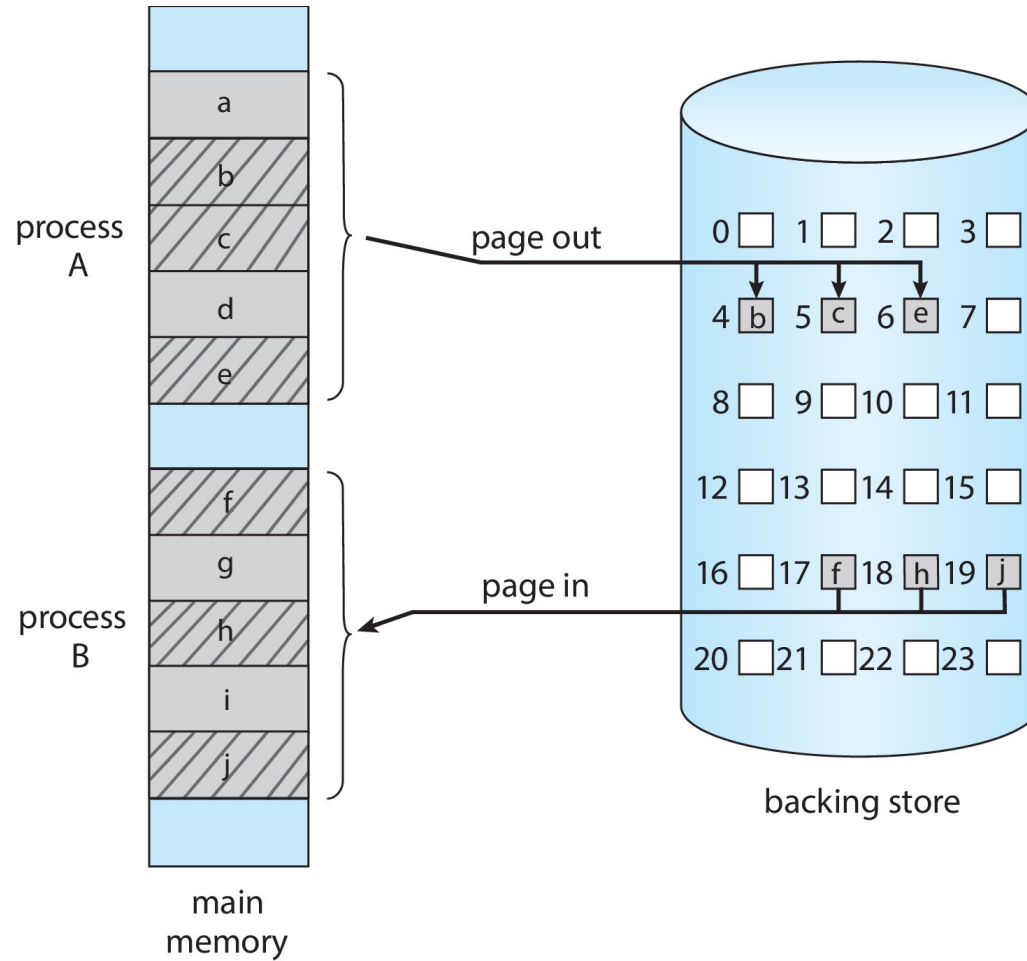
Swapping

- Standard Swapping
- A process can be **swapped** temporarily out of memory to a backing store, and then brought **back** into memory for continued execution
 - Total physical memory space of processes can exceed physical memory
- **Backing store** – fast disk large enough to accommodate copies of all memory images for all users; must provide direct access to these memory images
- **Roll out, roll in** – swapping variant used for priority-based scheduling algorithms; lower-priority process is swapped out so higher-priority process can be loaded and executed
- Major part of swap time is transfer time; total transfer time is directly proportional to the amount of memory swapped
- System maintains a **ready queue** of ready-to-run processes which have memory images on disk

Schematic View of Swapping



Swapping with Paging



Swapping on Mobile Systems

- Not typically supported
 - Flash memory based
 - ▶ Small amount of space
 - ▶ Limited number of write cycles
 - ▶ Poor throughput between flash memory and CPU on mobile platform
- Instead use other methods to free memory if low
 - iOS **asks** apps to voluntarily relinquish allocated memory
 - ▶ Read-only data thrown out and reloaded from flash if needed
 - ▶ Failure to free can result in termination
 - Android terminates apps if low free memory, but first writes **application state** to flash for fast restart
 - Both OSes support paging as discussed below

สรุป

- Memory Management
- Page Table
- TLB
- Page Table Implementations